Neo Redfish/Swordfish REST API

Dan Matthews
Tim Morneau
May 14, 2024

Table of Contents

References: 8
1. Introduction 9
2. High Level Usage 9
2.1. API Access 9
2.1.1. API TLS/HTTPS details 10
2.2. Session Authentication 10
2.3. Discover details of SessionService 11
2.4. Discover Active Sessions 11
2.5. Discover details of EventService 11
2.6. Event Subscription 11
2.7. Setup EventDestination 12
2.8. Discover Event history 12
2.9. Event Details 12
2.10. ServiceRoot 13
2.11. Available Resources 13
2.12. Detailed Resource Discovery 14
2.13. Resource Cross Referencing 15
2.14. Resource Collections 15
2.15. Resource Dynamics 19
2.16. Connecting Events to API resources 19
2.17. Resource tracking 19
2.18. Inventory resources using Events 20
2.19. API Session Termination 20
2.20. Unsubscribe an EventDestination 20
3. Token based API Authentication 20
3.1. Access Denied Before Authenticating 21
3.2. Login POST request 21
3.3. Authentication Token Use 25
3.4. SessionService Resource 28
3.5. Viewing Active Sessions 28
3.6. Deleting active Session 29

4. AccountService Resource 33
4.1 Role Based Access with Role Resource 34
4.1.1 The readonly Role 37
4.1.2 The limited Role 37
4.1.1 The full Role 38
4.2 External Accounts with ExternalAccountProvider Resource 38
4.2.1 S3 Lustre User Mapping Representation and Management 38
4.2.1.1 Creating an ExternalAcountProvider Resource to Represent an S3 User Mapping.. 38
4.2.1.2 Getting an ExternalAcountProvider Resource to Represent an S3 User Mapping ... 39
4.2.1.3 Removing an ExternalAcountProvider Resource to Represent an S3 User Mapping40
5. Redfish/Swordfish API ServiceRoot 42
5.1. ServiceRoot Resource: /redfish/v1 43
6. Cluster StorageSystems 44
6.1. Storage Node Representation 46
6.1.1. ComputerSystem Collections resources 46
6.1.2. ComputerSystemId 46
6.1.3. Example ComputerSystem populated Resource 47
6.2. NetworkInterfaces – NetworkInterfaceCollection Resource 50
6.3. NetworkInterface Representation 51
6.3.1 NetworkInterface Representation for a Physical Interface 52
6.3.2 NetworkInterface Representation for a Virtual Interface 53
7. ComputerSystem and Top-level service Resources 56
7.1 ComputerSystem.Links Chassis Collection 56
7.2 ComputerSystem.HostedServices.Services StorageService Collection 56
8. Cluster Chassis 57
8.1. Enclosure Representation as Chassis resource 58
8.1.1. Collections that are offered by a Chassis resource 59
8.1.2. ChassisId 59
8.2. The Assembly Resource 61
8.3. The Power Resource 64
8.4. Thermal Resource 67
8.5. Status.State fields 71
8.6. AssemblyData Resource 72
8.7. The PowerSupply Resource 74

8.8. The Fan Resource 77
9. Cluster StorageServices 80
9.1. Enclosure Services 81
9.1.1. StorageServiceId: 82
9.2. StoragePoolCollection Resource 83
9.3. StoragePool Resource 84
9.4. StoragePool CapacitySourceCollection Resource 87
9.5. StoragePool CapacitySource Resource 88
9.6. CapacitySource ProvidingDrives - DriveCollection Resource 90
9.7. Drive Resource 92
9.8. VolumeCollection Resource 94
9.9. Volume Resource 95
9.10. FileSystemCollection Resource 98
9.11. FileSystem Resource 100
10. Cluster Managers 103
10.1 Storage Node Manager 105
Manager Resource: /redfish/v1/Manager/{ManagerId} 105
10.2 HA Resource Group Representation – Redundancy 109
Redundancy Resource: /redfish/v1/Manager/{ManagerId}/Redundancy/{RedundancyId} 109
10.3 Manager Action Requests Which Operate on Redundancy Resources 113
10.3.1 Manager.StartFilesystem Action Request 114
/redfish/v1/Manager/{ManagerId}/Actions/Manager.StartFilesystem 114
10.3.2 Manager.StopFilesystem Action Request 130
/redfish/v1/Manager/{ManagerId}/Actions/Manager.StopFilesystem 130
10.3.3 Manager.MoveFilesystem Action Request 142
/redfish/v1/Manager/{ManagerId}/Actions/Manager.MoveFilesystem 142
10.4 Manager Action Requests Which Operate on Storage Node Power State 160
10.4.1 Manager.GetPowerState Action Request 160
/redfish/v1/Manager/{ManagerId}/Actions/Manager.GetPowerState 160
10.4.2 Manager.PowerOn Action Request 161
/redfish/v1/Manager/{ManagerId}/Actions/Manager.PowerOn 161
10.4.3 Manager.PowerOff Action Request 161
/redfish/v1/Manager/{ManagerId}/Actions/Manager.PowerOff 161
10.4.4 Manager.Reset Action Request 163

/redfish/v1/Manager/{ManagerId}/Actions/Manager.Reset 163
11. Top-Level Lustre FileSystem 164
11.1. Lustre FileSystem StorageService 165
11.2. Lustre FileSystem FileSystemCollection 166
11.3. Lustre FileSystem resource 167
12. RFSF API EventService and Events 169
12.1. EventService Resource 172
12.1.1 Updating the EventService resource 174
12.1.1.1 EventService PUT request 175
12.1.1.2 EventService PATCH request 178
12.2. Event Subscriptions 180
12.2.1. Event Resource Filtering 183
12.2.1.1 Event Resource Filtering Using EventDestination.ResourceTypes 184
12.2.1.2 Event Resource Filtering Using EventDestination.OriginResources 184
12.2.2 Updating an Event Subscription 185
12.2.2.1 Updating an Event Subscription via a PUT request 186
12.2.2.1 Updating an Event Subscription via a PATCH request 189
12.2.3 – Retiring of EventDestination subscriptions which no longer receive Events 193
12.2.4 – Restricting the Number of Queued Events/MetricReports During Delivery Retry 194
12.3. Runtime Event History 195
12.3.1. EventCollection Resource /redfish/v1/Events 195
12.3.2. Event resource /redfish/v1/Events/{EventId} 196
12.3.3 Rotating of runtime Event History 198
12.4. IcingaServices Events 198
12.5. Types of Events 199
12.6. Resources Events can occur on 200
12.7. Subscribers Receiving Events 201
13. MetricReport data Streaming Subscriptions 205
13.1 Available MetricReport Data Sets 205
13.1.1 LustreStats Data Set 206
13.1.2 LinuxStats Data Set 208
13.1.3 JobStats Data Set 209
13.1.4 EnvironmentalStats Data Set 213
13.1.5 TieringStats Data Set 214

13.2. MetricReport Event Subscriptions 215
13.2.1 MetricReport Event Subscriptions – EventDesetination.MetricReportDefinitions 216
13.3. MetricReport Subscription Restrictions and Behavior 216
13.3.1 MetricReport Subscription Generic Restrictions 217
13.3.2 MetricReport Subscription MetricReportDefinition/MetricDefinition specific Restrictions and Behavior 217
13.3.2.1 JobStats MetricReportDefinition Subscription specific Restrictions and Behavior . 217
13.3.2.1 TieringStats MetricReportDefinition Subscription specific Restrictions and Behavior 219
13.3.3 MetricReport subscription failed thread callback Events 220
13.4. MetricReport Subscription Example 221
13.5. Updating MetricReport Subscriptions 224
13.5.1 Updating MetricReport Subscription via a PUT request 224
13.5.2 Updating MetricReport Subscription via a PATCH request 228
13.6. Canceling MetricReport Subscriptions 230
14. TelemetryService Resource 232
14.1 MetricDefinitions Collection Resource 232
14.2 MetricDefinition Resource 234
14.2.1 Updating MetricDefinition Resources 234
14.2.1.1 Updating MetricDefinition Resource via a PUT request 235
14.2.1.2 Updating MetricDefinition Resource via a PATCH request 237
14.2.1.2 Restrictions for Updating MetricDefinition Resources 239
14.2.2 Creating MetricDefinition Resources 239
14.2.2 Removing MetricDefinition Resources 242
14.3 MetricReportDefinition Resources 244
14.3.1 MetricReportDefinitionCollection Resource 244
14.3.2 MetricReportDefinition resource 246
14.3.2.1 MetricReportDefinition.MetricReportDefinitionEnabled Property 247
14.3.2.2 MetricReportDefinition.MetricReportDefinitionType Property 248
14.3.2.3 MetricReportDefinition.Metrics Property 248
14.3.2.4 MetricReportDefinition.ReportActions Property 249
14.3.2.5 MetricReportDefinition.ReportUpdates Property 250
14.3.2.6 MetricReportDefinition.Schedule Property 251
14.4 MetricReport Resources 251
14.4.1 MetricReportCollection Resource 252

14.4.2 MetricReport Resource 253
14.6 Triggers Resources 255
14.6.1 TriggersCollection Resource 255
14.6.2 Triggers Resources 256
15. UpdateService Resource 258
15.1 SoftwareInventoryCollection Resource 259
15.2 SoftwareInventory Resource 260
16. rfsf-api-client: Client-side libraries to interact with the RFSF API 261
16.1. Populating the base config file to setup API destination and API user 262
16.2 Authenticate a user 263
16.3 Rfsf_api_client.inventory resource_tree discovery 265
16.4 Event Subscription with rfsf_api_client.subscription library 266
16.5 LustreStats MetricReport Events Subscription 269
16.6 LinuxStats MetricReport Events Subscription 273
16.7 JobStats MetricReport Subscription 277
16.8 Resource Tree Iteration Examples 281
16.9 Example Event based monitoring setup 284
Appendix 1. Client Notes 287

Version	Author	Description
.01	Tim Morneau	Initial conversion via confluence
1.0.0	Dan Matthews, Tim Morneau	• Updated all code samples for visuals and techpubs conversion tools • Added Event management detail • General cleanup
1.1.0	Dan Matthews	• Add Update and Software Inventory
1.2.0	Dan Matthews	• Add event subscription update PUT/PATCH operations
1.3.0	Dan Matthews	• Add Updating EventService section with PUT/PATCH operations • Add Retiring of EventDestinations section • Add Rotating of runtime Event History section
1.4.0	Dan Matthews	• Tiering statistics
1.5.0	Dan Matthews Tim Morneau	• Added Dan's client helps Appendix entry.
1.6.0	Dan Matthews	• Added ManagerCollection resource • Added Manager resource • Added Manager Action ower state requests • Added Redundancy resources • Added Manager Action Redundancy requests • Added EventService.Oem.EventDeliveryRetryQueueSize documentation • Update Eventing description to indicate Events are published via gRPC POST and not dbus message

References:

Swordfish spec: https://www.snia.org/sites/default/files/technical-work/swordfish/release/v1.2.3/html/Specification/Swordfish_v1.2.3_Specification.html

Swordfish schema definitions: https://redfish.dmtf.org/schemas/swordfish/

Redfish spec: https://www.dmtf.org/sites/default/files/standards/documents/DSP0266_1.11.1.pdf

Redfish schema definitions: https://redfish.dmtf.org/schemas/v1/

1. Introduction

The NEO Redfish/Swordfish REST API is designed to provide a Redfish/Swordfish based RESTful interface which will initially be used for cluster monitoring. The API is based on Redfish, a management standard that uses data model representation with a RESTful interface. Being RESTful, Redfish is easy to use and implement, being model-oriented, it can express the relationships between components in the cluster, and the semantics of the Redfish services and components within them. All components the API manages are represented as "resources", all resources are accessible through a "resource tree" structure, with a well-known starting URI as the root. Redfish also provides a model which is easy to extend. By requiring JSON representation, Redfish enables easy integration with programming environments, and is easy to interpret by humans. The API is extended by Swordfish, which defines a RESTful interface and a standardized data model to provide a scalable, customer-centric interface for managing storage and related data services. Swordfish is an extension to Redfish, and therefore follows the same model-oriented, JSON represented semantics that Redfish standardizes.

The API is implemented in a way that the URI and "resource tree" structure will always stay the same, regardless of the backend data sources the API populates the resources with. This model makes the API easily extendable without changing the overall structure. The initial implementation of the API will be used for monitoring purposes and is readonly, however, may be extended in the future for additional management actions.

The following document describes how use the API for ClusterStor monitoring purposes.

2. High Level Usage

The following will describe how to effectively use the NEO Redfish/Swordfish API at a high level for the purposes of cluster monitoring. Several steps to set up the API for client usage will be described, as well as suggested methods to inventory all resources provided by the API, and mechanisms to periodically monitor their state and status. Sections of this document that provide more detail will be referenced throughout.

2.1. API Access

Access, Deployment and Initial Access Details

The NEO RFSF API main RESTful server listens on port 8081 on the primary MGMT node in a ClusterStor system and can be accessed either internally OR externally to the cluster. Internally, the API can be accessed on localhost if running from the primary mgmt node itself, or across the private mgmt network from any other node in the cluster. Externally, the API can be accessed across the public or customer network, the same network used for ClusterStor GUI or login, via the configured public IP addresses for the current primary mgmt node. The API initially will use https protocol as a read-only first iteration.

The API is deployed as a set of 6 systemd services under HA control that are required to run on the MGMT primary node which provides access to all databases and other services that are stored in the md64 shared management array. This set of 6 systemd HA primitive services form a HA service group called "grp-rfsf-api" that can be failed back and forth between the two mgmt nodes following the md64-group.

The NEO RFSF API provides only two endpoints that are accessible without an authentication token granted to a valid ClusterStor admins user. You can either access the ServiceRoot endpoint, or the endpoint to authenticate a user and create a session. See below for more details.

2.1.1. API TLS/HTTPS details

The API is secured by SSL certificate. It uses the same self signed certificate that Apache is using for CSM GUI. Customer has the option to upload the certificate signed by a trusted **certificate authority** (CA). The API can be accessed using HTTPS. In case of self signed certificate, "insecure connection" must be accepted, in that, certificate validation is bypassed. For example, using the "-k, --insecure" option in curl:

```
       -k, --insecure
              (TLS)  By default, every SSL connection curl makes is verified to be secure.
This option allows curl to proceed and
              operate even for server connections otherwise considered insecure.
```

Certificate validation bypass options should not be necessary if a properly CA signed certificate specific to that ClusterStor system is uploaded by the customer.

2.2. Session Authentication

By definition of the Redfish Specification, the only endpoints which do not require authentication if an authentication mechanism is implemented is the /redfish/v1 (also known as the Service Root endpoint), as well as the /redfish/v1/SessionService/Sessions endpoint to create a Session and authenticate a user. To access the rest of the API, an Authentication Token is required. To Authenticate a user and receive a token, you must first create a ClusterStor admin user (that is not the admin user there by default) via the cscli admins command. This user should have a "readonly" role, and a password that meets the criteria of a user password defined by the cscli admins command. Once the user is created, a POST request to /redfish/v1/SessionService/Sessions which passes user credentials in the body will grant a user an access token, if the request is valid. A successful POST request will create an active Session

fChasor that user. **Section 3 "Token based API Authentication"** provides more detail on how this authentication request and the granted token work.

2.3. Discover details of SessionService

The SessionService resource provides more detail about Sessions that are created, as well as a list of created Sessions. To view details of the SessionService resource, run a GET request to /redfish/v1/SessionService. **Section 3.4 "SessionService Resource"** provides more detail on this.

2.4. Discover Active Sessions

Once authenticated, it is possible to view a list of Active Sessions via a GET request to the /redfish/v1/SessionService/Sessions endpoint. To view details of a specific Session, run a GET request to /redfish/v1/SessionService/Sessions/{SessionId} for the SessionId of interest. This will provide detail of which user the Session is active for, as well as information about when the Session was created, when it expires, and how much time remains. **Section 3.5 "Viewing Active Sessions and Individual Session Details"** provides more detail on how to look at active Sessions.

2.5. Discover details of EventService

The EventService resource provides details about all EventType's and ResourcesTypes that the API will send Events on. It will also provide a list of active EventService Subscriptions. To view details of the EventService resource, run a GET request to /redfish/v1/EventService. **Section 10.1 "EventService Resource"** provides more details on this.

2.6. Event Subscription

Subscribe to Events or MetricReports from EventService

The NEO Redfish/Swordfish API provides an EventService which will receive Events from various services which describe a change or provide metrics to a service or resource. The EventService will then Broadcast these Events to all subscribed EventDestinations, which are then free to do what they see fit with these Events. To subscribe to receive Events from the API EventService, you must send a POST request to /redfish/v1/EventService/Subscriptions/{EventDestinationId}, where EventDestinationId is some unique Id. The POST request body must be an EventDestination resource structure, which provides a Destination field of an endpoint that EventService will broadcast Events to. **Section 10.2 "Event Subscriptions"** provides more detail on how to subscribe to receive Events from EventService. Sections 10.4 “IcingaServices Events”, 10.5 “Types of Events”, and 10.6 “Resources Events Can Occur On”, provide additional detail on NEO RFSF specific Events that may not be contained fully within the EventService resource. Section 11 “MetricReport Event Subscriptions for Metric Data Streaming” also provides details that differentiate a MetricReport

subscription from a generic Event subscription, including the EventDestination request body requirements, and full details of how data streaming is achieved. Please see these sections for full NEO RFSF Event details.

2.7. Setup EventDestination

The EventDestination must implement an Event Receiver service, i.e., http/https service.

As mentioned above, once subscribed to receive Events from API EventService at the endpoint specified in the subscription POST request body Destination field, clients must implement an Event Receiver service that accepts http POST requests on the specific address and port specified in that Destination field, in order to receive broadcasts from the API EventService. The API EventService broadcasts Events simply by sending a POST request with the Event json body it received as the payload to all Subscribers known to the API in /redfish/v1/EventService/Subscriptions collection. In order to always receive Events, the Event Receiver service to handle these POST requests must always be running, and the port specified in the EventDestination must be open to accept tcp connections on the server of the IP address specified in the EventDestination field. Once clients are receiving Events, it is up to the user what they would like the Receiver Service to do with them. **Section 10.7 "Subscribers Receiving Events"** provides more detail as well as an example on how to set this up.

2.8. Discover Event history

Examine individual historical Events

While it is up to the Event Receiver to decide what they want to do with Events, the NEO RFSF API will save off history of Events for the runtime of RFSF API service. If the service is stopped, restarted, or failed over for any reason, all Event runtime history will be lost. For this reason it is suggested that Event Receiver services save off Events in some persistent manner. To see all runtime Events that have occurred, you can run a GET request to /redfish/v1/Events. To see a single runtime Event that has occurred, you can run a GET request to /redfish/v1/Events/{EventId}. **Section 10.3 "Runtime Event History"** provides more details about this, as well as **Section 10.3.1 "EventCollection Resource /redfish/v1/Events"** and **Section 10.3.2 "Event resource /redfish/v1/Events/{EventId}".**

2.9. Event Details

Interpret the details of and react to an Event

Whether looking at the request body received on the Event Receiver side, or re-examining a particular Event that occurred in the past, there are several useful informational fields in the Event resource body to describe what the Event was that occurred, when that Event occurred, what type of Event occurred, and what resource and resource type that Event occurred on.

Reacting to the Event, the Event receiver service will likely send some type of alert itself, such as an email notification, to whatever client monitoring system may be in use - the details of the Event resource body can provide content of that email (or whatever) alert that will be useful for service. The client monitoring process can also be told to go run another GET request to the resource which the Event occurred on to confirm the status change the Event may have highlighted. It could continue to monitor the status of that resource until it returns to "Ok". **Section 10.3.1 "Event resource /redfish/v1/Events/{EventId}"** and **Section 10.7 "Subscribers Receiving Events"** provide more detail on this topic.

2.10. ServiceRoot

Discover ServiceRoot to determine all resource service entry points

The ServiceRoot request serves as a top-level entry point to access all API resources. It displays the sub-section Service entry points where all underlying resources exist, which shows the start of the various paths of the API to traverse when discovering resources. As mentioned before, by definition, the ServiceRoot request does not require an authenticated Session, but anything below the ServiceRoot (except for Session creation endpoint) does. To view the ServiceRoot, run a GET request to /redfish/v1. **Section 4 "Redfish/Swordfish API Service Root"** provides more details on this.

2.11. Available Resources

Discover top-level resource collections and determine what they represent

As mentioned above, all of the sub-service entry points shown in the ServiceRoot resource provide paths to all underlying resources that exist in the API. These top-level service entry points are known as Collections, which provide a collection of all resource URIs of that collection type that exist in the API. Also seen in the ServiceRoot resource, are the EventService and SessionService entry point, which are not necessarily collections, but provide the paths to collections of EventDestinations and Sessions respectively. The next step to discovering all API resources after looking at the ServiceRoot, is to discover all resources that exist in these top-level Collections. Below are the top-level Collections offered by the NEO RFSF API.

/redfish/v1/Events - EventCollection

- **Section 10.3 "Runtime Event History" - provides more details on this**

/redfish/v1/StorageSystems - the ComputerSystemCollection resource

- **Section 5 "Cluster StorageSystems - StorageSystemCollection Resource /redfish/v1/StorageSystems" - provides more details on this**

/redfish/v1/StorageServices - the StorageServiceCollection resource

- **Section 8 "Cluster StorageServices - StorageServiceCollection Resource /redfish/v1/StorageServices" - provides more details on this**

/redfish/v1/Chassis - the ChassisCollection resource

- **Section 7 "Cluster Chassis - ChassisCollection Resource /redfish/v1/Chassis" - provides more details on this**

/redfish/v1/EventService/Subscriptions - Provides path to the EventDestinationCollection resource

- **Section 10.1 "EventService Resource" - provides more details on this**

/redfish/v1/SessionService/Sessions - Provides path to the SessionCollection resource

- **Section 3.4 "SessionService Resource" - provides more details on this**

2.12. Detailed Resource Discovery

Discover each resource within a top-level resource collection and determine what it represents, and the data it offers

As mentioned above, all of the top-level Collections resources listed above provide a collection of all resources URIs of that collection type that exist in the API. All of these individual resources URIs provide data about the component they represent or some service the component they represent provides. Because there are multiple top-level resources of the same type in each collection, each top-level resource must be uniquely identifiable by some Id, which can be represented as a number of data points taken from the system that identifies the component each resource represents, for example: node_id, serial_number, etc. The next step to discovering all API resources after looking at the top-level Collections, is to run GET request to each top-level resource in each Collection to see the data that it offers. Below are the top-level resources within each Collection offered by the NEO RFSF API.

/redfish/v1/Events/{EventId} - the Event resource

- **Section 10 "RFSF API EventService and Events" - provides more details on this**

/redfish/v1/StorageSystems/{ComputerSystemId} - the ComputerSystem resource

- **Section 5.1 "Storage Node Representation - ComputerSystem Resource: /redfish/v1/StorageSystems/{ComputerSystemId}" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId} - the StorageService resource

- **Section 8.1 "Enclosure Representation - StorageService - StorageService Resource: /redfish/v1/StorageServices/{StorageServiceId}" - provides more details on this**

/redfish/v1/Chassis/{ChassisId} - the Chassis resource

- **Section 7.1 "Enclosure Representation - Chassis Resource: /redfish/v1/Chassis/{ChassisId}" - provides more details on this**

/redfish/v1/EventService/Subscriptions/{EventDestinationId} - the EventDestination resource

- **Section 10.2 "Event Subscriptions" - provides more details on this**

/redfish/v1/SessionService/Sessions/{SessionId} - the Session resource

- **Section 3.5 "Viewing Active Sessions and Individual Session Details" - provides more details on this**

2.13. Resource Cross Referencing

Determine how top-level resources are linked to or associated with each other

The main link of top-level resources to highlight here is the ability to associate a ComputerSystem resource (which represents a Storage node) to the Chassis and StorageService resources it is attached to (which represents the physical Storage enclosure(s) the Storage node is attached to). There are two main associations to highlight which can be seen within the ComputerSystem json structures:

- ComputerSystem.HostedServices.StorageServices - shows the associated StorageService resources with this ComputerSystem
- ComputerSystem.Links.Oem - shows the associated Chassis resources with this ComputerSystem
- **Section 6 "ComputerSystem Relationships to Top-level service Resources" provides more details**

Another important piece to highlight which is all Chassis resources have a matching StorageService resource where ChassisId == StorageServiceId. This is because both resources represent the same Storage enclosure, but different services and sub-structure resources are provided under each service entry point: /redfish/v1/Chassis and /redfish/v1/StorageService respectively. **Sections 7.1.2 ChassisId and 8.1.1 StorageServiceId** provide more detail on this.

2.14. Resource Collections

Discover all resources which exist within sub-structures of each resource within a top-level resource collection

Much like the cluster components the above top-level resources represent, i.e. a Storage Node or Storage Enclosure, each of these top-level resources contain several resources within themselves to represent additional software and hardware components provided by that cluster component. For example, a Storage Node may contain NetworkInterface cards, and a Storage Enclosure may contain several Drives which provide software components such as Volumes or FileSystems. After discovering all individual top-level resources as mentioned above, the next step to discovering all API resources is to examine the data available within each top-level resource and continue to traverse the paths down their endpoints to discover all resources which exist within sub-structures of these top-level resources. Below are all resource which exist within the sub-structures of these top-level resources which represent cluster components - ComputerSystem, StorageService and Chassis resources:

ComputerSystem sub-structure resources:
/redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces - the NetworkInterfaceCollection resource

- **Section 5.2 "NetworkInterfaces - NetworkInterfaceCollection Resource /redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces" - provides more details on this**

/redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces/{NetworkInterfaceId} - the NetworkInterface resource

- **Section 5.3 "NetworkInterface Resource - /redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces/{NetworkInterfaceId}" - provides more details on this**

StorageService sub-structure resources:

/redfish/v1/StorageServices/{StorageServiceId}/FileSystems - the FileSystemCollection resource

- **Section 8.10 "FileSystemCollection Resource /redfish/v1/StorageServices/{StorageServiceId}/FileSystems" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId}/FileSystems/{FIleSystemId} - the FileSystem resource

- **Section 8.11 "FileSystem Resource /redfish/v1/StorageServices/{StorageServiceId}/FileSystems/{FileSystemId}" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId}/Volumes - the VolumeCollection resource

- **Section 8.8 "VolumeCollection Resource /redfish/v1/StorageServices/{StorageServiceId}/Volumes" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId}/Volumes/{VolumeId} - the Volume resource

- **Section 8.9 "Volume Resource /redfish/v1/StorageServices/{StorageServiceId}/Volumes/{VolumeId}" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools - the StoragePoolCollection resource

- **Section 8.2 "StoragePoolCollection Resource /redfish/v1/StorageServices/{StorageServiceId}/StoragePools" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePooId} - the StoragePool resource

- **Section 8.3 "StoragePool Resource /redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePool Id}" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePooId}/CapacitySources - the CapacitySourceCollection resource

- **Section 8.4 "StoragePool CapacitySourceCollection Resource /redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePool Id}/CapacitySources" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePooId}/CapacitySources/{CapacitySourceId} - the Capacity resource

- **Section 8.5 "StoragePool CapacitySource Resource /redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePool Id}/CapacitySources/{CapacitySourceId}" - provides more details on this**

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePooId}/CapacitySources/{CapacitySourceId}/ProvidingDrives - the DriveCollection resource

- **Section 8.6 "CapacitySource ProvidingDrives - DriveCollection Resource /redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePool**

Id}/CapacitySources/{CapacitySourceId}/ProvidingDrives" - provides more details on this

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePooId}/CapacitySources/{CapacitySourceId}/ProvidingDrives/{DriveId} - the Drive resource

- **Section 8.7 "ProvidingDrives Drive Resource /redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePoolId}/CapacitySources/{CapacitySourceId}/ProvidingDrives/{DriveId}" - provides more details on this**

Chassis sub-structure resources:

/redfish/v1/Chassis/{ChassisId}/Assembly - the Assembly resource

- **Section 7.2 "The Assembly Resource /redfish/v1/Chassis/{ChassisId}/Assembly" - provides more details on this**

/redfish/v1/Chassis/{ChassisId}/Power - the Power resource

- **Section 7.3 "The Power Resource /redfish/v1/Chassis/{ChassisId}/Power" - provides more details on this**

/redfish/v1/Chassis/{ChassisId}/Thermal - the Thermal resource

- **Section 7.4 "The Thermal Resource /redfish/v1/Chassis/{ChassisId}/Thermal" - provides more details on this**

/redfish/v1/Chassis/{ChassisId}/Power/PowerSupplies/{PowerSupplyId}/Assembly/{AssemblyDataId} - the AssemblyData resource

- **Section 7.6 "The AssemblyData Resource /redfish/v1/Chassis/{ChassisId}/Power/PowerSupplies/{PowerSupplyId}/Assembly/{AssemblyDataId}" provides more details on this**

/redfish/v1/Chassis/{ChassisId}/Power/PowerSupplies/{PowerSupplyId} - the PowerSupply resource

- **Section 7.7 "The PowerSupply Resource /redfish/v1/Chassis/{ChassisId}/Power/PowerSupplies/{PowerSupplyId}" provides more details on this**

/redfish/v1/Chassis/{ChassisId}/Thermal/Fans/{FanId} - the Fan resource

- **Section 7.8 "The Fan Resource /redfish/v1/Chassis/{ChassisId}/Thermal/Fans/{FanId}" - provides more details on this**

2.15. Resource Dynamics

Determine how various fields for each resource change when back-end cluster components change

All the various resources in the NEO RFSF API are intended to represent actual cluster components that exist in a ClusterStor system. The resource fields not only show static information to identify a cluster component, but most of them also contain dynamic fields that represent the state or status of that cluster component at the time the GET request is run. For all the individual resources identified above, each section has a "**Details of the response"** section to highlight important fields and what they represent / are populated with, as well as a "**Dynamic Fields"** section, which highlights the specific fields for each resource that can change when the backend cluster component they represent changes. Often, an Event is broadcasted when these dynamic fields change, indicating to the client that they should run another GET request on that resource to see the change in the data. Not all "Dynamic fields" currently have Events implemented for them. Please view the **"Details of the response" and "Dynamic Fields"** section of each resource for more detail, as details will differ from resource to resource.

2.16. Connecting Events to API resources

One good way to determine the different types of Events that NEO RFSF API EventService can be sent and broadcast to subscribers is by looking at the EventService resource, **section 10.1 "EventService Resource"** provides more details on this. Not all Events that the RFSF API supports may be seen in the EventService resource however, as some are extensions for ClusterStor specific Events, for example, we extended the EventService to handle Icinga2 alert-based Events, **section 10.4 "IcingaService Events"** provides more details on this. Sections 10.5 “Types of Events”, and 10.6 “Resources Events Can Occur On”, also provide additional details on the Events that RFSF API supports.

2.17. Resource tracking

Determine mechanism to periodically inventory all resources

While it is up to the client implementation, one example monitoring use case to periodically inventory all resources is as follows (See some examples using the rfsf-api-client libraries in section 12):

1) General inventory loop monitoring: Client discovers full resource tree and runs GET requests to all resource endpoints in that tree, saving off the “cached” state of the resource. Periodically, another inventory of GET requests is done where the fresh data is

compared against the cached data if anything has changed – notify of the change and on what resource it occurred.

2.18. Inventory resources using Events

While it is up to the client implementation, one example monitoring use case to receive and react to Events that occur on resources is as follows (See some examples using the rfsf-api-client libraries in section 12):

1) Event based monitoring: Client subscribes to receive Events from API EventService at some EventDestination. Each time an Event is received, the client code reacts to the Event by parsing out the ResourceId the Event occurred on from the Event body. The client code then runs a GET request to that resource endpoint to determine what data has changed in the resource.

2.19. API Session Termination

While Sessions by default last 12 hours, and the time left shows in GET request to /redfish/v1/SessionService/Sessions/{SessionId} endpoint for a particular session, it is standard to provide a logout mechanism that immediately expires the Session and authentication token. To do this, a DELETE request should be sent to /redfish/v1/SessionService/Sessions/{SessionId}. **Section 3.6 "Deleting active Session to log user out from API"** provides more details.

2.20. Unsubscribe an EventDestination

When clients want to stop receiving Events, generic or MetricReports, on a particular EventDestination, a DELETE request can be sent to the /redfish/v1/EventService/Subscriptions/{EventDestinationId} endpoint for some EventDestinationId that was created earlier to subscribe. Section “10.8 Deleting EventDestinationId subscription to no longer receive Events at some EventDestination” provides more details on unsubscribing from generic Events. Section 11.6 “Send DELETE request to cancel MetricReport EventDestination Subscription” provides more details on unsubscribing from a MetricReport subscription, though the DELETE request is the same, the two actions differ in their own ways.

3. Token based API Authentication

Users can access the Neo Redfish/Swordfish REST API via a JWT (JSON Web Token) which is granted to a user via a POST request to **/redfish/v1/SessionService/Sessions** endpoint. The POST request must pass UserName/Password credentials in the request body which will be checked against local cluster LDAP and validate that request was made for an existing admins user created with the standard method to add an admin user in NEO with "cscli admins add". If

user credentials are validated, a JWT token is returned in the header of the response to the POST request, which returns a Session resource. By default, a Session is valid for 12 hours - we may support a configurable session time in the future.

Session resource schema: https://redfish.dmtf.org/schemas/v1/Session.v1_4_0.json

Redfish session login authentication details can be found in section 14.3.3 of the spec here: https://www.dmtf.org/sites/default/files/standards/documents/DSP0266_1.11.1.pdf

The following provides an example of authenticating an existing admin user on the cluster to use the API via JWT based access - this example is using the "external" server kjlmo12-oem to access API running on kjlmo12, over the kjlmo1200 pub0 address 172.26.137.72 on dp-api-server port 8081:

3.1. Access Denied Before Authenticating

Before authenticating, a user will only be exposed to the login endpoint: /redfish/v1/SessionService/Sessions - if a request is sent to the API trying to access anything else, a http 401 Unauthorized will be returned.

401 Unauthorized response before authenticating

```
# curl https://172.26.137.72:8081/redfish/v1/StorageServices | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   146  100   146    0     0   142k      0 --:--:-- --:--:-- --:--:--  142k
{
  "error": {
    "code": "401",
    "message": "No Auth Token found in request header. User is NOT AUTHORIZED to use the API."
  }
}
```

3.2. Login POST request

Target /redfish/v1/SessionService/Sessions to authenticate a user

To authenticate an existing admin user, a POST request to /redfish/v1/SessionService/Sessions endpoint must be sent, passing the UserName / Password fields in the request body. These user credentials will be validated against local cluster LDAP instance, to ensure the user is an existing admin user on the cluster - created by NEO standard method to add an admin user, "cscli admins add".

POST request body must contain the following fields:

- UserName: <username>

- Password: <password>

Sessions POST request body example

```
{
  "UserName": "<username>",
  "Password": "<password>"
}
```

Example POST request seen with error response if user information cannot be validated against local cluster LDAP when user doesn't exist:

Sessions invalid user POST request and response

```
# curl -v -X POST -H "Content-Type: application/json-rpc" -d @test_credentials.json
https://172.26.137.72:8081/redfish/v1/SessionService/Sessions | jq
Note: Unnecessary use of -X or --request, POST is already inferred.
*   Trying 127.0.0.1...
* TCP_NODELAY set
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Connected to 127.0.0.1 (127.0.0.1) port 8081 (#0)
> POST /redfish/v1/SessionService/Sessions HTTP/1.1
> Host: 127.0.0.1:8081
> User-Agent: curl/7.61.1
> Accept: */*
> Content-Type: application/json-rpc
> Content-Length: 54
>
} [54 bytes data]
* upload completely sent off: 54 out of 54 bytes
< HTTP/1.1 401 Unauthorized
< Content-Type: application/json; charset=UTF-8
< Date: Mon, 13 Jun 2022 22:01:29 GMT
< Content-Length: 150
<
{ [150 bytes data]
100   204  100   150  100    54   146k  54000 --:--:-- --:--:-- --:--:--  199k
* Connection #0 to host 127.0.0.1 left intact
{
  "error": {
    "code": "401",
    "message": "Failed to authorize Session for user: dmatthews. No LDAP entry for user dmatthews"
  }
}
```

Example POST request seen with error response if user information cannot be validated against local cluster LDAP when user password is invalid:

Sessions invalid password POST request and response

```
# curl -v -X POST -H "Content-Type: application/json-rpc" -d @test_credentials.json
https://172.26.137.72:8081/redfish/v1/SessionService/Sessions | jq
Note: Unnecessary use of -X or --request, POST is already inferred.
*   Trying 127.0.0.1...
* TCP_NODELAY set
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Connected to 127.0.0.1 (127.0.0.1) port 8081 (#0)
> POST /redfish/v1/SessionService/Sessions HTTP/1.1
> Host: 127.0.0.1:8081
> User-Agent: curl/7.61.1
> Accept: */*
> Content-Type: application/json-rpc
> Content-Length: 54
>
} [54 bytes data]
* upload completely sent off: 54 out of 54 bytes
< HTTP/1.1 401 Unauthorized
< Content-Type: application/json; charset=UTF-8
< Date: Mon, 13 Jun 2022 22:01:29 GMT
< Content-Length: 150
<
{ [150 bytes data]
100   204  100   150  100    54   146k  54000 --:--:-- --:--:-- --:--:--  199k
* Connection #0 to host 127.0.0.1 left intact
{
  "error": {
    "code": "401",
    "message": "Failed to authorize Session for user: dmatthews. LDAP Result Code 49 \"Invalid Credentials\": "
  }
}
```

Example POST request and response to successfully authenticate existing user: dmatthews.

See that user already exists:

cscli admins list

```
[root@kjlmo1200 ~]# cscli admins list
-----------------------------------------------------------------
 Username   Role      Uid   SSH Enabled  Web Enabled  Policy
-----------------------------------------------------------------
 dmatthews  readonly  1016     True         True      default
-----------------------------------------------------------------
```

POST request and response:

Sessions POST request and response

```
[root@kjlmo12-oem rest_api]# curl -v -H "Content-Type: application/json-rpc" -d @authentication/test_creds.json
https://172.26.137.72:8081/redfish/v1/SessionService/Sessions | jq
*   Trying 127.0.0.1...
* TCP_NODELAY set
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Connected to 127.0.0.1 (127.0.0.1) port 8081 (#0)
> POST /redfish/v1/SessionService/Sessions HTTP/1.1
> Host: 127.0.0.1:8081
> User-Agent: curl/7.61.1
> Accept: */*
> Content-Type: application/json-rpc
> Content-Length: 53
>
} [53 bytes data]
* upload completely sent off: 53 out of 53 bytes
< HTTP/1.1 200 OK
< Content-Type: application/json; charset=UTF-8
< Location: /redfish/v1/SessionService/Sessions/dk9eDAWu4eCBZxLZabv6HC
< X-Auth-Token:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJVc2VyTmFtZSI6ImRtYXR0aGV3cyIsImV4cCI6MTY1NDc3MzE4N30.3YRjS-
3TANRqjfxjhjQoys_C-9iF0PPhZP2p_pXtAZ0
< Date: Wed, 08 Jun 2022 23:13:07 GMT
< Content-Length: 307
<
{ [307 bytes data]
100   360  100   307  100    53   149k  26500 --:--:-- --:--:-- --:--:--  351k
* Connection #0 to host 127.0.0.1 left intact
{
  "@odata.context": "Session.Session",
  "@odata.id": "/redfish/v1/SessionService/Sessions/dk9eDAWu4eCBZxLZabv6HC",
  "@odata.type": "Session.v1_3_0.Session",
  "Description": "Session for user dmatthews",
  "Id": "dk9eDAWu4eCBZxLZabv6HC",
  "Name": "dmatthews Session",
  "Oem": {
    "StartTime": "Tue Jul 26 12:37:31 MDT 2022",
    "ExpirationTime": "Wed Jul 27 00:37:31 MDT 2022",
    "TimeLeft": "11h59m59.999936652s"
  },
  "UserName": "dmatthews"
}
```

Details of the response:

- Id matches the SessionId in /redfish/v1/SessionService/Sessions/{SessionId} endpoint

- UserName is the authenticated User
- Oem
  - StartTime: Time Session was created
  - ExpirationTime: Time Session expires
  - TimeLeft: Time until session expires

Note the following important information returned in the header:

- `Location: /redfish/v1/SessionService/Sessions/dk9eDAWu4eCBZxLZabv6HC`
  - This is the location of the active Session resource with 12-hour expiration period on it. Once session expires and / or dp-api.service is failed over or restarted, this session will be automatically purged.
- `X-Auth-Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJVc2VyTmFtZSI6ImRtYXR0aGV3cyIsImV4cCI6MTY1NDc3MzE4N30.3YRjS-3TANRqjfxjhjQoys_C-9iF0PPhZP2p_pXtAZ0`
  - This is the authentication token string, which must be passed in the header of any subsequent request to access the API.
  - More details on passing the X-Auth-Token in the header seen below

3.3. Authentication Token Use

If user information is successfully validated and a valid JWT token is returned in the X-Auth-Token field of the header of Sessions POST response, as well as a valid Session is created, the user can then access the rest of the API using the JWT token.

The token string generated and returned in X-Auth-Token field, must be passed in subsequent GET requests in the header in the same way it was returned:

- A key, value pair as follows:
  - `'{"X-Auth-Token": "<tokenString>"}'`

- Best practice is to set an env variable AUTH="<tokenString>" for subsequent authorized requests:
- Example: `export AUTH="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJVc2VyTmFtZSI6ImRtYXR0aGV3cyIsImV4cCI6MTY1NDc3MzE4N30.3YRjS-3TANRqjfxjhjQoys_C-9iF0PPhZP2p_pXtAZ0"`

Example GET request passing granted X-Auth-Token in header:

GET request authenticated with X-Auth-Token

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageServices | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
```

```
                                 Dload  Upload   Total   Spent    Left  Speed
100   812  100   812    0     0   396k      0 --:--:-- --:--:-- --:--:--  396k
{
  "@odata.context": "StorageServiceCollection.StorageServiceCollection",
  "@odata.id": "/redfish/v1/StorageServices",
  "@odata.type": "StorageServiceCollection.v1_0_0.StorageServiceCollection",
  "Description": "Collection of references StorageService Resources.",
  "Members": [
    {
      "enclosure_id: 6": "/redfish/v1/StorageServices/MXE30000122VS00A"
    },
    {
      "enclosure_id: 1": "/redfish/v1/StorageServices/MXE30000122VS00D"
    },
    {
      "enclosure_id: 2": "/redfish/v1/StorageServices/MXE30000122VS00E"
    },
    {
      "enclosure_id: 8": "/redfish/v1/StorageServices/SGFGD2044527EF1"
    },
    {
      "enclosure_id: 7": "/redfish/v1/StorageServices/SGFGD2044527F7C"
    },
    {
      "Lustre Filesystem": "/redfish/v1/StorageServices/lustre-kjlmo12"
    }
  ],
  "Members@odata.count": 6,
  "Name": "StorageServiceCollection"
}
```

The X-Auth-Token string must be an exact match for the session created, if the string is off by any characters - the following error response will be seen:

GET request with invalid X-Auth-Token

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/SessionService/Sessions/dk9eDAWu4eCBZxLZabv6HC | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   128  100   128    0     0   125k      0 --:--:-- --:--:-- --:--:--  125k
{
  "error": {
    "code": "401",
    "message": "User is NOT AUTHORIZED to use the API. signature is invalid"
  }
}
```

If the JWT token has expired, the following error response will be seen when trying to make a request using the expired token, and the session will be automatically purged.

GET request with expired X-Auth-Token

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/SessionService/Sessions/dk9eDAWu4eCBZxLZabv6HC | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   128  100   128    0     0   125k      0 --:--:-- --:--:-- --:--:--  125k
{
  "error": {
    "code": "401",
    "message": "User is NOT AUTHORIZED to use the API. token is expired by 5m24s. Deleting Session!"
  }
}
```

If dp-api service is restarted or failed over to another node, because all Session resources are tracked in memory all previous existing sessions will be gone and the following error response will be seen:

GET request with X-Auth-Token that dp-api.service is no longer aware of

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageServices/lustre-
kjlmo12/FileSystems/kjlmo12 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   157  100   157    0     0   153k      0 --:--:-- --:--:-- --:--:--  153k
{
  "error": {
    "code": "401",
    "message": "User is NOT AUTHORIZED to use the API. No session found to auth token sent with request!"
  }
}
```

3.4. SessionService Resource

The SessionService resource lives directly under the ServiceRoot /redfish/v1 can be seen by issuing a GET request to /redfish/v1/SessionService. It will show a list of active Sessions as well as details about the SessionTimeout value which will be set for all created Sessions.

SessionService GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.73:8081/redfish/v1/SessionService | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   457  100   457    0     0   446k      0 --:--:-- --:--:-- --:--:--  446k
{
  "@odata.context": "SessionService.SessionService",
```

```
  "@odata.id": "/redfish/v1/SessionService",
  "@odata.type": "SessionService.v1_1_8.SessionService",
  "Description": "SessionService for RFSF API",
  "Id": "SessionService",
  "Name": "SessionService",
  "ServiceEnabled": true,
  "SessionTimeout": "12h0m0s",
  "Sessions": {
    "@odata.id": "/redfish/v1/SessionService/Sessions"
  },
  "Status": {}
}
```

3.5. Viewing Active Sessions and Individual Session Details

Once a valid session is created, a user can view all active Sessions via a GET request to /redfish/v1/SessionService/Sessions

GET request to view ALL active Sessions

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/SessionService/Sessions | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   441  100   441    0     0   430k      0 --:--:-- --:--:-- --:--:--  430k
{
  "@odata.context": "SessionCollection.SessionCollection",
  "@odata.id": "/redfish/v1/SessionService/Sessions",
  "@odata.type": "SessionCollection.v1_0_0.SessionCollection",
  "Description": "Collection of active API login sessions Ids",
  "Members": [
    {
      "dk9eDAWu4eCBZxLZabv6HC": "/redfish/v1/SessionService/Sessions/dk9eDAWu4eCBZxLZabv6HC"
    }
  ],
  "Members@odata.count": 1,
  "Name": "SessionCollection"
}
```

Once a valid session is created, a user can view a specific active Session via a GET request to /redfish/v1/SessionService/Sessions/{SessionId}

GET request to view single active Session

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/SessionService/Sessions/dk9eDAWu4eCBZxLZabv6HC | jq
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   307  100   307    0     0   299k      0 --:--:-- --:--:-- --:--:--  299k
{
  "@odata.context": "Session.Session",
  "@odata.id": "/redfish/v1/SessionService/Sessions/dk9eDAWu4eCBZxLZabv6HC",
  "@odata.type": "Session.v1_3_0.Session",
  "Description": "Session for user dmatthews",
  "Id": "dk9eDAWu4eCBZxLZabv6HC",
  "Name": "dmatthews Session",
  "Oem": {
    "StartTime": "Tue Jul 26 12:37:31 MDT 2022",
    "ExpirationTime": "Wed Jul 27 00:37:31 MDT 2022",
    "TimeLeft": "11h52m57.842041602s"
  },
  "UserName": "dmatthews"
}
```

3.6. Deleting active Session

By default, the granted Authentication token from a successful /redfish/v1/SessionService/Sessions POST request is set to expire after 12 hours. A user should also be able to logout at any point before that 12-hour expiration period when they are done using the API. This can be accomplished via a DELETE request to /redfish/v1/SessionService/Sessions/{SessionId} for that active Session.

Details of the response:

- SessionId must match an active Session in dp-api SessionService
- There is no request body permitted for the DELETE request
- Once the DELETE request is sent, the AUTH token will no longer be valid
- User must re-authenticate to receive a new valid token in order to use the API again

Example - below shows 2 active sessions, one for user dmatthews and one for user dmatthews2:

- dmatthews is authenticated by ${AUTH} token
- dmatthews2 is authenticated by ${AUTH2} token

List of active Sessions before:

Active Sessions before DELETE

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.143.246:8081/redfish/v1/SessionService/Sessions | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   554  100   554    0     0   541k      0 --:--:-- --:--:-- --:--:--  541k
{
```

```
  "@odata.context": "SessionCollection.SessionCollection",
  "@odata.id": "/redfish/v1/SessionService/Sessions",
  "@odata.type": "SessionCollection.v1_0_0.SessionCollection",
  "Description": "Collection of active API login sessions Ids",
  "Members": [
    {
      "STrJWYypdmrDWEUcNMzUrE": "/redfish/v1/SessionService/Sessions/STrJWYypdmrDWEUcNMzUrE"
    },
    {
      "fS3ybZhLy45hM24fgfpFJF": "/redfish/v1/SessionService/Sessions/fS3ybZhLy45hM24fgfpFJF"
    }
  ],
  "Members@odata.count": 2,
  "Name": "SessionCollection"
}
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.143.246:8081/redfish/v1/SessionService/Sessions/STrJWYypdmrDWEUcNMzUrE | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   307  100   307    0     0   299k      0 --:--:-- --:--:-- --:--:--  299k
{
  "@odata.context": "Session.Session",
  "@odata.id": "/redfish/v1/SessionService/Sessions/STrJWYypdmrDWEUcNMzUrE",
  "@odata.type": "Session.v1_3_0.Session",
  "Description": "Session for user dmatthews",
  "Id": "STrJWYypdmrDWEUcNMzUrE",
  "Name": "dmatthews Session",
  "Oem": {
    "StartTime": "Tue Jul 26 12:37:31 MDT 2022",
    "ExpirationTime": "Wed Jul 27 00:37:31 MDT 2022",
    "TimeLeft": "11h52m57.842041602s"
  },
  "UserName": "dmatthews"
}
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.143.246:8081/redfish/v1/SessionService/Sessions/fS3ybZhLy45hM24fgfpFJF | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   310  100   310    0     0   302k      0 --:--:-- --:--:-- --:--:--  302k
{
  "@odata.context": "Session.Session",
  "@odata.id": "/redfish/v1/SessionService/Sessions/fS3ybZhLy45hM24fgfpFJF",
  "@odata.type": "Session.v1_3_0.Session",
  "Description": "Session for user dmatthews2",
  "Id": "fS3ybZhLy45hM24fgfpFJF",
  "Name": "dmatthews2 Session",
  "Oem": {
    "StartTime": "Tue Jul 26 12:37:31 MDT 2022",
```

```
    "ExpirationTime": "Wed Jul 27 00:37:31 MDT 2022",
    "TimeLeft": "11h52m57.842041602s"
  },
  "UserName": "dmatthews2"
}
```

DELETE request to remove a session and log a user dmatthews out:

DELETE request to end an active Session

```
# curl -X DELETE -H "X-Auth-Token: ${AUTH}"
https://172.26.143.246:8081/redfish/v1/SessionService/Sessions/STrJWYypdmrDWEUcNMzUrE | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
```

Now ${AUTH} token for user dmatthews is no longer valid:

Not Authorized Response for user dmatthews

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.143.246:8081/redfish/v1/SessionService/Sessions  | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   157  100   157    0     0   153k      0 --:--:-- --:--:-- --:--:--  153k
{
  "error": {
    "code": "401",
    "message": "User is NOT AUTHORIZED to use the API. No session found to auth token sent with request!"
  }
}
```

Using still logged in dmatthews2 user and ${AUTH2 token} - see that only one active session remains, and /redfish/v1/SessionService/Sessions/STrJWYypdmrDWEUcNMzUrE session for dmatthews user has ended.

Active Sessions after DELETE

```
# curl -H "X-Auth-Token: ${AUTH2}" https://172.26.143.246:8081/redfish/v1/SessionService/Sessions  | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   441  100   441    0     0   430k      0 --:--:-- --:--:-- --:--:--  430k
{
  "@odata.context": "SessionCollection.SessionCollection",
  "@odata.id": "/redfish/v1/SessionService/Sessions",
  "@odata.type": "SessionCollection.v1_0_0.SessionCollection",
  "Description": "Collection of active API login sessions Ids",
  "Members": [
    {
      "fS3ybZhLy45hM24fgfpFJF": "/redfish/v1/SessionService/Sessions/fS3ybZhLy45hM24fgfpFJF"
```

```
    }
  ],
  "Members@odata.count": 1,
  "Name": "SessionCollection"
}
# curl -H "X-Auth-Token: ${AUTH2}"
https://172.26.143.246:8081/redfish/v1/SessionService/Sessions/fS3ybZhLy45hM24fgfpFJF | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   310  100   310    0     0   302k      0 --:--:-- --:--:-- --:--:--  302k
{
  "@odata.context": "Session.Session",
  "@odata.id": "/redfish/v1/SessionService/Sessions/fS3ybZhLy45hM24fgfpFJF",
  "@odata.type": "Session.v1_3_0.Session",
  "Description": "Session for user dmatthews2",
  "Id": "fS3ybZhLy45hM24fgfpFJF",
  "Name": "dmatthews2 Session",
  "Oem": {
    "StartTime": "Tue Jul 26 12:37:31 MDT 2022",
    "ExpirationTime": "Wed Jul 27 00:37:31 MDT 2022",
    "TimeLeft": "11h50m45.842041602s"
  },
  "UserName": "dmatthews2"
}
```

4. AccountService Resource

The AccountService resource is the redfish service which is used to manage various internal and external user information, including but not limited to:

- Internally managed User accounts in the /redfish/v1/AccountService/Accounts collection
- Externally managed User accounts in the /redfish/v1/AccountService/ExternalAccountProviders
- User roles and privileges in the /redfish/v1/AccountService/Roles collection

The resource lives directly under the ServiceRoot /redfish/v1 can be seen by issuing a GET request to /redfish/v1/AccountService. Because ClusterStor historically provides its own means of user management, for initial purposes, the ClusterStor Redfish/Swordfish API will implement the /redfish/v1/AccountService/Roles and /redfish/v1/AccountService/ExternalAccountProviders collections only, with future plans to migrate ClusterStor user management more fully into /redfish/v1/AccountService, including, representing and managing actual users via the /redfish/v1/AccountService/Accounts collection, as well as other various uses of ExternalAccountProviders.

Schema: https://redfish.dmtf.org/schemas/v1/AccountService.v1_18_0.json

Example GET response of the /redfish/v1/AccountService endpoint:

```
~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/AccountService | jq
{
  "@odata.context": "AccountService.AccountService",
  "@odata.id": "/redfish/v1/AccountService",
  "@odata.type": "AccountService.v1_9_0.AccountService",
  "Accounts": {
    "@odata.id": "/redfish/v1/AccountService/Accounts"
  },
  "ActiveDirectory": {
    "@odata.id": "",
    "@odata.type": "",
    "Authentication": {},
    "LDAPService": {
      "SearchSettings": {}
    },
    "Id": "",
    "TACACSplusService": {}
  },
  "AdditionalExternalAccountProviders": {
    "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders"
  },
  "Description": "AccountService for ClusterStor DP-API",
  "Id": "AccountService",
  "LDAP": {
    "@odata.id": "",
    "@odata.type": "",
    "Authentication": {},
    "LDAPService": {
      "SearchSettings": {}
    },
    "Id": "",
    "TACACSplusService": {}
  },
  "Name": "AccountService",
  "Roles": {
    "@odata.id": "/redfish/v1/AccountService/Roles"
  },
  "ServiceEnabled": true,
  "Status": {},
  "TACACSplus": {
    "@odata.id": "",
    "@odata.type": "",
    "Authentication": {},
    "LDAPService": {
      "SearchSettings": {}
```

```
    },
    "Id": "",
    "TACACSplusService": {}
  }
}
```


4.1 Role Based Access with Role Resource

Role based access for the authenticated Sessions to use the ClusterStor Redfish/Swordfish API is managed via the redfish Role resource under the /redfish/v1/AccountService/Roles collection.

Schema: https://redfish.dmtf.org/schemas/v1/Role.v1_3_3.json

For initial implementation, the API will automatically generate Role resources to represent the ClusterStor admins roles available to the "cscli admins add" command:

```
~]# cscli admins add -h
usage: cscli admins add [-h] --username USERNAME --role {full,limited,readonly} [--firstname FIRSTNAME] [--
lastname LASTNAME]
                [--enable-ssh | --disable-ssh] [--enable-web | --disable-web] [--password PASSWD] [--stream-api]
                [--password-policy {strong,weak,default}]

optional arguments:
...
  --role {full,limited,readonly}
                Displays available roles; Select one from the list:- full: Full admin rights, limited: Provides more
                restricted access than the Full Admin role, but is not read only and provides greater access to CSCLI
                commands than the Read Only Admin role, readonly: Provides read only access.
...
```

As seen above, the available ClusterStor admins user roles are:

- full
- limited
- readonly

These three ClusterStor admins user roles are all represented as a /redfish/v1/AccountService/Roles/{RoleId} resource which are generated on startup and cannot be removed or modified. As seen below, these are the only three Roles which exist in the /redfish/v1/AccountService/Roles collection:

```
~]# curl -Ssk -H "X-Auth-Token":$RFSF_API_TOKEN https://localhost:8081/redfish/v1/AccountService/Roles | jq
{
  "@odata.context": "RoleCollection.RoleCollection",
```

```
  "@odata.id": "/redfish/v1/AccountService/Roles",
  "@odata.type": "RoleCollection.v1_0_0.RoleCollection",
  "Description": "Collection of Role resources for DP-API AccountService",
  "Members": [
    {
      "@odata.id": "/redfish/v1/AccountService/Roles/full"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/Roles/limited"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/Roles/readonly"
    }
  ],
  "Members@odata.count": 3,
  "Name": "RoleCollection"
}
```


When a User runs a /redfish/v1/SessionService/Sessions POST request with passed in user credentials, the API SessionService is able to determine the role of that Username via LDAP lookup, and sets the Role of that User in the /redfish/v1/Session/SessionService/Sessions/{SessionId} resource for the created Session:


```
  ~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
  https://localhost:8081/redfish/v1/SessionService/Sessions/gNLfidiiNpc5exFFYuVmkg | jq
  {
    "@odata.context": "Session.Session",
    "@odata.id": "/redfish/v1/SessionService/Sessions/gNLfidiiNpc5exFFYuVmkg",
    "@odata.type": "Session.v1_7_1.Session",
    "Description": "Session for user apiUser",
    "Id": "gNLfidiiNpc5exFFYuVmkg",
    "Links": {},
    "Name": "apiUser Session",
    "Oem": {
      "StartTime": "Mon Jul 14 15:50:13 MDT 2025",
      "ExpirationTime": "Tue Jul 15 03:59:53 MDT 2025",
      "LastAccessed": "Mon Jul 14 15:59:53 MDT 2025"
    },
    "Roles": [
      "limited"
    ],
    "UserName": "apiUser"
  }
```

For any subsequent request, when the token passed in the header is checked for a valid Session (as described above in section 3) – the API SessionService will also verify that the User Role for the associated Session has the correct privilege level to access the endpoint and make the request they are attempting to. If the user for that Session does not have the correct permissions, a 403 permission denied error response will be returned. See below for more details on what each of the native ClusterStor roles mean for API access.

As mentioned before, because these roles are defined by ClusterStor management software itself, and not via the API at this time, these Role resources cannot be removed or modified. In the future once all ClusterStor user management has migrated under API control, we will support additional management of Roles.

4.1.1 The readonly Role

As the name and cscli admins add help statement suggests, the readonly role is for users with readonly access. For the API, this means that Sessions for users of this role may only execute requests which are readonly, so may only execute GET requests. If it is detected that a Session for a user with readonly Role attempts to run any sort of write request, the following 403 error response will be seen:

```
~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/EventService/Subscriptions/test-event-receiver -d
@metric_report_subscription.json -X POST | jq
{
  "error": {
    "code": "403",
    "message": "permission denied: user with readonly Role is forbidden to run requests which are not GET
requests"
  }
}
```

4.1.2 The limited Role

As the name and cscli admins add help statement suggests, users with a role of limited have less permissions than a full admin user, but more permissions than a readonly user, in that – they are not totally restricted to readonly operations. For the API, this means that Sessions for users of this role are allowed to make requests which perform write operations, but are restricted from some administrative Actions requests which perform administrative operations on the backend. If it is detected that a Session for a user with limited Role attempts to run such an administrative Action request, the following 403 error response will be seen:

```
~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/Managers/kjlmo17n:Node-R1C1-18U-A/Actions/Manager.PowerOff -X
POST | jq
{
  "error": {
    "code": "403",
    "message": "permission denied: user with limited Role is forbidden to run Manager Action requests"
  }
}
```

4.1.1 The full Role

As the name and cscli admins add help statement suggests, users with a role of full have full administrative access to the system. For the API, this means that Sessions for users of this role are allowed to access all endpoints in the resource tree, and execute all requests available to the API, there is no restriction.

4.2 External Accounts with ExternalAccountProvider Resource

The /redfish/v1/AccountService/ExternalAccountProviders collection provides a means to represent users or accounts which are managed by external systems, such as, LDAP, as well as their means of access to the system managed by the local redfish instance, such as certificates, credentials, etc. as an ExternalAccountProvider resource.

Schema: https://redfish.dmtf.org/schemas/v1/ExternalAccountProvider.v1_8_1.json

For the initial purposes of the ClusterStor Redfish/Swordfish API, we will use ExternalAccountProvider resources to represent S3 gateway user mappings to Lustre users, but this will be expanded in the future for additional use cases.

4.2.1 S3 Lustre User Mapping Representation and Management

In order to provide users access to the Lustre filesystem via the S3 gateway protocol, user mappings must be created on the backend S3 API which tie a Lustre username/UID/GID so an S3 access account. These user mappings are represented as /redfish/v1/AccountService/ExternalAccountProviders/{ExternalAccountProviderId} resource

which can be created, read, updated and deleted. For more information on the S3 gateway protocol, please view the ClusterStor S3 network gateway documentation.

4.2.1.1 Creating an ExternalAcountProvider Resource to Represent an S3 User Mapping

In order to create a User mapping so that lustre users may access the filesystem via S3 gateway protocol, users may run a POST request to the /redfish/v1/AccountService/ExternalAccountProviders endpoint with a request such as the following:

```
{
  "AccountProviderType": "S3",
  "Authentication": {
    "Username": "dmatthews"
  }
}
```

NOTE: the AccountProviderType must be set to “S3” and the Autthentication field must have a valid Username which you would like to create the mapping for.

Running the POST request:

```
~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/AccountService/ExternalAccountProviders -d @s3_user_mapping.json -X POST
| jq
{
  "@odata.context": "AccountService.ExternalAccountProvider",
  "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/ZAM4fksvr8A9DDZk2GBeFc",
  "@odata.type": "AccountService.v1_9_0.ExternalAccountProvider",
  "AccountProviderType": "S3",
  "Authentication": {
    "Username": "dmatthews"
  },
  "LDAPService": {
    "SearchSettings": {}
  },
  "Id": "ZAM4fksvr8A9DDZk2GBeFc",
  "TACACSplusService": {}
}
```

This request creates an S3 user mapping via the S3 API on the backend which can be represented and accessed as a /redfish/v1/AccountService/ExternalAccountProviders/{ExternalAccountProviderId} resource.

4.2.1.2 Getting an ExternalAcountProvider Resource to Represent an S3 User Mapping

To get information about a previously created ExternalAccountProvider resource which represents an S3 User Mapping, run a GET request to the desired /redfish/v1/AccountService/ExternalAccountProviders/{ExternalAccountProviderId} endpoint:

Example GET request:

```
~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/AccountService/ExternalAccountProviders/ZAM4fksvr8A9DDZk2GBeFc | jq
{
  "@odata.context": "AccountService.ExternalAccountProvider",
  "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/ZAM4fksvr8A9DDZk2GBeFc",
  "@odata.type": "AccountService.v1_9_0.ExternalAccountProvider",
  "AccountProviderType": "S3",
  "Authentication": {
    "Username": "dmatthews"
  },
  "LDAPService": {
    "SearchSettings": {}
  },
  "Id": "ZAM4fksvr8A9DDZk2GBeFc",
  "TACACSplusService": {}
}
```

4.2.1.3 Removing an ExternalAcountProvider Resource to Represent an S3 User Mapping

To remove an ExternalAccountProvider resource which represents an S3 User Mapping – run a DELETE request to the desired /redfish/v1/AccountService/ExternalAccountProviders/{ExternalAccountProviderId} endpoint. This will remove the resource from the /redfish/v1/AccountService/ExternalAccountProviders collection, and remove the user mapping via the S3 API on the backend:

```
~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/AccountService/ExternalAccountProviders | jq
{
  "@odata.context": "ExternalAccountProviderCollection.ExternalAccountProviderCollection",
  "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders",
  "@odata.type": "ExternalAccountProviderCollection.v1_0_0.ExternalAccountProviderCollection",
  "Description": "Collection of ExternalAccountProvider resources for DP-API AccountService",
  "Members": [
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/2RhZzAvwXuRkMB2iXNEemT"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/GTNds4VsfKckcPYRYatKrX"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/JQ6pcJfkyzvXNFadRXwY7G"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/SmrRu4A7bWKsPEUaFKQHgo"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/ZAM4fksvr8A9DDZk2GBeFc"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/emN4QaNGz7b7JXVJHeQkiy"
    }
  ],
  "Members@odata.count": 6,
  "Name": "ExternalAccountProviderCollection"
}
~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/AccountService/ExternalAccountProviders/ZAM4fksvr8A9DDZk2GBeFc -X DELETE
| jq
{
  "@odata.context": "AccountService.ExternalAccountProvider",
  "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/ZAM4fksvr8A9DDZk2GBeFc",
  "@odata.type": "AccountService.v1_9_0.ExternalAccountProvider",
  "AccountProviderType": "S3",
  "Authentication": {
    "Username": "dmatthews"
  },
  "LDAPService": {
    "SearchSettings": {}
  },
  "Id": "ZAM4fksvr8A9DDZk2GBeFc",
  "TACACSplusService": {}
}
```

```
~]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/AccountService/ExternalAccountProviders | jq
{
  "@odata.context": "ExternalAccountProviderCollection.ExternalAccountProviderCollection",
  "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders",
  "@odata.type": "ExternalAccountProviderCollection.v1_0_0.ExternalAccountProviderCollection",
  "Description": "Collection of ExternalAccountProvider resources for DP-API AccountService",
  "Members": [
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/2RhZzAvwXuRkMB2iXNEemT"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/GTNds4VsfKckcPYRYatKrX"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/JQ6pcJfkyzvXNFadRXwY7G"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/SmrRu4A7bWKsPEUaFKQHgo"
    },
    {
      "@odata.id": "/redfish/v1/AccountService/ExternalAccountProviders/emN4QaNGz7b7JXVJHeQkiy"
    }
  ],
  "Members@odata.count": 5,
  "Name": "ExternalAccountProviderCollection"
}
```

5. Redfish/Swordfish API ServiceRoot

5.1. ServiceRoot Resource: /redfish/v1

The ServiceRoot endpoint /redfish/v1 describes the entry points for all resources that exist in the API. All other resources provided by the API can be found under one of the provided service Entry points seen in the ServiceRoot resource. Authentication is NOT required to retrieve the ServiceRoot via a GET request to /redfish/v1.

Schema: https://redfish.dmtf.org/schemas/v1/ServiceRoot.v1_14_0.json

Example of ServiceRoot GET request:

GET /redfish/v1 Example

```
# curl https://172.26.143.246:8081/redfish/v1  | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   839  100   839    0     0   819k      0 --:--:-- --:--:-- --:--:--  819k
{
  "@odata.context": "ServiceRoot.ServiceRoot",
  "@odata.id": "/redfish/v1",
  "@odata.type": "ServiceRoot.v1_10_0.ServiceRoot",
  "Chassis": {
    "@odata.id": "/redfish/v1/Chassis"
  },
  "EventService": {
    "@odata.id": "/redfish/v1/EventService"
  },
  "Id": "ServiceRoot",
  "Links": {},
  "Name": "NEO RedfishSwordfish API ServiceRoot",
  "Oem": {
    "Events": {
      "@odata.id": "/redfish/v1/Events"
    }
  },
  "ProtocolFeaturesSupported": {
    "DeepOperations": {}
  },
  "SessionService": {
    "@odata.id": "/redfish/v1/SessionService"
  },
  "StorageServices": {
    "@odata.id": "/redfish/v1/StorageServices"
  },
  "StorageSystems": {
    "@odata.id": "/redfish/v1/StorageSystems"
  },
  "UUID": "cf5d0f9f-b946-4b77-8975-7994423a8031"
}
```

Details of the Response:

- **odata.id** is always set to ServiceRoot endpoint
- **Id:** name of the resource ServiceRoot
- **Chassis:** link to Chassis service entry point
- **Oem.Events:** link to Events service entry point
- **SessionService:** link to SessionService service entry point

- **StorageServices:** link to StorageServices service entry point
- **StorageSystems**: link to StorageSystems service entry point

Dynamic Fields:

- No dynamic fields

Service Entry Points:

- /redfish/v1/Chassis
  - Resources: Assembly, Power, Thermal, AssemblyData, PowerSupply, Fan
- /redfish/v1/EventService
  - Resources: EventDestinationCollection, EventDestination
- /redfish/v1/Events
  - Resources: EventCollection, Event
- /redfish/v1/SessionService
  - Resources: SessionCollection, Session
- /redfish/v1/StorageServices
  - Resources: StorageServiceCollection, FileSystemCollection, StoragePoolCollection, VolumeCollection, FileSystem, StoragePool, Volume, CapacitySourceCollection, CapacitySource, DriveCollection, Drive
- /redfish/v1/StorageSystems
  - Resources: ComputerSystemCollection, ComputerSystem, NetworkInterfaceCollection, NetworkInterface

6. Cluster StorageSystems

StorageSystemCollection Resource /redfish/v1/StorageSystems

From a Redfish/Swordfish (rfsf) REST API perspective, a NEO based ClusterStor cluster will be represented by a StorageSystemCollection Resource:

Schema: https://redfish.dmtf.org/schemas/swordfish/StorageSystemCollection.json:

This Collection represents a set of Storage based ComputerSystem resources, which represent ClusterStor Storage nodes themselves (seen below).

Each StorageSystemCollection resource, uniquely identified by some Id, will be the main ServiceRoot entry point for 5the Neo RFSF REST API. The requirements for each StorageSystemCollection resource from the specification state that:

- StorageSystemCollection.Members: The value of each member entry shall reference a ComputerSystem resource that shall have a HostingRoles entry with a value of 'StorageServer'.
- These `ComputerSystem` resources represent storage servers that support Swordfish storage management services
- Example StorageSystemCollection.Members URI: /redfish/v1/StorageSystems/{StorageSystemId}/{ComputerSystemId}

Populated StorageSystemsCollection resource to represent a cluster

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageSystems | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   819  100   819    0     0   266k      0 --:--:-- --:--:-- --:--:--  266k
{
  "@odata.context": "StorageSystemCollection.StorageSystemCollection",
  "@odata.id": "7757953e-dd1e-11ec-b945-941882897e68",
  "@odata.type": "StorageSystemCollection.v1_0_0.StorageSystemCollection",
  "Description": "Collection of references to storage based ComputerSystem resources.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-40U-A"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-40U-B"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-38U-A"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-38U-B"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-12U-B"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-12U-A"
    }
  ],
  "Members@odata.count": 6,
  "Name": "kjlmo12"
}
```

Details of the response:

- **@odata.id:** the UUID of the cluster

- **Members:** Collection of OdataId which represent endpoints of the Storage based ComputerSystem resources that exist in the cluster
- **Members@odata.count:** the total count of ComputerSystem resources that exist in the cluster
- **Name:** the name of the cluster

Dynamic Fields:

- Members may add or remove a member from the list if a Storage node is replaced, added or removed
- Members@odata.count will change with the Members list

6.1. Storage Node Representation

ComputerSystem Resource: /redfish/v1/StorageSystems/{ComputerSystemId}

From a Redfish/Swordfish (rfsf) REST API perspective, a NEO based ClusterStor Storage Node will be represented by a ComputerSystem Resource with **a HostingRoles entry with a value of 'StorageServer'.** This will in turn represent a Storage node in the Cluster:

Schema: http://redfish.dmtf.org/schemas/v1/ComputerSystem.v1_17_0.json#/definitions/ComputerSystem

6.1.1. ComputerSystem Collections resources

- /redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces

6.1.2. ComputerSystemId

Must be uniquely identifiable at the storage node level - will use node_id:

- node_id: id field from t0db.node table will be the ComputerSystemId for each ComputerSystem resource
- Example: kjlmo12:Node-R1C1-40U-A

6.1.3. Example ComputerSystem populated Resource

Using SSU-D node with NVMe head enclosure + 2 SAS EBOD 4U106 enclosures attached

/redfish/v1/StorageSystems/{ComputerSystemId} GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-B | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
```

```
100  3050    0  3050    0     0   3900      0 --:--:-- --:--:-- --:--:--  3900
s/kjlmo12:Node-R1C1-36U-B  | jq
{
  "@odata.context": "ComputerSystem.ComputerSystem",
  "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-B",
  "@odata.type": "ComputerSystem.v1_15_0.ComputerSystem",
  "Boot": {},
  "BootProgress": {},
  "GraphicalConsole": {},
  "HostName": "kjlmo1205",
  "HostWatchdogTimer": {
    "FunctionEnabled": false,
    "Status": {},
    "TimeoutAction": ""
  },
  "HostedServices": {
    "StorageServices": [
      {
        "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A"
      },
      {
        "@odata.id": "/redfish/v1/StorageServices/SGFGD2044527EF1"
      },
      {
        "@odata.id": "/redfish/v1/StorageServices/SGFGD2044527F7C"
      }
    ]
  },
  "HostingRoles": [
    "StorageServer"
  ],
  "Id": "kjlmo12:Node-R1C1-36U-B",
  "Links": {
    "Oem": [
      {
        "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A"
      },
      {
        "@odata.id": "/redfish/v1/Chassis/SGFGD2044527EF1"
      },
      {
        "@odata.id": "/redfish/v1/Chassis/SGFGD2044527F7C"
      }
    ]
  },
  "Manufacturer": "Viking Enterprise Solutions",
  "MemorySummary": {
    "Status": {}
  },
  "Model": "viking-VSSEP1EC_hybrid_esu2",
  "Name": "kjlmo12:Node-R1C1-36U-B ComputerSystem",
  "NetworkInterfaces": {
    "@odata.context": "NetworkInterfaceCollection.NetworkInterfaceCollection",
```

```
  "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-B/NetworkInterfaces",
  "@odata.type": "NetworkInterfaceCollection.v1_0_0.NetworkInterfaceCollection",
  "Description": "NetworkInterfaceCollection for kjlmo12:Node-R1C1-36U-B",
  "Members": [
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-B/NetworkInterfaces/meth0"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-B/NetworkInterfaces/eth1"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-B/NetworkInterfaces/mgmt0"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-B/NetworkInterfaces/cxi0"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-B/NetworkInterfaces/cxi1"
    }
  ],
  "Members@odata.count": 5,
  "Name": "NetworkInterfaceCollection"
},
"Oem": {
  "IcingaServices": {},
  "LinuxStats": {
    "Available Memory (%)": "46.1640",
    "Available Memory (GB)": "111.3061",
    "Buffered Memory (GB)": "20.2147",
    "CPU Guest Nice Time (%)": "0.0000",
    "CPU Guest Time (%)": "0.0000",
    "CPU IOWait Time (%)": "0.0218",
    "CPU IRQ Time (%)": "0.0237",
    "CPU Idle Time (%)": "98.1021",
    "CPU Nice Time (%)": "0.0085",
    "CPU SoftIRQ Time (%)": "0.0143",
    "CPU Steal time (%)": "0.0000",
    "CPU System Time (%)": "0.6379",
    "CPU User Time (%)": "1.2135",
    "CPU Utilization (%)": "1.8979",
    "Cached Memory (GB)": "45.3352",
    "Context Switches per second": "9685.4932",
    "Free Memory (GB)": "50.1603",
    "Free Swap Space (%)": "99.8932",
    "Free Swap Space (GB)": "15.9829",
    "Interrupts per second": "4683.9425",
    "Load Average (15m)": "0.87",
    "Load Average (1m)": "0.44",
    "Load Average (5m)": "0.62",
    "Memory Utilization (%)": "55.5931",
    "Number of CPUs": "32",
    "Total Memory (GB)": "250.6502",
    "Total Swap Space (GB)": "16.0000"
  }
```

```
  },
  "PowerState": "On",
  "ProcessorSummary": {
    "Status": {}
  },
  "SerialConsole": {
    "IPMI": {},
    "SSH": {},
    "Telnet": {}
  },
  "SerialNumber": "MXE3000011TVT074",
  "Status": {},
  "SystemType": "oss",
  "VirtualMediaConfig": {}
}
```


Details of the response:

- Each ComputerSystem resource will be uniquely identified by some Id.
- Each ComputerSystem resource will contain a set of HostedStorageServices identified by: /redfish/v1/StorageServices/{StorageServiceId}. Each StorageServiceId will represent an enclosure that provides a set of Storage Services.
- Each ComputerSystem resource will contain a set of Links to Chassis resources identified by: /redfish/v1/Chassis/{ChassisId}. Each ChassisId will represent the physical HW components and FRUs contained in an enclosure.
- Each storage enclosure will have a /redfish/v1/StorageServices/{StorageServiceId} and /redfish/v1/Chassis/{ChassisId} where StorageServiceId and ChassisId are defined by the same enclosure serial number.
- Each ComputerSystem resource will contain a Set of NetworkInterfaces to represent the available Network Interface Connections on the node.
- Each ComputerSystem resource will contain a set of IcingaServices with applicable Icinga Service output, name, and status.
- Each ComputerSystem resource will contain a PowerState field to represent the Power state of the node

Dynamic Fields:

- HostedServices.StorageServices - StorageService endpoints may change their StorageServiceId if an enclosure replacement occurs
- Links.Oem - Chassis endpoints may change their ChassisId if a enclosure replacement occurs
- NetworkInterfaces - list may grow or shrink if interfaces are added or removed upon controller replacement (only HSN interfaces typically)
- Oem.IcingaServices.Services - any IcingaService State and Output fields may change depending on the status of the actual Icinga service

- PowerState - may change between On, Off and Unknown depending on whether the node's BMC response
- SerialNumber - may change if a node gets replaced

6.2. NetworkInterfaces – NetworkInterfaceCollection Resource

/redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces

Each ComputerSystem will have a NetworkInterfaceCollection resource that lists all the available NetworkInterface resources for that node. This includes details about external-facing interfaces, internal-facing interfaces, and public-facing interfaces on management nodes, like pub0, which is used for login and other external cluster tasks. Currently, there are no NetworkInterface resources for IPMI interfaces.

Schema: https://redfish.dmtf.org/schemas/v1/NetworkInterfaceCollection.json

/redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageSystems/kjmo5:Node-R1C1-34U-A/NetworkInterfaces | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   925  100   925    0     0   301k      0 --:--:-- --:--:-- --:--:--  301k
{
"@odata.context": "NetworkInterfaceCollection.NetworkInterfaceCollection",
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces",
"@odata.type": "NetworkInterfaceCollection.v1_0_0.NetworkInterfaceCollection",
"Description": "NetworkInterface Collection for kjlmo5:Node-R1C1-34U-A",
"Members": [
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/icl0"
},
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/mgmt0"
},
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/heth0"
},
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/eth0"
},
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/pub1"
},
{
```

```
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/pub3"
},
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/meth0"
},
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/pub0"
},
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/pub2"
},
{
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/meth1"
}
],
"Members@odata.count": 10,
"Name": "kjlmo5:Node-R1C1-34U-A NetworkInterfaces"
}
```


Details of the response:

- **Members:** Collection of OdataId which represent endpoints of all the NetworkInterface resources that exist for this ComputerSystem
- **Members@odata.count:** the total count of NetworkInterface resources that exist in this ComputerSystem
- **Name:** highlights the name of the ComputerSystem this collection belongs too.

Dynamic Fields:

- Members - list may grow or shrink if interfaces are added or removed upon controller replacement (only HSN interfaces typically)
- Members@odata.count - will grow or shrink with Members list

6.3. NetworkInterface Representation

NetworkInterface Resource - /redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces/{NetworkInterfaceId}

Each NetworkInterface available in a ComputerSystem's NetworkInterfaceCollection can be queried to get hardware, virtual, state, configuration, and other details about the interface. Resources can either represent a physical Network Interface Card contained within a Storage Node or a virtual network interface made up of one or more underlying physical members, as seen below.

Schema: https://redfish.dmtf.org/schemas/v1/NetworkInterface.v1_2_0.json

6.3.1 NetworkInterface Representation for a Physical Interface

As mentioned above, NetworkInterface resources can represent either a physical single interface or a virtual interface made up of one or more physical underlying members. The response of the NetworkInterface request which represents a physical interface contains details about that interface only, whether it is part of an over-arching virtual interface or not. The following example shows a NetworkInterface response which represents a physical network interface card.

/redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces/{NetworkInterfaceId} GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/heth0 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1102  100  1102    0     0   5565      0 --:--:-- --:--:-- --:--:--  5565
{
"@odata.context": "NetworkInterface.NetworkInterface",
"@odata.id": "/redfish/v1/StorageSystems/kjlmo5:Node-R1C1-34U-A/NetworkInterfaces/heth0",
"@odata.type": "NetworkInterface.v1_2_1.NetworkInterface",
"Description": "Network Interface heth0 for ComputerSystem kjlmo5:Node-R1C1-34U-A",
"Id": "heth0",
"Links": {},
"Name": "kjlmo5:Node-R1C1-34U-A heth0",
"Oem": {
"MacAddress": "94:40:c9:b3:3b:40",
"IpAddresses": [
{
"Address": "10.230.79.128",
"SubNetMask": "255.255.224.0",
"Type": "IPv4"
}
],
"NetworkType": "System",
"InterfaceType": "Ethernet",
"MTUSize": 9000,
"SpeedMbps": 100000,
"Statistics": {
"BytesReceived": "0",
"BytesSent": "300",
"DiscardedReceived": "0",
"DiscardedSent": "0",
"ErrorsReceived": "0",
"ErrorsSent": "0",
"PacketsReceived": "0",
"PacketsSent": "5"
```

```
}
},
"Status": {
"State": "up"
}
}
```


Details of the response:

- **Id:** The NetworkInterface.Id is set to the name of the interface
- **Oem.MacAddress:** the mac address of the interface
- **Oem.IpAddresses:** the list of IP addresses of the interface if configured
- **O**em.Net**workType:** the network type which this interface is connected to
- **Oem.InterfaceType:** type of the interface like Ethernet, InfiniBand
- **Oem.MTUSize:** maximum transmission unit of the interface
- **Oem.SpeedMbps:** speed of the interface
- **Oem.Statistics:** various data, settings, and stats about this interface
- **Status.State:** the current operstate value of the interface

Dynamic Fields

- Oem.MacAddress - may change in card is replaced
- Oem.IpAddress - may change if a new HSN range is configured
- Statistics - constantly changes based on network traffic across the interface
- Status.State - may change based on operstate of the interface

6.3.2 NetworkInterface Representation for a Virtual Interface

As mentioned above, NetworkInterface resources can represent either a physical single interface or a virtual interface made up of one or more physical underlying members. The response of the NetworkInterface request which represents a virtual interface contains information about what type of virtual interface it is (teamd or bonded) as well as various configuration details about that virtual interface, such as, its underlying physical member(s), what physical member is active vs. standby, and other configuration details specific to the type of virtual interface it is. The following example shows a NetworkInterface response which represents a virtual bonded network interface.

NetworkInterface Resource
- /redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces/{NetworkInterfaceId}

Each Virtual NetworkInterface available in a ComputerSystem's NetworkInterfaceCollection can be queried to get Bond or Teaming Mode, state, Active Slave, Link to members , and other details about the interface.

Schema: https://redfish.dmtf.org/schemas/v1/NetworkInterface.v1_2_0.json

/redfish/v1/StorageSystems/{ComputerSystemId}/NetworkInterfaces/{NetworkInterfaceId}
GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageSystems/kjlmo16:Node-R1C1-36U-A/NetworkInterfaces/ib0 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1102  100  1102    0     0   5565      0 --:--:-- --:--:-- --:--:--  5565
{
        "@odata.context": "NetworkInterface.NetworkInterface",
        "@odata.id": "/redfish/v1/StorageSystems/kjlmo16:Node-R1C1-36U-A/NetworkInterfaces/ib0",
        "@odata.type": "NetworkInterface.v1_2_1.NetworkInterface",
        "Description": "Network Interface bond0 for ComputerSystem kjlmo16:Node-R1C1-36U-A",
        "Id": "ib0",
        "Links": {
                "Oem": [
                {
                        "@odata.id": "/redfish/v1/StorageSystems/kjlmo16:Node-R1C1-36U-
                A/NetworkInterfaces/ib1"
                },
                {
                        "@odata.id": "/redfish/v1/StorageSystems/kjlmo16:Node-R1C1-36U-
                A/NetworkInterfaces/ib2"
                }
        ]
},
"Name": "kjlmo16:Node-R1C1-36U-A ib0",
"Oem": {
        "MacAddress": "b6:4e:84:b0:7f:8f",
        "NetworkType": "System",
        "InterfaceType": "Bond",
        "Mode": "802.3ad",
        "XmitHashPolicy": "layer2+3",
        "MTUSize": 2400,
        "Statistics": {
                "BytesReceived": "0",
                "BytesSent": "0",
                "DiscardedReceived": "0",
                "DiscardedSent": "0",
                "ErrorsReceived": "0",
                "ErrorsSent": "0",
                "PacketsReceived": "0",
```

```
                "PacketsSent": "0"
        }
},
"Status": {
        "State": "down"
}
}
```


Details of the response:

- **Id:** The NetworkInterface.Id is set to the name of the interface
- **Oem.MacAddress:** the mac address of the interface
- **Oem.IpAddresses:** the list of IP addresses of the interface if configured
- **Oe**m.Net**workType:** the network type which this interface is connected to
- **Oem.InterfaceType:** type of the interface like Bond, Team
- **Oem.MTUSize:** maximum transmission unit of the interface
- **Oem.Mode:** bonding mode or teaming mode
- **Oem.XmitHashPolicy:** transmit hash poilcy of the bonding interface
- **Oem.ActiveSlave:** active port/ active slave
- **Oem.Statistics:** various data, settings, and stats about this interface
- **Status.State:** the current operstate value of the interface

Dynamic Fields

- Oem.MacAddress - may change in card is replaced
- Oem.IpAddress - may change if a new HSN range is configured
- Statistics - constantly changes based on network traffic across the interface
- Status.State - may change based on operstate of the interface
- Mode – when user changes bonding mode via cscli lustre_network bonding mode set
- ActiveSlave – when current active slave/port operational status goes down
- XmitHashPoilcy – when user changes xmit policy via cscli lustre_network bonding hash

7. ComputerSystem and Top-level service Resources

Each ComputerSystem resource will provide a link to the collection of top-level /redfish/v1/Chassis and /redfish/v1/StorageServices resources which belong to or are associated with it. These top-level resources linked to the computersystem represent the enclosure(s) the ComputerSystem reside in or are attached to (in the case of EBOD enclosures). The association can be seen as follow:

7.1 ComputerSystem.Links Chassis Collection

All /redfish/v1/Chassis/{Chassis} resources associated with a particular ComputerSystem can be seen in the ComputerSystem.Links section:

ComputerSystem.Links Chassis Collection

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-A | jq '.Links'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  3054    0  3054    0     0   3910      0 --:--:-- --:--:-- --:--:--  3910
{
  "Oem": {
    "enclosure_id: 3": "/redfish/v1/Chassis/MXE30000122VS00A",
    "enclosure_id: 4": "/redfish/v1/Chassis/SGFGD2044527F7C",
    "enclosure_id: 5": "/redfish/v1/Chassis/SGFGD2044527EF1"
  }
}
```

NOTE: each Chassis resource seen in the ComputerSystem.Links section represents a /redfish/v1/Chassis service entry point for each enclosure that storage node the ComputerSystem represents is attached to. The ChassisId will be the **same** as the StorageServiceId for each enclosure as seen below

7.2 ComputerSystem.HostedServices.Services StorageService Collection

All /redfish/v1/StorageServices/{StorageServiceId} resources associated with a particular ComputerSystem can be seen in the ComputerSystem.HostedStorageServices.Services section:

ComputerSystem.HostedServices.Services StorageService Collection

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-36U-A | jq '.HostedServices'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  3054    0  3054    0     0   3856      0 --:--:-- --:--:-- --:--:--  3851
{
  "StorageServices": {
    "enclosure_id: 3": "/redfish/v1/StorageServices/MXE30000122VS00A",
    "enclosure_id: 4": "/redfish/v1/StorageServices/SGFGD2044527F7C",
    "enclosure_id: 5": "/redfish/v1/StorageServices/SGFGD2044527EF1"
  }
}
```

NOTE: each StorageService resource seen in the ComputerSystem.HostedServices.Services section represents a /redfish/v1/StorageServices service entry point for each enclosure that storage node the ComputerSystem represents is attached to. The StorageServiceId will be the **same** as the StorageServiceId for each enclosure as seen above.

8. Cluster Chassis

- ChassisCollection Resource /redfish/v1/Chassis

The ChassisCollection resource provides a Collection of OdataId's which represent endpoints of all Chassis resources that exist in the cluster. Each individual Chassis resource within the ChassisCollection represents a Storage enclosure in the cluster.

Each Chassis resource, uniquely identified by some Id, will be the service entry point to access several other resources which may represent components of an enclosure, such as Fans, PSU, Assemblies, etc.

Schema: https://redfish.dmtf.org/schemas/v1/ChassisCollection.json

/redfish/v1/Chassis GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/Chassis | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   672  100   672    0     0   328k      0 --:--:-- --:--:-- --:--:--  328k
{
  "@odata.context": "ChassisCollection.ChassisCollection",
  "@odata.id": "/redfish/v1/Chassis",
  "@odata.type": "ChassisCollection.v1_0_0.ChassisCollection",
  "Description": "Collection of references to storage Chassis resources.",
  "Members": [
    {
      "enclosure_id: 3": "/redfish/v1/Chassis/MXE30000122VS00A"
    },
    {
      "enclosure_id: 1": "/redfish/v1/Chassis/MXE30000122VS00D"
    },
    {
      "enclosure_id: 2": "/redfish/v1/Chassis/MXE30000122VS00E"
    },
    {
      "enclosure_id: 5": "/redfish/v1/Chassis/SGFGD2044527EF1"
    },
    {
      "enclosure_id: 4": "/redfish/v1/Chassis/SGFGD2044527F7C"
```

```
    }
  ],
  "Members@odata.count": 5,
  "Name": "kjlmo12 Storage Chassis Collection"
}
```


Details of the response:

- **Members:** Collection of OdataId which represent endpoints of the Chassis resources that exist in the cluster
- **Members@odata.count:** the total count of Chassis resources that exist in the cluster
- **Name:** contains the name of the cluster

Dynamic Fields:

- Members may add or remove a member from the list if a Storage unit, added or removed
- Members OdataId fields may change if an enclosure is replaced
- Members@odata.count will change with the Members list

8.1. Enclosure Representation as Chassis resource

- Chassis Resource: /redfish/v1/Chassis/{ChassisId}

The Chassis resource is a top-level resource that represents a Storage enclosure. It provides the service entry point to access several other resources and collections of resources which may represent components of an enclosure. While there are two top-level resources that represent an enclosure, Chassis and StorageService, the Chassis resource focusing more on Hardware components of the enclosure in most cases, including FRUs such as Fans, PSUs, and PAB (Power Assembly's).

Schema https://redfish.dmtf.org/schemas/v1/Chassis.v1_19_0.json:

8.1.1. Collections that are offered by a Chassis resource

- /redfish/v1/Chassis/{ChassisId}/Power
- /redfish/v1/Chassis/{ChassisId}/Assembly
- /redfish/v1/Chassis/{ChassisId}/Thermal

8.1.2. ChassisId

- ChassisId is represented by an enclosure serial number
- ChassisId is the same as /redfish/v1/StorageServices/{StorageServiceId} StorageServiceId for an enclosure

/redfish/v1/Chassis/{ChassisId} GET request

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Chassis/MXE30000122VS009 |
jq
{
  "@odata.context": "Chassis.v1_16_0.Chassis",
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009",
  "@odata.type": "Chassis.v1_16_0.Chassis",
  "Assembly": [
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Power/PowerSupplies/PSU-1/Assembly/PAB-1"
    },
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Power/PowerSupplies/PSU-2/Assembly/PAB-2"
    }
  ],
  "ChassisType": "IPMI",
  "Description": "Storage Chassis MXE30000122VS009",
  "Id": "MXE30000122VS009",
  "Links": {},
  "Location": {
    "PartLocation": {},
    "Placement": {},
    "PostalAddress": {}
  },
  "Manufacturer": "Viking Enterprise Solutions",
  "Model": "VSSEP1EC",
  "Name": "Storage Chassis MXE30000122VS009",
  "Oem": {
    "ServerSlots": [
      {
        "ComputerSystemId": "kjlmo17n:Node-R1C1-30U-A",
        "SlotNumber": "0",
        "Postition": "left",
        "ServerPresent": true
      },
      {
        "ComputerSystemId": "kjlmo17n:Node-R1C1-30U-B",
        "SlotNumber": "1",
        "Postition": "right",
        "ServerPresent": true
      }
    ]
  },
  "Power": [
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Power/PowerSupplies/PSU-1"
    },
```

```
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Power/PowerSupplies/PSU-2"
    }
  ],
  "Status": {},
  "Thermal": [
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Thermal/Fans/Cooling-1"
    },
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Thermal/Fans/Cooling-2"
    },
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Thermal/Fans/Cooling-3"
    },
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Thermal/Fans/Cooling-4"
    },
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Thermal/Fans/Cooling-5"
    },
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS009/Thermal/Fans/Cooling-6"
    }
  ]
}
```


Details of the response:

- **Assembly**: contains collection of OdataId's to access AssemblyData resources which represent PAB (Power Assembly Units) in the enclosure
    - not all Chassis have Power Assembly FRUs, this Assembly Collection is only seen those that do
    - For Chassis that do have Power Assembly FRUs, there should be a 1 to 1 mapping of each AssemblyData resource to its associated PowerSupply resource
- ChassisType: This is the type of Chassis the resource represents, can either be: IPMI, SES, or redfish depending on the supported backend monitoring/management interface of the platform
- **Id**: This is the enclosure serial number, and must match the ChassisId in the OdataId field: /redfish/v1/Chassis/{ChassisId}
- **Manufacturer**: the hardware manufacturer of the Storage enclosure the Chassis resource represents
- **Model**: the VPD string of the Storage enclosure the Chassis resource represents
- **Oem**.ServerSlots: If the platform supports physical enclosure server slot detection sensors, this section will show:

        - **SeverSlot**.ComputerSystemId: the Id of the ComputerSystem resource which resides in this enclosure server slot
        - ServerSlot.SlotNumber: the enumerated slot number in the enclosure
        - ServerSlot.Position: the physical position of the server slot (left/right or top/bottom depending on the platform)
        - ServerSlot.ServerPresent: whether the server presence sensor detects physical presence of the server in this slot
    - **Power**: contains collection of OdataId's to access PowerSupply resources which represent PSU (Power Supply Units) in the enclosure
        - if the Chassis resource contains Assembly FRUs, there should be a 1 to 1 mapping of each PowerSupply resource to its associated AssemblyData resource
    - **Thermal**: contains collection of OdataId's to access Fan resources which represent Cooling units in the enclosure

Dynamic Fields:

- Id - may change if enclosure is replaced
- Oem.ServerSlots may change as servers are removed or replaced

8.2. The Assembly Resource

/redfish/v1/Chassis/{ChassisId}/Assembly

The Assembly resource provides access to the Assemblies list for a Chassis resource which will contain the individual AssemblyData resources for that Chassis. This represents a set of Power Assembly Units (PABs) that exist in the Storage enclosure the Chassis represents. If that Storage enclosure does not contain PAB units as FRUs, then a 404 response will be returned indicating there are no AssemblyData units for this Chassis.

Schema: https://redfish.dmtf.org/schemas/v1/Assembly.v1_3_0.json

Example GET request on Chassis that does have AssemblyData resources:

/redfish/v1/Chassis/{ChassisId}/Assembly GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Assembly |
jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1044  100  1044    0     0   339k      0 --:--:-- --:--:-- --:--:--  339k
{
  "@odata.context": "Assembly.Assembly",
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Assembly",
  "@odata.type": "Assembly.v1_3_0.Assembly",
  "Assemblies": [
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-1/Assembly/PAB-1",
```

```
    "Actions": {},
    "Location": {
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
    "MemberId": "1",
    "Model": "00",
    "Name": "PAB-1",
    "PartNumber": "ASM-02904-01-A",
    "Producer": "VikingEnterpriseSolutions",
    "SerialNumber": "MXE3000011TVU098",
    "Status": {
      "State": "Ok"
    },
    "Vendor": "VikingEnterpriseSolutions",
    "Version": "00"
  },
  {
    "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-2/Assembly/PAB-2",
    "Actions": {},
    "Location": {
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
    "MemberId": "2",
    "Name": "PAB-2",
    "Status": {
      "State": "Not Installed"
    }
  }
],
"Assemblies@odata.count": 2,
"Id": "MXE30000122VS00A",
"Name": "Assembly"
}
```


Details of the response:

- Assemblies: the list of all AssemblyData resources that exist for this Chassis
  - Note: by schema definition, this list contains populated AssemblyData resources, not just OdataId's
- Each AssemblyData in Assemblies list
  - Note: not all Assemblies in all enclosures can read the same FRU data, each field varies by enclosure if it exists or not.
  - MemberId: the FRU index of the AssemblyData unit

- Model: the HW model of the AssemblyData unit
- Name: the name of the AssemblyData unit
  - combination of FRU type and FRU index
- PartNumber: the HW part number of the AssemblyData unit
- Producer: the HW manufacturer of this AssemblyData
- SerialNumber: the HW serial number of the AssemblyData unit
- Status.State: the over status of the AssemblyData unit
  - note that for the AssemblyData resource with Status.State of "Not Installed" - the FRU data cannot be read
- Vendor: the enclosure Vendor that contains this AssemblyData unit
- Version: the HW revision of this AssemblyData unit

- Assemblies@odata.count: total number of Assemblies for that Chassis

Dynamic Fields

- Assemblies
  - PartNumber, Producer, SerialNumber, Model, Vendor, Version - could all change if a PAB unit is replaced
  - Status.State - could change if status of PAB changes (see section 8.5)

NOTE: GET requests to AssemblyData /redfish/v1/Chassis/{ChassisId}/Power/PowerSupplies/{PowerSupplyId}/Assembly/{AssemblyData} resources directly may provide more detailed information and statistics for that particular unit

Example GET request on Chassis that does not have AssemblyData resources:

Example Chassis with no Assembly FRUs

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/Chassis/SGFGD2044527EF1/Assembly | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   105  100   105    0     0  35000      0 --:--:-- --:--:-- --:--:-- 35000
{
  "error": {
    "code": "404",
    "message": "No Assembly Units found for ChassisId SGFGD2044527EF1."
  }
}
```

Details of the response:

- A RedfishError response is returned here with 404 response code

- this is because not Assembly units exist for this enclosure

8.3. The Power Resource

/redfish/v1/Chassis/{ChassisId}/Power

The Power resource provides access to the PowerSupplies list for a Chassis resource which will contain the individual PowerSupply resources for that Chassis. This represents a set of Power Supply Units (PSUs) that exist in the Storage enclosure the Chassis represents. If that Storage enclosure this Chassis represents contains AssemblyData resources, each PowerSupply unit will contain an Assembly list which provides the OdataId of the associated Assembly resource for that PowerSupply. If the Storage enclosure this Chassis represents does not contain AssemblyData resources, no Assembly links will be seen in each PowerSupply resource

Schema: https://redfish.dmtf.org/schemas/v1/Power.v1_6_3.json

Example GET request on Chassis that does contain AssemblyData resources:

Populated Power Resource

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Power | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1185  100  1185    0     0   289k      0 --:--:-- --:--:-- --:--:--  289k
{
  "@odata.context": "Power.Power",
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power",
  "@odata.type": "Power.v1_7_1.Power",
  "Id": "MXE30000122VS00A",
  "Name": "Power",
  "PowerSupplies": [
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-1",
      "Actions": {},
      "Assembly": {
        "PAB-1": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-1/Assembly/PAB-1"
      },
      "Location": {
        "PartLocation": {},
        "Placement": {},
        "PostalAddress": {}
      },
      "Manufacturer": "3YPOWER",
      "MemberId": "1",
      "Model": "EM-2A01P10",
      "Name": "PSU-1",
      "Oem": {},
```

```
      "PartNumber": "YSEF1600",
      "SerialNumber": "S0A00A3032029000204",
      "Status": {
        "State": "Ok"
      }
    },
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-2",
      "Actions": {},
      "Assembly": {
        "PAB-2": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-2/Assembly/PAB-2"
      },
      "Location": {
        "PartLocation": {},
        "Placement": {},
        "PostalAddress": {}
      },
      "MemberId": "2",
      "Name": "PSU-2",
      "Oem": {},
      "Status": {
        "State": "Unknown"
      }
    }
  ],
  "PowerSupplies@odata.count": 2
}
```


Example GET request on Chassis that does not contain AssemblyData resources:

Populated Power Resource

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/Chassis/SGFGD2044527EF1/Power | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   995  100   995    0     0   323k      0 --:--:-- --:--:-- --:--:--  323k
{
  "@odata.context": "Power.Power",
  "@odata.id": "/redfish/v1/Chassis/SGFGD2044527EF1/Power",
  "@odata.type": "Power.v1_7_1.Power",
  "Id": "SGFGD2044527EF1",
  "Name": "Power",
  "PowerSupplies": [
    {
      "@odata.id": "/redfish/v1/Chassis/SGFGD2044527EF1/Power/PowerSupplies/PSU-0",
      "Actions": {},
      "Location": {
```

```
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
    "MemberId": "0",
    "Model": "0109",
    "Name": "PSU-0",
    "Oem": {},
    "PartNumber": "727656301-02",
    "SerialNumber": "M644X402MKAMP",
    "Status": {
      "State": "Ok"
    }
  },
  {
    "@odata.id": "/redfish/v1/Chassis/SGFGD2044527EF1/Power/PowerSupplies/PSU-2",
    "Actions": {},
    "Location": {
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
    "MemberId": "2",
    "Model": "0109",
    "Name": "PSU-2",
    "Oem": {},
    "PartNumber": "727656301-02",
    "SerialNumber": "M644X402MMAMP",
    "Status": {
      "State": "Ok"
    }
  }
 ],
 "PowerSupplies@odata.count": 2
}
```


Details of the response:

- **PowerSupplies:** the list of all PowerSupply resources that exist for this Chassis
  - Note: by schema definition, this list contains populated PowerSupply resources, not just OdataId's
- Each PowerSupply in PowerSupplies list
  - Note: not all PowerSupplies in all enclosures can read the same FRU data, each field varies by enclosure if it exists or not.
  - Manufacturer: the HW manufacturer of this PowerSupply
  - MemberId: the FRU index of the PowerSupply unit
  - Model: the HW model of the PowerSupply unit

        - Name: the name of the PowerSupply unit
            - combination of FRU type and FRU index
        - PartNumber: the HW part number of the PowerSupply unit
        - SerialNumber: the HW serial number of the PowerSupply unit
        - Status.State: the over status of the PowerSupply unit
    - PowerSupplies@odata.count: the total number of Power Supplies for that Chassis

Dynamic Fields

- PowerSupplies
    - Manufacturer, Model, PartNumber, SerialNumber - could all change if a PSU unit is replaced
    - Status.State - could change if status of PSU changes (see section 8.5)

NOTE: GET requests to PowerSupply /redfish/v1/Chassis/{ChassisId}/Power/PowerSupplies/{PowerSupplyId} resources directly may provide more detailed information and statistics for that unit

8.4. Thermal Resource

/redfish/v1/Chassis/{ChassisId}/Thermal

The Thermal resource provides access to the Fans list for a Chassis resource which will contain the individual Fan resources for that Chassis. This represents a set of Fans or Cooling Units that exist in the Storage enclosure the Chassis represents

Schema: https://redfish.dmtf.org/schemas/v1/Thermal.v1_7_0.json

Example GET request on Chassis containing 6 Fans:

Populated Thermal Resource

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Thermal |
jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  2906    0  2906    0     0   567k      0 --:--:-- --:--:-- --:--:--  567k
{
  "@odata.context": "Thermal.Thermal",
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal",
  "@odata.type": "Thermal.v1_7_1.Thermal",
  "Actions": {},
  "Fans": [
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-1",
      "Actions": {},
      "FanName": "Cooling-1",
```

```
    "HotPluggable": true,
    "Location": {
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
    "Manufacturer": "N/A",
    "MemberId": "1",
    "Model": "N/A",
    "Name": "Cooling-1",
    "Oem": {},
    "PartNumber": "N/A",
    "SerialNumber": "N/A",
    "Status": {
      "State": "Ok"
    }
  },
  {
    "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-2",
    "Actions": {},
    "FanName": "Cooling-2",
    "HotPluggable": true,
    "Location": {
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
    "Manufacturer": "N/A",
    "MemberId": "2",
    "Model": "N/A",
    "Name": "Cooling-2",
    "Oem": {},
    "PartNumber": "N/A",
    "SerialNumber": "N/A",
    "Status": {
      "State": "Ok"
    }
  },
  {
    "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-3",
    "Actions": {},
    "FanName": "Cooling-3",
    "HotPluggable": true,
    "Location": {
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
```

```
    "Manufacturer": "N/A",
    "MemberId": "3",
    "Model": "N/A",
    "Name": "Cooling-3",
    "Oem": {},
    "PartNumber": "N/A",
    "SerialNumber": "N/A",
    "Status": {
      "State": "Ok"
    }
  },
  {
    "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-4",
    "Actions": {},
    "FanName": "Cooling-4",
    "HotPluggable": true,
    "Location": {
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
    "Manufacturer": "N/A",
    "MemberId": "4",
    "Model": "N/A",
    "Name": "Cooling-4",
    "Oem": {},
    "PartNumber": "N/A",
    "SerialNumber": "N/A",
    "Status": {
      "State": "Ok"
    }
  },
  {
    "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-5",
    "Actions": {},
    "FanName": "Cooling-5",
    "HotPluggable": true,
    "Location": {
      "PartLocation": {},
      "Placement": {},
      "PostalAddress": {}
    },
    "Manufacturer": "N/A",
    "MemberId": "5",
    "Model": "N/A",
    "Name": "Cooling-5",
    "Oem": {},
    "PartNumber": "N/A",
```

```
      "SerialNumber": "N/A",
      "Status": {
        "State": "Ok"
      }
    },
    {
      "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-6",
      "Actions": {},
      "FanName": "Cooling-6",
      "HotPluggable": true,
      "Location": {
        "PartLocation": {},
        "Placement": {},
        "PostalAddress": {}
      },
      "Manufacturer": "N/A",
      "MemberId": "6",
      "Model": "N/A",
      "Name": "Cooling-6",
      "Oem": {},
      "PartNumber": "N/A",
      "SerialNumber": "N/A",
      "Status": {
        "State": "Ok"
      }
    }
  ],
  "Fans@odata.count": 6,
  "Id": "MXE30000122VS00A",
  "Name": "Thermal",
  "Status": {}
}
```


Details of the response:

- **Fans:** the list of all Fan resources that exist for this Chassis
    - Note: by schema definition, this list contains populated Fan resources, not just OdataId's
- Each Fan in Fans list
    - Note: not all Fans in all enclosures can read the same FRU data, each field varies by enclosure if it exists or not.
        - If no data is available, the value is sometimes populated with "N/A"
    - Manufacturer: the HW manufacturer of this Fan
    - MemberId: the FRU index of the Fan unit
    - Model: the HW model of the Fan unit
    - Name: the name of the Fan unit
        - combination of FRU type and FRU index

    - PartNumber: the HW part number of the Fan unit
    - SerialNumber: the HW serial number of the Fan unit
    - Status.State: the over status of the Fan unit
- Fans@odata.count: the total number of Fans for that Chassis

Dynamic Fields

- Fans
    - Status.State - could change if status of PSU changes (see section 8.5)

NOTE: GET requests to Fan /redfish/v1/Chassis/{ChassisId}/Thermal/Fans/{FanId} resources directly may provide more detailed information and statistics for that particular unit

8.5. Status.State fields

Status.State fields of all individual resources which have them can be mapped to the following values. Generally, if a resource changes status - and Event will be sent indicating the change.

SES Element Status Table

```
[root@kjlmo12-oem ~]# mysql -e 'select * from ses_element_status' t0db
+----+---------------+
| id | description   |
+----+---------------+
|  2 | Critical      |
|  3 | Non-Critical  |
|  7 | Not available |
|  5 | Not Installed |
|  1 | Ok            |
|  6 | Unknown       |
|  4 | Unrecoverable |
|  0 | Unsupported   |
+----+---------------+
[root@kjlmo12-oem ~]#
```

8.6. AssemblyData Resource

/redfish/v1/Chassis/{ChassisId}/Power/PowerSupplies/{PowerSupplyId}/Assembly/{AssemblyDataId}

The AssemblyData resource provides access to information on PAB (Power Assembly unit) FRUs if they exist in the Storage enclosure that the Chassis resource represents. The resource structure provides data to capture the name and index of the Assembly unit, any hardware data

that is readable from the unit, as well as a Status of the unit which can change, triggering an Event.

Schema: definition for the AssemblyData schema is seen within the Assembly schema here: https://redfish.dmtf.org/schemas/v1/Assembly.v1_3_0.json

AssemblyDataId: the Id for this resource is generated from the following:

- FRU type
- FRU index
- Example: FRU type = PAB, FRU index = 1 - AssemblyDataId = PAB-1
- Note that FRU index for associated Assembly and PowerSupply resources is the same

Example GET request of AssemblyData unit with "Ok" status

Populated AssemblyData Resources

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-1/Assembly/PAB-1
| jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   450  100   450    0     0   146k      0 --:--:-- --:--:-- --:--:--  146k
{
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-1/Assembly/PAB-1",
  "Actions": {},
  "Location": {
    "PartLocation": {},
    "Placement": {},
    "PostalAddress": {}
  },
  "MemberId": "1",
  "Model": "00",
  "Name": "PAB-1",
  "PartNumber": "ASM-02904-01-A",
  "Producer": "VikingEnterpriseSolutions",
  "SerialNumber": "MXE3000011TVU098",
  "Status": {
    "State": "Ok"
  },
  "Vendor": "VikingEnterpriseSolutions",
  "Version": "00"
}
```

Details of the response:

- MemberId: the FRU index of the AssemblyData unit
- Model: the HW model of the AssemblyData unit
- Name: the name of the AssemblyData unit which matches the AssemblyDataId
- PartNumber: the HW part number the AssemblyData unit
- SerialNumber: the serial number of the AssemblyData unit
- Status.State: the status of the AssemblyData unit
    - See section 8.5 for details
- Vendor: the manufacturer of the enclosure the AssemblyData unit is contained in
- Version: the HW revision the AssemblyData unit

Dynamic Fields

- PartNumber, Producer, SerialNumber, Model, Vendor, Version - could all change if a PAB unit is replaced
- Status.State - could change if status of PAB changes (see section 8.5)

Example GET request of AssemblyData unit with "Not Installed" status

Populated AssemblyData Resources

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-2/Assembly/PAB-2
| jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   275  100   275    0     0  68750      0 --:--:-- --:--:-- --:--:-- 68750
{
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-2/Assembly/PAB-2",
  "Actions": {},
  "Location": {
    "PartLocation": {},
    "Placement": {},
    "PostalAddress": {}
  },
  "MemberId": "2",
  "Name": "PAB-2",
  "Status": {
    "State": "Not Installed"
  }
```

Details of the response:

- notice that none of the HW FRU data is readable when the status shows up as not installed

8.7. The PowerSupply Resource

/redfish/v1/Chassis/{ChassisId}/Power/PowerSupplies/{PowerSupplyId}

The PowerSupply resource provides access to information on PSU (Power Supply unit) FRUs if they exist in the Storage enclosure that the Chassis resource represents. The resource structure provides data to capture the name and index of the PSU, any hardware data that is readable from the unit, Status of the unit which can change, triggering an Event, as well as various sensor data and other statistics if it is available.

PowerSupplyId: the Id for this resource is generated from the following:

- FRU type:
- FRU index
- Example: FRU type = PSU, FRU index = 1 - PowerSupplyId = PSU-1
- Note that FRU index for associated AssemblyData and PowerSupply resources are the same

Example GET request of PowerSupply unit with Ok status

Populated PSU resource

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-1 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   978  100   978    0     0   4770      0 --:--:-- --:--:-- --:--:--  4770
{
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-1",
  "Actions": {},
  "Assembly": {
    "PAB-1": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-1/Assembly/PAB-1"
  },
  "Location": {
    "PartLocation": {},
    "Placement": {},
    "PostalAddress": {}
  },
  "Manufacturer": "3YPOWER",
  "MemberId": "1",
  "Model": "EM-2A01P10",
  "Name": "PSU-1",
  "Oem": {
    "Statistics": {
      "Power Supply 1 fault_sen": "0",
      "Power Supply 1 manufacturer": "Viking Enterprise Solutions",
      "Power Supply 1 mfg_datetime": "2021.02.10 23:37:00",
```

```
      "Power Supply 1 overall_health": "ok",
      "Power Supply 1 part_number": "PCA-01119-01-C",
      "Power Supply 1 powerprst": "1",
      "Power Supply 1 product_name": "1600W PSU Assembly",
      "Power Supply 1 prsnt": "1",
      "Power Supply 1 serial_number": "CNE850000CAVE146",
      "Power Supply 1 warn_sen": "0"
    }
  },
  "PartNumber": "YSEF1600",
  "SerialNumber": "S0A00A3032029000204",
  "Status": {
    "State": "Ok"
  }
}
```


Details of the response:

- Assembly: Link to the AssemblyData resource associated with this PowerSupply unit
- Manufacturer: the HW Manufacturer of the PowerSupply unit
- MemberId: the FRU index of the PowerSupply unit
- Model: the HW model of the PowerSupply unit
- Name: the name of the PowerSupply unit which matches the PowerSupplyId
- Oem.Statistics:
    - These are additional statistics on the PowerSupply unit pulled as available
    - This includes several sensor readings, and additional health checks and hardware data
    - Note: this is not available for all FRUs and is based on what is collected by Zabbix Agents on the nodes
- PartNumber: the HW part number the PowerSupply unit
- SerialNumber: the serial number of the PowerSupply unit
- Status.State: the status of the PowerSupply unit
    - See section 8.5 for details

Dynamic Fields

- Manufacturer, Model, PartNumber, SerialNumber - could all change if a PSU unit is replaced
- Status.State - could change if status of PAB changes (see section 8.5)
- Oem.Statistics - could change based on the value of PSU sensors, etc.

Example GET request of PowerSupply unit with Unknown status - notice that this is the PowerSupply unit associated with AssemblyData unit above that had status of "Not Installed"

Populated PSU resource

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-2 | jq
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   486  100   486    0     0   2382      0 --:--:-- --:--:-- --:--:--  2370
{
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-2",
  "Actions": {},
  "Assembly": {
    "PAB-2": "/redfish/v1/Chassis/MXE30000122VS00A/Power/PowerSupplies/PSU-2/Assembly/PAB-2"
  },
  "Location": {
    "PartLocation": {},
    "Placement": {},
    "PostalAddress": {}
  },
  "MemberId": "2",
  "Name": "PSU-2",
  "Oem": {
    "Statistics": {
      "Power Supply 2 overall_health": "no_ac_power",
      "Power Supply 2 powerprst": "2"
    }
  },
  "Status": {
    "State": "Unknown"
  }
}
```

Details of the response:

- notice no HW FRU data and much less statistics are read for the PowerSupply unit when it has a Status.State of Unknown

8.8. The Fan Resource

/redfish/v1/Chassis/{ChassisId}/Thermal/Fans/{FanId}

The Fan resource provides access to information on Fan or Cooling unit FRUs if they exist in the Storage enclosure that the Chassis resource represents. The resource structure provides data to capture the name and index of the Fan, any hardware data that is readable from the unit, Status of the unit which can change, triggering an Event, as well as various sensor data and other statistics if it is available.

FanId: the Id for this resource is generated from the following:

- FRU type:
- FRU index

- Example: FRU type = Cooling, FRU index = 1 - FanId = Cooling-1

Note that most Fans have no readable FRU data such as:

- SerialNumber
- PartNumber
- Manufacturer
- Model

Example GET request for Fan unit with Ok Status

Populated Fan Resource

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-1 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   562  100   562    0     0   2768      0 --:--:-- --:--:-- --:--:--  2768
{
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-1",
  "Actions": {},
  "FanName": "Cooling-1",
  "HotPluggable": true,
  "Location": {
    "PartLocation": {},
    "Placement": {},
    "PostalAddress": {}
  },
  "Manufacturer": "N/A",
  "MemberId": "1",
  "Model": "N/A",
  "Name": "Cooling-1",
  "Oem": {
    "Statistics": {
      "Cooling 1 fan_prsnt": "1",
      "Cooling 1 fan_tach": "9900",
      "Cooling 1 overall_health": "ok",
      "Cooling 1 part_number": "102192600"
    }
  },
  "PartNumber": "N/A",
  "SerialNumber": "N/A",
  "Status": {
    "State": "Ok"
  }
}
```

Details of the response:

- Manufacturer: "N/A" is set because it is not readable
- MemberId: the FRU index of the Fan unit
- Model: "N/A" is set because it is not readable
- Name: the name of the Fan unit which matches the FanId
- Oem.Statistics:
  - These are additional statistics on the Fan unit pulled as available
  - This includes several sensor readings, and additional health checks and hardware data
  - Note: this is not available for all FRUs and is based on what is collected by Zabbix Agents on the nodes
- PartNumber: "N/A" is set because it is not readable
- SerialNumber: "N/A" is set because it is not readable
- Status.State: the status of the Fan unit
  - See section 8.5 for details

Dynamic Fields

- Status.State - could change if status of Fan changes (see section 8.5)
- Oem.Statistics - could change based on the value of Fan sensors, etc.

Example GET request for Fan unit with Critical status

Populated Fan Resource

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-4 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   562  100   562    0     0   2796      0 --:--:-- --:--:-- --:--:--  2810
{
  "@odata.id": "/redfish/v1/Chassis/MXE30000122VS00A/Thermal/Fans/Cooling-4",
  "Actions": {},
  "FanName": "Cooling-4",
  "HotPluggable": true,
  "Location": {
    "PartLocation": {},
    "Placement": {},
    "PostalAddress": {}
  },
  "Manufacturer": "N/A",
  "MemberId": "4",
  "Model": "N/A",
  "Name": "Cooling-4",
  "Oem": {
    "Statistics": {
      "Cooling 4 fan_prsnt": "1",
      "Cooling 4 fan_tach": "0",
      "Cooling 4 overall_health": "critical",
```

```
      "Cooling 4 part_number": "102192600"
    }
  },
  "PartNumber": "N/A",
  "SerialNumber": "N/A",
  "Status": {
    "State": "Critical"
  }
}
```


Details of the response:

- notice all of the FRU data fields are the same because they are not readable anyway
- The important thing to note is the change in Oem.Statistics sensor values and Status.State

Dynamic Fields

- Status.State - could change if status of Fan changes (see section 8.5)
- Oem.Statistics - could change based on the value of Fan sensors, etc.

9. Cluster StorageServices

StorageServiceCollection Resource /redfish/v1/StorageServices

The StorageServiceCollection resource provides a Collection of OdataId's which represent endpoints of all StorageService resources that exist in the cluster. Each individual StorageService resource within the StorageServiceCollection represents a Storage enclosure in the cluster.

Each StorageService resource, uniquely identified by some Id, will be the service entry point to access several other resources which may represent storage components of an enclosure, such as StoragePools, Drives, Volumes, FileSystems, etc.

Schema: https://redfish.dmtf.org/schemas/swordfish/HostedStorageServices.json

/redfish/v1/StorageServices GET request

```
[root@kjlmo17-oem rest_api]# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   812  100   812    0     0   264k      0 --:--:-- --:--:-- --:--:--  264k
{
```

```
  "@odata.context": "StorageServiceCollection.StorageServiceCollection",
  "@odata.id": "/redfish/v1/StorageServices",
  "@odata.type": "StorageServiceCollection.v1_0_0.StorageServiceCollection",
  "Description": "Collection of references StorageService Resources.",
  "Members": [
    {
      "enclosure_id: 3": "/redfish/v1/StorageServices/MXE30000122VS00A"
    },
    {
      "enclosure_id: 1": "/redfish/v1/StorageServices/MXE30000122VS00D"
    },
    {
      "enclosure_id: 2": "/redfish/v1/StorageServices/MXE30000122VS00E"
    },
    {
      "enclosure_id: 5": "/redfish/v1/StorageServices/SGFGD2044527EF1"
    },
    {
      "enclosure_id: 4": "/redfish/v1/StorageServices/SGFGD2044527F7C"
    },
    {
      "Lustre Filesystem": "/redfish/v1/StorageServices/lustre-kjlmo12"
    }
  ],
  "Members@odata.count": 6,
  "Name": "StorageServiceCollection"
}
```


Details of the response:

- **Members:** Collection of OdataId which represent endpoints of the StorageService resources that exist in the cluster
- **Members@odata.count:** the total count of StorageService resources that exist in the cluster

Dynamic Fields:

- Members may add or remove a member from the list if a Storage unit is added or removed
- Members StorageServiceId may change for one Member if enclosure is replaced
- Members@odata.count will change with the Members list

9.1. Enclosure Services

StorageService Resource: /redfish/v1/StorageServices/{StorageServiceId}

The StorageService resource is a top-level resource that represents a Storage enclosure. It provides the service entry point to access several other resources and collections of resources which may represent storage components of an enclosure. While there are two top-level resources that represent an enclosure, Chassis and StorageService, the StorageService resource focuses more on storage specific components of the enclosure, including StoragePools, Drives, Volumes, and FileSystems.

Collections that are offered by a StorageService are either all or a subset of the following:

- /redfish/v1/StorageServices/{StorageServiceId}/StoragePools
- /redfish/v1/StorageServices/{StorageServiceId}/Volumes
- /redfish/v1/StorageServices/{StorageServiceId}/FileSystems

9.1.1. StorageServiceId:

- StorageServiceId is represented by an enclosure serial number
- StorageServiceId is the same as /redfish/v1/Chassis/{ChassisId} ChassisId for an enclosure

Schema: https://redfish.dmtf.org/schemas/swordfish/StorageService.v1_5_0.json

Example StorageService GET request:

/redfish/v1/StorageServices/{StorageServiceId} GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00A | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   651  100   651    0     0   317k      0 --:--:-- --:--:-- --:--:--  317k
{
  "@odata.context": "StorageService.StorageService",
  "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A",
  "@odata.type": "StorageService.v1_5_0.StorageService",
  "Description": "Collection of Storage Services provided by enclosure MXE30000122VS00A id 3",
  "FileSystems": {
    "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems"
  },
  "Id": "MXE30000122VS00A",
  "Identifier": {},
  "Links": {},
  "Name": "viking-VSSEP1EC",
  "Status": {},
  "StoragePools": {
```

```
    "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools"
  },
  "Volumes": {
    "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/Volumes"
  }
}
```


Details of the response:

- FileSystems: Collection of FileSystem resources available to this StorageService
- Id: Id of this StorageService that is equal to the StorageServiceId in the resource endpoint
- Name: the name of the StorageService is set to the VPD string of the enclosure it represents
- StoragePools: Collection of StoragePool resources available to this StorageService
- Volumes: Collection of Volume resources available to this StorageService

Dynamic Fields:

- Id - may change if enclosure is replaced

9.2. StoragePoolCollection Resource

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools

The StoragePoolCollection resource provides a representation of "pools" of available storage provided by a particular StorageService. For our purposes, each individual StorageService will have just one available StoragePool that provides a path to see all Drive resources available to the StoragePool, as well as access individual Drives available to the StoragePool

Schema: https://redfish.dmtf.org/schemas/swordfish/StoragePoolCollection.json

StoragePoolCollection GET request


```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   498  100   498    0     0   243k      0 --:--:-- --:--:-- --:--:--  243k
{
  "@odata.context": "StoragePoolCollection.StoragePoolCollection",
  "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools",
  "@odata.type": "StoragePoolCollection.v1_0_0.StoragePoolCollection",
  "Description": "Collection of StoragePools for StorageService MXE30000122VS00E",
  "Members": [
    {
      "enclosure_id 2": "/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E"
    }
```

```
  ],
  "Members@odata.count": 1,
  "Name": "MXE30000122VS00E StoragePool Collection"
}
```

Details of the response:

- **Members:** this shows the endpoint to the StoragePool resource within the StoragePoolCollection
- Members@odata.count: this shows the total number of StoragePool resources in the StoragePoolCollection
- Name: the StorageServiceId is contained within the name

Dynamic Fields:

- Members - StorageServiceId or StoragePoolId may change if enclosure is replaced

9.3. StoragePool Resource

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePoolId}

The StoragePool resource provides a representation of a "pool" of available storage provided by a particular StorageService. While StoragePool resources provide several other things beyond Drives, for our purposes, the StoragePool will provide access to all available Drive resources for that enclosure in the StoragePools.CapacitySources.ProvidingDrives section. Each StoragePool provides a path to see all Drive resources available to the StoragePool, as well as access individual Drives available to the StoragePool.

StoragePoolId:

- StoragePoolId will be represent again by the enclosure serial number
- StoragePoolId will be equal to the higher level StorageServiceId

Schema: https://redfish.dmtf.org/schemas/swordfish/StoragePool.v1_7_0.json

StoragePool GET request

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  6170    0  6170    0     0  1506k      0 --:--:-- --:--:-- --:--:-- 1506k
{
  "@odata.context": "StoragePool.StoragePool",
  "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A",
  "@odata.type": "StoragePool.v1_6_0.StoragePool",
  "CapacitySources": [
```

{
"@odata.context": "Capacity.CapacitySource",
"@odata.id":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A",
"@odata.type": "Capacity.v1_2_0.CapacitySource",
"Description": "CapacitySource for StoragePool MXE30000122VS00A",
"Id": "MXE30000122VS00A",
"Name": "MXE30000122VS00A CapacitySource",
"ProvidedCapacity": {
"Data": {},
"Metadata": {},
"Snapshot": {}
},
"ProvidingDrives": {
"@odata.context": "DriveCollection.DriveCollection",
"@odata.id":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A/ProvidingDrives",
"@odata.type": "DriveCollection.v1_0_0.DriveCollection",
"Description": "Providing Drives for CapacitySource MXE30000122VS00A",
"Members": [
{
"Bay 1":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A/ProvidingDrives/S4YPNE0NA05073"
},
{
"Bay 2":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A/ProvidingDrives/S4YPNE0NC03831"
},
.
.
.

{
"Bay 23":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A/ProvidingDrives/S4YPNE0N900143"
},
{
"Bay 24":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A/ProvidingDrives/S4YPNE0N900114"
}
],
"Members@odata.count": 24,

```
      "Name": "MXE30000122VS00A ProvidingDrives"
    },
    "ProvidingMemory": {
      "@odata.id": "",
      "@odata.type": "",
      "Members": null,
      "Members@odata.count": 0,
      "Name": ""
    },
    "ProvidingMemoryChunks": {
      "@odata.id": "",
      "@odata.type": "",
      "Members": null,
      "Members@odata.count": 0,
      "Name": ""
    },
    "ProvidingPools": {
      "@odata.id": "",
      "@odata.type": "",
      "Members": null,
      "Members@odata.count": 0,
      "Name": ""
    },
    "ProvidingVolumes": {
      "@odata.id": "",
      "@odata.type": "",
      "Members": null,
      "Members@odata.count": 0,
      "Name": ""
    }
  }
],
"CapacitySources@odata.count": 1,
"Description": "StoragePool provided by StorageService MXE30000122VS00A",
"Id": "MXE30000122VS00A",
"Identifier": {},
"Links": {},
"NVMeEnduranceGroupProperties": {
  "EndGrpLifetime": {}
},
"NVMeProperties": {},
"NVMeSetProperties": {},
"Name": "MXE30000122VS00A StoragePool",
"Status": {}
}
```

Details of the response:

- **Id:** The Id of the StoragePool, must be equal to StoragePoolId in the resource endpoint AND the StorageServiceId of the StorageService that provides the StoragePool
- **CapacitySources:** Provides a list of CapacitySources resources which provides access to ProvidingDrives resource collection
    - ProvidingDrives: Provides a DriveCollection resource which lists all Drive resource OdataId's available to this StoragePool

Dynamic Fields:

- Id: may change if enclosure is replaced
- ProvidingDrives.Members - DriveId of members may change if a Drive is replaced

9.4. StoragePool CapacitySourceCollection Resource

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePoolId}/CapacitySources

The StoragePool CapacitySourcesCollection resource will provide a list of CapacitySources available for a particularStoragePool. For our purposes, each StoragePool will have only 1 CapacitySource. Each CapacitySource provides access to the ProvidingDrives collection which lists the OdataId's of all Drive resources available to this StoragePool

Schema: https://redfish.dmtf.org/schemas/swordfish/CapacitySourceCollection.json

StoragePool CapacitySourceCollection GET request

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   491  100   491    0     0   159k      0 --:--:-- --:--:-- --:--:--  159k
{
  "@odata.context": "CapacitySourceCollection.CapacitySourceCollection",
  "@odata.id":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources",
  "@odata.type": "CapacitySourceCollection.v1_0_0.CapacitySourceCollection",
  "Members": [
    {
      "@odata.id":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A"
    }
```

```
  ],
  "Members@odata.count": 1,
  "Name": "MXE30000122VS00A CapacitySourceCollection"
}
```

Details of the response:

- **Members:** this shows the endpoint to the CapacitySource resource within the CapacitySources collection
- Members@odata.count: this shows the total number of CapacitySource resources in the CapacitySources collection

Dynamic Fields:

- Id: may change if enclosure is replaced

9.5. StoragePool CapacitySource Resource

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePoolId}/CapacitySources/{CapacitySourceId}

The CapacitySource resource provides a representation of the CapacitySource of a particular StoragePool provided by a particular StorageService. While CapacitySource provides many things, for our purposes, CapacitySource will provide access to ProvidingDrives of a StoragePool. The ProvidingDrives list will contain OdataId's for each Drive resource available to the StoragePool

Schema: https://redfish.dmtf.org/schemas/swordfish/Capacity.v1_2_0.json

CapacitySourceId:

- CapacitySourceId will be represent again by the enclosure serial number
- CapacitySourceId will be equal to the higher level StoragePoolId and StorageServiceId

StoragePool CapacitySource GET request

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  5370    0  5370    0     0  1748k      0 --:--:-- --:--:-- --:--:-- 1748k
{
  "@odata.context": "Capacity.CapacitySource",
  "@odata.id":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A",
```

```
  "@odata.type": "Capacity.v1_2_0.CapacitySource",
  "Description": "CapacitySource for StoragePool MXE30000122VS00A",
  "Id": "MXE30000122VS00A",
  "Name": "MXE30000122VS00A CapacitySource",
  "ProvidedCapacity": {
    "Data": {},
    "Metadata": {},
    "Snapshot": {}
  },
  "ProvidingDrives": {
    "@odata.context": "DriveCollection.DriveCollection",
    "@odata.id":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000
122VS00A/ProvidingDrives",
    "@odata.type": "DriveCollection.v1_0_0.DriveCollection",
    "Description": "Providing Drives for CapacitySource MXE30000122VS00A",
    "Members": [
      {
        "Bay 1":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000
122VS00A/ProvidingDrives/S4YPNE0NA05073"
      },
      {
        "Bay 2":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000
122VS00A/ProvidingDrives/S4YPNE0NC03831"
      },
      .
      .
      .
      {
        "Bay 23":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000
122VS00A/ProvidingDrives/S4YPNE0N900143"
      },
      {
        "Bay 24":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000
122VS00A/ProvidingDrives/S4YPNE0N900114"
      }
    ],
    "Members@odata.count": 24,
    "Name": "MXE30000122VS00A ProvidingDrives"
  },
  "ProvidingMemory": {
    "@odata.id": "",
    "@odata.type": "",
    "Members": null,
```

```
    "Members@odata.count": 0,
    "Name": ""
  },
  "ProvidingMemoryChunks": {
    "@odata.id": "",
    "@odata.type": "",
    "Members": null,
    "Members@odata.count": 0,
    "Name": ""
  },
  "ProvidingPools": {
    "@odata.id": "",
    "@odata.type": "",
    "Members": null,
    "Members@odata.count": 0,
    "Name": ""
  },
  "ProvidingVolumes": {
    "@odata.id": "",
    "@odata.type": "",
    "Members": null,
    "Members@odata.count": 0,
    "Name": ""
  }
}
```


Details of the response:

- **Id:** The Id of the CapacitySource, must be equal to CapacitySourceId in the resource endpoint AND the StoragePoolId AND the StorageServiceId of the StorageService that provides the StoragePool
- ProvidingDrives: Provides a DriveCollection resource which lists all Drive resource OdataId's available to this StoragePool
- Members@odata.count: Provides the total number of Drive resources in the DriveCollection available to the StoragePool
- NOTE: the CapcitySource resource is a sub-structure of the StoragePool resource

Dynamic Fields:

- Id: may change if enclosure is replaced
- ProvidingDrives.Members - DriveId of members may change if a Drive is replaced

9.6. CapacitySource ProvidingDrives - DriveCollection Resource

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePoolId}/CapacitySources/{CapacitySourceId}/ProvidingDrives

The CapacitySource ProvidingDrives DriveCollection resource will provide a list of Drive resources available for a particular StoragePool. For our purposes, each CapacitySource will have only 1 ProvidingDrives resource The ProvidingDrives resource is represent by a DriveCollection resource. The DriveCollection resource will contain a list of OdataId's for Drive resources available to the StoragePool.

Note: The links seen in the DriveCollection resource will provide information on which enclosure drive bay or slot each drive is contained in.

Schema: https://redfish.dmtf.org/schemas/v1/DriveCollection.json

CapacitySource ProvidingDrives GET request

```
[root@kjlmo12-oem rest_api ~]# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/Ca
pacitySources/MXE30000122VS00E/ProvidingDrives | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  4320    0  4320    0     0   234k      0 --:--:-- --:--:-- --:--:--  234k
{
  "@odata.context": "DriveCollection.DriveCollection",
  "@odata.id":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives",
  "@odata.type": "DriveCollection.v1_0_0.DriveCollection",
  "Description": "Providing Drives for CapacitySource MXE30000122VS00E",
  "Members": [
    {
      "Bay 1":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives/S4YPNE0NA04612"
    },
    {
      "Bay 2":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives/S4YPNE0NA04615"
    },
    {
      "Bay 3":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives/S4YPNE0NA04613"
    },
    {
```

```
      "Bay 4":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives/S4YPNE0NA04614"
    },
    {
      "Bay 5":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives/S4YPNE0NA04600"
    },
    {
      "Bay 6":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives/S4YPNE0NA04597"
    },
    {
      "Bay 7":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives/S4YPNE0NA04602"
    },
    .
    .
    .
    {
      "Bay 24":
"/redfish/v1/StorageServices/MXE30000122VS00E/StoragePools/MXE30000122VS00E/CapacitySources/MXE30000
122VS00E/ProvidingDrives/S4YPNE0NA04567"
    }
  ],
  "Members@odata.count": 24,
  "Name": "MXE30000122VS00E ProvidingDrives"
}
```


Details of the response:

- **Members:** this shows the list of OdataId's of all Drive resources available to the DriveCollection
- Members@odata.count: this shows the total number of Drive resources in the DriveCollection

Dynamic Fields:

- Id: may change if enclosure is replaced
- ProvidingDrives.Members - DriveId of members may change if a Drive is replaced

9.7. Drive Resource

/redfish/v1/StorageServices/{StorageServiceId}/StoragePools/{StoragePoolId}/CapacitySources/{CapacitySourceId}/ProvidingDrives/{DriveId}

Drive resource will represent an actual enclosure drive available to that StoragePool.

DriveId

- DriveId will be represented by the Drive Serial Number
- Drive resource will contain information about which enclosure drive bay or slot the drive is contained in

ProvidingDrives Drive GET request

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A/ProvidingDrives/S4YPNE0NC03831 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   585  100   585    0     0   142k      0 --:--:-- --:--:-- --:--:--  142k
{
  "@odata.context": "Drive.Drive",
  "@odata.id":
"/redfish/v1/StorageServices/MXE30000122VS00A/StoragePools/MXE30000122VS00A/CapacitySources/MXE30000122VS00A/ProvidingDrives/S4YPNE0NC03831",
  "@odata.type": "Drive.v1_12_1.Drive",
  "Id": "S4YPNE0NC03831",
  "Links": {},
  "Manufacturer": "SAMSUNG",
  "MediaType": "NVME",
  "Model": "MZWLJ3T8HBLS-00007",
  "Name": "Drive S4YPNE0NC03831 in bay 2",
  "Oem": {},
  "PhysicalLocation": {
    "PartLocation": {},
    "Placement": {},
    "PostalAddress": {}
  },
  "Protocol": "NVME",
  "SerialNumber": "S4YPNE0NC03831",
  "Status": {
    "State": "Ok"
  }
}
```

Details of the response:

- **Id:** Id of the Drive which is equal to the DriveId in the resource endpoint. Set to the Drive Serial Number

- Manufacturer: the HW manufacturer of the Drive
- MediaType: the drive Media type - i.e. SAS, SATA, NVMe, etc.
- Model: the HW model number of the Drive
- Name: contains the Serial number of the drive and the Bay which it resides in
- Protocol: the Drive controller protocol
- SerialNumber: the Drive serial number
- Status.State: the current state of the Drive
    - see section 8.5 for more details on available states
    - more drive states may be available in the future, but for now treated as a FRU state

Dynamic Fields:

- Id - may change if Drive is replaced
- Status.State - may change depending on state of the Drive (see section 8.5)

9.8. VolumeCollection Resource

/redfish/v1/StorageServices/{StorageServiceId}/Volumes

The VolumeCollection resource provides a representation of available Volume resources provided by a particular StorageService. Volume resources represent arrays in NEO RFSF API, and depending on the type of the ComputerSystem resource pair associated with the StorageService (any non mgmt types), will have an associated FileSystem resource to represent the lustre target target that is formatted on the array.

Schema: https://redfish.dmtf.org/schemas/swordfish/VolumeCollection.json

/redfish/v1/StorageServices/{StorageServiceId}/Volumes GET request

```
[root@kjlmo17-oem rest_api]# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00E/Volumes | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   572  100   572    0     0  63555      0 --:--:-- --:--:-- --:--:-- 63555
{
  "OdataContext": "VolumeCollection.VolumeCollection",
  "OdataId": "/redfish/v1/StorageServices/MXE30000122VS00E/Volumes",
  "OdataType": "VolumeCollection.v1_0_0.VolumeCollection",
  "Description": "Collection of Volume resources for StorageServiceId MXE30000122VS00E",
  "Members": [
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00E/Volumes/md65"
    },
    {
```

```
    "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00E/Volumes/md66"
   }
  ],
  "MembersOdataCount": 2,
  "Name": "MXE30000122VS00E VolumeCollection"
 }
```


Details of the response:

- **Members:** this shows the endpoints to the Volume resources within the VolumeCollection
- Members@odata.count: this shows the total number of Volume resources in the VolumeCollection
- Name: the StorageServiceId is contained within the name

Dynamic Fields:

- no dynamic fields

9.9. Volume Resource

/redfish/v1/StorageServices/{StorageServiceId}/Volumes/{VolumeId}

The Volume resource will be used to represent, and individual Volume provided by a particular StorageService, in NEO rfsf API, Volumes represent underlying arrays on the system. If the StorageService represents a particular enclosure, such as an MDU, then the Volume resources provided by that StorageService will show data about the underlying arrays assembled on the drives in that enclosure. Each Volume resource (except for StorageService associated with ComputerSystem resources which represent mgmt nodes) will have an associated FileSystem resource which represents the lustre target formatted on the array the Volume resource represent. This relation can be seen in the Links.Oem.FileSystem section of the Volume resource. Each should have only 1 associated FileSystem (except mgmt StorageServices which have 0).

VolumeId:

- The VolumeId of each Volume resource will be represented by the Array or Pool name (ZFS) it represents
  - Ex: /dev/md65
    - VolumeId: md65
  - Ex: pool-mds65/mdt65
    - VolumeId: pool-mds65

Schema: https://redfish.dmtf.org/schemas/swordfish/Volume.v1_7_0.json

Example GET request to md65 Volume resource

/redfish/v1/StorageServices/{StorageServiceId}/Volumes/{VolumeId} GET request

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00E/Volumes/md65 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1367  100  1367    0     0  18226      0 --:--:-- --:--:-- --:--:-- 18226
{
  "OdataContext": "Volume.Volume",
  "OdataId": "/redfish/v1/StorageServices/MXE30000122VS00E/Volumes/md65",
  "OdataType": "Volume.v1_6_2.Volume",
  "Description": "Volume resource to represent Array md65",
  "Id": "md65",
  "Links": {
    "Oem": {
      "FileSystem": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-MDT0000"
    }
  },
  "Name": "Volume md65",
  "Oem": {
    "HomeHost": "kjlmo1202",
    "IcingaServices": {
      "Services": [
        {
          "Name": "Arrays and Disk Status",
          "State": "0",
          "Output": "No arrays found, T10PI: Not relevant on this system\\nTotal number of disk slots available:
24\\nTotal number of disks found: 24\\nslot:   1, wwn: eui.345950304ea04612002538450000004, cap:
3840755982336, dev: nvme13n1, parts:        0, status: Hot Spare, t10: 11110711121, type: ssd, sect: 4096, reason:
normal, dwpd: 1\\nslot:  13, wwn: eui.345950304ea04578002538450000004, cap:        3840755982336, dev:
nvme11n1, parts:        0, status: Hot Spare, t10: 11110711121, type: ssd, sect: 4096, reason: normal, dwpd:
1\\n\\nMD RAID to Lustre mapping\\n\\n\\n\\n\\nDegraded Array information:\\n\\nAll arrays are in clean state on
node kjlmo1202"
        }
      ]
    }
  },
  "RAIDType": "RAID10",
  "Status": {
    "State": "ok"
  }
}
```

Details of the response:

- Id: the Id of the resource which is derived from the array name on the system it represents and matches VolumeId in the resource endpoint
- Links.Oem.FileSystem: link to the associated FileSystem resource which represents the target formatted on this Volume
- Oem
  - HomeHost: the host this Volume should start on by default
  - IcingaServices.Services
    - Shows Name, State and Output of the Icinga2 "Arrays and Disk Status" service to provide information on the state of the Array and underlying Disks
- RAIDType: the raid type of the array, i.e. RAID10, RAID6, GRIDRAID dRAID2, etc.
- Status.State: the current state of the array, based on the Icinga2 "Arrays and Disk Status" service state

Dynamic Fields:

- IcingaServices.Services
  - State and Output of Arrays and Disks Status service will change as the state of Array and underlying drives change
- Status.State - will change as the status of the underlying Array changes

Example GET request to md64 Volume in mgmt node StorageService which shows there is no associated FileSystem resource

- this is NOT because there is no FileSystem at all on the md64 array
- this IS because for initial delivery, we are only monitoring Lustre FileSystem resources

/redfish/v1/StorageServices/{StorageServiceId}/Volumes/{VolumeId} GET request

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00D/Volumes/md64 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1303  100  1303    0     0  18884      0 --:--:-- --:--:-- --:--:-- 18884
{
  "OdataContext": "Volume.Volume",
  "OdataId": "/redfish/v1/StorageServices/MXE30000122VS00D/Volumes/md64",
  "OdataType": "Volume.v1_6_2.Volume",
  "Description": "Volume resource to represent Array md64",
  "Id": "md64",
  "Name": "Volume md64",
  "Oem": {
    "HomeHost": "kjlmo1200",
    "IcingaServices": {
      "Services": [
        {
```

```
        "Name": "Arrays and Disk Status",
        "State": "0",
        "Output": "All arrays are operating normally, T10PI: Not relevant on this system\\nTotal number of disk slots available: 24\\nTotal number of disks found: 5\\nslot:   5, wwn: eui.34594e304e9012810025384500000001, cap: 1600321314816, dev: nvme3n1, parts:        0, status: Hot Spare, t10: 11110711121, type: ssd, sect: 4096, reason: normal, dwpd: 3\\nArray: md64, status: Ok, t10: enabled\\nArray: md67, status: Ok, t10: enabled\\nArray: md511, status: Ok, t10: not-applicable\\n\\nMD RAID to Lustre mapping\\n\\nArray /dev/md/kjlmo1200:md64 doesn't have associated WIB array \\nArray /dev/md/kjlmo1200:md67 doesn't have associated WIB array \\n\\n\\n\\nDegraded Array information:\\n\\nAll arrays are in clean state on node kjlmo1200"
      }
    ]
  }
},
"RAIDType": "RAID10",
"Status": {
  "State": "ok"
}
```


Details of the response:

- Id: the Id of the resource which is derrived from the array name on the system it represents and matches VolumeId in the resource endpoint
- **NOTE: there are NO Links.Oem.FileSystem entries**
- Oem
    - HomeHost: the host this Volume should start on by default
- RAIDType: the raid type of the array, i.e. RAID10, RAID6, GRIDRAID dRAID2, etc.
- Status.State: the current state of the array, based on the Icinga2 "Arrays and Disk Status" service state

9.10. FileSystemCollection Resource

/redfish/v1/StorageServices/{StorageServiceId}/FileSystems

The FileSystemCollection resource provides a representation of available FileSystem resources provided by a particular StorageService. FileSystem resources represent lustre targets in NEO RFSF API, and depending on the type of the ComputerSystem resource pair associated with the StorageService, the FileSystem collection may not exist (i.e. for mgmt node StorageService).

Schema: https://redfish.dmtf.org/schemas/swordfish/FileSystemCollection.json

GET request to StorageService which contains FileSystem resources (non-mgmt)

/redfish/v1/StorageServices/{StorageServiceId}/FileSystems GET request

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   628  100   628    0     0   204k      0 --:--:-- --:--:-- --:--:--  204k
{
  "OdataContext": "FileSystemCollection.FileSystemCollection",
  "OdataId": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems",
  "OdataType": "FileSystemCollection.v1_0_0.FileSystemCollection",
  "Description": "Collection of FileSystem resources for StorageService MXE30000122VS00E",
  "Members": [
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-MDT0000"
    },
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-MDT0001"
    }
  ],
  "MembersOdataCount": 2,
  "Name": "MXE30000122VS00E FileSystemCollection"
}
```

Details of the response:

- **Members:** this shows the endpoints to the FileSystem resources within the StorageService
- Members@odata.count: this shows the total number of StoragePool resources in the StorageService
- Name: the StorageServiceId is contained within the name

Dynamic Fields:

- no dynamic fields

GET request to StorageService which does NOT contain any FileSystem resources (mgmt)

/redfish/v1/StorageServices/{StorageServiceId}/FileSystems GET request

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00D/FileSystems | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   124  100   124    0     0  31000      0 --:--:-- --:--:-- --:--:-- 31000
{
  "error": {
    "code": "404",
    "message": "No FileSystems present for StorageService Id MXE30000122VS00D"
  }
```

}

Details of the response:

- a RedfishError containing a 404 (NOTFOUND) response code is returned with an error message
- this is NOT because there is no FileSystem at all on the md64 array
- this IS because for initial delivery, we are only monitoring Lustre FileSystem resources

9.11. FileSystem Resource

/redfish/v1/StorageServices/{StorageServiceId}/FileSystems/{FileSystemId}

The FileSystem resource will be used to represent an individual FileSystem provided by a particular StorageService. If the StorageService represents a particular enclosure, such as an MDU, then the FileSystem resources provided by that StorageService will show data about the individual Lustre targets hosted by the node pair that reside in that enclosure. Each FileSystem resource will have an associated Volume resource which represents the array the lustre target is formatted on. This relation can be seen in the CapacitySources.ProvidingVolumes section of the FileSystem resource. Each FileSystem should only have 1 associated ProvidingVolume.

FileSystemId:

- the FileSystemId of each FileSystem resource will be represented by:
  - <fsname>-<target name>
  - where fsname == lustre filesystem name
  - target name == lustre target name
  - this should match what is actually formatted on-disk for this lustre target, and can be seen with various tools such as blkid, tunefs.lustre
- The FileSystemId also provides a means of associating each individual Lustre target FileSystem resource to the over-arching Lustre FileSystem resource
  - this will be provided by another StorageService that is not enclosure specific.

Schema: https://redfish.dmtf.org/schemas/swordfish/FileSystem.v1_2_0.json

FileSystem GET request for resource representing MDT0000

```
# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-MDT0000 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  3377    0  3377    0     0   8726      0 --:--:-- --:--:-- --:--:--  8703
{
  "OdataContext": "FileSystem.FileSystem",
  "OdataId": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-MDT0000",
```

```
  "OdataType": "FileSystem.v1_2_2.FileSystem",
  "CapacitySources": [
    {
      "OdataContext": "Capacity.CapacitySource",
      "OdataId": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-
MDT0000/CapacitySources/kjlmo12-MDT0000",
      "OdataType": "Capacity.v1_2_0.CapacitySource",
      "Description": "CapacitySource resource for FileSystem kjlmo12-MDT0000",
      "Id": "kjlmo12-MDT0000",
      "Name": "kjlmo12-MDT0000 CapacitySource",
      "ProvidingVolumes": {
        "OdataContext": "VolumeCollection.VolumeCollection",
        "OdataId": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-
MDT0000/CapacitySources/kjlmo12-MDT0000/ProvidingVolumes",
        "OdataType": "VolumeCollection.v1_0_0.VolumeCollection",
        "Description": "VolumeCollection resource for FileSystem kjlmo12-MDT0000",
        "Members": [
          {
            "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00E/Volumes/md65"
          }
        ],
        "MembersOdataCount": 1,
        "Name": "kjlmo12-MDT0000 ProvidingVolumes"
      }
    }
  ],
  "CapacitySourcesOdataCount": 1,
  "Description": "FileSystem resource to represent Lustre target kjlmo12-MDT0000",
  "Id": "kjlmo12-MDT0000",
  "Name": "FileSystem kjlmo12-MDT0000",
  "Oem": {
    "FsType": "Lustre",
    "State": "Started",
    "Lustre": {
      "MgsNIDS": "90@kfi,56@kfi:57@kfi,120@kfi",
      "FailoverNIDS": "57@kfi,120@kfi",
      "TargetType": "MGS|MDT",
      "TargetIndex": "0",
      "TargetName": "kjlmo12-MDT0000",
      "BackFsType": "ldiskfs",
      "FsName": "kjlmo12",
      "Device": "/dev/md65",
      "Statistics": {
        "MDT0000 used inodes": "289",
        "MDT0000 used space": "1978368",
        "MDT0000 close": "0",
        "MDT0000 connect": "0",
        "MDT0000 create": "0",
```

```
      "MDT0000 destroy": "0",
      "MDT0000 disconnect": "0",
      "MDT0000 free inodes": "4294122689",
      "MDT0000 total inodes": "4294122984",
      "MDT0000 getattr": "0",
      "MDT0000 getxattr": "0",
      "MDT0000 available space": "16267129159680",
      "MDT0000 free space": "16477245935616",
      "MDT0000 total space": "16477247987712",
      "MDT0000 link": "0",
      "MDT0000 llog_init": "0",
      "MDT0000 mkdir": "0",
      "MDT0000 mknod": "0",
      "MDT0000 notify": "0",
      "MDT0000 num exports": "0",
      "MDT0000 open": "0",
      "MDT0000 process_config": "0",
      "MDT0000 quotactl": "0",
      "MDT0000 reconnect": "0",
      "MDT0000 recovery completed clients": "0",
      "MDT0000 recovery status": "INACTIVE",
      "MDT0000 recovery time remaining": "0",
      "MDT0000 rename": "0",
      "MDT0000 rmdir": "0",
      "MDT0000 setattr": "0",
      "MDT0000 statfs": "0",
      "MDT0000 unlink": "0"
    }
  },
  "MountDir": "/data/kjlmo1202:md65",
  "Hostname": "kjlmo1202",
  "HaPeer": "kjlmo1203",
  "MountedHost": "kjlmo1202",
  "FailedOver": false
 }
}
```


Details of the response:

- CapacitySources.ProvidingVolumes.Members: will show a link to the underlying Volume resource which represents the array which the target is formatted on
- CapacitySourcesOdataCount: the number of ProvidingVolumes, should only ever be 1
- Id: Id of the FileSystem resource which is equal to the FileSystemId in the endpoint
- Oem.State will show the state of the target: Started, Stopped, Unknown
- Oem.FsType: the top-level FileSystem type, will be set to Lustre always
- Oem.Lustre will contain several Lustre specific details about the target including:
    - MGSNids: lustre nids of the MGS servers who can host MDT0000

- FailoverNids: lustre nids of the HA partner which this target can failover to
- TargetType: Lustre target type: MDT, MGT, MGTMDT, OST, etc.
- TargetIndex: Index of the Lustre target for that Target type in the FileSystem
- TargetName: blkid name of the Lustre target
- BackFsType: backing filesystem type of the lustre target: ldiskfs or zfs
- FsName: top-level lustre filesystem name
- Device: underlying array or pool device which this target is formatted on (represented by Volume resource in FileSystem.CapacitySources.ProvidingVolumes.Members)
- Statistics: various Lustre statistics for this target
- MountDir: the directory which serves as a mountpoint for the target
- Hostname: the primary HostName where this target should be started
- HaPeer: the hostname of the HA peer this target could be failed over to
- MountedHost: the hostname the target is currently mounted on (if it is started)
- FailedOver: a Boolean that indicates whether a target is failed over
    - True if started on its HaPeer host
    - False if started on its primary Hostname host.

Dynamic Fields:

- Oem
    - FsName: may change if Lustre Filesystem is renamed
    - State: will change as the lustre target is mounted or unmounted
    - Statistics: will change with changes in I/O activity to the lustre target
    - MountedHost: will change depending which host the lustre target gets mounted on
    - FailedOver: will change depending if lustre target is mounted on its primary HostName or its HaPeer hostname

10. Cluster Managers

ManagerCollection Resource /redfish/v1/Managers

From a Redfish/Swordfish (rfsf) REST API perspective, a NEO based ClusterStor cluster can also be represented by a ManagerCollection Resource:

Schema: https://redfish.dmtf.org/schemas/v1/ManagerCollection.json

This Collection represents a set of Manager resources which represent ClusterStor Storage nodes, similar to how a storage based ComputerSystem resource represents a ClusterStor Storage node, but with a few unique purposes (seen below in this section):

- Manager resources provide representation of highavailability (HA) storage node pairs

- Manager resources provide paths to highavailability (HA) resource groups represented as Redundancy resources
- Manager resources provide Actions to operate on Redundancy resources
- Manager resources provide Actions to remotely get and operate on storage node power state

Each Manager resource within the ManagerCollection resource must have a unique Id. Each Manager resource within the ManagerCollection resource will have an Id that matches some associated ComputerSystem resource in the StorageSystemCollection resource, as they represent the same underlying ClusterStor storage node.

Populated ManagerCollection resource to represent a Cluster:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://10.214.128.240:8081/redfish/v1/Managers | jq
{
  "@odata.context": "/redfish/v1/$metadata#ManagerCollection.ManagerCollection",
  "@odata.id": "/redfish/v1/Managers",
  "@odata.type": "#ManagerCollection.ManagerCollection",
  "Members": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-40U-A"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-40U-B"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-38U-A"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-38U-B"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B"
    }
  ],
  "Members@odata.count": 6,
  "Name": "HA Managers"
}
```

Details of the response:

- @odata.id: the URI endpoint of the resource
- @odata.type: ManagerCollection type reference

- Members: Collection of Manager resource odata.id endpoints based on the Manager resources that exists in the cluster
- Members@odata.count: the total count of Manager resources that exist in the cluster
- Name: name of the ManagerCollection resource (indicative of the highavailability HA representation)

Dynamic fields:

- Members may add or remove a member from the list if the Storage node is replaced, added, or removed
- Members@odata.count will change with the list

10.1 Storage Node Manager

Manager Resource: /redfish/v1/Manager/{ManagerId}

From a Redfish/Swordfish (rfsf) REST API perspective, a ClusterStor storage node as part of a highly available (HA) pair, will be represented as a Manager resource. The ClusterStor RFSF API implementation has extended the Manager resource schema to represent one node in an HA pair, so that it can manage underlying Redundancy resources, which represent HA resource groups (see Redundancy resource section below).

Schema: https://redfish.dmtf.org/schemas/v1/Manager.v1_19_1.json

Each resource captures Oem information about the node it represents, as well as its HA partner node. Each resource also provides a Redundancy list of Redundancy resources it manages, as well as a list of Action request endpoints which go through this Manager.

Example GET request to a Manager resource:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq
{
  "@odata.context": "Manager.Manager",
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A",
  "@odata.type": "Manager.v1_12_0.Manager",
  "Actions": {
    "#Manager.GetPowerState": {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.GetPowerState"
    },
    "#Manager.MoveFilesystem": {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem"
    },
    "#Manager.PowerOff": {
```

```
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.PowerOff"
  },
  "#Manager.PowerOn": {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.PowerOn"
  },
  "#Manager.Reset": {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.Reset"
  },
  "#Manager.StartFilesystem": {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem"
  },
  "#Manager.StopFilesystem": {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem"
  }
},
"CommandShell": {},
"Description": "Primary HA Manager",
"FirmwareVersion": "10.22",
"GraphicalConsole": {},
"Id": "kjlmo12:Node-R1C1-36U-A",
"Links": {},
"Location": {
  "PartLocation": {},
  "Placement": {},
  "PostalAddress": {}
},
"ManagerType": "HA",
"Manufacturer": "Viking Enterprise Solutions",
"Model": "VSSEP1EC",
"Name": "Primary HA Manager",
"Oem": {
  "IsPrimary": true,
  "Hostname": "kjlmo1204",
  "HostIP": "172.16.2.7/16",
  "IpmiAddr": "172.16.0.105",
  "IpmiSecAddr": "169.254.1.1",
  "ChassisSerialNumber": "MXE30000122VS00A",
  "PartnerInfo": {
    "Hostname": "kjlmo1205",
    "HostIP": "172.16.2.8/16",
    "Id": "kjlmo12:Node-R1C1-36U-B",
    "IpmiAddr": "172.16.0.106",
    "IpmiSecAddr": "169.254.1.2",
    "SerialNumber": "MXE3000011TVT074",
    "ChassisSerialNumber": "MXE30000122VS00A",
    "DaemonInfo": {}
  },
  "DaemonInfo": {},
```

```
    "NodeRole": [
      "oss"
    ]
  },
  "PartNumber": "ASM-02903-01-B",
  "PowerState": "On",
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ],
  "Redundancy@odata.count": 3,
  "SerialConsole": {},
  "SerialNumber": "MXE3000011TVT076",
  "Status": {},
  "UUID": "09090000-3d00-003d-0606-00eded000707"
}
```


Details of the response:

- @odata.id: the URI endpoint of the resource
- @odata.type: Manager type reference
- Actions: list of Manager Action request endpoints available (see Manager Actions request section below)
- FirmwareVersion: the BMC firmware version of the underlying storage node's BMC
- Id: unique resource Id
- ManagerType: set to Oem ManagerType "HA" to indicate it represents a node in a highly available node pair
- Manufacturer: Manufacturer of underlying storage node
- Model: Model of underlying storage node
- Name: resource name
- Oem: oem extensions to this resource
- Oem.IsPrimary: indication whether this storage node is in the primary slot of an enclosure or not
- Oem.Hostname: hostname of the storage node
- Oem.HostIP: Host IP address and subnet on the LMN network for this storage node
- Oem.IpmiAddr: primary IPMI address of this storage node
- Oem.IpmiSecAddr: secondary IPMI address of this storage node
- Oem.ChassisSerialNumber: Chassis serial number of the enclosure this storage node is contained in – used to determine partner node match
- Oem.PartnerInfo: information about the HA partner of this storage node

- Oem.PartnerInfo.Hostname: hostname of the HA partner of this storage node
- Oem.PartnerInfo.HosIP: Host IP address and subnet on the LMN network of the HA partner of this storage node
- Oem.PartnerInfo.Id: resource Id of the Manager resource which represents the HA partner of this storage node
- Oem.PartnerInfo.IpmiAddr: primary IPMI address of the HA partner of this storage node
- Oem.PartnerInfo.IpmiSecAddr: secondary IPMI address of the HA partner of the HA partner of this storage node
- Oem.PartnerInfo.SerialNumber: serial number of the HA partner of this storage node
- Oem.PartnerInfo.ChassisSerialNumber: Chassis serial number of the enclosure the HA partner of this storage node is contained in – used to determine partner node match
- Oem.NodeRole: list of ClusterStor node roles of this storage node
- PartNumber: part number of this storage node
- PowerState: IPMI power state of this storage node
- Redundancy: List of Redundancy resource endpoints managed by this storage node
- Redundancy@odata.count: total number of Redundancy resources managed by this storage node
- SerialNumber: serial number of this storage node
- UUID: universally unique identifier of this storage node

Dynamic fields:

- FirmwareVersion: subject to change when storage node BMC firmware is flashed
- Oem.Hostname: subject to change if cluster rename operation is invoked (Same for PartnerInfo fields)
- Oem.HostIP, IpmiAddr, IpmiSecAddr: subject to change if cluster mgmt network is re IP'd (same for PartnerInfo fields)
- Oem.ChassisSerialNumber: subject to change if a Chassis replacement occurs (Same for PartnerInfo field)
- Oem.PartnerInfo.SerialNumber: subject to change if HA partner storage node is replaced
- PartNumber, Manufacturer, Model: subject to change if storage node is replaced
- PowerState: subject to change as storage node IPMI power state changes
- Redundancy, Redundancy@odata.count: subject to change as Redundancy resources are failed over or failed back (moved) between HA partner storage node, whether this happens via Action request (see below) or on the underlying system by HA services
- SerialNumber: subject to change if storage node is replaced
- UUID: subject to change if storage node is replaced

10.2 HA Resource Group Representation – Redundancy

Redundancy Resource:
/redfish/v1/Manager/{ManagerId}/Redundancy/{RedundancyId}

From a Redfish/Swordfish (rfsf) REST API perspective, ClusterStor HA resource groups which manage both Lustre targets and other management Filesystems and services via underlying HA services pacemaker and corosync, will be represented as Redundancy resources.

Schema: https://redfish.dmtf.org/schemas/v1/Redundancy.v1_4_2.json

The ClusterStor RFSF API implementation has extended the Redundancy resource schema in order to capture information about these underlying HA resource groups, including: status, primary, secondary and current locations of a resource group, underlying primitive resources contained within a resource group as well as their status and location, and underlying system components these HA CRM groups manage, including links to other API resources which represent these system components in the Redunduncy.RedundancySet field (i.e. Volume and FileSystem resources). The state of these HA resources groups are constantly being monitored so that the Redundancy resources, as well as which Manager resource owns them, is constantly up to date and accurately reflected.

Example GET request to a Redundancy resource currently owned by Manager resource: kjlmo12:Node-R1C1-36U-A:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group
| jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group",
  "Id": "kjlmo1204_md0-group",
  "MemberId": "",
  "MinNumNeeded": 0,
  "Mode": "",
  "Name": "Redundancy kjlmo1204_md0-group",
  "Oem": {
    "TargetType": "OST",
    "TargetName": "kjlmo12-OST0000",
    "FsName": "kjlmo12",
    "ArrayName": "md0",
    "ResourceGroupName": "kjlmo1204_md0-group",
    "ResourceGroupStatus": "started",
    "PrimaryLocation": "kjlmo1204",
    "SecondaryLocation": "kjlmo1205",
    "Location": "kjlmo1204",
    "FailedOver": false,
    "ResourceGroupAttrs": [
      {
        "Name": "kjlmo1204_md0-wibr",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
```

```
      {
        "Name": "kjlmo1204_md0-jnlr",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md0-wibs",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md0-raid",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md0-fsys",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md0-stop",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      }
    ],
    "HaNetworking": {
      "LNetInfo": {}
    },
    "StorageServiceId": "MXE30000122VS00A"
  },
  "RedundancySet": [
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/Volumes/md0"
    },
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0000"
    }
  ],
  "RedundancySet@odata.count": 2,
  "Status": {}
}
```

Details of the response

- @odata.id: the URI endpoint of the resource
- Id: unique Id of the resource
- Name: name of the resource
- Oem: Oem extensions to represent underlying HA resource group
- Oem.TargetType: the Lustre target type this HA resource group manages
- Oem.TargetName: the Lustre target name this HA resource manages – NOTE: this will match an existing FileSystem resource Id
- Oem.FsName: the top-level Lustre filesystem name the target this HA resource manages is a part of
- Oem.ArrayName: array or zpool name of the RAID volume this HA resource manages – NOTE: this will match an existing Volume resource Id
- Oem.ResourceGroupName: the actual name of the HA resource group managed by corosync/pacemaker
- Oem.ResourceGroupStatus: the status of the HA resource group (started, stopped)
- Oem.PrimaryLocation: the primary location this HA resource group should start (if not failed over)
- Oem.SecondaryLocation: the secondary location this HA resource group should start (if failedover)
- Oem.Location: the current location this HA resource group is started, or will start next (if currently stopped)
- Oem.FailedOver: indicates whether this HA resource groups is failed over or not, i.e. if Oem.Location != Oem.PrimaryLocation – then Oem.FailedOver: true
- Oem.ResourceGroupAttrs: collection of underlying HA primitive resources that are part of an HA resource group
- Oem.ResourceGroupAttrs.Name: name of the underlying HA primitive resource
- Oem.ResourceGroupAttrs.Status: status of the underlying HA primitive resource (started, stopped)
- Oem.ResourceGroupAttrs.ErrorMessage: contains any failed action or error information surrounding the underlying HA primitive resource
- Oem.ResourceGroupAttrs.Location: current location of the underlying HA primitive resource
- Oem.ResourceGroupAttrs.FailedOver: set to true if the Location of the underlying HA primitive resource != the HA resource group's PrimaryLocation
- Oem.StorageServiceId: The Id of the StorageService resource which represents the enclosure the two Manager resources are contained in
- RedundancySet: resource URI endpoints that represent underlying components this HA resource group manages, i.e. Volume and FileSystem resources
- RedundancySet@odata.count: number of resources in the RedundancySet

Dynamic Fields:

- ResourceGroupStatus: subject to change as underlying HA resource group is started/stopped (Same for ResourceGroupAttrs field for underlying HA primitive resources in the group)
- Location: subject to change as underlying HA resource group is failed over or failed back (Same for ResourceGroupAttrs field for underlying HA primitive resources in the group)
- FailedOver: subject to change as underlying HA resource group is failed over or failed back (Same for ResourceGroupAttrs field for underlying HA primitive resources in the group)

Redundancy resource monitoring behavior:

Because Redundancy resources represent underlying HA resource groups on the system, we must monitor these HA resource groups in case the HA services that manage them are instructed to start, stop, or move these resource groups from one host to the another in within the HA pair. The status and location of these groups may change due to administrative actions either through the API or other ClusterStor interfaces, or by failover of resources when a node encounters a problem that prevents it from hosting the resources in the group any longer, also known as "fencing and stonith" in the pacemaker/corosync world: https://clusterlabs.org/pacemaker/doc/crm_fencing.html . Additionally, we must capture any failed action errors in the Oem.ResourceGroupAttrs.ErrorMessage field when a fencing or stonith occurs.

In order to most accurately reflect the status and location of the underlying HA resource groups, the ClusterStor RFSF API implements a monitoring thread which will periodically inventory the state of all HA resource groups, and update each associated Redundancy resource, as well as the owning Manager resources, via PUT or PATCH requests between storage nodes in the HA pair as these HA resource groups change status or location.

If a Redundancy resource moves location from one host to another in the HA pair, the associated Manager.Redundancy lists will be updated to reflect which Manager resource currently owns that Redundancy resource. An example of this can be seen in below section explaining the Mangaer.MoveFilesystem action request, but this can also occur as part of the period monitoring interval. If resources move locations, or change status, an Event will be broadcasted indicating what happened.

10.3 Manager Action Requests Which Operate on Redundancy Resources

As previously mentioned, Redundancy resources represent HA resource groups that manage Lustre targets and their underlying components on the system. These HA resource groups can be instructed to start, stop, or move from one host in an HA pair to another via the underlying pacemaker and corosync HA services. ClusterStor RFSF API implements a RESTful interface

to instruct these HA resources groups to start, stop or move locations via the following OEM extension Manager Action requests:

- /redfish/v1/Managers/{ManagerId}/Actions/Manager.StartFilesystem
- /redfish/v1/Managers/{ManagerId}/Actions/Manager.StopFilesystem
- /redfish/v1/Managers/{ManagerId}/Actions/Manager.MoveFilesystem

By definition of an action request, each of these requests are a POST request. Each of these Action requests can operate on one or more Redundancy resource, specified by details past request body. While the Action is being carried out on the system under the hood, any subsequent GET request to the Redundancy endpoint being operated on will return a Task resource with details on the status of the underlying operation. More details about redfish Tasks can be seen in the redfish spec.

Task Schema: https://redfish.dmtf.org/schemas/v1/Task.v1_7_4.json

Additionally, an Event for each Redundancy resource operated on will be broadcasted to subscribers upon success or failure of the request.

See details below on each of the mentioned requests, including:

- Request body requirements
- Manager ownership requirements
- Current Status and Location requirements for Redundancy resources being operated on
- Task handling while operation is in progress
- Events broadcasted for failure and success
- Redundancy Status and Location change upon success
- Manager Redundancy list updates (for Redundancy groups being moved)

10.3.1 Manager.StartFilesystem Action Request

/redfish/v1/Manager/{ManagerId}/Actions/Manager.StartFilesystem

The Manager.StartFilesystem Action POST request is used to start a specified set of Redundancy resources in the request body. This request will instruct the corosync and pacemaker services to start the target HA resource group that each Redundancy resource represents.

Request Body:

The Manager.StartFilesystem POST request requires that you pass a set of Redundancy resource endpoint(s) to operate on. At least one Redundancy resource endpoint is required, but multiple can be passed to a single request.

Example of a Manager.StartFilesystem request to start three Redundancy resources which represent Lustre OST targets owned by a Manager resource of a given host:

```
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ]
}
```

NOTE: the request body can be derived from the current Redundancy list of the target Manager resource:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq '.Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
  }
]
```

As noted before, Redundancy Manager Action requests can only operate on Redundancy resources that a Manager currently owns. For example, if a Manager.StartFilesystem request is made to a Manager resource which does not own the Redundancy resource(s) passed in the request body, the following error will be seen:

```
# cat start_md0_2_4_group.json | jq
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
```

```
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ]
}

# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B | jq '.Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md1-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md3-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md5-group"
  }
]
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -d @start_md0_2_4_group.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Actions/Manager.StartFilesystem | jq
{
  "error": {
    "code": "400",
    "message": "Unable to forward request to host kjlmo1205. Received invalid response code 400 from request
to endpoint: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Actions/Manager.StartFilesystem. Failed to find
Redundancy resource by Id kjlmo1204_md0-group. rpc error: code = NotFound desc = Redundancy Resource
kjlmo1204_md0-group is not found in Manager kjlmo12:Node-R1C1-36U-B Redundancy list {\n\t\"error\":
{\n\t\t\"code\": \"400\",\n\t\t\"message\": \"Failed to find Redundancy resource by Id kjlmo1204_md0-group.
rpc error: code = NotFound desc = Redundancy Resource kjlmo1204_md0-group is not found in Manager
kjlmo12:Node-R1C1-36U-B Redundancy list\"\n\t}\n}"
  }
}
```

Because Manager resource kjlmo12:Node-R1C1-36U-B did not currently own redundancy resources kjlmo1204_md0-group, kjlmo1204_md2-group, kjlmo1204_md4-group – the Manager.StartFilesystem request cannot be handled by kjlmo12:Node-R1C1-36U-B, so a error response indicating so is returned.

Example of a successful Manager.StartFilesystem request to Manager kjlmo12:Node-R1C1-36U-A which owns the Redundancy resources passed in request body:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq '.Redundancy'
```

```
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
  }
]
# cat start_md0_2_4_group.json | jq
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ]
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -d @start_md0_2_4_group.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines start Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-
36U-A/Actions/Manager.StartFilesystem on RedundancyId kjlmo1204_md4-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "start-kjlmo1204_md4-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Started"
    }
  ],
  "Name": "start Redundancy kjlmo1204_md4-group",
  "Payload": {},
  "StartTime": "2024-08-09T11:57:12.159797517-06:00",
  "TaskState": "New"
}
```

When a request is accepted, it begins concurrent redfish Task operations for each of the 3 specified Redundancy groups. NOTE: because more than one Redundancy resource was specified in the request body, the Task resource of the *last* Redundancy Task created is returned as a response. NOTE: All Task resources are created with a TaskState of “New”.

When GET requests are run to each Redundancy resource being operated on while the underlying operation is still being carried out, a Task resource is returned to indicate the operation is still in progress:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-
group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines start Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-
36U-A/Actions/Manager.StartFilesystem on RedundancyId kjlmo1204_md0-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "start-kjlmo1204_md0-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Started"
    },
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Running"
    }
  ],
  "Name": "start Redundancy kjlmo1204_md0-group",
  "Payload": {},
  "StartTime": "2024-08-09T11:57:12.159093731-06:00",
  "TaskState": "Running"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-
group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group",
  "@odata.type": "#Task.v1_5_0.Task",
```

```
  "Description": "This Task Resource defines start Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem on RedundancyId kjlmo1204_md2-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "start-kjlmo1204_md2-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md2-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Started"
    },
    {
      "MessageArgs": [
        "kjlmo1204_md2-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Running"
    }
  ],
  "Name": "start Redundancy kjlmo1204_md2-group",
  "Payload": {},
  "StartTime": "2024-08-09T11:57:12.15939805-06:00",
  "TaskState": "Running"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines start Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem on RedundancyId kjlmo1204_md4-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "start-kjlmo1204_md4-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Started"
    },
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Running"
    }
  ],
  "Name": "start Redundancy kjlmo1204_md4-group",
```

```
  "Payload": {},
  "StartTime": "2024-08-09T11:57:12.159797517-06:00",
  "TaskState": "Running"
}
```


NOTE: the TaskState has been updated from “New” to “Running” for all Task resources involved.

When the underlying operation completes successfully, an Event is broadcasted for each Redundancy group being operated on to show that each Task for action request has completed successfully:


```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/617165cd-8cf2-43b9-a6a7-a04c7e8c0a88 | jq
{
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
  "Context": "ha-ec",
  "Events": [
    {
      "Actions": {},
      "EventId": "617165cd-8cf2-43b9-a6a7-a04c7e8c0a88",
      "EventTimestamp": "2024-08-09T11:57:44-06:00",
      "EventType": "",
      "MemberId": "0",
      "MessageArgs": [
        "kjlmo1204_md0-group",
        "kjlmo1204_md0-group",
        "started",
        "kjlmo1204"
      ],
      "MessageId": "Manager.StartFilesystem Success"
    }
  ],
  "Events@odata.count": 1,
  "Id": "617165cd-8cf2-43b9-a6a7-a04c7e8c0a88",
  "Name": "HA Reundancy Event"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/9065b987-a6c4-43e6-953c-ccdc8e4b631e | jq
{
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
  "Context": "ha-ec",
  "Events": [
    {
      "Actions": {},
```

```
    "EventId": "9065b987-a6c4-43e6-953c-ccdc8e4b631e",
    "EventTimestamp": "2024-08-09T11:57:45-06:00",
    "EventType": "",
    "MemberId": "0",
    "MessageArgs": [
      "kjlmo1204_md2-group",
      "kjlmo1204_md2-group",
      "started",
      "kjlmo1204"
    ],
    "MessageId": "Manager.StartFilesystem Success"
  }
 ],
 "Events@odata.count": 1,
 "Id": "9065b987-a6c4-43e6-953c-ccdc8e4b631e",
 "Name": "HA Reundancy Event"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/cb13359c-c10d-4705-bd72-af7a85942f0e | jq
{
 "@odata.context": "/redfish/v1/$metadata/#Event.Event",
 "@odata.type": "#Event.v1_5_0.Event",
 "Context": "ha-ec",
 "Events": [
  {
    "Actions": {},
    "EventId": "cb13359c-c10d-4705-bd72-af7a85942f0e",
    "EventTimestamp": "2024-08-09T11:57:48-06:00",
    "EventType": "",
    "MemberId": "0",
    "MessageArgs": [
      "kjlmo1204_md4-group",
      "kjlmo1204_md4-group",
      "started",
      "kjlmo1204"
    ],
    "MessageId": "Manager.StartFilesystem Success"
  }
 ],
 "Events@odata.count": 1,
 "Id": "cb13359c-c10d-4705-bd72-af7a85942f0e",
 "Name": "HA Reundancy Event"
}
```


An Event is an indication that a Task has completed for the associated resource. Upon successful completion of that Task, the Task resource will be automatically removed and

the next GET request to the associated Redundancy resource will return the Redundancy resource as expected.

Once the Task completes for each Redundancy resource, the state of the HA resources groups should be accurately reflected in each Redundancy resource GET request to indicate that they have been started, as well as the location they are started in, if they are failed over, etc.:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md0-group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group",
  "Id": "kjlmo1204_md0-group",
  "MemberId": "",
  "MinNumNeeded": 0,
  "Mode": "",
  "Name": "Redundancy kjlmo1204_md0-group",
  "Oem": {
    "TargetType": "OST",
    "TargetName": "kjlmo12-OST0000",
    "FsName": "kjlmo12",
    "ArrayName": "md0",
    "ResourceGroupName": "kjlmo1204_md0-group",
    "ResourceGroupStatus": "started",
    "PrimaryLocation": "kjlmo1204",
    "SecondaryLocation": "kjlmo1205",
    "Location": "kjlmo1204",
    "FailedOver": false,
    "ResourceGroupAttrs": [
      {
        "Name": "kjlmo1204_md0-wibr",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md0-jnlr",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md0-wibs",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
```

```
      },
      {
        "Name": "kjlmo1204_md0-raid",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md0-fsys",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md0-stop",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      }
    ],
    "HaNetworking": {
      "LNetInfo": {}
    },
    "StorageServiceId": "MXE30000122VS00A"
  },
  "RedundancySet": [
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/Volumes/md0"
    },
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0000"
    }
  ],
  "RedundancySet@odata.count": 2,
  "Status": {}
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md2-group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group",
  "Id": "kjlmo1204_md2-group",
  "MemberId": "",
  "MinNumNeeded": 0,
  "Mode": "",
  "Name": "Redundancy kjlmo1204_md2-group",
  "Oem": {
    "TargetType": "OST",
```

"TargetName": "kjlmo12-OST0001",
"FsName": "kjlmo12",
"ArrayName": "md2",
"ResourceGroupName": "kjlmo1204_md2-group",
"ResourceGroupStatus": "started",
"PrimaryLocation": "kjlmo1204",
"SecondaryLocation": "kjlmo1205",
"Location": "kjlmo1204",
"FailedOver": false,
"ResourceGroupAttrs": [
  {
    "Name": "kjlmo1204_md2-wibr",
    "Status": "started",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md2-jnlr",
    "Status": "started",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md2-wibs",
    "Status": "started",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md2-raid",
    "Status": "started",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md2-fsys",
    "Status": "started",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md2-stop",
    "Status": "started",
    "Location": "kjlmo1204",
    "FailedOver": false
  }
],
"HaNetworking": {

```
      "LNetInfo": {}
    },
    "StorageServiceId": "MXE30000122VS00A"
  },
  "RedundancySet": [
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/Volumes/md2"
    },
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0001"
    }
  ],
  "RedundancySet@odata.count": 2,
  "Status": {}
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md4-group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group",
  "Id": "kjlmo1204_md4-group",
  "MemberId": "",
  "MinNumNeeded": 0,
  "Mode": "",
  "Name": "Redundancy kjlmo1204_md4-group",
  "Oem": {
    "TargetType": "OST",
    "TargetName": "kjlmo12-OST0002",
    "FsName": "kjlmo12",
    "ArrayName": "md4",
    "ResourceGroupName": "kjlmo1204_md4-group",
    "ResourceGroupStatus": "started",
    "PrimaryLocation": "kjlmo1204",
    "SecondaryLocation": "kjlmo1205",
    "Location": "kjlmo1204",
    "FailedOver": false,
    "ResourceGroupAttrs": [
      {
        "Name": "kjlmo1204_md4-wibr",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md4-jnlr",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
```

```
      },
      {
        "Name": "kjlmo1204_md4-wibs",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md4-raid",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md4-fsys",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md4-stop",
        "Status": "started",
        "Location": "kjlmo1204",
        "FailedOver": false
      }
    ],
    "HaNetworking": {
      "LNetInfo": {}
    },
    "StorageServiceId": "MXE30000122VS00A"
  },
  "RedundancySet": [
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/Volumes/md4"
    },
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0002"
    }
  ],
  "RedundancySet@odata.count": 2,
  "Status": {}
}
```


If a Manager.StartFilesystem request is sent for a Redundancy resource which is already started, the request will be initially accepted and return a Task resource with TaskState of new as expected:

```
# cat start_md0_group.json | jq
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-
group"
    }
  ]
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md0-group | jq '.Oem.ResourceGroupStatus'
"started"
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -d @start_md0_group.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.StartFilesystem | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines start Manager.Action: /redfish/v1/Managers/kjlmo12:Node-
R1C1-36U-A/Actions/Manager.StartFilesystem on RedundancyId kjlmo1204_md0-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "start-kjlmo1204_md0-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem
Started"
    }
  ],
  "Name": "start Redundancy kjlmo1204_md0-group",
  "Payload": {},
  "StartTime": "2024-08-09T12:33:04.749543936-06:00",
  "TaskState": "New"
}
```

However, the Redundancy resource passed will be checked to see the current state of the underlying HA resource group and notice that it is already started, causing the next GET request to that Redundancy resource to return the Task resource, but with a TaskState of “Exception” indicating the operation failed, with a message as to why it failed:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md0-group | jq
```

```
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines start Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem on RedundancyId kjlmo1204_md0-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "start-kjlmo1204_md0-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Started"
    },
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StartFilesystem Running"
    },
    {
      "MessageArgs": [
        "kjlmo1204_md0-group",
        "CRM group kjlmo1204_md0-group is already started"
      ],
      "MessageId": "Manager.StartFilesystem Failure"
    }
  ],
  "Name": "start Redundancy kjlmo1204_md0-group",
  "Payload": {},
  "StartTime": "2024-08-09T12:33:04.749543936-06:00",
  "TaskState": "Exception"
}
```


As seen above, this Task failed because the underlying HA resource group was already started. Similarly, an Event will be broadcasted indicating this Task has failed:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/Events/714b81c2-a882-4cad-96b0-60b345970cd9 | jq
{
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
```

```
  "Context": "ha-ec",
  "Events": [
    {
      "Actions": {},
      "EventId": "714b81c2-a882-4cad-96b0-60b345970cd9",
      "EventTimestamp": "2024-08-09T12:33:05-06:00",
      "EventType": "",
      "MemberId": "0",
      "MessageArgs": [
        "kjlmo1204_md0-group",
        "CRM group kjlmo1204_md0-group is already started"
      ],
      "MessageId": "Manager.StartFilesystem Failure"
    }
  ],
  "Events@odata.count": 1,
  "Id": "714b81c2-a882-4cad-96b0-60b345970cd9",
  "Name": "HA Reundancy Event"
}
```


Task failures can also occur if the underlying operation being carried out fails along the way for any other reason, and similarly, a GET request to the Redundancy resource will return a Task with TaskState Exception and a message indicating what failed, and an Event will be broadcasted with a message indicating what failed.

After the Redundancy GET request for a failed operation returns the Task with TaskState Exception *one time,* the failed Task resource will be automatically removed and the next GET request will return that Redundancy resource again.

10.3.2 Manager.StopFilesystem Action Request

/redfish/v1/Manager/{ManagerId}/Actions/Manager.StopFilesystem

The Manager.StopFilesystem Action POST request is used to stop a specified set of Redundancy resources in the request body. This request will instruct the corosync and pacemaker services to stop the target HA resource group that each Redundancy resource represents.

Request Body:

Similar to the Manager.StartFileSystem request, the Manager.StopFilesystem POST request requires that you pass a set of Redundancy resource endpoint(s) to operate on. At least one Redundancy resource endpoint is required, but multiple can be passed to a single request.

Example of a Manager.StopFilesystem request to stop three Redundancy resources which represent Lustre OST targets owned by a Manager resource of a given host:

```
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ]
}
```

NOTE: the request body can be derived from the current Redundancy list of the target Manager resource:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq '.Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
  }
]
```

As noted before, Redundancy Manager Action requests can only operate on Redundancy resources that a Manager currently owns. For example, if a Manager.StopFilesystem request is made to a Manager resource which does not own the Redundancy resource(s) passed in the request body, the following error will be seen:

```
# cat start_md0_2_4_group.json | jq
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
```

```
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ]
}

# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B | jq '.Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md1-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md3-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md5-group"
  }
]
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -d @start_md0_2_4_group.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Actions/Manager.StopFilesystem | jq
{
  "error": {
    "code": "400",
    "message": "Unable to forward request to host kjlmo1205. Received invalid response code 400 from request
to endpoint: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Actions/Manager.StopFilesystem. Failed to find
Redundancy resource by Id kjlmo1204_md0-group. rpc error: code = NotFound desc = Redundancy Resource
kjlmo1204_md0-group is not found in Manager kjlmo12:Node-R1C1-36U-B Redundancy list {\n\t\"error\":
{\n\t\t\"code\": \"400\",\n\t\t\"message\": \"Failed to find Redundancy resource by Id kjlmo1204_md0-group.
rpc error: code = NotFound desc = Redundancy Resource kjlmo1204_md0-group is not found in Manager
kjlmo12:Node-R1C1-36U-B Redundancy list\"\n\t}\n}"
  }
}
```

Similar to the example above with Manager.StartFilesystem rqeuests, because Manager resource kjlmo12:Node-R1C1-36U-B did not currently own redundancy resources kjlmo1204_md0-group, kjlmo1204_md2-group, kjlmo1204_md4-group – the Manager.StopFilesystem request cannot be handled by kjlmo12:Node-R1C1-36U-B, so a error response indicating so is returned.

Example of a successful Manager.StopFilesystem request to Manager kjlmo12:Node-R1C1-36U-A which owns the Redundancy resources passed in request body:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq '.Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
```

```
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
  }
]
# cat start_md0_2_4_group.json | jq
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ]
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -d @start_md0_2_4_group.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines stop Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-
36U-A/Actions/Manager.StopFilesystem on RedundancyId kjlmo1204_md4-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "stop-kjlmo1204_md4-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem Started"
    }
  ],
  "Name": "stop Redundancy kjlmo1204_md4-group",
  "Payload": {},
  "StartTime": "2024-08-09T11:57:12.159797517-06:00",
  "TaskState": "New"
}
```

When a request is accepted, it begins concurrent redfish Task operations for each of the 3 specified Redundancy groups. NOTE: because more than one Redundancy resource was specified in the request body, the Task resource of the *last* Redundancy Task created is returned as a response. NOTE: All Task resources are created with a TaskState of “New”.

When GET requests are run to each Redundancy resource being operated on while the underlying operation is still being carried out, a Task resource is returned to indicate the operation is still in progress:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines stop Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem on RedundancyId kjlmo1204_md0-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "stop-kjlmo1204_md0-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem Started"
    },
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem Running"
    }
  ],
  "Name": "stop Redundancy kjlmo1204_md0-group",
  "Payload": {},
  "StartTime": "2024-08-09T11:57:12.159093731-06:00",
  "TaskState": "Running"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines stop Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.SopFilesystem on RedundancyId kjlmo1204_md2-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "stop-kjlmo1204_md2-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md2-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem Started"
    },
    {
      "MessageArgs": [
```

```
        "kjlmo1204_md2-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem
Running"
    }
  ],
  "Name": "stop Redundancy kjlmo1204_md2-group",
  "Payload": {},
  "StartTime": "2024-08-09T11:57:12.15939805-06:00",
  "TaskState": "Running"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-
group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines stop Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-
36U-A/Actions/Manager.StopFilesystem on RedundancyId kjlmo1204_md4-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "stop-kjlmo1204_md4-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem Started"
    },
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.StopFilesystem
Running"
    }
  ],
  "Name": "stop Redundancy kjlmo1204_md4-group",
  "Payload": {},
  "StartTime": "2024-08-09T11:57:12.159797517-06:00",
  "TaskState": "Running"
}
```


NOTE: the TaskState has been updated from “New” to “Running” for all Task resources involved.

When the underlying operation completes successfully, an Event is broadcasted for each Redundancy group being operated on to show that each Task for action request has completed successfully:


```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/617165cd-8cf2-
43b9-a6a7-a04c7e8c0a88 | jq
{
```

```
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
  "Context": "ha-ec",
  "Events": [
    {
      "Actions": {},
      "EventId": "617165cd-8cf2-43b9-a6a7-a04c7e8c0a88",
      "EventTimestamp": "2024-08-09T11:57:44-06:00",
      "EventType": "",
      "MemberId": "0",
      "MessageArgs": [
        "kjlmo1204_md0-group",
        "kjlmo1204_md0-group",
        "stopped",
        "kjlmo1204"
      ],
      "MessageId": "Manager.StopFilesystem Success"
    }
  ],
  "Events@odata.count": 1,
  "Id": "617165cd-8cf2-43b9-a6a7-a04c7e8c0a88",
  "Name": "HA Reundancy Event"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/9065b987-a6c4-43e6-953c-ccdc8e4b631e | jq
{
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
  "Context": "ha-ec",
  "Events": [
    {
      "Actions": {},
      "EventId": "9065b987-a6c4-43e6-953c-ccdc8e4b631e",
      "EventTimestamp": "2024-08-09T11:57:45-06:00",
      "EventType": "",
      "MemberId": "0",
      "MessageArgs": [
        "kjlmo1204_md2-group",
        "kjlmo1204_md2-group",
        "stopped",
        "kjlmo1204"
      ],
      "MessageId": "Manager.StopFilesystem Success"
    }
  ],
  "Events@odata.count": 1,
  "Id": "9065b987-a6c4-43e6-953c-ccdc8e4b631e",
  "Name": "HA Reundancy Event"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/cb13359c-c10d-4705-bd72-af7a85942f0e | jq
{
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
```

```
  "Context": "ha-ec",
  "Events": [
    {
      "Actions": {},
      "EventId": "cb13359c-c10d-4705-bd72-af7a85942f0e",
      "EventTimestamp": "2024-08-09T11:57:48-06:00",
      "EventType": "",
      "MemberId": "0",
      "MessageArgs": [
        "kjlmo1204_md4-group",
        "kjlmo1204_md4-group",
        "stopped",
        "kjlmo1204"
      ],
      "MessageId": "Manager.StopFilesystem Success"
    }
  ],
  "Events@odata.count": 1,
  "Id": "cb13359c-c10d-4705-bd72-af7a85942f0e",
  "Name": "HA Reundancy Event"
}
```


An Event is an indication that a Task has completed for the associated resource. Upon successful completion of that Task, the Task resource will be automatically removed and the next GET request to the associated Redundancy resource will return the Redundancy resource as expected.

Once the Task completes for each Redundancy resource, the state of the HA resources groups should be accurately reflected in each Redundancy resource GET request to indicate that they have been started, as well as the location they are started in, if they are failed over, etc.:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-
group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group",
  "Id": "kjlmo1204_md0-group",
  "MemberId": "",
  "MinNumNeeded": 0,
  "Mode": "",
  "Name": "Redundancy kjlmo1204_md0-group",
  "Oem": {
    "TargetType": "OST",
    "TargetName": "kjlmo12-OST0000",
    "FsName": "kjlmo12",
    "ArrayName": "md0",
    "ResourceGroupName": "kjlmo1204_md0-group",
    "ResourceGroupStatus": "stopped",
    "PrimaryLocation": "kjlmo1204",
```

```
  "SecondaryLocation": "kjlmo1205",
  "Location": "kjlmo1204",
  "FailedOver": false,
  "ResourceGroupAttrs": [
    {
      "Name": "kjlmo1204_md0-wibr",
      "Status": "stopped",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-jnlr",
      "Status": "stopped",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-wibs",
      "Status": "stopped",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-raid",
      "Status": "stopped",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-fsys",
      "Status": "stopped",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-stop",
      "Status": "stopped",
      "Location": "kjlmo1204",
      "FailedOver": false
    }
  ],
  "HaNetworking": {
    "LNetInfo": {}
  },
  "StorageServiceId": "MXE30000122VS00A"
},
"RedundancySet": [
  {
```

```
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/Volumes/md0"
    },
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0000"
    }
  ],
  "RedundancySet@odata.count": 2,
  "Status": {}
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-
group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group",
  "Id": "kjlmo1204_md2-group",
  "MemberId": "",
  "MinNumNeeded": 0,
  "Mode": "",
  "Name": "Redundancy kjlmo1204_md2-group",
  "Oem": {
    "TargetType": "OST",
    "TargetName": "kjlmo12-OST0001",
    "FsName": "kjlmo12",
    "ArrayName": "md2",
    "ResourceGroupName": "kjlmo1204_md2-group",
    "ResourceGroupStatus": "stopped",
    "PrimaryLocation": "kjlmo1204",
    "SecondaryLocation": "kjlmo1205",
    "Location": "kjlmo1204",
    "FailedOver": false,
    "ResourceGroupAttrs": [
      {
        "Name": "kjlmo1204_md2-wibr",
        "Status": "stopped",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md2-jnlr",
        "Status": "stopped",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md2-wibs",
        "Status": "stopped",
        "Location": "kjlmo1204",
        "FailedOver": false
```

```
      },
      {
        "Name": "kjlmo1204_md2-raid",
        "Status": "stopped",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md2-fsys",
        "Status": "stopped",
        "Location": "kjlmo1204",
        "FailedOver": false
      },
      {
        "Name": "kjlmo1204_md2-stop",
        "Status": "stopped",
        "Location": "kjlmo1204",
        "FailedOver": false
      }
    ],
    "HaNetworking": {
      "LNetInfo": {}
    },
    "StorageServiceId": "MXE30000122VS00A"
  },
  "RedundancySet": [
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/Volumes/md2"
    },
    {
      "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0001"
    }
  ],
  "RedundancySet@odata.count": 2,
  "Status": {}
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-
group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group",
  "Id": "kjlmo1204_md4-group",
  "MemberId": "",
  "MinNumNeeded": 0,
  "Mode": "",
  "Name": "Redundancy kjlmo1204_md4-group",
  "Oem": {
    "TargetType": "OST",
```

"TargetName": "kjlmo12-OST0002",
"FsName": "kjlmo12",
"ArrayName": "md4",
"ResourceGroupName": "kjlmo1204_md4-group",
"ResourceGroupStatus": "stopped",
"PrimaryLocation": "kjlmo1204",
"SecondaryLocation": "kjlmo1205",
"Location": "kjlmo1204",
"FailedOver": false,
"ResourceGroupAttrs": [
  {
    "Name": "kjlmo1204_md4-wibr",
    "Status": "stopped",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md4-jnlr",
    "Status": "stopped",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md4-wibs",
    "Status": "stopped",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md4-raid",
    "Status": "stopped",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md4-fsys",
    "Status": "stopped",
    "Location": "kjlmo1204",
    "FailedOver": false
  },
  {
    "Name": "kjlmo1204_md4-stop",
    "Status": "stopped",
    "Location": "kjlmo1204",
    "FailedOver": false
  }
],
"HaNetworking": {

```
    "LNetInfo": {}
  },
  "StorageServiceId": "MXE30000122VS00A"
 },
 "RedundancySet": [
  {
    "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/Volumes/md4"
  },
  {
    "@odata.id": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0002"
  }
 ],
 "RedundancySet@odata.count": 2,
 "Status": {}
}
```


Similar to a Manager.StartFilesystem request, if a Manager.StopFilesystem request is sent for a Redundancy resource which is already stopped, the same behavior will be expected in that:

- The request will be initially accepted and return a Task resource with a TaskState of new
- The Redundancy resource passed will be checked, and it will be determined that the current state of the underlying HA resource group is already stopped
- The next GET request to that Redundancy resource will return a Task resource with a TaskSate of "Exception" indicating the operation failed, with a message as to why it failed
- An Event will be broadcasted indicating this Task has failed

Task failures can also occur if the underlying operation being carried out fails along the way for any other reason and similarly, a GET request to the Redundancy resource will return a Task with TaskState Exception and a message indicating what failed, and an Event will be broadcasted with a message indicating what failed.

After the Redundancy GET request for a failed operation returns the Task with TaskState Exception *one time,* the failed Task resource will be automatically removed and the next GET request will return that Redundancy resource again.

10.3.3 Manager.MoveFilesystem Action Request

/redfish/v1/Manager/{ManagerId}/Actions/Manager.MoveFilesystem

The Manager.StopFilesystem Action POST request is used to move a a specified set of Redundancy resources in the request body from one Manager resource (which represents a

hostname in an HA pair) to its HA peer Manager resource. This request will instruct the corosync and pacemaker services to stop HA resource group(s) specified on the current host they are started on, and start them on the partner host. This effectively accomplishes a failover or failback of resources depending on if the destination host is the primary or secondary location for that HA resource group.

Request body:

Similar to both the Manager.StartFilesystem and Manager.StopFilesystem requests, the Manager.MoveFilesystem POST request requires that you pass a set of Redundancy resource endpoint(s) to operate on. At least one Redundancy resource endpoint is required, but multiple can be passed to a single request. Additionally, the Manager.MoveFilesystem POST request requires that you specify the current hostname (where the Redundancy resources being operated on currently are started at) and the destination hostname (where the Redundancy resources being operated on will be moved to) as the following fields:

- Location – the current location of the Redundancy resource(s)
- Destination – the destination location of the Redundancy resource(s)

Example of a Manager.MoveFilesystem request body to move 3 Redundancy resource from Location to Destination:

```
{
  "Redundancy": [
   {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
   },
   {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
   },
   {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
   }
  ],
  "Location": "kjlmo1204",
  "Destination": "kjlmo1205"
}
```

Similar to Manager.StartFilesystem and Manager.StopFilesystem requests, Manager.MoveFilesystem can only operate on Redundancy resources that a Manager currently owns. A similar error will be seen for Manager.MoveFilesystem if a Redundancy set is passed that is not in that current Manager.Redundancy list of that Manager.

There are also some restrictions regarding the Location and Destination specifications in the Manager.MoveFilesystem request body. The Location parameter must be set to the hostname of the node which the underlying HA resource groups are currently started on, and

ther Destination parameter must be set to the HA peer hostname. This information can be determined all within the Manager resource:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq .'Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
  }
]
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq .'Oem'
{
  "IsPrimary": true,
  "Hostname": "kjlmo1204",
  "HostIP": "172.16.2.7/16",
  "IpmiAddr": "172.16.0.105",
  "IpmiSecAddr": "169.254.1.1",
  "ChassisSerialNumber": "MXE30000122VS00A",
  "PartnerInfo": {
    "Hostname": "kjlmo1205",
    "HostIP": "172.16.2.8/16",
    "Id": "kjlmo12:Node-R1C1-36U-B",
    "IpmiAddr": "172.16.0.106",
    "IpmiSecAddr": "169.254.1.2",
    "SerialNumber": "MXE3000011TVT074",
    "ChassisSerialNumber": "MXE30000122VS00A",
    "DaemonInfo": {}
  },
  "DaemonInfo": {},
  "NodeRole": [
    "oss"
  ]
}
```

The output above indicates that the kjlmo12:Node-R1C1-36U-A Manager represents kjlmo1204 hostname, and that its HA partner hostname is kjlmo1205. Additonally, k jlmo12:Node-R1C1-36U-A Manager is the current owner of kjlmo1204_md0-group, kjlmo1204_md2-group, and kjlmo1204_md4-group Redundancy resources which are to be operated on by the Manager.MoveFilesystem request.

Additionally, each Redundancy resource can be checked to see its current Status and Location. Using kjlmo1204_md0-group as an example:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-
group | jq '.Oem'
{
  "TargetType": "OST",
  "TargetName": "kjlmo12-OST0000",
  "FsName": "kjlmo12",
  "ArrayName": "md0",
  "ResourceGroupName": "kjlmo1204_md0-group",
  "ResourceGroupStatus": "started",
  "PrimaryLocation": "kjlmo1204",
  "SecondaryLocation": "kjlmo1205",
  "Location": "kjlmo1204",
  "FailedOver": false,
  "ResourceGroupAttrs": [
    {
      "Name": "kjlmo1204_md0-wibr",
      "Status": "started",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-jnlr",
      "Status": "started",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-wibs",
      "Status": "started",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-raid",
      "Status": "started",
      "Location": "kjlmo1204",
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-fsys",
      "Status": "started",
      "Location": "kjlmo1204",
```

```
      "FailedOver": false
    },
    {
      "Name": "kjlmo1204_md0-stop",
      "Status": "started",
      "Location": "kjlmo1204",
      "FailedOver": false
    }
  ],
  "HaNetworking": {
    "LNetInfo": {}
  },
  "StorageServiceId": "MXE30000122VS00A"
}
```


The above output indicates that kjlmo1204_md0-group is started at Location kjlmo1204, which matches that resource group's PrimaryLocation. Additionally, it shows that its SecondaryLocation is kjlmo1205. Since the Redundancy resource is started in its PrimaryLocation we can move it to its SecondaryLocation using the parameters:

- Location: kjlmo1204
- Destination: kjlmo1205

If the Location or Destination parameters are not defined at all, in that, neither of them are defined or just one of them is not defined – the request will not be accepted:

```
# cat move_md0_2_4_group.json | jq
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ],
  "Location": "",
  "Destination": "kjlmo1204"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -d @move_md0_2_4_group.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem |
jq
{
  "error": {
```

```
    "code": "400",
    "message": "Unable to forward request to host kjlmo1204. Received invalid response code 400 from
request to endpoint: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem.
Not enough information passed in /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem request body: . Location, Destination and Redundancy fields must all be
populated. {\n\t\"error\": {\n\t\t\"code\": \"400\",\n\t\t\"message\": \"Not enough information passed in
/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem request body: .
Location, Destination and Redundancy fields must all be populated.\"\n\t}\n}"
  }
}
```

Even if the Location and Destination parameters are both populated, there are still some restrictions on the values specified that will cause the request body to not be accepted. These are:

- The Location and Destination are set to the same hostname
- The Location or Destination are unknown to the targeted Redundancy resources (i.e. not a defined Redundancy.Oem.PrimaryLocation or Redundancy.Oem.SecondaryLocation)
- The Destination hostname is the same as the current Redundancy.Oem.Location

If any of these conditions are not met, similar to a Manager.StartFilesystem or Manager.StopFilesystem Task action failure – initially, the request will be accepted and return a Task resource with TaskState new. Next, the request body will be checked against each specified Redundancy resource to validate Location and Destination fields, if the fields are deemed invalid, a Task error will be raised and an Event indicating the failure will be broadcasted. For example:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq .'Oem.Hostname'
"kjlmo1204"
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq
.'Oem.PartnerInfo.Hostname'
"kjlmo1205"
# cat move_md0_2_4_group.json | jq
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
```

```
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
    }
  ],
  "Location": "kjlmo1202",
  "Destination": "kjlmo1203"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -d @move_md0_2_4_group.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem |
jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines move Manager.Action: /redfish/v1/Managers/kjlmo12:Node-
R1C1-36U-A/Actions/Manager.MoveFilesystem on RedundancyId kjlmo1204_md4-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "move-kjlmo1204_md4-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem
Started"
    }
  ],
  "Name": "move Redundancy kjlmo1204_md4-group",
  "Payload": {},
  "StartTime": "2024-08-15T11:48:18.879420851-06:00",
  "TaskState": "New"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-
group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines move Manager.Action: /redfish/v1/Managers/kjlmo12:Node-
R1C1-36U-A/Actions/Manager.MoveFilesystem on RedundancyId kjlmo1204_md0-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "move-kjlmo1204_md0-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
```

"MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem Started"
},
{
"MessageArgs": [
"move-kjlmo1202"
],
"MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem Running"
},
{
"MessageArgs": [
"kjlmo1204_md0-group",
"rpc error: code = InvalidArgument desc = Location kjlmo1202 specified in request body is unknown to HA Manager kjlmo12:Node-R1C1-36U-A"
],
"MessageId": "Manager.MoveFilesystem Failure"
}
],
"Name": "move Redundancy kjlmo1204_md0-group",
"Payload": {},
"StartTime": "2024-08-15T11:48:18.879021003-06:00",
"TaskState": "Exception"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group | jq
{
"@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group",
"@odata.type": "#Task.v1_5_0.Task",
"Description": "This Task Resource defines move Manager.Action: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem on RedundancyId kjlmo1204_md2-group.",
"EndTime": "0001-01-01T00:00:00Z",
"Id": "move-kjlmo1204_md2-group",
"Messages": [
{
"MessageArgs": [
"kjlmo1204_md2-group"
],
"MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem Started"
},
{
"MessageArgs": [
"move-kjlmo1202"
],
"MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem Running"

```
    },
    {
      "MessageArgs": [
        "kjlmo1204_md2-group",
        "rpc error: code = InvalidArgument desc = Location kjlmo1202 specified in request body is unknown to
HA Manager kjlmo12:Node-R1C1-36U-A"
      ],
      "MessageId": "Manager.MoveFilesystem Failure"
    }
  ],
  "Name": "move Redundancy kjlmo1204_md2-group",
  "Payload": {},
  "StartTime": "2024-08-15T11:48:18.879202092-06:00",
  "TaskState": "Exception"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-
group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines move Manager.Action: /redfish/v1/Managers/kjlmo12:Node-
R1C1-36U-A/Actions/Manager.MoveFilesystem on RedundancyId kjlmo1204_md4-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "move-kjlmo1204_md4-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem
Started"
    },
    {
      "MessageArgs": [
        "move-kjlmo1202"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem
Running"
    },
    {
      "MessageArgs": [
        "kjlmo1204_md4-group",
        "rpc error: code = InvalidArgument desc = Location kjlmo1202 specified in request body is unknown to
HA Manager kjlmo12:Node-R1C1-36U-A"
      ],
      "MessageId": "Manager.MoveFilesystem Failure"
    }
```

```
  ],
  "Name": "move Redundancy kjlmo1204_md4-group",
  "Payload": {},
  "StartTime": "2024-08-15T11:48:18.879420851-06:00",
  "TaskState": "Exception"
}
# curl -k -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/25fc36b2-186c-4cc2-95f7-887fbfea1b6b | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   741  100   741    0     0   9623      0 --:--:-- --:--:-- --:--:--  9623
{
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
  "Context": "ha-ec",
  "Events": [
    {
      "Actions": {},
      "EventId": "25fc36b2-186c-4cc2-95f7-887fbfea1b6b",
      "EventTimestamp": "2024-08-15T11:48:18-06:00",
      "EventType": "",
      "MemberId": "0",
      "MessageArgs": [
        "kjlmo1204_md0-group",
        "rpc error: code = InvalidArgument desc = Location kjlmo1202 specified in request body is unknown to HA Manager kjlmo12:Node-R1C1-36U-A"
      ],
      "MessageId": "Manager.MoveFilesystem Failure"
    }
  ],
  "Events@odata.count": 1,
  "Id": "25fc36b2-186c-4cc2-95f7-887fbfea1b6b",
  "Name": "HA Reundancy Event"
}
# curl -k -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/7cf595fe-0d3f-4d79-89e8-9f172ea1edd8 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   741  100   741    0     0   9750      0 --:--:-- --:--:-- --:--:--  9750
{
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
  "Context": "ha-ec",
  "Events": [
    {
      "Actions": {},
      "EventId": "7cf595fe-0d3f-4d79-89e8-9f172ea1edd8",
      "EventTimestamp": "2024-08-15T11:48:18-06:00",
```

```
    "EventType": "",
    "MemberId": "0",
    "MessageArgs": [
      "kjlmo1204_md2-group",
      "rpc error: code = InvalidArgument desc = Location kjlmo1202 specified in request body is unknown to
HA Manager kjlmo12:Node-R1C1-36U-A"
    ],
    "MessageId": "Manager.MoveFilesystem Failure"
  }
 ],
 "Events@odata.count": 1,
 "Id": "7cf595fe-0d3f-4d79-89e8-9f172ea1edd8",
 "Name": "HA Reundancy Event"
}
# curl -k -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/Events/402d01fb-cdb1-
4e66-8e98-636aed676d15 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   741  100   741    0     0   9623      0 --:--:-- --:--:-- --:--:--  9750
{
 "@odata.context": "/redfish/v1/$metadata/#Event.Event",
 "@odata.type": "#Event.v1_5_0.Event",
 "Context": "ha-ec",
 "Events": [
  {
    "Actions": {},
    "EventId": "402d01fb-cdb1-4e66-8e98-636aed676d15",
    "EventTimestamp": "2024-08-15T11:48:18-06:00",
    "EventType": "",
    "MemberId": "0",
    "MessageArgs": [
      "kjlmo1204_md4-group",
      "rpc error: code = InvalidArgument desc = Location kjlmo1202 specified in request body is unknown to
HA Manager kjlmo12:Node-R1C1-36U-A"
    ],
    "MessageId": "Manager.MoveFilesystem Failure"
  }
 ],
 "Events@odata.count": 1,
 "Id": "402d01fb-cdb1-4e66-8e98-636aed676d15",
 "Name": "HA Reundancy Event"
}
```


Similar to Manager.StartFilesystem and Manager.StopFilesystem requests, Task failures can also occur if the underlying operation being carried out fails along the way for any other reason, and similarly, a GET request to the Redundancy resource will return a Task

with TaskState Exception and a message indicating what failed, and an Event will be broadcasted with a message indicating what failed.

After the Redundancy GET request for a failed operation returns the Task with TaskState Exception *one time,* the failed Task resource will be automatically removed and the next GET request will return that Redundancy resource again.

Example of a successful request to move kjlmo1204_md0-group, kjlmo1204_md2-group, kjlmo1204_md4-group Redundancy resources from kjlmo1204 to kjlmo1205:

Manager.Redundancy lists before:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq '.Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-group"
  }
]
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B | jq '.Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md1-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md3-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B/Redundancy/kjlmo1204_md5-group"
  }
]
```

Redundancy ResourceGroupStatus, Location, and FailedOver for kjlmo1204_md0-group, kjlmo1204_md2-group, kjlmo1204_md4-group Redundancy resources before:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md0-group | jq '.Oem | "\(.ResourceGroupStatus) \(.Location)"'
"started kjlmo1204"
```

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md0-group | jq '.Oem | "\(.ResourceGroupStatus) \(.Location) \(.FailedOver)"'
"started kjlmo1204 false"
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md2-group | jq '.Oem | "\(.ResourceGroupStatus) \(.Location) \(.FailedOver)"'
"started kjlmo1204 false"
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md4-group | jq '.Oem | "\(.ResourceGroupStatus) \(.Location) \(.FailedOver)"'
"started kjlmo1204 false"
```

Valid Request body:

```
# cat move_md0_2_4_group.json | jq
{
  "Redundancy": [
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-
group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-
group"
    },
    {
      "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-
group"
    }
  ],
  "Location": "kjlmo1204",
  "Destination": "kjlmo1205"
}
```

Similar to Manager.StartFileSystem and Manager.StopFilesystem, when a request is accepted, it begins concurrent redfish Task operations for each of the 3 specified Redundancy groups. NOTE: because more than one Redundancy resource was specified in the request body, the Task resource of the *last* Redundancy Task created is returned as a response. NOTE: All Task resources are created with a TaskState of “New”.

Starting the request:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -d @move_md0_2_4_group.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem | jq
```

```
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-
group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines move Manager.Action:
/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem on
RedundancyId kjlmo1204_md4-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "move-kjlmo1204_md4-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem Started"
    }
  ],
  "Name": "move Redundancy kjlmo1204_md4-group",
  "Payload": {},
  "StartTime": "2024-08-15T15:28:34.453973161-06:00",
  "TaskState": "New"
}
```


When GET requests are run to the Each Redundancy resource being operated on while the underlying operation is still being carried out, a Task resource is returned to indicate the operation is still in progress.

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md0-group | jq
```


```
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md0-
group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines move Manager.Action:
/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem on
RedundancyId kjlmo1204_md0-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "move-kjlmo1204_md0-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md0-group"
      ],
```

```
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem Started"
    },
    {
      "MessageArgs": [
        "move-kjlmo1204"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem Running"
    }
  ],
  "Name": "move Redundancy kjlmo1204_md0-group",
  "Payload": {},
  "StartTime": "2024-08-15T15:28:34.453140683-06:00",
  "TaskState": "Running"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md2-group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md2-
group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines move Manager.Action:
/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem on
RedundancyId kjlmo1204_md2-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "move-kjlmo1204_md2-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md2-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem Started"
    },
    {
      "MessageArgs": [
        "move-kjlmo1204"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem Running"
    }
  ],
  "Name": "move Redundancy kjlmo1204_md2-group",
  "Payload": {},
  "StartTime": "2024-08-15T15:28:34.453720037-06:00",
  "TaskState": "Running"
```

```
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md4-group | jq
{
  "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Redundancy/kjlmo1204_md4-
group",
  "@odata.type": "#Task.v1_5_0.Task",
  "Description": "This Task Resource defines move Manager.Action:
/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.MoveFilesystem on
RedundancyId kjlmo1204_md4-group.",
  "EndTime": "0001-01-01T00:00:00Z",
  "Id": "move-kjlmo1204_md4-group",
  "Messages": [
    {
      "MessageArgs": [
        "kjlmo1204_md4-group"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem Started"
    },
    {
      "MessageArgs": [
        "move-kjlmo1204"
      ],
      "MessageId": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Actions/Manager.MoveFilesystem Running"
    }
  ],
  "Name": "move Redundancy kjlmo1204_md4-group",
  "Payload": {},
  "StartTime": "2024-08-15T15:28:34.453973161-06:00",
  "TaskState": "Running"
}
```

NOTE: the TaskState for each Redundancy Resource has move from New to Running.

After successful Task completion, each Manager.Redundancy list should be updated accordingly, showing the cls123451204_md0-group, cls123451204_md2-group, cls123451204_md4-group endpoints moving from Manager cls1234512:Node-R1C1-36U-A to Manager cls1234512:Node-R1C1-36U-B:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A | jq
'.Redundancy'
```

```
[]
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-B | jq
'.Redundancy'
[
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
B/Redundancy/kjlmo1204_md0-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
B/Redundancy/kjlmo1204_md1-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
B/Redundancy/kjlmo1204_md2-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
B/Redundancy/kjlmo1204_md3-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
B/Redundancy/kjlmo1204_md4-group"
  },
  {
    "@odata.id": "/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
B/Redundancy/kjlmo1204_md5-group"
  }
]
```

And the ResourceGroupStatus, Location, and FailedOver fields should also be updated accordingly for each Redundancy resource:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
B/Redundancy/kjlmo1204_md0-group | jq '.Oem | "\(.ResourceGroupStatus) \(.Location)
\(.FailedOver)"'
"started kjlmo1205 true"
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
B/Redundancy/kjlmo1204_md2-group | jq '.Oem | "\(.ResourceGroupStatus) \(.Location)
\(.FailedOver)"'
"started kjlmo1205 true"
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
```

```
B/Redundancy/kjlmo1204_md4-group | jq '.Oem | "\(.ResourceGroupStatus) \(.Location)
\(.FailedOver)"'
"started kjlmo1205 true"
```

NOTE: you can no longer access the Redundancy resources on Manager: kjlmo12:Node-R1C1-36U-A now that they have been moved to Manager: kjlmo12:Node-R1C1-36U-B. If you try to send a GET request to a Redundancy endpoint under the Manager resource they were moved from, the following error response will be seen:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://10.214.128.240:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md0-group | jq
{
  "error": {
    "code": "404",
    "message": "Unable to forward request to host kjlmo1204. Received invalid response
code 404 from request to endpoint: /redfish/v1/Managers/kjlmo12:Node-R1C1-36U-
A/Redundancy/kjlmo1204_md0-group. Redundancy Resource kjlmo1204_md0-group is
not found in Manager kjlmo12:Node-R1C1-36U-A Redundancy list {\n\t\"error\":
{\n\t\t\"code\": \"404\",\n\t\t\"message\": \"Redundancy Resource kjlmo1204_md0-group
is not found in Manager kjlmo12:Node-R1C1-36U-A Redundancy list\"\n\t}\n}"
  }
}
```

10.4 Manager Action Requests Which Operate on Storage Node Power State

The ClusterStor RFSF API provides some oem extension Action endpoints through all Manager resources which either provide information on, or operate on a underlying cluster node's power state. Each endpoint provides a proxy to a cluster node's BMC through an IPMI interface in order to provide this information. By definition of an Action route, each of the routes below provide POST requests only, some of which require a request body depending on the underlying operation being caried out.

10.4.1 Manager.GetPowerState Action Request

/redfish/v1/Manager/{ManagerId}/Actions/Manager.GetPowerState

The Manager.GetPowerState request will return the current chassis power state of a cluster node. It requires no request body, as no parameters to the request will change its behavior.

Example Manager.GetPowerState request:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -X POST
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.GetPowerState | jq
{
  "Response": "Chassis Power is on"
}
```

Details of the response:

- Response: this is the chassis power status response from the BMC indicating the current power state of the host

10.4.2 Manager.PowerOn Action Request

/redfish/v1/Manager/{ManagerId}/Actions/Manager.PowerOn

The Manager.PowerOn request will send an ipmi chassis power on to the target cluster node BMC. It requires no request body, as no parameters to the request will change its behavior.

Example Manager.GetPowerState request:

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -X POST
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.PowerOn | jq
{
  "Response": "Chassis Power Control: Up/On"
}
```

Details of the response:

- Response: this is the chassis power on resposne from the BMC. Up/On indicates that the host was powered up.

10.4.3 Manager.PowerOff Action Request

/redfish/v1/Manager/{ManagerId}/Actions/Manager.PowerOff

The Manager.PowerOff request will send an ipmi chassis power off, or ipmi chassis power soft to the target cluster node BMC. It requires you pass a request body which specifies the type of PowerOff action to invoke: hard or soft.

The PowerOff action type is specified by passing the following in the request body:

```
# cat power_action_type.json | jq
{
  "Type": "<soft|hard>"
}
```

Where Type can either have a value of “soft” or “hard”.

Soft power off will run an ipmi chassis power soft – which invokes a “clean” or “soft” shutdown, allowing the OS to cleanly shutdown before power down.

Hard power off will run an ipmi chassis power off – which invokes a “hard” shutdown, similar to a physical power pull.

Example of a soft PowerOff request:

```
# cat power_action_type.json | jq
{
  "Type": "soft"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -X POST -d @power_action_type.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.PowerOff | jq
{
  "Response": "Chassis Power Control: Soft"
}
```

Example of a hard PowerOff request:

```
# cat power_action_type.json | jq
{
  "Type": "hard"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -X POST -d @power_action_type.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.PowerOff | jq
{
  "Response": "Chassis Power Control: Off"
}
```

10.4.4 Manager.Reset Action Request

/redfish/v1/Manager/{ManagerId}/Actions/Manager.Reset

The Manager.Reset request will send an ipmi chassis power cycle, or ipmi chassis power reset to the target cluster node BMC. It requires you pass a request body which specifies the type of Reset action to invoke: hard or soft.

The PowerOff action type is specified by passing the following in the request body:

```
# cat power_action_type.json | jq
{
  "Type": "<soft|hard>"
}
```

Where Type can either have a value of "soft" or "hard".

Soft reset will run an ipmi chassis power reset – which invokes a "clean" or "soft" reset, allowing the OS to cleanly shutdown before power down.

Hard power off will run an ipmi chassis power cycle – which invokes a "hard" reset, similar to a physical power pull and subsequent power restore.

Example of a soft Reset request:

```
# cat power_action_type.json | jq
{
  "Type": "soft"
}
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -X POST -d @power_action_type.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.Reset | jq
{
  "Response": "Chassis Power Control: Reset"
}
```

Example of a hard Reset request:

```
# cat power_action_type.json | jq
{
  "Type": "hard"
}
```

```
# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN" -X POST -d @power_action_type.json
https://localhost:8081/redfish/v1/Managers/kjlmo12:Node-R1C1-36U-A/Actions/Manager.Reset | jq
{
  "Response": "Chassis Power Control: Cycle"
}
```

11. Top-Level Lustre FileSystem

The top-level Lustre Filesystem will be represented by a FileSystem resource in Redfish/Swordfish schema in the same way that individual Lustre target (OST/MDT/MGT) resources are represented, but at a different StorageService level than the individual Lustre Target FileSystem resources which reside at the "enclosure" StorageService level. We still want to use the same /redfish/v1/StorageServices service root for the API to access the Lustre FileSystem data, however, in the URI /redfish/v1/StorageServices/{StorageServiceId}, the StorageServiceId will be represented a bit differently than the enclosure level StorageServices which are represented by an enclosure serial number.

StorageServices GET request showing enclosure level and Lustre FileSystem level services

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageServices/lustre-kjlmo12 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   479  100   479    0     0   233k      0 --:--:-- --:--:-- --:--:--  233k
{
  "@odata.context": "StorageService.StorageService",
  "@odata.id": "/redfish/v1/StorageServices/lustre-kjlmo12",
  "@odata.type": "StorageService.v1_5_0.StorageService",
  "Description": "Storage Service used to access the top level Lustre FileSystem kjlmo12",
  "FileSystems": {
    "@odata.id": "/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems"
  },
  "Id": "lustre-kjlmo12",
  "Identifier": {},
```

```
  "Links": {},
  "Name": "Lustre FileSystem kjlmo12 StorageServce",
  "Status": {}
}
```

Details of the response:

- notice the "Lustre Filesystem" entry with a StorageService Id in its OdataId that does not look like a serial number

Dynamic Fields:

- Lustre Filesystem OdataId StorageServiceId may change if the Lustre FS is renamed

11.1. Lustre FileSystem StorageService

/redfish/v1/StorageServices/{StorageServiceId}

- StorageServiceId will indicate this StorageService provides access to the Lustre FileSystem of name "filesystem name"
- Format: <filesystem type>-<filesystem name>
- Where:
  - filesystem type == lustre
  - filesystem name == name of the lustre filesystem provided by this cluster
- Example: /redfish/v1/StorageServices/lustre-kjlmo12
  - filesystem type == lustre
  - filesystem name = kjlmo12

Unlike enclosure level StorageServices which provide access to StoragePools, Volumes and FileSystems for that enclosure, the Lustre FileSystem StorageService will only provide access to FileSystems resources.

Schema: https://redfish.dmtf.org/schemas/swordfish/StorageService.v1_5_0.json

Lustre FileSystem StorageService GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   505  100   505    0     0   164k      0 --:--:-- --:--:-- --:--:--  164k
{
  "OdataContext": "FileSystemCollection.FileSystemCollection",
  "OdataId": "/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems",
  "OdataType": "FileSystemCollection.v1_0_0.FileSystemCollection",
  "Description": "Collection of Lustre FileSystem resources for StorageService lustre-kjlmo12",
  "Members": [
```

```
    {
      "@odata.id": "/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems/kjlmo12"
    }
  ],
  "MembersOdataCount": 1,
  "Name": "kjlmo12 Lustre FileSystemCollection"
}
```


Details of the response:

- Only FileSystems collection is available in the Lustre Filesystem StorageService
- Id: generated to represent the name of the lustre filesystem and equal to the StorageServiceId in the resource endpoint

Dynamic Fields:

- Id - may change if the Lustre FS is renamed

11.2. Lustre FileSystem FileSystemCollection

/redfish/v1/StorageServices/{StorageServiceId}/FileSystems

This will show all Lustre FileSystem resources provided by the Lustre FileSystem StorageService. There will only be one FileSystem in the collection, which is the FileSystem resource that represents the actual Lustre FileSystem

Schema: https://redfish.dmtf.org/schemas/swordfish/FileSystemCollection.json

Lustre FileSystem StorageService FileSystems GET request


```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   505  100   505    0     0   164k      0 --:--:-- --:--:-- --:--:--  164k
{
  "OdataContext": "FileSystemCollection.FileSystemCollection",
  "OdataId": "/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems",
  "OdataType": "FileSystemCollection.v1_0_0.FileSystemCollection",
  "Description": "Collection of Lustre FileSystem resources for StorageService lustre-kjlmo12",
  "Members": [
    {
      "@odata.id": "/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems/kjlmo12"
```

```
    }
  ],
  "MembersOdataCount": 1,
  "Name": "kjlmo12 Lustre FileSystemCollection"
}
```


Details of the response:

- **Members:** this shows the endpoint to the FileSystem resource within the StorageService that represent the top level Lustre Filesystem
- Members@odata.count: this shows the total number of Filesystem resources in the Lustre FileSystem StorageService
    - there will only ever be 1

Dynamic Fields:

- Members - FileSystemId may change if the Lustre FS is renamed

11.3. Lustre FileSystem resource

/redfish/v1/StorageServices/{StorageServiceId}/FileSystems/{FileSystemId}

This resource provides access to all top-level Lustre FileSystem data. FileSystemId is represented by the actual Lustre fsname taken from the cluster. This resource provides links to all underlying Lustre Target resources This resource will contain any type of Lustre statistical data we care about monitoring

FileSystemId:

- the FileSystemId is the actual name of the Lustre filesystem

Schema: https://redfish.dmtf.org/schemas/swordfish/FileSystem.v1_2_0.json

Lustre FileSystem GET request

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems/kjlmo12 | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  2086    0  2086    0     0   6622      0 --:--:-- --:--:-- --:--:--  6601
{
  "OdataContext": "FileSystem.FileSystem",
  "OdataId": "/redfish/v1/StorageServices/lustre-kjlmo12/FileSystems/kjlmo12",
  "OdataType": "FileSystem.v1_2_2.FileSystem",
  "Description": "FileSystem resource to represent top-level Lustre FileSystem kjlmo12",
  "IOStatistics": {},
  "Id": "kjlmo12",
```

```
  "Links": {
    "Oem": {
      "kjlmo12-OST0003": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0003",
      "kjlmo12-OST0004": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0004",
      "kjlmo12-OST0005": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0005",
      "kjlmo12-OST0000": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0000",
      "kjlmo12-OST0001": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0001",
      "kjlmo12-OST0002": "/redfish/v1/StorageServices/MXE30000122VS00A/FileSystems/kjlmo12-OST0002",
      "kjlmo12-MDT0000": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-MDT0000",
      "kjlmo12-MDT0001": "/redfish/v1/StorageServices/MXE30000122VS00E/FileSystems/kjlmo12-MDT0001"
    }
  },
  "Name": "FileSystem kjlmo12",
  "Oem": {
    "FsType": "Lustre",
    "Lustre": {
      "MgsNIDS": "90@kfi,56@kfi:57@kfi,120@kfi",
      "FsName": "kjlmo12",
      "Statistics": {
        "OST Total": "6",
        "Total Available FS Space Percentage": "98.95134735118445",
        "Total FS Space Available": "687265421684736",
        "Total FS Space": "694548826349568",
        "Total FS Read": "0",
        "Total FS Space Used": "279183257600",
        "Total FS Write": "0",
        "Avg (120 sec) aggregated all OST(s) read_bytes": "0",
        "Avg (120 sec) aggregated all OST(s) write_bytes": "0",
        "Max (24 hour) aggregated all OST(s) read_bytes": "2207842302",
        "Max (24 hour) aggregated all OST(s) write_bytes": "2333278206",
        "Total FS MD ops": "0"
      }
    }
  },
}
```


Details of the response:

- Id: equal to the lustre filesystem name
- Links: provides links to all underlying Lustre target FileSystem resources which make up the whole Lustre FileSystem
- FileSystem.Oem.Lustre will contain several Lustre specific details about the target including:
  - MGSNids: lustre nids of the MGS servers who can host MDT0000
  - FsName: top-level lustre filesystem name
  - Statistics: various Lustre statistics for the Lustre filesystem as a whole from a client perspective

Dynamic Fields:

- Id: may change if Lustre Filesystem is renamed
- Links.Oem - may change if targets are added or removed from the FileSystem
- Oem.Lustre
    - FsName: may change if Lustre Filesystem is renamed
    - Statistics: will change as I/O to the top-level Lustre FileSystem changes

12. RFSF API EventService and Events

dp-api-server runs an EventService which receives Events from other gRPC services via gRPC POST requests. These Events are sent in the form of an Event redfish resource which is then broadcasted to anyone subscribed to receive Events via a POST request to a subscribers Destination endpoint. Redfish Events are runtime Events, so we will not persistently store all Events dp-api receives, but rather store all Events received in memory for the runtime of dp-api-server. It is up to the subscriber to implement a basic service to receive these POST requests on the endpoint provided in their EventDestination (this will be detailed later). The subscriber can then choose to do whatever they would like with the Events the receive, whether that be store them persistently, send email notifications, etc.

The following list provides a comprehensive summary of all conditions or scenarios that can trigger an Event in ClusterStor runtime operation:

- Changes in FRU data, status change, or dropout of a PowerSupply resource
- Changes in FRU data, status change, or dropout of a Assembly resource
- Changes in FRU data, status change, or dropout of a Fan resource
- Changes in hardware or firmware data of a Chassis resource
- Changes in hardware or firmware data of a ComputerSystem resource
- Changes in FRU data, status change, or dropout of a physical NetworkInterface resource
- Changes in bonding or teaming config of a virtual NetworkInterface resource
- Changes in active vs. passive or members of a virtual bonded or teamd NetworkInterface resource
- Triggers thresholds are exceeded for CPU Usage for a ComputerSystem resource
- Triggers thresholds are exceeded for Memory Usage for a ComputerSystem resource
- Triggers thresholds are exceeded for Per CPU Load for a ComputerSystem resource
- Triggers thresholds are exceeded for Swap Usage for a ComputerSystem resource
- Triggers thresholds are exceeded for Lack of Available Memory for a ComputerSystem resource
- Manager resources are initialized
- Manager.Redundancy lists have changed between HA peers (indicating HA resources have failedover or failed back)
- An HA CRM group status change is noticed in Redundancy resource
- An HA CRM group location change is noticed in Redundancy resource

- A Manager.StartFilesystem Action request to start a set of Redudnacy resources succeeds
- A Manager.StartFilesystem Action request to start a set of Redudnacy resources fails
- A Manager.StopFilesystem Action request to stop a set of Redudnacy resources succeeds
- A Manager.StopFilesystem Action request to stop a set of Redudnacy resources fals
- A Manager.MoveFilesystem Action request to move a set of Redudnacy resources from local to partner Manager resource succeeds
- A Manager.MoveFilesystem Action request to move a set of Redudnacy resources from local to partner Manager resource fails
- IcingaServices checks associated with ComputerSystem resource for node status raise an alert
- IcingaServices checks associated with ComputerSystem resource for DIMM status raise an alert
- IcingaServices checks associated with ComputerSystem resource for Current Load raise an alert
- IcingaServices checks associated with ComputerSystem resource for Current Users raise an alert
- IcingaServices checks associated with ComputerSystem resource for RAM usage raise an alert
- IcingaServices checks associated with ComputerSystem resource for filesystem space raise an alert
- IcingaServices checks associated with ComputerSystem resource for Swap Space raise an alert
- IcingaServices checks associated with ComputerSystem resource for CPU usage raise an alert
- IcingaServices checks associated with ComputerSystem resource for HW profile fingerprint failure raise an alert
- IcingaServices checks associated with StoragePools resource for Array status raise an alert
- IcingaServices checks associated with StoragePools resource for Disk status raise an alert
- IcingaServices checks associated with StoragePools resource for NVMe speed and connection status raise an alert
- IcingaServices checks associated with Assembly resource for PSU Assembly status raise an alert
- IcingaServices checks associated with PowerSupply resource for PSU status raise an alert
- IcingaServices checks associated with Fan resource for Fan status raise an alert
- IcingaServices checks associated with Fan resource for Fan statistics raise an alert
- IcingaServices checks associated with Chassis resource for Enclosure Electronics module status raise an alert
- IcingaServices checks associated with Chassis resource for Power statistics raise an alert
- IcingaServices checks associated with Chassis resource for Thermal statistics raise an alert
- IcingaServices checks associated with Chassis resource for Voltage statistics raise an alert
- IcingaServices checks associated with Chassis resource for Current statistics raise an alert
- IcingaServices checks associated with Chassis resource for SAS heatlh raise an alert
- A new node has been discovered, adding a ComputerSystem resource to the collection

- A node has been removed, removing a ComputerSystem resource from the collection
- A node has been replaced, updating a ComputerSystem resource
- More Events TBD in future releases

Schema: https://redfish.dmtf.org/schemas/v1/Event.v1_7_0.json

Example of a populated Event resource indicating a status change in a Fan in enclosure SGFGD190843A3A9:

Event resource example


```
{
  "@odata.type": "",
  "Events": [
   {
    "Actions": {},
    "EventId": "054b421a-1266-11ed-9413-00093d0693ca",
    "EventTimestamp": "1659446490",
    "EventType": "Alert",
    "MemberId": "",
    "Message": "Status changed for cooling",
    "MessageId": "",
    "Oem": {
     "Hpe": {
      "attribute": "overall_health",
      "enclosure_serial_number": "SGFGD190843A3A9",
      "index": 3,
      "timestamp": 1659446490,
      "type": "cooling"
     },
     "ResourceId": "/redfish/v1/Chassis/SGFGD190843A3A9/Thermal/Fans/Cooling-0"
    },
    "Severity": "Ok"
   }
  ],
  "Id": "054b421a-1266-11ed-9413-00093d0693ca",
  "Name": "Events"
}
```


Details of the response:

- EventId: unique generated Id to identify a particular Event that occurred
- EventTimestamp: the time an Event occurred
- Message: message about the Event
- Oem.Hpe
    - attribute: the type of status or sensor reading that changed

        - enclosure_serial_number: the enclosure serial number which the resource the Event occurred on resides in
        - index: the FRU index of the resource
        - type: the FRU type of the resource
    - ResourceId: the Resource OdataId of the resource the Event occurred on
    - Id: Equal to the EventId both in the resource and endpoint

12.1. EventService Resource

The EventService resource lives directly under the ServiceRoot /redfish/v1 and can be seen by issuing a GET request to /redfish/v1/EventService. It will show all available EventType's for subscribers, as well as all ResourceType's subscribers can receive Events on. It will also show the endpoint where a list of active Subscriptions can be seen - which will persist through service restart or failover.

While most of the EventService resource content is immutable, the EventService resource also provides some controlling parameters: DeliveryRetryAttempts, DeliveryRetryIntervalSeconds, RequestMaxEventsToQueue, EventDeliveryRetryQueueSize that can be updated or configured to preferred values via a PUT or PATCH request to the /redfish/v1/EventService endpoint. Details of what each of these parameters control can be seen below in the "Details of the response" of the EventService GET request.

Schema: https://redfish.dmtf.org/schemas/v1/EventService.v1_8_0.json

EventService GET request

```
# curl -k -H "X-Auth-Token: $RFSF_API_TOKEN" https://localhost:8081/redfish/v1/EventService | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   884  100   884    0     0  11631      0 --:--:-- --:--:-- --:--:-- 11631
{
  "@odata.context": "EventService.EventService",
  "@odata.id": "/redfish/v1/EventService",
  "@odata.type": "EventService.v1_7_1.EventService",
  "DeliveryRetryAttempts": 100,
  "DeliveryRetryIntervalSeconds": 10,
  "Description": "EventService for RFSF API",
  "EventTypesForSubscription": [
    "Alert",
    "StatusChange",
    "MetricReport"
  ],
  "Id": "EventService",
  "Name": "EventService",
  "Oem": {
```

```
    "RequestedMaxEventsToQueue": 1000,
    "EventDeliveryRetryQueueSize": 10
  },
  "ResourceTypes": [
    "PowerSupply",
    "AssemblyData",
    "Cooling",
    "Drive",
    "ComputerSystem",
    "Volume",
    "FileSystem"
  ],
  "SMTP": {},
  "SSEFilterPropertiesSupported": {},
  "Status": {},
  "Subscriptions": {
    "@odata.id": "/redfish/v1/EventService/Subscriptions"
  }
}
```


Details of the response:

- DeliveryRetryAttempts - The number of times that the POST of an event is retried before the subscription terminates. This retry occurs at the service level, which means that the HTTP POST to the event destination fails with an HTTP `4XX` or `5XX` status code or an HTTP timeout occurs this many times before the event destination subscription terminates.
- DeliveryRetryIntervalSeconds - The interval, in seconds, between retry attempts for sending any event.
- EventTypesForSubscription - shows all possible Event types that subscribers could receive
- Oem.RequestedMaxEventsToQueue: the number of Events the API EventService will store in memory before rotating them
- Oem.EventDeliveryRetryQueueSize – The number of incoming Events or MetricReports that each EventDestination thread is allowed to queue up while it is retrying to send to an EventDestination and the DeliveryRetryAttempts has not been exceeded. If an attempt to resend an Event/MetricReport succeeds within the DeliveryRetryAttempts threshold, the entirety of the queue will be sent to the client EventDestination to replay the history of Events/MetricReports queued up while thread was attempting to retry reaching the client EventDestination endpoint.
- ResourceTypes - shows all possible API resource types that subscribers could receive Event's on
- Subscriptions - shows list of all active Subscriptions

12.1.1 Updating the EventService resource

As mentioned before, while the EventService resource largely provides informational fields that are immutable, there are four parameters seen within the EventService resource body that provide a means of configuring the behavior of the running API EventService. Those four parameters are:

- DeliveryRetryIntervalSeconds - The interval, in seconds, between retry attempts for sending any event.
- EventTypesForSubscription - shows all possible Event types that subscribers could receive
- Oem.RequestedMaxEventsToQueue: the number of Events the API EventService will store in memory before rotating them
- Oem.EventDeliveryRetryQueueSize: the number of incoming Events/MetricReports a thread for a given EventDestination is allowed to queue up while retrying to send a failed Event/MetricReport to a given client EventDestination

The API creates the EventService resource with default values for these parameters on startup, but one may wish to change the values of these based on the needs and size of the system. The following two mechanisms provide a way to update the EventService to configure these parameters.

NOTE: ALL other fields of the EventService resource are immutable, if a PUT or PATCH request attempts to update any values outside of the three configurable parameters mentioned, a 400 BadRequest response will be received with an error.

For more information on what these configurable parameters control, see sections:

- 11.2.3 – Retiring of EventDestination subscriptions which no longer receive Events
- 11.3.3 – Rotating of runtime Event history

12.1.1.1 EventService PUT request

One mechanism that can be used to update the existing EventService resource is to send a PUT request to the /redfish/v1/EventService. By definition, the HTTP PUT request method creates a new resource, or replaces a representation of the target resource entirely with the request payload. For the API /redfish/v1/EventService PUT request implementation, it is expected that the request payload will replace an existing target resource. Because of this, it is required that the request body contains a fully populated EventService model with updates to the existing resource.

For example, if there exists an EventService resource with configurable params with the following values:

- "DeliveryRetryAttempts": 100
- "DeliveryRetryIntervalSeconds": 5
- "RequestedMaxEventsToQueue": 1000
- "EventDeliveryRetryQueueSize": 10

That needs to be updated to change the configurable params to updated values of:

- "DeliveryRetryAttempts": 10
- "DeliveryRetryIntervalSeconds": 10
- "RequestedMaxEventsToQueue": 5000
- "EventDeliveryRetryQueueSize": 100

The PUT request payload must contain the entire existing EventService model with changes to the params within:

Example PUT request body with above changes to configurable params:


```
{
  "@odata.context": "EventService.EventService",
  "@odata.id": "/redfish/v1/EventService",
  "@odata.type": "EventService.v1_7_1.EventService",
  "DeliveryRetryAttempts": 10,
  "DeliveryRetryIntervalSeconds": 10,
  "Description": "EventService for RFSF API",
  "EventTypesForSubscription": [
    "Alert",
    "StatusChange",
    "MetricReport"
  ],
  "Id": "EventService",
  "Name": "EventService",
  "Oem": {
    "RequestedMaxEventsToQueue": 5000,
    "EventDeliveryRetryQueueSize": 100,
  },
  "ResourceTypes": [
    "PowerSupply",
    "AssemblyData",
    "Cooling",
    "Drive",
    "ComputerSystem",
    "Volume",
    "FileSystem"
  ],
  "SMTP": {},
  "SSEFilterPropertiesSupported": {},
  "Status": {},
  "Subscriptions": {
    "@odata.id": "/redfish/v1/EventService/Subscriptions"
  }
}
```

Example PUT request to /redfish/v1/EventService with the request body shown above:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}" -d
@event_service_udpates/event_service_put.json -X PUT
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/EventService | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1565  100   840  100   725  14237  12288 --:--:-- --:--:-- --:--:-- 26083
{
  "@odata.context": "EventService.EventService",
  "@odata.id": "/redfish/v1/EventService",
  "@odata.type": "EventService.v1_7_1.EventService",
  "DeliveryRetryAttempts": 10,
  "DeliveryRetryIntervalSeconds": 10,
  "Description": "EventService for RFSF API",
  "EventTypesForSubscription": [
    "Alert",
    "StatusChange",
    "MetricReport"
  ],
  "Id": "EventService",
  "Name": "EventService",
  "Oem": {
    "RequestedMaxEventsToQueue": 5000,
    "EventDeliveryRetryQueueSize": 100
  },
  "ResourceTypes": [
    "PowerSupply",
    "AssemblyData",
    "Cooling",
    "Drive",
    "ComputerSystem",
    "Volume",
    "FileSystem"
  ],
  "SMTP": {},
  "SSEFilterPropertiesSupported": {},
  "Status": {},
  "Subscriptions": {
    "@odata.id": "/redfish/v1/EventService/Subscriptions"
  }
}
```

Now the next GET request to /redfish/v1/EventService will show updates to those params:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/EventService | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
```

```
                                 Dload  Upload   Total   Spent    Left  Speed
100  1565  100   840  100   725  14237  12288 --:--:-- --:--:-- --:--:-- 26083
{
  "@odata.context": "EventService.EventService",
  "@odata.id": "/redfish/v1/EventService",
  "@odata.type": "EventService.v1_7_1.EventService",
  "DeliveryRetryAttempts": 10,
  "DeliveryRetryIntervalSeconds": 10,
  "Description": "EventService for RFSF API",
  "EventTypesForSubscription": [
    "Alert",
    "StatusChange",
    "MetricReport"
  ],
  "Id": "EventService",
  "Name": "EventService",
  "Oem": {
    "RequestedMaxEventsToQueue": 5000,
    "EventDeliveryRetryQueueSize": 100
  },
  "ResourceTypes": [
    "PowerSupply",
    "AssemblyData",
    "Cooling",
    "Drive",
    "ComputerSystem",
    "Volume",
    "FileSystem"
  ],
  "SMTP": {},
  "SSEFilterPropertiesSupported": {},
  "Status": {},
  "Subscriptions": {
    "@odata.id": "/redfish/v1/EventService/Subscriptions"
  }
}
```

12.1.1.2 EventService PATCH request

Another mechanism that can be used to update the existing EventService resource is to send a PATCH request to the /redfish/v1/EventService. By definition, the HTTP PATCH request method creates *partial* changes to an existing resource. For the API /redfish/v1/EventService PATCH request implementation, , it is expected that the request payload will replace some number of fields in an existing resource, but not the entire resource like PUT does.

For example, if there exists an EventService resource with the following DeliveryRetryAttempts value:

- "DeliveryRetryAttempts": 100

That needs to be updated to change the configurable params to updated a value of:

- "DeliveryRetryAttempts": 10

The PATCH request payload must contain the field key/value pair to be updatedd:

Example PATCH request body to change only the DeliveryRetryAttempts field:

```
{
  "DeliveryRetryAttempts": 10
}
```

Example PATCH request to /redfish/v1/EventService with the request body shown above:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}" -d
@event_service_udpates/event_service_patch.json -X PATCH
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/EventService | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1565  100   840  100   725  14237  12288 --:--:-- --:--:-- --:--:-- 26083
{
  "@odata.context": "EventService.EventService",
  "@odata.id": "/redfish/v1/EventService",
  "@odata.type": "EventService.v1_7_1.EventService",
  "DeliveryRetryAttempts": 10,
  "DeliveryRetryIntervalSeconds": 5,
  "Description": "EventService for RFSF API",
  "EventTypesForSubscription": [
    "Alert",
    "StatusChange",
    "MetricReport"
  ],
  "Id": "EventService",
  "Name": "EventService",
  "Oem": {
    "RequestedMaxEventsToQueue": 1000,
    "EventDeliveryRetryQueueSize": 10
  },
  "ResourceTypes": [
    "PowerSupply",
    "AssemblyData",
    "Cooling",
    "Drive",
    "ComputerSystem",
```

```
    "Volume",
    "FileSystem"
  ],
  "SMTP": {},
  "SSEFilterPropertiesSupported": {},
  "Status": {},
  "Subscriptions": {
    "@odata.id": "/redfish/v1/EventService/Subscriptions"
  }
}
```


Now the next GET request to /redfish/v1/EventService will show updates to those params:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/EventService | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1565  100   840  100   725  14237  12288 --:--:-- --:--:-- --:--:-- 26083
{
  "@odata.context": "EventService.EventService",
  "@odata.id": "/redfish/v1/EventService",
  "@odata.type": "EventService.v1_7_1.EventService",
  "DeliveryRetryAttempts": 10,
  "DeliveryRetryIntervalSeconds": 5,
  "Description": "EventService for RFSF API",
  "EventTypesForSubscription": [
    "Alert",
    "StatusChange",
    "MetricReport"
  ],
  "Id": "EventService",
  "Name": "EventService",
  "Oem": {
    "RequestedMaxEventsToQueue": 1000,
    "EventDeliveryRetryQueueSize": 10
  },
  "ResourceTypes": [
    "PowerSupply",
    "AssemblyData",
    "Cooling",
    "Drive",
    "ComputerSystem",
    "Volume",
    "FileSystem"
  ],
  "SMTP": {},
  "SSEFilterPropertiesSupported": {},
  "Status": {},
```

```
  "Subscriptions": {
    "@odata.id": "/redfish/v1/EventService/Subscriptions"
  }
}
```


12.2. Event Subscriptions

To begin receiving Events from dp-api, you must subscribe to receive Events with a POST request to dp-api to the /redfish/v1/EventService/Subscriptions/{EventDestinationId} endpoint. The EventDestinationId should be something unique, generated with a tool such as uuidgen. The POST request body will contain information used by EventService to send Events to the Destination specified.

Example POST request body to subscribe to receive events to the following endpoint: http://172.16.0.1:8082/eventreceiver/

Example EventDestination POST request body


```
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/869291a5-5c0e-44c8-92b8-2efa6799c4cb",
  "Id": "869291a5-5c0e-44c8-92b8-2efa6799c4cb",
  "Destination": "http://172.16.0.1:8082/eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Context": "Public",
}
```


More details on the request body can be seen here DP-API /EventService but some details to highlight:

- the Destination field species the destination address (or hostname), port, and endpoint to receive Events from dp-api on
- HttpHeaders to pass with request should be set to JSON specific headers
- @odata.id field must match full endpoint request is sent to
- Id field must match the last field of the @odata.id field

Example POST request to register for events to the following endpoint: http://172.16.0.1:8082/eventreceiver/.

This will show:

- before the POST request when there are 0 subscriptions with a GET request to /redfish/v1/EventService/Subscriptions
- the POST request itself
- after the POST request showing the new subscription with a GET request to /redfish/v1/EventService/Subscriptions
- a GET request to /redfish/v1/EventService/Subscriptions/{EventDestinationId} to show subscription details

Event subscription POST example

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/EventService/Subscriptions | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   397  100   397    0     0   387k      0 --:--:-- --:--:-- --:--:--  387k
{
  "@odata.context": "EventDestinationCollection.EventDestinationCollection",
  "@odata.id": "/redfish/v1/EventService/Subscriptions",
  "@odata.type": "EventDestinationCollection.v1_0_0.EventDestinationCollection",
  "Description": "Collection of EventDestinations, locations for event notifications.",
  "Members": [],
  "Members@odata.count": 0,
  "Name": "EventDestinationCollection"
}

[root@kjlmo12-oem rest_api]# curl -X POST -H "Content-Type: application/json-rpc" -d
@events/test_subscription.json -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/EventService/Subscriptions/869291a5-5c0e-44c8-92b8-2efa6799c4cb | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1151  100   650  100   501   634k   489k --:--:-- --:--:-- --:--:-- 1124k
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/869291a5-5c0e-44c8-92b8-2efa6799c4cb",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8082/eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "869291a5-5c0e-44c8-92b8-2efa6799c4cb",
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

```
[root@kjlmo1200 event_registration]# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/EventService/Subscriptions | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   517  100   517    0     0   504k      0 --:--:-- --:--:-- --:--:--  504k
{
  "@odata.context": "EventDestinationCollection.EventDestinationCollection",
  "@odata.id": "/redfish/v1/EventService/Subscriptions",
  "@odata.type": "EventDestinationCollection.v1_0_0.EventDestinationCollection",
  "Description": "Collection of EventDestinations, locations for event notifications.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/EventService/Subscriptions/869291a5-5c0e-44c8-92b8-2efa6799c4cb"
    }
  ],
  "Members@odata.count": 1,
  "Name": "EventDestinationCollection"
}

[root@kjlmo1200 event_registration]# curl -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/EventService/Subscriptions/869291a5-5c0e-44c8-92b8-2efa6799c4cb | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   650  100   650    0     0   634k      0 --:--:-- --:--:-- --:--:--  634k
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/869291a5-5c0e-44c8-92b8-2efa6799c4cb",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8082/eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "869291a5-5c0e-44c8-92b8-2efa6799c4cb",
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

Now every time dp-api receives an Event, it will be forwarded to

```
"Destination": "http://172.16.0.1:8082/eventreceiver/"
```

via a POST which must implement a request handler to receive the Events, more details on this below.

12.2.1. Event Resource Filtering

Sometimes when subscribing to receive Events from RFSF API EventService, you may want to create a subscription that is more focused that a general subscription to receive Events of all types on all resources the API is tracking. This could be for organizational purposes, to send Events on different ResourceTypes or different sections of the ResourceTree to different Destination endpoints, or to track a concerning few resources to a particular Destination endpoint to watch more closely. RFSF API provides two mechanisms to filter Events based on the Resources you are interested in, based on the following two EventDestination resource attributes that can be sent in the request body with a subscription request:

- ResoureTypes
- OriginResources

The following sub-sections will describe what each is and how they can be used. The Redfish spec, as well as the EventDestination schema mentioned above, provide more details on these two attributes and how they can be used. It is worth mentioning that these two attributes are not mutually exclusive, they can be passed alone or together in the request body.

12.2.1.1 Event Resource Filtering Using EventDestination.ResourceTypes

One mechanism provided by Redfish Eventing to filter the Events a client Destination endpoint may receive is to specify a list of ResourceTypes in the EventDestination request body. Example:

```
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-resource-types",
  "Id": "filter-on-resource-types",
  "Destination": "http://172.16.0.1:9084/eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Context": "Public",
  "ResourceTypes": [
    "Cooling",
    "PowerSupply"
  ]
}
```

The above EventDestination specifies that it wants to receive Events on resources of type ResourceType “Cooling” and “PowerSupply” only. If the Event the resource occurs on being

processed by the API EventService is not of one of the two of these ResourceTypes, that Event will not be sent to the this Destination. You may specify as many ResourceTypes as are supported here. To see a full list of available ResourceTypes you may filter on, see the /redfish/v1/EventService resource, which contains a list of supported ResourceTypes.

12.2.1.2 Event Resource Filtering Using EventDestination.OriginResources

Another mechanism provided by Redfish Eventing to filter the Events a client Destination endpoint may receive is to specify a list of OriginResources in the EventDestination request body. OriginResources are defined as a list of Resource URIs. This can be either a URI which contains a subsection of the ResourceTree, such as a top-level collection, the URI of one specific resource, or anything in between.

Example:

```
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-origin-resources",
  "Id": "filter-on-origin-resources",
  "Destination": "http://172.16.0.1:9082/eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Context": "Public",
  "OriginResources": [
    {
      "@odata.id": "/redfish/v1/StorageSerivces"
    },
    {
      "@odata.id": "/redfish/v1/StorageSystems/kjlmo12:Node-R1C1-38U-B"
    },
    {
      "@odata.id": "/redfish/v1/Chassis/SGFGD2044527F7C/Power/PowerSupplies/PSU-0"
    }
  ]
}
```

The above EventDestination specifies three different OriginResource filters. The first defines that it wants to receive Events on *all resources* that exist under the StroageServices collection, this can be any resource under this collection or sub collections within the structure. The second defines that it wants to receive Events on all resources underneath, and including the kjlmo12:Node-R1C1-38U-B ComputerSystem resource. The third defines that it wants to

receive Events on PSU-0 in Chassis SGFGD2044527F7C only. It may not make sense to combine this specific set of OriginResources filters, but this example is meant to show that you can specify any number of OriginResource filters here. If the Event the resource occurs on being processed by the API EventService is not on or within one of these OriginResources, that Event will not be sent to the this Destination.

12.2.2 Updating an Event Subscription

Throughout the lifetime of an EventDestination resource, it is possible to update the subscription parameters to receive Events by altering the EvenDestination resource initially created with a PUT or PATCH request to an existing /redfish/v1/EventService/Subscriptions/{EventDestinationId} resource. The PUT or PATCH request body should contain the information to update in the existing EventDestination resource, that *results in a change* to the resource. If no new information is contained in the request body, then a 400 "BadRequest" response will be received.

12.2.2.1 Updating an Event Subscription via a PUT request

One mechanism that can be used to update an existing Event subscription, is to send a PUT request to the associated EventDestination resource at the /redfish/v1/EventService/Subscriptions/{EventDestinationId} endpoint. By definition, the HTTP PUT request method creates a new resource, or replaces a representation of the target resource entirely with the request payload. For the /redfish/v1/EventService/Subscriptions/{EventDestinationId} PUT request implementation, it is expected that the request payload will replace an existing target resource. Because of this, it is required that the request body contains a fully populated EventDestination model with updates to the existing resource.

For example, say you have an existing Event subscription with EventDestination.Id "filter-on-resource-types":

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-resource-types",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:9084/eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
```

```
  ],
  "Id": "filter-on-resource-types",
  "Name": "",
  "ResourceTypes": [
    "Cooling",
    "PowerSupply"
  ],
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```


And would like to update the Event filtering mechanism to use the OriginResources filter, as well as the Destination that Events are received on to a new endpoint. To update this subscription with a PUT request, the request body should contain the entire existing EventDestination model in the payload, with the changes you'd like to apply contained within.

Example PUT request payload:


```
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-resource-types",
  "Id": "filter-on-resource-types",
  "Destination": "http://172.16.0.1:9099/storagesystems-eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Context": "Public",
  "OriginResources": [
    {
      "@odata.id": "/redfish/v1/StorageSystems"
    }
  ]
}
```


Details of the PUT request payload:

- The EventDestination.Id is the same as the target resource: "filter-on-resource-types"
- The EventDestination.@odata.id is the same as the target resource: "/redfish/v1/EventService/Subscriptions/filter-on-resource-types"
- The Destination endpoint has changed to http://172.16.0.1:9099/storagesystems-eventreceiver/
- The Event filtering mechanism has changed from ResourceTypes to OriginResources

The PUT request when sent, if all requirements are met, will then entirely update and replace the target resource.

Example successful PUT request using payload above:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" -d @events/test_subscription_filter_resource_types_put_update.json -
X PUT https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-resource-types",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:9099/storagesystems-eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "filter-on-resource-types",
  "Name": "",
  "OriginResources": [
    {
      "@odata.id": "/redfish/v1/StorageSystems"
    }
  ],
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

Now, a GET request to the same filter-on-resource-types EventDestination resource will reflect the updates taken from the PUT request body:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}"
https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-resource-types",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:9099/storagesystems-eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "filter-on-resource-types",
  "Name": "",
  "OriginResources": [
```

```
    {
      "@odata.id": "/redfish/v1/StorageSystems"
    }
  ],
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

As mentioned before, there are a couple requirements in order for a PUT request to /redfish/v1/EventService/Subscriptions/{EventDestinationId} to be successful. The main ones are:

- The EventDestination.Id in the request payload must match the EventDestinaton.Id of the target resource
- The request payload must update the target EventDestination resource

Examples of some "BadRequest" responses received if these conditions are not met:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" -d
@events/test_subscription_filter_resource_types_put_update.json -X PUT
https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "error": {
    "code": "400",
    "message": "Requested URI uuid filter-on-resource-types does not match EventDestination.Id filter-on-
resource-types2 in request body."
  }
}
# curl -Ssk -H "X-Auth-Token: ${AUTH}" -d
@events/test_subscription_filter_resource_types_put_update.json -X PUT
https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "error": {
    "code": "400",
    "message": "Request body does not differ from saved resource:
/mnt/mgmt/var/lib/xyratex/rfsf_resources//Events/Destinations/filter-on-resource-types.json"
  }
}
```

12.2.2.1 Updating an Event Subscription via a PATCH request

Another mechanism that can be used to update an existing Event subscription, is to send a PATCH request to the associated EventDestination resource at the /redfish/v1/EventService/Subscriptions/{EventDestinationId} endpoint. By definition, the HTTP PATCH request method creates *partial* changes to an existing resource. For the

/redfish/v1/EventService/Subscriptions/{EventDestinationId} PATCH request implementation, it is expected that the request payload will replace some number of fields in an existing resource, but not the entire resource like PUT does.

For example, say you have an existing Event subscription with EventDestination.Id “filter-on-resource-types”:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-resource-types",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:9099/storagesystems-eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "filter-on-resource-types",
  "Name": "",
  "OriginResources": [
    {
      "@odata.id": "/redfish/v1/StorageSystems"
    }
  ],
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

And you’d like to update just a single field, OriginResources, to add another URI endpoint to filter on. This can be done by sending a PATCH request to the existing resource endpoint with a request payload containing changes to just the OriginResources field:

Example request payload:

```
{
  "OriginResources": [
    {
      "@odata.id": "/redfish/v1/StorageSystems"
    },
    {
      "@odata.id": "/redfish/v1/Chassis"
    }
  ]
```

```
}
```

Details of the PATCH request payload

- This adds an additional OriginResources filter on /redfish/v1/Chassis to the existing OriginResources field
- This payload updates OriginResources *only*

Example successful PATCH request using payload above:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" -d @events/test_subscription_filter_resource_types_patch_update.json -
X PATCH https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-resource-types",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:9099/storagesystems-eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "filter-on-resource-types",
  "Name": "",
  "OriginResources": [
    {
      "@odata.id": "/redfish/v1/StorageSystems"
    },
    {
      "@odata.id": "/redfish/v1/Chassis"
    }
  ],
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

Now, a GET request to the same filter-on-resource-types EventDestination resource will reflect the updates taken from the PATCH request body:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}"
https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/filter-on-resource-types",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:9099/storagesystems-eventreceiver/",
```

```
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "filter-on-resource-types",
  "Name": "",
  "OriginResources": [
    {
      "@odata.id": "/redfish/v1/StorageSystems"
    },
    {
      "@odata.id": "/redfish/v1/Chassis"
    }
  ],
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```


PATCH requests are a bit more flexible than a PUT request in terms of allowed requests since it is only a partial update of a resource. The main restriction is as follows:

- The request payload must update the target EventDestination resource

If the resulting resource is not changed by the PATCH request payload, a “BadResposne” is received. Example of a “BadRequest” response received when payload does not update the resource:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" -d @events/test_subscription_filter_resource_types_patch_update.json -
X PATCH https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/filter-on-resource-types | jq
{
  "error": {
    "code": "400",
    "message": "Request body does not differ from saved resource:
/mnt/mgmt/var/lib/xyratex/rfsf_resources//Events/Destinations/filter-on-resource-types.json"
  }
}
```

12.2.3 – Retiring of EventDestination subscriptions which no longer receive Events

As mentioned in section 11.1 – the EventService resource contains the following configurable values:

- DeliveryRetryAttempts - The number of times that the POST of an event is retried before the subscription terminates. This retry occurs at the service level, which means that the HTTP POST to the event destination fails with an HTTP `4XX` or `5XX` status code or an HTTP timeout occurs this many times before the event destination subscription terminates.
- DeliveryRetryIntervalSeconds - The interval, in seconds, between retry attempts for sending any event.

These parameters are global values for all EventDestination subscriptions that define behavior in the API EventService related to sending Events to subscribers that do not go through. For example, if an subscription exists where the API cannot reach the target EventDestination.Destination endpoint, it will be allowed up to, but not more than *DeliveryRetryAttempts* number of attempts to resend the Event with a period of *DeliveryRetryIntervalSeconds* in between each attempt. If the number of attempts goes over the *DeliveryRetryAttempts* configured value, than the API will automically terminate this description with a DELETE operation for the associated /redfish/v1/EventService/Subscriptions/{EventDestinationId} endpoint.

Example logs where this occurs from dp-api service:

```
Apr 05 12:18:25 kjlmo1200 dp-api-server[1800598]: ERRO[2023-04-05T12:18:25-06:00] Attempt 9 of 10 failed to
send Event notificat
ion -- Err: Post "http://172.16.0.1:8082/metricsreceiver/": dial tcp 172.16.0.1:8082: connect: connection refused.
Retrying in 5
 seconds...
Apr 05 12:18:30 kjlmo1200 RFSF-API[1800598]: ERRO[2023-04-05T12:18:30-06:00] Attempt 10 of 10 failed to
send Event notification
-- Err: Post "http://172.16.0.1:8082/metricsreceiver/": dial tcp 172.16.0.1:8082: connect: connection refused.
Retrying in 5 sec
onds...
Apr 05 12:18:35 kjlmo1200 RFSF-API[1800598]: ERRO[2023-04-05T12:18:35-06:00] Failed to send Event
notification to Destination ht
tp://172.16.0.1:8082/metricsreceiver/ after 10 attempts. Removing this subscription from EventService!
Apr 05 12:18:35 kjlmo1200 RFSF-API[1800598]: INFO[2023-04-05T12:18:35-06:00] Sending dbus message to
stop MetricReports for Even
tDestination Id test-metrics-report.
```

Once this happens, clients will have to re-subscribe via a POST request in order to begin receiving Events on this destination again.

12.2.4 – Restricting the Number of Queued Events/MetricReports During Delivery Retry

As mentioned in section 11.1 – the EventService contains the following configurable parameter:

- Oem.EventDeliveryRetryQueueSize: the number of incoming Events/MetricReports a thread for a given EventDestination is allowed to queue up while retrying to send a failed Event/MetricReport to a given client EventDestination

This parameter was introduced as an Oem extension the redfish EventService to control the number of incoming Events/MetricReports that a given thread is allowed to queue up while it is in a DeliveryRetry loop. If a client EventDestination endpoint stops listening for Event / MetricReport POST requests from the API EventService, something in the network topology causes the Destination endpoint to become unreachable, or anything else that may prevent API EventService from reaching a given client EventDestination occurs, a given EventDestination thread will enter a "DeliveryRetry" loop. When an EventDestination thread enters this DeliveryRetry loop, it will attempt to resend a failed Event/MetricReport at an interval of time in seconds defined by the EventService.DeliveryRetryIntervalSeconds parameter until either:

- The client EventDestination becomes reachable again
- The EventService.DeliveryRetryAttempts threshold is succeed

While a given EventDestination thread is in a DeliveryRetry loop, more Events or MetricReports may come in. In an effort to maintain history, each EventDestination thread maintains a queue of incoming Events / MetricReports. If the client EventDestination never becomes reachable, and many Events or MetricReports start to get queued up, the memory consumption of each thread can start to grow large, particularly on a large system. In an effort to control this and reduce the resource consumption of dp-api-server, the EventService.Oem.EventDeliveryRetryQueueSize can be set accordingly to control the number of items allowed in each of these queues.

This parameter controls the queue size as follows: as incoming Events / MetricReports are queued up, if the number of items in a given queue exceeds the EventDeliveryRetryQueueSize defined, items will be popped off the queue until the queue size is below this threshold before allowing another item to be placed on the queue. This means that the number of Events or MetricReports in a given queue will sit right around the EventDeliveryRetryQueueSize defined until a client EventDestination becomes reachable again and the queue is unloaded, or the EventServiceDeliveryRetryAttempts threshold is exceeded, and the subscription is automatically terminated.

The default value of the EventService.Oem.EventDeliveryRetryQueueSize is 10, but this parameter can be configured via a PUT or PATCH request to /redfish/v1/EventService to fit the behavior desired for a given system. It is important to consider the size of a system when configuring this parameter, more nodes in a given cluster means that more Events and MetricReports can be generated, which can fill up these queues during an DeliveryRetry

loop quicker and quicker. It is better to maintain a stable resource consumption for dp-api-server during these Event DeliveryRetry loops, and determine the root of the problem why a given client EventDestination cannot be reached, rather than allow for larger queue sizes to try to retain history.

12.3. Runtime Event History

Events in the Redfish spec are intended to be runtime occurrences. Because we realize that people may want to track history of Events, or recall information of a specific Event, we have implemented runtime in memory storage of Events to track history. If the subscriber wants to persistently store Events they receive, they must do so on their Destination host described above.

The following GET requests can be used to track runtime Event history:

12.3.1. EventCollection Resource /redfish/v1/Events

To see the collection of all runtime Events that the API EventService has been tracking, you can run a GET request to /redfish/v1/Events. It is important to note that this resource collection is runtime only. If dp-api service is stopped, restarted, or failed over for any reason, the runtime collection will be lost. It is up to the client running an EventReceiver service that dp-api EventService broadcasts Events to, to save the Event history persistently if they choose to.

GET /redfish/v1/Events example (output truncated)

```
[root@kjlmo17-oem rest_api]# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/Events | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 16024    0 16024    0     0  15.2M      0 --:--:-- --:--:-- --:--:-- 15.2M
{
  "@odata.context": "EventCollection.EventCollection",
  "@odata.id": "/redfish/v1/Events",
  "@odata.type": "EventCollection.v1_0_0.EventCollection",
  "Description": "Collection of Events from dp-api in-memory storage",
  "Members": [
    {
      "@odata.id": "/redfish/v1/Events/e7e807e8-1289-11ed-b9b8-b4969102730c"
    },
    .
    .
    .
    {
      "@odata.id": "/redfish/v1/Events/fefe1712-12a0-11ed-b9b8-b4969102730c"
    }
  ],
  "Members@odata.count": 162,
  "Name": "EventCollection"
}
```

Details of the response:

- Members: shows collection of URIs for all Events that have been sent to dp-api EventService since it has been running
- Members@odata.count: the total number of Events send to dp-api EventService since it has been running

12.3.2. Event resource /redfish/v1/Events/{EventId}

The Event resource is used to capture the details of s single Event. Details of the Event resource fields will be shown below. Each Event will contain a unique EventId.

EventId:

- a unique Id that is generated by the monitoring service that sends the Event to dp-api.

Schema: https://redfish.dmtf.org/schemas/v1/Event.v1_7_0.json

GET /redfish/v1/Events/{EventId} example

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/Events/fefe1712-12a0-11ed-b9b8-b4969102730c | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   689  100   689    0     0   672k      0 --:--:-- --:--:-- --:--:--  672k
{
  "@odata.type": "",
  "Events": [
    {
      "Actions": {},
      "EventId": "fefe1712-12a0-11ed-b9b8-b4969102730c",
      "EventTimestamp": "1659471819",
      "EventType": "Alert",
      "MemberId": "",
      "Message": "Status changed for cooling",
      "MessageId": "",
      "Oem": {
        "Hpe": {
          "attribute": "overall_health",
          "enclosure_serial_number": "MXE30000122VS00E",
          "index": "6",
          "timestamp": 1659471819,
          "type": "cooling"
        }
      },
      "ResourceId": "/redfish/v1/Chassis/MXE30000122VS00E/Thermal/Fans/Cooling-6",
      "Severity": "Ok"
    }
  ],
  "Id": "fefe1712-12a0-11ed-b9b8-b4969102730c",
  "Name": "Events"
}
```

Details of the response:

- EventTimestamp: the timestamp the Event occurred
- EventType: the type of Event sent
  - the supported Event types can be seen in the EventService resource via a GET request to /redfish/v1/EventService
- Message: message about what occurred for this Event
- Oem.Hpe
  - attribute: the type of alert received from the backend monitoring service that sent the event
  - enclosure_serial_number: the enclosure which contains the resource this Event came from

        - this would map to a specific ChassisId - /redfish/v1/Chassis/{ChassisId} where enclosure_serial_number == ChassisId
    - index: the FRU index of the resource the Event was sent against
        - this is applicable to Events on FRU type resources only, i.e. Drive, PSU, PAB, Fan
    - type: the FRU type of the resource the Event was sent against
        - this is applicable to Events on FRU type resources only, i.e. Drive, PSU, PAB, Fan
- ResourceId: the OdataId of the resource this Event was sent against
    - this is intended to allow clients to quickly parse the Event response body and run a GET request to the ResourceId to see the change in data reflected
- Severity: this reflects the status of the resource the Event occurred on after it occurred
- Id: the unique Id to identify this Event
    - matches EventId in /redfish/v1/Events/{EventId}
    - generated by the sender

12.3.3 Rotating of runtime Event History

As mentioned in section 10.1 – the EventService resource contains the following configurable value

- Oem.RequestedMaxEventsToQueue: the number of Events the API EventService will store in memory before rotating them

This parameter defines the number Events that API EventService will retain before rotating them out. Since all Events in runtime history are stored in memory and not persistenly, by Redfish spec definition, when Events history gets rotated, Events are then lost. This is an effort to help prevent the memory foot print of the API from becoming too large.

Events should be stored externally to the cluster if persistent storage is required. The EventService.Oem.RequestedMaxEventsToQueue parameter can be changed in /redfish/v1/EventService to an appropriate value for that system via a PATCH or PUT request.

Example logs from dp-api showing runtime Event history being rotated:

```
Apr 04 11:46:47 kjlmo1200 dp-api-server[1124914]: INFO[2023-04-04T11:46:47-06:00] In memory EventCollection has reached maximum
length of 1000! Rotating EventCollection...
```

12.4. IcingaServices Events

The NEO RFSF API provides some custom Events that are triggered based on change in state or output of an Icinga2 monitoring service running on the cluster. When these Events occur, they will be sent in the same Event resource structure and Broadcasted the same to all subscribers as any other Event. Here is a list of all Icinga2 services which may invoke Events and which

resources the occur on. Please see existing ClusterStor Manager and Monitoring documentation for more details on each of these Icinga2 services.

NOTE: most all of these services reside in the ComputerSystem resource, as they are all node specific. We may choose to break them out into other Redfish/Swordfish resources in the future.

ComputerSystem IcingaServices which trigger Events:
Node Status
DIMMs Status
Current Load
Current Users
RAM usage
Total Processes
Free Space
Swap Usage
CPU Usage
Node HW Fingerprint Status

StoragePools IcingaServices which trigger Events:
Array Status
Disk Status
NVMe Connection

Chassis IcingaServices which trigger Events:
FRU PSU Assembly Units Status
FRU Power Supply Status
FRU Enclosure Electronics Module Status
FRU Fan Status
Fan Statistics
Power Statistics
Thermal Statistics
Voltage Statistics
Current Statistics
Health Check
SAS Health

RAS Service Events

12.5. Types of Events

The Redfish specification specifies the types of Events that EventService can receive and Broadcast as the following (please see the Redfish Specification for more details):

- StatusChange

- ResourceUpdated
- ResourceAdded
- ResourceRemoved
- Alert
- MetricReport
- Other

For initial delivery, the NEO RFSF API EventService will support 3 types of Events. These are defined in the EventService resource. To see this, look at the EventService.EventTypesForSubscription field.

Supported Event Types

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/EventService | jq
'.EventTypesForSubscription'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   791  100   791    0     0   772k      0 --:--:-- --:--:-- --:--:--  772k
[
  "Alert",
  "StatusChange",
  "MetricReport"
]
```

12.6. Resources Events can occur on

The EventService resource also describes the ResourceTypes that Events can occur on. Not all resources tracked by NEO RFSF API are contained in this list, but more may be added in the future - to see this list, look at the EventService.ResourceTypes field.

Supported Event ResourceTypes

```
# curl -H "X-Auth-Token: ${AUTH}" https://172.26.137.72:8081/redfish/v1/EventService | jq '.ResourceTypes'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   791  100   791    0     0   772k      0 --:--:-- --:--:-- --:--:--  772k
[
  "PowerSupply",
  "AssemblyData",
  "Cooling",
  "Drive",
  "ComputerSystem",
  "Volume",
  "FileSystem",
  "StoragePool"
]
```

12.7. Subscribers Receiving Events

As mentioned above, dp-api only provides the capability for users to subscribe to receive Events, as well as forward any Events it receives from other API services to all subscriber destinations. It is then up to the describer to implement a simple service to listen on some port at some endpoint for POST requests from dp-api. Once the subscriber receives these POST request each time an Event occurs, they can choose to do whatever they want with these Events - such as:

- persistently store all Event json somewhere
- send email notifications about the Event
- schedule service related to Event
- whatever

Example simple python service to receive events on http://172.16.0.1:8082/eventreceiver/ forwarded via POST requests from dp-api as EventService receives them. This service simply logs information about the request, including the request body which contains the Event itself.

NOTE: This is not my code, it is a slightly modified example taken from https://gist.github.com/mdonkers/63e115cc0c79b4f6b8b3a6b797e485c7

Event receiver POST request handler service

```
#!/usr/bin/env python3.9
"""
Very simple HTTP server in python for logging requests
Usage::
    ./server.py [<port>]
"""
from http.server import BaseHTTPRequestHandler, HTTPServer
import logging

class S(BaseHTTPRequestHandler):
    def _set_response(self):
        self.send_response(200)
        self.send_header('Content-type', 'text/html')
        self.end_headers()

    def do_POST(self):
        content_length = int(self.headers['Content-Length']) # <--- Gets the size of data
        post_data = self.rfile.read(content_length) # <--- Gets the data itself
        logging.info("Received Event:\nDestination: %s\nHeaders:\n%s\n\nBody:\n%s\n",
                str(self.path), str(self.headers), post_data.decode('utf-8'))

        self._set_response()
        self.wfile.write("POST request for {}".format(self.path).encode('utf-8'))
```

```
def run(server_class=HTTPServer, handler_class=S, port=8080):
    logging.basicConfig(level=logging.INFO)
    server_address = ('', port)
    httpd = server_class(server_address, handler_class)
    logging.info('Starting httpd...\n')
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        pass
    httpd.server_close()
    logging.info('Stopping httpd...\n')

if __name__ == '__main__':
    from sys import argv

    if len(argv) == 2:
        run(port=int(argv[1]))
    else:
        run()
```

Full example of dp-api EventService receiving an Event and forwarding it to the example service above, which I've named test_event_receiver.py. I've specified this service to handler POST requests to http://172.16.0.1:8082/eventreceiver/ and log what it receives.

First, dp-api-server receives an Event from another API service:

dp-api-server logs

```
May 25 11:29:18 kjlmo1200 dp-api-server[2988909]: INFO[2022-05-25T11:29:18-06:00] Received dbus Signal --
Body: {"Id": "3506dce0-dc50-11ec-8738-b4969102730c", "Events": [{"EventId": "3506dce0-dc50-11ec-8738-
b4969102730c", "EventType": "Alert", "EventTimestamp": "sometime", "Message": "Test Event!!", "Severity":
"Critical", "Oem": {"Hpe": {"type": "Alert", "timestamp": "somedate", "enclosure_serial_number": "XXXXXXXXXXXX",
"attribute": "FAN"}}}], "OdataType": "#Event.1.0.0.Event", "Name": "Events"}
```

Second, dp-api saves Event to runtime history:

dp-api-server logs

```
May 25 11:39:17 kjlmo1200 dp-api-server[3025730]: INFO[2022-05-25T11:39:17-06:00] Saving Event 99daf7fe-
dc51-11ec-b279-b4969102730c to runtime history...
```

Next, dp-api sends Event to all subscribed destinations. You will see one of two messages in logs based off if Event receiver service is running listening on the correct port specified in Destination field of POST request to subscribe.

dp-api-server logs when receiver service is NOT running

```
May 25 11:39:00 kjlmo1200 RFSF-API[3025730]: ERRO[2022-05-25T11:39:00-06:00] Could not send event
notification -- Body: {"Id": "901bec78-dc51-11ec-9f17-b4969102730c", "Events": [{"EventId": "901bec78-dc51-11ec-
9f17-b4969102730c", "EventType": "Alert", "EventTimestamp": "sometime", "Message": "Test Event!!", "Severity":
```

"Critical", "Oem": {"Hpe": {"type": "Alert", "timestamp": "somedate", "enclosure_serial_number": "XXXXXXXXXXX", "attribute": "FAN"}}}], "OdataType": "#Event.1.0.0.Event", "Name": "Events"}
Err: Post "http://172.16.0.1:8082/eventreceiver/": dial tcp 172.16.0.1:8082: connect: connection refused

dp-api-server logs when receiver service is running

May 25 11:39:17 kjlmo1200 RFSF-API[3025730]: INFO[2022-05-25T11:39:17-06:00] Event notification sent -- Dest: http://172.16.0.1:8082/eventreceiver/
Body: {"Id": "99daf7fe-dc51-11ec-b279-b4969102730c", "Events": [{"EventId": "99daf7fe-dc51-11ec-b279-b4969102730c", "EventType": "Alert", "EventTimestamp": "sometime", "Message": "Test Event!!", "Severity": "Critical", "Oem": {"Hpe": {"type": "Alert", "timestamp": "somedate", "enclosure_serial_number": "XXXXXXXXXXX", "attribute": "FAN"}}}], "OdataType": "#Event.1.0.0.Event", "Name": "Events"}
May 25 11:39:17 kjlmo1200 dp-api-server[3025730]: INFO[2022-05-25T11:39:17-06:00] Event notification sent -- Dest: http://172.16.0.1:8082/eventreceiver/
May 25 11:39:17 kjlmo1200 dp-api-server[3025730]: Body: {"Id": "99daf7fe-dc51-11ec-b279-b4969102730c", "Events": [{"EventId": "99daf7fe-dc51-11ec-b279-b4969102730c", "EventType": "Alert", "EventTimestamp": "sometime", "Message": "Test Event!!", "Severity": "Critical", "Oem": {"Hpe": {"type": "Alert", "timestamp": "somedate", "enclosure_serial_number": "XXXXXXXXXXX", "attribute": "FAN"}}}], "OdataType": "#Event.1.0.0.Event", "Name": "Events"}

Finally, event receiver service receives the Event POST from dp-api and logs it.

test_event_receiver.py logs

```
[root@kjlmo12-oem rest_api]# ./test_event_receiver.py 8082
INFO:root:Starting httpd...

INFO:root:Received Event:
Destination: /eventreceiver/
Headers:
Host: 172.16.0.1:8082
User-Agent: Go-http-client/1.1
Content-Length: 445
Accept-Encoding: gzip
```

Body:
"{\"Id\": \"99daf7fe-dc51-11ec-b279-b4969102730c\", \"Events\": [{\"EventId\": \"99daf7fe-dc51-11ec-b279-b4969102730c\", \"EventType\": \"Alert\", \"EventTimestamp\": \"sometime\", \"Message\": \"Test Event!!\", \"Severity\": \"Critical\", \"Oem\": {\"Hpe\": {\"type\": \"Alert\", \"timestamp\": \"somedate\", \"enclosure_serial_number\": \"XXXXXXXXXXX\", \"attribute\": \"FAN\"}}}], \"OdataType\": \"#Event.1.0.0.Event\", \"Name\": \"Events\"}"

```
172.16.2.3 - - [25/May/2022 17:39:17] "POST /eventreceiver/ HTTP/1.1" 200 -
```

The point here is that subscribers can do anything they want with their service to receive events, such as send emails, save events persistently, etc.

12.8 Deleting EventDestinationId subscription

Events at some EventDestination

Sometimes administrators may want to stop receiving Events at some specific EventDestinationId that was earlier subscribed for. The proper way to do this is to send a DELETE request to the EventDestination endpoint the subscription was created with: /redfish/v1/EventService/Subscriptions/{EventDestinationId}. This will remove dp-api EventService's knowledge of the EventDestination resource, so Events will no longer be broadcasted there.

Example:

Send the DELETE request to unsubscribe the "test-event-subscription" EventDestination resource:

```
[root@kjlmo17-oem ~]# curl -k -H "X-Auth-Token: ${AUTH}" -X DELETE
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/EventService/Subscriptions/test-event-subscription | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-event-subscription",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8082/eventreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "test-event-subscription,
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

Now that the "test-event-subscription" EventDestination resource was deleted, EventService will no longer send Events to the "http://172.16.0.1:8082/eventreceiver/" destination.

13. MetricReport data Streaming Subscriptions

MetricReport data streaming subscriptions are a special type of subscription where clients subscribe to receive a certain set of Metric data at a specified interval. Similar to a generic Event subscription, clients must send a POST request to the /redfish/v1/EventService/Subscriptions/{EventDestinationId} endpoint where EventDestinationId must be unique. Also similar to general Event subscriptions, an Event Receiver service must be setup to handle POST request from dp-api which will come in the form of a MetricReport Redfish resource. To designate a particular subscription request as a MetricReport subscription, EventDestination.EventTypes must be set to "MetricReport", and EventDestination.MetricReportDefinitions must be defined. See below for more details.

MetricReport Schema: https://redfish.dmtf.org/schemas/v1/MetricReport.v1_5_0.json

Each MetricReport contains an array of MetricValue resources, which provide individual Metric names, values, and other metadata. The schema for MetricValues can be seen within the MetricReport schema above.

13.1 Available MetricReport Data Sets

To view the available data sets which a client can subscribe to receive, clients must run a GET request to the /redfish/v1/TelemetryService/MetricDefinitions endpoint to view the MetricDefinitions collection. Examplel GET request to view the MetricDefinitions collection:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions | jq
{
  "@odata.context": "MetricDefinitionCollection.MetricDefinitionCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions",
  "@odata.type": "MetricDefinitionCollection.v1_0_0.MetricDefinitionCollection",
  "Description": "MetricDefinitionCollection defines the collection of supported MetricDefinition resources, which
describe all supported API Metrics available.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/JobStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats"
    },
```

```
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/TieringStats"
    }
  ],
  "Members@odata.count": 5,
  "Name": "MetricDefinitionCollection"
}
```


The MetricDefinitions collection defines the individual MetricDefinition resources which represent a particular data set that a client can subscribe to receive. The EventDestination.MetricReportDefinitions field must be set to a valid /redfish/v1/TelemetryService/MetricReportDefinitions/{MetricReportDefinitionId} endpoint which provides the desired MetricDefinition in its MetricReportDefinition.Metrics list in order to be considered a valid request. To find more info on each individual MetricDefinition resource, a GET request to the particular /redfish/v1/TelemetryService/MetricDefinitions/{MetricDefinitionId} resource endpoint can be made. To find more info on each individual MetricReportDefinition resource, a GET request to the particular /redfish/v1/TelemetryService/MetricReportDefinitions/{MetricReportDefinitionId} resource endpoint can be made. Section 14 contains more details about the /redfish/v1/TelemetryService, /redfish/v1/TelemetryService/MetricDefinitions, /redfish/v1/TelemetryService/MetricReportDefinitions, /redfish/v1/TelemetryService/MetricReportDefinitions/{MetricReportDefinitionId} and /redfish/v1/TelemetryService/MetricDefinitions/{MetricDefinitionId} endpoints and resources and their relationships.

13.1.1 LustreStats Data Set

The LustreStats data set defined by the /redfish/v1/TelemetryService/MetricDefinitions/LustreStats MetricDefinition resource provides a data set of all Lustre filesystem statistics, both at the top-level Filesystem and individual Lustre target levels. The default LustreStats MetricReportDefinition: /redfish/v1/TelemetryService/MetricReportDefinitions/LustreStats Provides the LustreStats MetricDefinition in its Metrics list and can be used to subscribe to receive the data. The following attributes are collected for each individual Lustre target (MDTs and OSTs), as well as the top-level Lustre filesystem.

NOTE: The LustreStats MetricDefinition is of Implementation type: “Reported”, meaning data is collected from external components and aggregated to a single location accessible to grp-rfsf-api HA group services on the mgmt. nodes only.

Individual MDT and OST statistics:

- available_space

- close
- connect
- create
- destroy
- disconnect
- free_inodes
- free_space
- getattr
- getxattr
- link
- llog_init
- mkdir
- mknod
- notify
- num_exports
- open
- process_config
- quotactl
- read
- reconnect
- recovery_time_remaining
- rename
- rmdir
- setattr
- statfs
- total_inodes
- total_space
- unlink
- write

Top-level Lustre FileSystem statistics:

- Avg_120_sec_aggregated_all_OSTs_read_bytes
- Avg_120_sec_aggregated_all_OSTs_write_bytes
- Max_24_hour_aggregated_all_OSTs_read_bytes
- Max_24_hour_aggregated_all_OSTs_write_bytes
- OST_Total
- Total_FS_MD_ops
- Total_FS_Read
- Total_FS_Space
- Total_FS_Space_Available
- Total_FS_Space_Used
- Total_FS_Write

13.1.2 LinuxStats Data Set

The LinuxStats data set defined by the /redfish/v1/TelemetryService/MetricDefinitions/LinuxStats MetricDefinition resource provides a data set of all resource consumption statistics available to a Linux host. The default LinuxStats MetricReportDefinition: /redfish/v1/TelemetryService/MetricReportDefinitions/LinuxStats Provides the LinuxStats MetricDefinition in its Metrics list and can be used to subscribe to receive the data. The following attributes are collected for each node in the cluster.

NOTE: The LinuxStats MetricDefinition is of Implementation type: "Reported", meaning data is collected from external components and aggregated to a single location accessible to grp-rfsf-api HA group services on the mgmt. nodes only.

Individual statistics:

AvailableMemory
AvailableMemoryPercent
BufferedMemory
CPUGuestNiceTime
CPUGuestTime
CPUIOWaitTime
CPUIRQTime
CPUIdleTime
CPUNiceTime
CPUSoftIRQTime
CPUStealTime
CPUSystemTime
CPUUserTime
CPUUtilization
CachedMemory
ContextSwitchesPerSecond
FreeMemory
FreeSwapSpace
FreeSwapSpacePercent
InterruptsPerSecond
LoadAverage15m
LoadAverage1m
LoadAverage5m
MemoryUtilization
NumberOfCPUs
TotalMemory
TotalSwapSpace

13.1.3 JobStats Data Set

The JobStats data set defined by the /redfish/v1/TelemetryService/MetricDefinitions/JobStats MetricDefinition resource provides a data set of all statistics related to on-going jobs running against the Lustre FileSystem. The default JobStats MetricReportDefinition: /redfish/v1/TelemetryService/MetricReportDefinitions/Jobstats Provides the JobStats MetricDefinition in its Metrics list and can be used to subscribe to receive the data. All of the attributes gathered are measured for each job running against each individual lustre target, then calculated and presented as statistics against the top-level Lustre FileSystem as a whole:

NOTE: The JobStats MetricDefinition is of Implementation type: “Reported”, meaning data is collected from external components and aggregated to a single location accessible to grp-rfsf-api HA group services on the mgmt. nodes only.

Individual Statistics:

- calc_metadata_ops
- calc_read_bytes_rate
- calc_write_bytes_rate
- close
- close_max
- close_min
- close_sum
- close_sumsq
- create
- create_max
- create_min
- create_sum
- create_sumsq
- crossdir_rename
- crossdir_rename_max
- crossdir_rename_min
- crossdir_rename_sum
- crossdir_rename_sumsq
- destroy
- destroy_max
- destroy_min
- destroy_sum
- destroy_sumsq
- fallocate
- fallocate_max
- fallocate_min
- fallocate_sum
- fallocate_sumsq
- get_info
- get_info_max

- get_info_min
- get_info_sum
- get_info_sumsq
- getattr
- getattr_max
- getattr_min
- getattr_sumgetattr_sumsq
- getxattr
- getxattr_max
- getxattr_min
- getxattr_sum
- getxattr_sumsq
- link
- link_max
- link_min
- link_sum
- link_sumsq
- migrate
- migrate_max
- migrate_min
- migrate_sum
- migrate_sumsq
- mkdir
- mkdir_max
- mkdir_min
- mkdir_sum
- mkdir_sumsq
- mknod
- mknod_max
- mknod_min
- mknod_sum
- mknod_sumsq
- open
- open_max
- open_min
- open_sum
- open_sumsq
- parallel_rename_dir
- parallel_rename_dir_max
- parallel_rename_dir_min
- parallel_rename_dir_sum
- parallel_rename_dir_sumsq
- parallel_rename_file
- parallel_rename_file_max

- parallel_rename_file_min
- parallel_rename_file_sum
- parallel_rename_file_sumsq
- prealloc
- prealloc_max
- prealloc_min
- prealloc_sum
- prealloc_sumsq
- punch
- punch_max
- punch_min
- punch_sum
- punch_sumsq
- quotactl
- quotactl_max
- quotactl_min
- quotactl_sum
- quotactl_sumsq
- read
- read_bytes
- read_bytes_max
- read_bytes_min
- read_bytes_sum
- read_bytes_sumsq
- read_max
- read_min
- read_sum
- read_sumsq
- rename
- rename_max
- rename_min
- rename_sum
- rename_sumsq
- rmdir
- rmdir_max
- rmdir_min
- rmdir_sum
- rmdir_sumsq
- samedir_rename
- samedir_rename_max
- samedir_rename_min
- samedir_rename_sum
- samedir_rename_sumsq
- set_info

- set_info_max
- set_info_min
- set_info_sum
- set_info_sumsq
- setattr
- setattr_max
- setattr_min
- setattr_sum
- setattr_sumsq
- setxattr
- setxattr_max
- setxattr_min
- setxattr_sum
- setxattr_sumsq
- statfs
- statfs_max
- statfs_min
- statfs_sum
- statfs_sumsq
- sync
- sync_max
- sync_min
- sync_sum
- sync_sumsq
- unlink
- unlink_max
- unlink_min
- unlink_sum
- unlink_sumsq
- write
- write_bytes
- write_bytes_max
- write_bytes_min
- write_bytes_sum
- write_bytes_sumsq
- write_max
- write_min
- write_sum
- write_sumsq

13.1.4 EnvironmentalStats Data Set

The EnvironmentalStats data set defined by the /redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats MetricDefinition resource provides a data set of all "environmental" metrics available from the system, i.e. any sort of metric to represent energy consumption from the system. The default EnvironmentalStats MetricReportDefinition: /redfish/v1/TelemetryService/MetricReportDefinitions/EnvironmentalStats Provides the EnvironmentalStats MetricDefinition in its Metrics list and can be used to subscribe to receive the data. These metric readings largely come from various IPMI and SES sensors available to ClusterStor nodes, depending on the HW config and attached enclosures available. Information may include the current sensor reading, units, and thresholds for each sensor, as well as any other information they may uniquely provide. The sensor "types" collected at a high level are listed for each enclosure type below.

NOTE: The EnvironmentalStats MetricDefinition is of Implementation type: "Sensor", meaning data is gathered from internal API services / components, accessible to those API services / components running directly on cluster nodes.

IPMI:

- Temperature
- Voltage
- Current
- Power Supply
- Fan

SES:

- PSU
- Cooling
- Temperature
- Enclosure_Electronics
- Enclosure
- Voltage_Sensor
- Current_Sensor
- Enclosure_Electronics_Power
- Enclosure_Electronics_Diagnostics

13.1.5 TieringStats Data Set

The TieringStats data set defined by the /redfish/v1/TelemetryService/MetricDefinitions/TieringStats MetricDefinition resource provides a data set of all statistics related to data movement between different tiers of the Lustre filesystem which are generated by the CDS (ClusterStor Data Services) mini-tiering services. The default TieringStats MetricReportDefinition: /redfish/v1/TelemetryService/MetricReportDefinitions/TieringStats Provides the TieringStats

MetricDefinition in its Metrics list and can be used to subscribe to receive the data. The following list captures the set of available metrics, for more information on what each of these metrics represent, please see the “auto-tiering” section of the Cray ClusterStor Admin Guide:

NOTE: The TieringStats MetricDefinition is of Implementation type: “Reported”, meaning data is collected from external components and aggregated to a single location accessible to grp-rfsf-api HA group services on the mgmt. nodes only.

Individual statistics

- commit.time
- commit.count
- index.full.count
- index.incremental.count
- index.changelog.count
- index.changelog.targets
- index.full.time
- index.incremental.time
- index.changelog.time
- commit.tasks
- commit.threshold_warns
- commit.errors
- commit.error_drops
- commit.db_init
- commit.file_add
- commit.file_delete
- commit.path_add
- commit.path_delete
- calls.lstat.count
- calls.lstat.time
- calls.ostlayout.count
- calls.ostlayout.errors
- calls.ostlayout.time
- calls.hsmattrs.count
- calls.hsmattrs.errors
- calls.hsmattrs.time
- changelog.record.index
- changelog.record.cleared
- changelog.record.count
- changelog.record.reduced
- changelog.filter.fidzero
- changelog.record.reduce_inuse
- changelog.record.reduce_spill
- query.count

- query.time
- query.bytes
- query.items
- query.processed
- query.processed_bytes

13.2. MetricReport Event Subscriptions

As mentioned above, in order to receive MetricReport streaming data for one of the defined MetricDefinition data sets from RFSF API EventService, clients must send a special EventDestination subscription POST request to the same endpoint used to subscribe for general Events: /redfish/v1/EventService/Subscriptions/{EventDestinationId}. The details of what makes this EventDestination POST request "special" or different from a general Event subscription request are contained within the request body:

Example MetricReport EventDestination POST request body:

```
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metric-report",
  "Id": "test-metric-report",
  "Destination": "http://172.16.0.1:8082/metricsreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Context": "Public",
  "EventTypes": [
    "MetricReport"
  ],
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    }
  ]
}
```

Details of the request body that differentiate a MetricReport EventDestination subscription from a generate EventDestination subscription request:

- EventTypes - this Array **must contain** the EventType of "MetricReport"

    - RFSF API EventService keys off this EventTypes field to kick off some special setup for the MetricReport data stream
- MetricReportDefinitions - This must contain a single valid /redfish/v1/TelemetryService/MetricReportDefinitions/{MetricReportDefinitionId} endpoint
    - Only one MetricReportDefinition resource may be subscribed to at a time
    - The data set received is defined by the MetricReportDefinition.Metrics list, which must contain a valid MetricDefinition endpoint
    - The rate at which data is received by the client is defined by the MetricReportDefinition.Schedule

13.2.1 MetricReport Event Subscriptions – EventDesetination.MetricReportDefinitions

As mentioned above, part of what makes a subscription request to /redfish/v1/EventService/Subscriptions/{EventDestinationId} endpoint unique compared to generic Event subscriptions, is the MetricReportDefinitions specified in the EventDestination request body payload. The request body must contain a MetricReportDefinitions list with:

- A single valid odata.id endpoint of an existing /redfish/v1/TelemetryService/MetricReportDefinitions/{MetricReportDefinitionId} resource

13.3. MetricReport Subscription Restrictions and Behavior

MetricReport Subscription requests do have some rules surrounding when a request is considered valid or not, checked both against the request endpoint, request body, and certain conditions on the system related to a data source being available or not. There are both generic restrictions that apply to MetricReport subscription request for *all* MetricReportDefintions's, as well as more specific restrictions for certain MetricReportDefintions's. The following sections will highlight both. It will also highlight some behavior that can occur when subscribing to receive certain MetricReportDefinitions's that depends on the activity related to the collection / propagation of the MetricDefinition data that MetricReportDefinition provides in its Metrics list on going on the cluster.

13.3.1 MetricReport Subscription Generic Restrictions

The following list highlights MetricReport subscription request restrictions that apply to all MetricReportDefinition's:

- Each MetricReport Subscription can only be for one target MetricReportDefinition endpoint
- There is no limit to the number of client subscribers subscribed to receive that MetricReportDefinition

- The rate at which clients receive data when subscribed is specified by the associated MetricReportDefinition.Schedule in the MetricReportDefinition resource passed in the request body
- The MetricReportDefinition.Schedule.RecurrenceInterval is set by the associated MetricDefinition.SensingInterval by default, but can be updated to change the rate at which data is received via a PUT or PATCH request to that MetricReportDefinition resource
- MetricReport Events are only sent to a EventDestinations subscribed to receive that MetricReportDefinition

13.3.2 MetricReport Subscription MetricReportDefinition/MetricDefinition specific Restrictions and Behavior

Some MetricReportDefinition's that may be requested in a MetricReport subscription request have some more rules surrounding whether or not a request is allowed to proceed or not. Most of these rules regard the capability or availability of the data source that pulls the MetricDefinition data specified by the MetricReportDefinition.Metrics list in the request body. Additionally, certain subscriptions for certain MetricReportDefinition's come with some expectations surrounding the behavior of the system in terms of when data may become available or not without terminating the subscription. The following sub-sections highlight restrictions and special behavior for each MetricReportDefinition subscription that has them.

13.3.2.1 JobStats MetricReportDefinition Subscription specific Restrictions and Behavior

When subscribing to receive JobStats data from the API, there are certain system conditions that must be satisfied in order for the request to be accepted.

First, jobstats collection must be enabled. The command to check and enable jobstats collection on the system is the following (see the CSCLI command reference document for more information on the cscli command):

```
[root@kjlmo1200 ~]# cscli lustre jobstats collection -h
usage: cscli lustre jobstats collection [-h] (--enable | --disable) [--fs FILESYSTEM]

optional arguments:
  -h, --help       show this help message and exit.
  --enable         Enable Lustre Job Statistics collection
  --disable        Disable Lustre Job Statistics collection.
  --fs FILESYSTEM  [optional] Specify the filesystem Default:kjlmo12
[root@kjlmo1200 ~]#
```

To check the current status of jobstats collection on the system, you can run “cscli lustre jobstats list”:

```
[root@kjlmo1200 ~]# cscli lustre jobstats list
---------------------------------------------------
 FSName   Collection  Frequency(in sec)  Scheduler
---------------------------------------------------
 kjlmo12  Disabled      -                  -
---------------------------------------------------
[root@kjlmo1200 ~]#
```

If “cscli lustre jobstats list” shows as disabled, and a client tries to make a request to subscribe to receive JobStats MetricReport data, an error response such as the following may be seen:

```
{
  "error": {
    "code": "400",
    "message": "JobStats Collection is not enabled on the system! Please enable with \"cscli lustre jobstats collection --enable\" before subscribing to receive JobStats MetricReports!"
  }
}
```

Unique behaviors of JobStats MetricReport subscription:

- If jobstats collection is enabled, and the request succeeds to create a subscription to receive JobStats MetricReport data, it is still possible that no data will come back to the client if there are no active jobs running on the system.
- If jobstats collection is disabled or any other dropped connection interrupts the data flow while a JobStats MetricReport subscription is active, a failed thread callback Event will automatically be sent to the client which will cancel the subscription (see section 13.3.3 for more details on this)

13.3.2.1 TieringStats MetricReportDefinition Subscription specific Restrictions and Behavior

When subscribing to receive TieringStats data from the API, there are certain system conditions that must be satisfied in order for the request to be accepted.

First, lustre tiering must be enabled. The command to check and enable lustre tiering on the system is the following (see the CSCLI command reference document for more information on the cscli command):

```
[root@kjlmo1200 ~]# cscli lustre tiering -h
```

```
usage: cscli lustre tiering [-h] {show,enable,disable} ...

Manage the Clusterstor tiering service

positional arguments:
  {show,enable,disable}
    show            Displays the current state of the Clusterstor tiering service
    enable          Enable the Clusterstor tiering service
    disable         Disable the Clusterstor tiering service

optional arguments:
  -h, --help        show this help message and exit.
```

In order to validate the lustre tiering is both enabled, and active – "cscli lustre tiering show" command output must show the following:

```
[root@kjlmo700 ~]# cscli lustre tiering show
State: Enabled
Status: Active
Client mount: Active
Lustre: Active
[root@kjlmo700 ~]#
```

The above output indicates that not only was the Lustre filesystem started with "cscli mount" (Lustre: Active) – but "cscli lustre tiering enable" has been run in order to activate auto-tiering (State: Enabled, Status: Active, Client mount: Active). If output of "cscli lustre tier show" does not show that it is both Enabled State and Active Status , and a client tries to make a request to subscribe to receive TieringStats MetricReport data, an error response such as the following may be seen:


```
{
 "error": {
   "code": "400",
   "message": "Lustre Tiering is not enabled on the system! Please enable with \"cscli lustre tiering enable\" before subscribing to receive TieringStats MetricReports!"
 }
}
```


Unique behaviors of TieringStats MetricReport subscription:

- If lustre tiering is enabled, and the request succeeds to create a subscription to receive TieringStats MetricReport data, it is still possible that no data will come back to the client if there is no on-going data movement or tiering operations against the filesystem

- If lustre tiering is disabled or any other dropped connection interrupts the data flow while a TieringStats MetricReport subscription is active, a failed thread callback Event will automatically be sent to the client which will cancel the subscription (see section 13.3.3 for more details on this)

13.3.3 MetricReport subscription failed thread callback Events

As mentioned above, throughout the lifetime of a MetricReport subscription for any MetricReportDefinition, it is possible that due to some system event or administrative action on the cluster, that the backend data source may be disabled, fail, or go away for some other reason. When this occurs, the backend EC which is the component collecting data from that data source will trigger a "failed thread callback" Event, indicating to the API and client that the subscription is being automatically removed and that clients will need to re-subscribe at a later time to receive data again for that MetricReportDefinition. This "failed thread callback" Event, will remove *all* subscribers for that MetricReportDefinition, no matter how many there are, and all client EventDestination endpoints will receive the Event indicating their subscription has been cancled.

An example of the Event body send to clients when a "failed thread callback" exception is hit, may look like the following:

```
Destination: /metricsreceiver/
Headers:
Host: 172.16.0.1:8083
User-Agent: Go-http-client/1.1
Content-Length: 655
Accept-Encoding: gzip
```


```
Body:
"{\"Id\": \"e494d23a-3df6-11ee-a743-b4969102730c\", \"Events\": [{\"EventId\": \"e494d23a-3df6-11ee-a743-b4969102730c\", \"EventType\": \"Alert\", \"EventTimestamp\": \"1692384110.127196\", \"Message\": \"TieringStats thread FAILED! This thread exited unexpectedly for EventDestination: test-metrics-report-tier. ALL subscribers for this DataSet will be removed. Clients must re-subscribe to receive data again.\", \"Severity\": \"Critical\", \"Oem\": {}, \"OriginOfCondition\": {\"e494d23a-3df6-11ee-a743-b4969102730c\": \"/redfish/v1/EventService/Subscriptions/test-metrics-report-tier\"}}], \"OdataType\": \"#Event.1.0.0.Event\", \"Name\": \"Events\"}"
```

As the message in the Event payload indicates, the EventDestination resource has been removed, and the associated client must re-subscribe to start receiving this data again.

13.4. MetricReport Subscription Example

The following example will show the full setup of a MetricReport EventDestination subscription, which is setup to receive the default LustreStats MetricReportDefinition, at a default Schedule.RecurrentInterval of 10 seconds, at the client side Destination: http://172.16.0.1:8082/metricsreceiver/. It will show the details of the request body, the POST request being executed by, and the client side EventReceiver service that will parse and print the data each time it receives a MetricReport Event:

Request body:

MetricReport EventDestination POST request body

```
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metric-report",
  "Id": "test-metric-report",
  "Destination": "http://172.16.0.1:8082/metricsreceiver/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Context": "Public",
  "EventTypes": [
    "MetricReport"
  ],
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    }
  ]
}
```

Request Body MetricReport details:

- `"EventTypes": ["MetricReport"] - specifies this is a subscription for MetricReports`
- `"MetricReportDefinitions": [{"@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/LustreStats"}] - specifies the MetricReportDefinitiont the client is subscribing to receive`

- NOTE: for more information about the MetricReportDefinition endpoint passed in the MetricReportDefinitionsList, a GET request can be run to the resource to see the details

POST request sent to dp-api:

MetricReport EventDestination POST request

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" -H "Content-Type: application/json-rpc" -d
@rest_api/events/metrics_subscription.json https://172.26.137.72:8081/redfish/v1/EventService/Subscriptions/test-
metrics-report | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8082/metricsreceiver/",
  "EventTypes": [
    "MetricReport"
  ],
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "test-metrics-report",
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    }
  ],
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

If the POST request was successfully received, the client Event Receiver running at http://172.16.0.1:8082/metricsreceiver/ will receive the data every MetricReportDefinition.Schedule.RecurrenceInteval seconds. See the following example of client Event Receiver code parsing and printing all MetricValues from the MetricReport each time it receives a MetricReport Event

Event Receiver service receives MetricReport every RecurrenceInteval and parses and outputs the MetricValues (truncated below):

```
INFO:root:Received test-metrics-report LustreStats
INFO:root:Metrics....
{'MetricDefinition': None, 'MetricId': 'Total MDT0000 metadata', 'MetricProperty': None, 'MetricValue': '0', 'Oem':
{'ns': '206808324', 'itemid': '51921', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp':
'1668549950'}
{'MetricDefinition': None, 'MetricId': 'Total MDT0001 metadata', 'MetricProperty': None, 'MetricValue': '0', 'Oem':
{'ns': '206815247', 'itemid': '51922', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp':
'1668549950'}
{'MetricDefinition': None, 'MetricId': 'MDT0000 close', 'MetricProperty': None, 'MetricValue': '0', 'Oem': {'ns': '18',
'itemid': '51925', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp': '1668549950'}
{'MetricDefinition': None, 'MetricId': 'MDT0000 connect', 'MetricProperty': None, 'MetricValue': '0', 'Oem': {'ns': '20',
'itemid': '51927', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp': '1668549950'}
{'MetricDefinition': None, 'MetricId': 'MDT0000 create', 'MetricProperty': None, 'MetricValue': '0', 'Oem': {'ns': '6',
'itemid': '51929', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp': '1668549950'}
{'MetricDefinition': None, 'MetricId': 'MDT0000 destroy', 'MetricProperty': None, 'MetricValue': '0', 'Oem': {'ns': '2',
'itemid': '51931', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp': '1668549950'}
{'MetricDefinition': None, 'MetricId': 'MDT0000 disconnect', 'MetricProperty': None, 'MetricValue': '0', 'Oem': {'ns':
'7', 'itemid': '51933', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp': '1668549950'}
{'MetricDefinition': None, 'MetricId': 'MDT0000 getattr', 'MetricProperty': None, 'MetricValue': '0', 'Oem': {'ns': '16',
'itemid': '51939', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp': '1668549950'}
.
.
.
INFO:root:Received test-metrics-report LustreStats
INFO:root:Metrics....
{'MetricDefinition': None, 'MetricId': 'Total FS Write', 'MetricProperty': None, 'MetricValue': '0', 'Oem': {'ns':
'219940683', 'itemid': '52001', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp':
'1668549971'}
{'MetricDefinition': None, 'MetricId': 'Total FS Read', 'MetricProperty': None, 'MetricValue': '0', 'Oem': {'ns':
'220488415', 'itemid': '52002', 'value': 'lustre', 'category': 'performance', 'type': 'filesystem'}, 'TimeStamp':
'1668549972'}
.
.
.
{'MetricDefinition': None, 'MetricId': 'Avg (120 sec) aggregated all OST(s) read_bytes', 'MetricProperty': None,
'MetricValue': '0', 'Oem': {'ns': '222142914', 'itemid': '52009', 'value': 'lustre', 'category': 'performance', 'type':
'filesystem'}, 'TimeStamp': '1668549974'}
{'MetricDefinition': None, 'MetricId': 'Avg (120 sec) aggregated all OST(s) read_bytes', 'MetricProperty': None,
'MetricValue': '0', 'Oem': {'ns': '224654908', 'itemid': '52009', 'value': 'lustre', 'category': 'performance', 'type':
'filesystem'}, 'TimeStamp': '1668549979'}
172.16.2.3 - - [15/Nov/2022 22:06:21] "POST /metricsreceiver/ HTTP/1.1" 200 -
```

13.5. Updating MetricReport Subscriptions

Similar to updating generic Event subscriptions in section 12.2.2, throughout the lifetime of an MetricReport EventDestination resource, it is possible to update the subscription parameters to

receive MetricReports by altering the EvenDestination resource initially created with a PUT or PATCH request to an existing /redfish/v1/EventService/Subscriptions/{EventDestinationId} resource. The PUT or PATCH request body should contain the information to update in the existing EventDestination resource, that *results in a change* to the resource. If no new information is contained in the request body, then a 400 "BadRequest" response will be received.

13.5.1 Updating MetricReport Subscription via a PUT request

One mechanism that can be used to update an existing MetricReport subscription, is to send a PUT request to the associated EventDestination resource at the /redfish/v1/EventService/Subscriptions/{EventDestinationId}endpiont. By definition, the HTTP PUT request method creates a new resource, or replaces a representation of the target resource entirely with the request payload. For the /redfish/v1/EventService/Subscriptions/{EventDestinationId} PUT request implementation, it is expected that the request payload will replace an existing target resource. Because of this, it is required that the request body contains a fully populated EventDestination model with updates to the existing resource.

For example, say you have an existing MetricReport subscription with EventDestination.Id "test-metrics-report-linux"

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/test-metrics-report-linux | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report-linux",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8085/metricsreceiver/",
  "EventTypes": [
    "MetricReport"
  ],
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "test-metrics-report-linux",
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    }
  ],
  "Name": "",
```

```
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```


And would like to update the MetricReportDefinitions endpoint to a MetricReportDefinition which as a lower RecurrenceInterval in order to receive LinuxStats metrics at a higher frequency, as well as update the Destination endpoint at which you'd like to receive them. To update this subscription with a PUT request, the request body should contain the entire existing EventDestination model in the payload, with the changes you'd like to apply contained within.

Example PUT request payload:


```
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report-linux",
  "Id": "test-metrics-report-linux",
  "Destination": "http://172.16.0.1:8085/metricsreceiver-slower/",
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Context": "Public",
  "EventTypes": [
    "MetricReport"
  ],
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats120s"
    }
  ],
}
```


Details of the request payload:

- The EventDestination.Id is the same as the target resource: test-metrics-report-linux
- Destination has been updated to: http://172.16.0.1:8085/metricsreceiver-slower/
- MetricReportDefinitions has been updated to a new MetricReportDefinition endpoint, which has a 120s RecurrenceInterval defined (to see this run a GET request to the `/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats120s` endpoint)

The PUT request when sent, if all requirements are met, will then entirely update and replace the target resource.

Example successful PUT request using payload above:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" -d @events/metrics_subscription_linux_stats_put_update.json -X PUT
https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/test-metrics-report-linux | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report-linux",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8085/metricsreceiver-slower/",
  "EventTypes": [
    "MetricReport"
  ],
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "test-metrics-report-linux",
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats120s"
    }
  ],
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

Now, a GET request to the same test-metrics-report-linux EventDestination resource will reflect the updates taken from the PUT request body:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}"
https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/test-metrics-report-linux | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report-linux",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8085/metricsreceiver-slower/",
  "EventTypes": [
    "MetricReport"
  ],
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
```

```
  "Id": "test-metrics-report-linux",
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats120s"
    }
  ],
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```


This means, that the client will now be receiving LinuxStats MetricReport Events at the http://172.16.0.1:8085/metricsreceiver-slower/ Destination at a slower interval of 120s, defined by the LinuxStats120s MetricReportDefinition resource.

13.5.2 Updating MetricReport Subscription via a PATCH request

Another mechanism that can be used to update an existing MetricReport subscription, is to send a PATCH request to the associated EventDestination resource at the /redfish/v1/EventService/Subscriptions/{EventDestinationId}endpiont. By definition, the HTTP PATCH request method creates *partial* changes to an existing resource. For the /redfish/v1/EventService/Subscriptions/{EventDestinationId} PATCH request implementation, it is expected that the request payload will replace some number of fields in an existing resource, but not the entire resource like PUT does.

For example, say you have an existing MetricReport subscription with EventDestination.Id “test-metrics-report-linux” :

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/test-metrics-report-linux | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report-linux",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8085/metricsreceiver-slower/",
  "EventTypes": [
    "MetricReport"
  ],
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
```

```
  "Id": "test-metrics-report-linux",
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    }
  ],
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```


And would like to update the MetricReportDefinitions send_interval to increase the interval at which you receive LinuxStats metrics. To update this subscription with a PATCH request, the request body can contain only the EventDestination.MetricReportDefintions structure in the payload with the changes you'd like to apply contained within:

Example PATCH request payload:


```
{
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats30s"
    }
  ]
}
```


Details of the request payload:

- Only the MetricReportDefintions endpoint is updated to LinuxStats30s endpoint, which specifies a RecurrenceInterval of 30s within.

Example PATCH request using the payload above:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" -d @events/metrics_subscription_linux_stats_patch_update.json -X PATCH
https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/test-metrics-report-linux | jq
```


```
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report-linux",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8085/metricsreceiver-slower/",
  "EventTypes": [
    "MetricReport"
  ],
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
```

```
    }
  ],
  "Id": "test-metrics-report-linux",
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats30s"
    }
  ],
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```


Now, a GET request to the same test-metrics-report-linux EventDestination resource will reflect the updates taken from the PATCH request body:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" https://10.214.132.158:8081/redfish/v1/EventService/Subscriptions/test-metrics-report-linux | jq
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report-linux",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8085/metricsreceiver-slower/",
  "EventTypes": [
    "MetricReport"
  ],
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "test-metrics-report-linux",
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats30s"
    }
  ],
  "Name": "",
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

This means, that the client will now be receiving LinuxStats MetricReport Events at the http://172.16.0.1:8085/metricsreceiver-slower/ Destination at a quicker interval of 30s, defined by the LinuxStats30s MetricReportDefinition resource.

13.6. Canceling MetricReport Subscriptions

Sometimes administrators may want to stop receiving MetricReport data once they are subscribed to do so. The proper way to do this would be to send a DELETE request to the EventDestination endpoint the subscription was created with: /redfish/v1/EventService/Subscriptions/{EventDestination}. This will remove dp-api EventService's knowledge of the EventDestination resource.

Example:

Send the DELETE request for test-metric-report EventDestination:

DELETE request to cancel MetricReport subscription

```
# curl -k -X DELETE -H "X-Auth-Token: ${AUTH}"
https://172.26.137.72:8081/redfish/v1/EventService/Subscriptions/test-metrics-report | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   785  100   785    0     0  46176      0 --:--:-- --:--:-- --:--:-- 46176
{
  "@odata.id": "/redfish/v1/EventService/Subscriptions/test-metrics-report",
  "@odata.type": "",
  "Context": "Public",
  "Destination": "http://172.16.0.1:8082/metricsreceiver/",
  "EventTypes": [
    "MetricReport"
  ],
  "HttpHeaders": [
    {
      "Content-Type": "Application/JSON",
      "OData-Version": "4.0"
    }
  ],
  "Id": "test-metrics-report",
  "MetricReportDefinitions": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    }
  ],
  "Name": "",
```

```
  "SNMP": {},
  "Status": {},
  "SubscriptionType": ""
}
```

Once the EventDestination resource has been removed, the client will no longer receive data at the specified EventDestination.Destination endpoint, and must re-subscribe in order to start receiving data again.

14. TelemetryService Resource

The TelemetryService resource lives directly under the ServiceRoot /redfish/v1 and can be seen by issuing a GET request to /redfish/v1/TelemetryService. It provides an entry point to the MetricDefinitions, MetricReportDefinitions, and Triggers resource collections that provide various implementations of Telemetry and metric data from the system. The initial use of the TelemetryService in the NEO RFSF API will be focused on MetricDefinitions, which will provide the set of supported Metrics available via the API.

Schema: https://redfish.dmtf.org/schemas/v1/TelemetryService.v1_3_2.json

Example TelemetryService GET request:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService | jq
{
  "@odata.context": "TelemetryService.TelemetryService",
  "@odata.id": "/redfish/v1/TelemetryService",
  "@odata.type": "TelemetryService.v1_3_1.TelemetryService",
  "Description": "TelemetryService for RFSF API",
  "Id": "TelemetryService",
  "MetricDefinitions": {
    "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions"
  },
  "Name": "TelemetryService",
  "Status": {}
}
```

Details of the response: There is not much detail provided in the response output. The important thing to note, is that it provides the path to the MetricDefinitions collection resource, which is initially the only TelemetryService collection implemented by the NEO RFSF API.

14.1 MetricDefinitions Collection Resource

The MetricDefinition Collection resource provides a collection of all supported MetricDefinition resources. Each MetricDefinition resource is intended to provide a description of a data set that one can subscribe to receive MetricReports from via the MetricReport data streaming subscriptions described in section 13. The MetricDefintion collection resource is also used to check against subscription request, that the EventDestination.MetricReportDefinitions "data_set" a client is subscribing for, is in fact a supported MetricDefinition. This means, that each MetricDefinition.Id seen in this collection, represents the name of an actual supported data set available from the API.

Schema: https://redfish.dmtf.org/schemas/v1/MetricDefinitionCollection.json

Example MetricDefinitions GET request:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions | jq
{
  "@odata.context": "MetricDefinitionCollection.MetricDefinitionCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions",
  "@odata.type": "MetricDefinitionCollection.v1_0_0.MetricDefinitionCollection",
  "Description": "MetricDefinitionCollection defines the collection of supported MetricDefinition resources, which
describe all supported API Metrics available.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/JobStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/TieringStats"
    }
  ],
  "Members@odata.count": 5,
  "Name": "MetricDefinitionCollection"
}
```

Details of the response:

- The Members field of this resource holds the collection of all supported MetricDefinition resource endpoints
- The Members@odata.count shows how many total MetricDefinition resources exist the the collection

14.2 MetricDefinition Resource

The MetricDefinition resource contains the definition, metadata, or characteristics for a metric. For the purposes of Metrics available via the NEO RFSF API, each MetricDefinition resource will provide a description and metadata of the data sets contained in MetricReports, which available for streaming via subscription requests to receive that data. Each MetricDefinition resource is currently only required to provide a Description and SensingInterval field, where the description field contains a high-leveled description of the data set provided by that MetricDefinition, and the SensingInterval contains the metadata of how often the data is collected on the backend. In some cases, the SensingInterval may be configurable.

Schema: https://redfish.dmtf.org/schemas/v1/MetricDefinition.v1_3_2.json

Example MetricDefintion GET request:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStat
s | jq
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats",
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Stastics for all measurable components that cover any Environmental factors such as Power,
Current, Voltage, etc. SensingInterval of 1800 seconds (30 minutes) by default.",
  "Id": "EnvironmentalStats",
  "Name": "EnvironmentalStats MetricDefinition",
  "SensingInterval": "1800"
}
```

Details of the response:

- Description: this provides a high level description of the full data set provided by this MetricDefinition resource
- Id: this provides a unique Id to define the data set provided by this MetricDefinition – when subscribing to receive this data set, the EventDestination.MetricReportDefinitions “data_set” field in the request body must contain a valid MetricDefinition.Id that exists

- SensingInterval: The time interval between when a metric is updated – this represents the backend interval at which data is collected, which is configurable for certain MetricDefinition resources and is not configurable for others. The unit is in seconds.

14.2.1 Updating MetricDefinition Resources

As mentioned above, there are certain MetricDefinition resources in the supported MetricDefinitions collection that *are configurable* and some that *are not configurable.* For the current set of MetricDefinition resources in the MetricDefinitions collection, the only resource which is configurable, is the EnvironmentalStats MetricDefinition resource. The other three MetricDefinition resources, LustreStats, LinuxStats, JobStats, are not configurable.

When configuring a MetricDefinition resource, there is really only one field that has any functional affect on the system when changed, and that is: MetricDefinition.SensingInterval. As mentioned above, the SensingInterval field represents the interval at which a data set is collected. Each MetricDefinition resource has a default SensingInterval value, which for the EnvironmentalStats MetricDefinition resource, can be updated within a minimum value restriction.

14.2.1.1 Updating MetricDefinition Resource via a PUT request

One mechanism to update an existing MetricDefinition resource, if it is configurable, is to send a PUT request to the resource with updates in the Request body. By definition, the HTTP PUT request method creates a new resource, or replaces a representation of the target resource entirely with the request payload. For the /redfish/v1/TelemetryService/MetricDefinitions/{MetricDefinitionId} PUT request implementation, it is expected that the request payload will replace an existing target resource.

Example GET request to resource before updates:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStat
s | jq
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats",
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Stastics for all measurable components that cover any Environmental factors such as Power,
Current, Voltage, etc. SensingInterval of 1800 seconds (30 minutes) by default.",
  "Id": "EnvironmentalStats",
  "Name": "EnvironmentalStats MetricDefinition",
  "SensingInterval": "1800"
}
```

Example Request body payload:

```
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats",
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Stastics for all measurable components that cover any Environmental factors such as Power,
Current, Voltage, etc. SensingInterval of 1800 seconds (30 minutes) by default.",
  "Id": "EnvironmentalStats",
  "Name": "EnvironmentalStats MetricDefinition",
  "SensingInterval": "300"
}
```

Note: the SensingInterval field is in the request body payload has been changed from the existing resource.

Example PUT request to the resource:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}" -X PUT -d
@envstats_metric_definition_put.json
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalSt
ats | jq
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats",
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Stastics for all measurable components that cover any Environmental factors such as Power,
Current, Voltage, etc. SensingInterval of 1800 seconds (30 minutes) by default.",
  "Id": "EnvironmentalStats",
  "Name": "EnvironmentalStats MetricDefinition",
  "SensingInterval": "300"
}
```

Example GET request to resource after updates:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalSt
ats | jq
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats",
```

```
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Stastics for all measurable components that cover any Environmental factors such as Power,
 Current, Voltage, etc. SensingInterval of 1800 seconds (30 minutes) by default.",
  "Id": "EnvironmentalStats",
  "Name": "EnvironmentalStats MetricDefinition",
  "SensingInterval": "300"
}
```

The SensingInterval of the EnvironmentalStats MetricDefinition has now been updated from 1800 to 300, changing the rate at which the data set is collected from the system from 30 minutes (1800s) default, to a 5 minute (300s) interval.

14.2.1.2 Updating MetricDefinition Resource via a PATCH request

Another mechanism to update an existing MetricDefinition resource, if it is configurable, is to send a PATCH request to the resource with updates in the Request body. By definition, the HTTP PATCH request method creates *partial* changes to an existing resource. For the /redfish/v1/TelemetryService/MetricDefinitions/{MetricDefinitionId} PATCH request implementation, it is expected that the request payload will replace some number of fields in an existing resource, but not the entire resource like PUT does.

Example GET request to resource before updates:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStat
s | jq
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats",
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Stastics for all measurable components that cover any Environmental factors such as Power,
Current, Voltage, etc. SensingInterval of 1800 seconds (30 minutes) by default.",
  "Id": "EnvironmentalStats",
  "Name": "EnvironmentalStats MetricDefinition",
  "SensingInterval": "1800"
}
```

Example Request body payload:

```
{
  "SensingInterval": "300"
}
```

Note: the SensingInterval field is in the request body payload has been changed from the existing resource, and is the only field being updated with the PATCH request

Example PUT request to the resource:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}" -X PATCH -d
@envstats_metric_definition_patch.json
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalSt
ats | jq
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats",
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Stastics for all measurable components that cover any Environmental factors such as Power,
Current, Voltage, etc. SensingInterval of 1800 seconds (30 minutes) by default.",
  "Id": "EnvironmentalStats",
  "Name": "EnvironmentalStats MetricDefinition",
  "SensingInterval": "300"
}
```

Example GET request to resource after updates:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalSt
ats | jq
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats",
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Stastics for all measurable components that cover any Environmental factors such as Power,
Current, Voltage, etc. SensingInterval of 1800 seconds (30 minutes) by default.",
  "Id": "EnvironmentalStats",
  "Name": "EnvironmentalStats MetricDefinition",
  "SensingInterval": "300"
}
```

The SensingInterval of the EnvironmentalStats MetricDefinition has now been updated from 1800 to 300, changing the rate at which the data set is collected from the system from 30 minutes (1800s) default, to a 5 minute (300s) interval.

14.2.1.2 Restrictions for Updating MetricDefinition Resources

There are a few basic restrictions for updating MetricDefinition resources via a PUT or PATCH request. If any of these conditions are broken, the request will not be successful and the appropriate error message and response code will be returned:

- Update request must be to an existing resource
- Update requests cannot update resources which are deemed not configurable (see above)
- Update request body must actually contain fields that update the existing resource (PUT or PATCH which results in no updates to existing resource is invalid)
- The MetricReport.Description field cannot be empty in the update request body payload
- The MetricDefinition.Id must match the {MetricDefinitionId} in the request endpoint
- The MetricDefintion.SensingInterval must be in seconds
- The MetricDefinition.SensingInterval cannot be < 60 seconds for EnvironmentalStats

14.2.2 Creating MetricDefinition Resources

MetricDefinition resources can be created to add to the MetricDefintions collection via a POST request to the /redfish/v1/TelemetryService/MetricDefinitions/{MetricDefintionId} endpoint where the request body payload defines the new MetricDefinition resource content. The MetricDefinition.Id should have a meaningful value that represents the data set that this MetricDefinition may represent, as well as a meaningful description of the data set that this MetricDefinition would represent. The MetricDefinition.SensingInterval should also be provided to set a backend collection interval of that data set.

Currently, clients may only decide to create a new MetricDefinition resources if they are planning to add their own backend data collector element controller to the API. Creation of this new MetricDefinition would be one of the steps that allows clients to subscribe to receive some new custom data set, which ties API knowledge of the new data set to the custom data collector. All of this is described more detail in another document, but the follow shows the actual creation of the new MetricDefinition resource via a POST request:

Example GET request to MetricDefinitions Collection resource before creating a new resource:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions | jq
{
  "@odata.context": "MetricDefinitionCollection.MetricDefinitionCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions",
  "@odata.type": "MetricDefinitionCollection.v1_0_0.MetricDefinitionCollection",
```

```
  "Description": "MetricDefinitionCollection defines the collection of supported MetricDefinition resources, which
describe all supported API Metrics available.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/JobStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats"
    }
  ],
  "Members@odata.count": 4,
  "Name": "MetricDefinitionCollection"
}
```


Request body payload:


```
  {
    "@odata.context": "MetricDefinition.MetricDefinition",
    "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/NetworkStats",
    "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
    "Description": "Network throughput and performance statistics from high speed network on the system.
  SensingInterval of 300 seconds by default.",
    "Id": "NetworkStats",
    "Name": "NetworkStats MetricDefinition",
    "SensingInterval": "300"
  }
```


Details of the request body:

- Id: provides a new unique Id to define this MetricDefinition, cannot previously exist
- Description: cannot be empty
- SensingsInterval: cannot be empty

Example POST request to create the NetworkStats MetricDefinition above:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}" -X POST -d
@networkstats_metric_definition.json
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/NetworkSt
ats | jq
{
```

```
    "@odata.context": "MetricDefinition.MetricDefinition",
    "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/NetworkStats",
    "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
    "Description": "Network throughput and performance statistics from high speed network on the system.
  SensingInterval of 300 seconds by default.",
    "Id": "NetworkStats",
    "Name": "NetworkStats MetricDefinition",
    "SensingInterval": "300"
  }
```


Example GET request to MetricDefinitions Collection resource after creating a new resource:


```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions | jq
{
  "@odata.context": "MetricDefinitionCollection.MetricDefinitionCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions",
  "@odata.type": "MetricDefinitionCollection.v1_0_0.MetricDefinitionCollection",
  "Description": "MetricDefinitionCollection defines the collection of supported MetricDefinition resources, which
describe all supported API Metrics available.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/JobStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/NetworkStats"
    }
  ],
  "Members@odata.count": 5,
  "Name": "MetricDefinitionCollection"
}
```


Note: the new NetworkStats MetricDefinition is now part of the MetricDefinitions collection.

14.2.2 Removing MetricDefinition Resources

MetricDefinition resources can be removed by sending a DELETE request to the target /redfish/v1/TelemetryService/MetricDefinitions/{MetricDefinitionId} resource you'd like to remove. DELETE requests do not require a request body. Removing a MetricDefinition resource can only be done on MetricDefinition resources that were created, beyond the default supported set of resources in the MetricDefinitions collection, i.e. – you cannot remove a MetricDefinition in the supported set of: JobStats, LustreStats, LinuxStats, EnvironmentalStats, but you can remove a MetricDefinition that was created and no longer needed.

Example GET request to MetricDefinitions Collection resource before removing the created NetworkStats MetricDefinition resource:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions | jq
{
  "@odata.context": "MetricDefinitionCollection.MetricDefinitionCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions",
  "@odata.type": "MetricDefinitionCollection.v1_0_0.MetricDefinitionCollection",
  "Description": "MetricDefinitionCollection defines the collection of supported MetricDefinition resources, which
describe all supported API Metrics available.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/JobStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/NetworkStats"
    }
  ],
  "Members@odata.count": 5,
  "Name": "MetricDefinitionCollection"
}
```

Example DELETE request to remove the created NetworkStats MetricDefinition resource:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}" -X DELETE
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions/NetworkStats
| jq
{
  "@odata.context": "MetricDefinition.MetricDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/NetworkStats",
  "@odata.type": "MetricDefinition.v1_2_0.MetricDefinition",
  "Description": "Network throughput and performance statistics from high speed network on the system.
SensingInterval of 300 seconds by default.",
  "Id": "NetworkStats",
  "Name": "NetworkStats MetricDefinition",
  "SensingInterval": "300"
}
```

Example GET request to MetricDefinitions Collection resource after removing the created NetworkStats MetricDefinition resource:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/MetricDefinitions | jq
{
  "@odata.context": "MetricDefinitionCollection.MetricDefinitionCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions",
  "@odata.type": "MetricDefinitionCollection.v1_0_0.MetricDefinitionCollection",
  "Description": "MetricDefinitionCollection defines the collection of supported MetricDefinition resources,
which describe all supported API Metrics available.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/JobStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats"
    }
  ],
  "Members@odata.count": 4,
  "Name": "MetricDefinitionCollection"
}
```

14.3 MetricReportDefinition Resources

In the TelemetryService resource, the MetricReportDefinitions collection holds MetricReportDefinition resources which define how and when MetricReports for data in a given MetricDefinition data set are generated by the API. A MetricReport which is created by a redfish service, can be transmitted using the Redfish EventService, stored locally for retrieval, or both. A MetricReport may be created periodically and propagated at a given Recurrence Interval, when a given reading value changes, or upon GET request. MetricReportDefinition resources provide the means to define and configure all of these details, and have a coupled relationship with MetricDefinition resources and MetricReport resources.

14.3.1 MetricReportDefinitionCollection Resource

The MetricReportDefinition Collection resource provides a collection of all supported MetricReportDefinition resources. Each MetricReportDefinition resource is intended to provide a definition of how and when MetricReports for the MetricDefinition in the given MetricReportDefinition.Metrics list are generated, as well as how these MetricReports are propagated or made available to clients. The MetricReportCollection resource is also used to check against subscription request, that the EventDestination.MetricReportDefinitions endpoint a client is subscribing for exists and is valid.

By default, dp-api TelemetryService will generate a MetricReportDefinition for each native MetricDefinition resource that is “bound” by that MetricDefintion resource, see section 14.5.1 “Default MetricReportDefinitions” below for more details.

Schema: https://redfish.dmtf.org/schemas/v1/MetricReportDefinitionCollection.json

Example GET request response:

```
]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/TelemetryService/MetricReportDefinitions | jq
{
  "@odata.context": "MetricReportDefinitionCollection.MetricReportDefinitionCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions",
  "@odata.type": "MetricDefinitionCollection.v1_0_0.MetricDefinitionCollection",
  "Description": "MetricReportCollection defines the collection of supported MetricReportDefinition
resources, which describe propagation of MetricReports generated by API services",
  "Members": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/LustreStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/LinuxStats"
    },
```

```
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/JobStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/EnvironmentalStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/TieringStats"
    }
  ],
  "Members@odata.count": 5,
  "Name": "MetricReportCollection"
}
```


Details of the response:

- The Members field of this resource holds the collection of all supported MetricReportDefinition resource endpoints
- The Members@odata.count shows how many total MetricReportDefinition resources exist in the collection

14.3.2 MetricReportDefinition resource

As mentioned before, MetricReportDefinition resources are intended to provide a definition of how and when MetricReports for the MetricDefinition in the given MetricReportDefinition.Metrics list are generated, as well as how these MetricReports are propagated or made available to clients. The following fields in the MetricReportDefinition must be defined in order to configure this:

- MetricReportDefinitionEnabled – Indicates whether generation of the defined MetricReport is enabled or not
- MetricReportDefinitionType – Indicates when the defined MetricReport is generated
- Metrics – Contains the list of Metrics this MetricReport will provide (this is set by a valid MetricDefinition endpoint)
- ReportActionsEnum – contains the set of actions for the service to perform when the defined MetricReport is generated
- ReportUpdates – defines the handling of subsequent MetricReports when a MetricReport of the same Id already exists
- Schedule – defines the recurrence interval of each MetricReport generation

Each of these properties will be further detailed for the ClusterStor Redfish/Swordfish API implementation in sub-sections below.

Schema: https://redfish.dmtf.org/schemas/v1/MetricReportDefinition.v1_4_7.json

Example GET response for a default MetricReportDefinition resource:

```
]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/TelemetryService/MetricReportDefinitions/LustreStats | jq
{
  "@odata.context": "MetricReportDefinition.MetricReportDefinition",
  "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/LustreStats",
  "@odata.type": "MetricReportDefinition.v1_4_1.MetricReportDefinition",
  "Description": "Default MetricReport resource definition for propagation of the LustreStats data set to the
/redfish/v1/TelemetryService/MetricReports collection to be consumed",
  "Id": "LustreStats",
  "Links": {},
  "MetricReportDefinitionEnabled": true,
  "MetricReportDefinitionType": "Periodic",
  "Metrics": [
    {
      "MetricId": "LustreStats",
      "MetricProperties": [
        "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
      ]
    }
  ],
  "Name": "LustreStats MetricReportDefinition",
  "ReportActions": [
    "RedfishEvent"
  ],
  "ReportUpdates": "Overwrite",
  "Schedule": {
    "Name": "default",
    "RecurrenceInterval": "5"
  },
  "Status": {}
}
```

Details of the response:

- MetricReportDefinitionEnabled – set to true, this means MetricReports defined in this resource will be generated
- MetricReportDefinitionType – set to “periodic”, this means that MetricReports defined in this resource will be generated at a periodic time interval, specified in the Schedule property

- Metrics – the Metrics list contains one Metric, defined by the LustreStats MetricDefintion resource, which will be provided in the generated reports
- ReportActions – set to “RedfishEvent” indicating EventService will send an Event message containing the MetricReport
- ReportUpdates – set to “Overwrite” indicating this MetricReport will be overwritten in the /redfish/v1/TelemetryService/MetricReports collection on each generation
- Schedule – has a Name of “default” indicating it is the “default” MetricReportDefinition that is bound by the associated MetricDefinition it provides, and RecurrenceInterval of 5 (set my LustreStats.SensingInterval from the MetricDefintion)

14.3.2.1 MetricReportDefinition.MetricReportDefinitionEnabled Property

The MetricReportDefinitionEnabled property is a boolean property that controls whether or not MetricReports defined by this MetricReportDefinition should be generated or not. For all default MetricReportDefinitions (which are bound by the associated MetricDefinition) – this is set to true on startup, but can be disabled with a PUT or PATCH request to the resource. For any subsequently created MetricReportDefinitons which are not part of the default set, this can be set to true or false depending on the desired behavior.

When MetricReportDefinitionEnabled is set to true, the MetricReports defined by this resource will be generated each Schedule period and propagated to the /redfish/v1/Telemetry/MetricRerports collection – if it is set to false, this report generation will stop.

14.3.2.2 MetricReportDefinition.MetricReportDefinitionType Property

TheMetricReportDefinitionType property defines when, or the conditions under which, the MetricReport defined by the MetricReportDefinition is generated. It is an enum with the following types available. From the MetricReport Schema:

```
"MetricReportDefinitionType": {
   "description": "Indicates when the metric report is generated.",
   "enum": [
      "Periodic",
      "OnChange",
      "OnRequest"
   ],
   "enumDescriptions": {
      "OnChange": "The metric report is generated when any of the metric values change.",
      "OnRequest": "The metric report is generated when a HTTP GET is performed on the specified
metric report.",
```

```
      "Periodic": "The metric report is generated at a periodic time interval, specified in the Schedule
property."
    },
    "longDescription": "This type shall specify when the metric report is generated.",
    "type": "string"
}
```

At this time, the ClusterStor Redfish/Swordfish API implements MetricReportDefinitions with a MetricReportDefinitionType of "Periodic" – which indicates that these reports will be generated at a periodic time interval defined by the Schedule property. Other types are currently not supported.

14.3.2.3 MetricReportDefinition.Metrics Property

The Metrics property specifies the set of Metrics which this report will provide. Each item in the list should be that of the Metric scheam which defines:

- MetricId – the Id of the Metric
- MetricProperties – the Properties of the Metric

At this time, only a single Metric may be specified in the Metrics list, and the MetricProperties list should contain a valid MetricDefinition endpoint which describes the data that this report will provide. For example:

```
  "Metrics": [
    {
      "MetricId": "LustreStats",
      "MetricProperties": [
        "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
      ]
    }
  ],
```

The above Metrics specification states that MetricReports generated by this MetricReportDefinition will contain data from the data set defined by the LustreStats MetricDefinition only. In the future, we may support allowing multiple Metrics to be defined in a single MetricReportDefinition, and part of a single MetricReport.

14.3.2.4 MetricReportDefinition.ReportActions Property

The ReportActions field specifies actions that will be performed when a MetricReport is generated. From the MetricReportDefinition Schema, the field is an enum with the following allowable values:

```
"ReportActionsEnum": {
   "description": "Actions to perform when a metric report is generated.",
   "enum": [
      "LogToMetricReportsCollection",
      "RedfishEvent"
   ],
   "enumDescriptions": {
      "LogToMetricReportsCollection": "Record the occurrence to the metric report collection.",
      "RedfishEvent": "Send a Redfish event message containing the metric report."
   },
   "enumLongDescriptions": {
      "LogToMetricReportsCollection": "This value shall indicate the service records the occurrence to the
metric report collection found under the telemetry service.  The service shall update the metric report
based on the setting of the ReportUpdates property.",
      "RedfishEvent": "This value shall indicate the service sends a Redfish event of type MetricReport to
subscribers in the event subscription collection of the event service."
   },
   "longDescription": "This type shall specify the actions to perform when a metric report is generated.",
   "type": "string"
},
```

At this time, the ClusterStor Redfish/Swordfish API implements the ReportActions of type “RedfishEvent”, such that, an Event is sent so subscribed clients with the content of the MetricReport as each one is generated.

14.3.2.5 MetricReportDefinition.ReportUpdates Property

The ReportUpdates property specifies the handling of subsequent MetricReports when a MetricReport exists. From the MetricReportDefinition Schema, the field is an enum with the following allowable values:

```
"ReportUpdatesEnum": {
   "description": "Handling of subsequent metric reports when a metric report exists.",
   "enum": [
      "Overwrite",
      "AppendWrapsWhenFull",
      "AppendStopsWhenFull",
      "NewReport"
   ],
   "enumDescriptions": {
```

"AppendStopsWhenFull": "New information is appended to the metric report. The service stops adding entries when the metric report has reached its maximum capacity.",
"AppendWrapsWhenFull": "New information is appended to the metric report. The metric report entries are overwritten with new entries when the metric report has reached its maximum capacity.",
"NewReport": "A new metric report is created, whose resource name is the metric report resource name concatenated with the timestamp.",
"Overwrite": "Overwrite the metric report."
},
"enumLongDescriptions": {
"AppendStopsWhenFull": "This value shall indicate the service appends new information to the metric report referenced by the MetricReport property. The service shall stop adding entries when the metric report has reached its maximum capacity. The State property within Status should be set to `Disabled` and the MetricReportDefinitionEnabled property should be set to `false` when the append limit is reached.",
"AppendWrapsWhenFull": "This value shall indicate the service appends new information to the metric report referenced by the MetricReport property. The service shall overwrite entries in the metric report with new entries when the metric report has reached its maximum capacity.",
"NewReport": "This value shall indicate the service creates a new metric report resource, whose resource name is the metric report resource name concatenated with the timestamp.",
"Overwrite": "This value shall indicate the service overwrites the metric report referenced by the MetricReport property."
},
"longDescription": "This type shall indicate how the service handles subsequent metric reports when a metric report exists.",
"type": "string"
},

At this time, the ClusterStor Redfish/Swordfish API implements the ReportUpdates type of "overwrite", such that, when a MetricReport is generated by and API service, it will overwrite the MetricReport resource in the /redfish/v1/TelemetryService/MetricReports collection which matches its MetricReport.Id. This overwrite occurs every Schedule Recurrence interval, such that, the latest published MetricReport is always available in the collection. At this time this is the only ReportUpdates value supported, but we may consider others in the future.

14.3.2.6 MetricReportDefinition.Schedule Property

The Scheule property defines a Schedule at which a MetricReport is generated. The Schedule should be populated with a name which identifies that Schedule, such that, it can be re used between multiple MetricReportDefinitions, as well as, a RecurrenceInterval which defines the interval of time between each report generation. For initial implementation, it is expected that the RecurrenceInterval be specified in seconds, but in the future, we plan to accept any such time format.

By default, on startup, the API will generate a “default” MetricReportDefinition for each “default MetricDefinition, such that, the MetricReportDefinition has a Schedule.Name of “default” and a Schedule.RecurrenceInterval set to the MetricDefinition.SensingInterval which the report provides (this way the MetricReport is generated each time the Metric data is collected on the backend). If users would like to receive data at a different rate, it is possible to create another MetricReportDefinition which specifies a different schedule. More support for this will be available in the future.

14.4 MetricReport Resources

In the TelemetryService resource, the MetricReports collection holds MetricReport resources which contain Metric information generated as a report by API services. If a MetricReport is propagated to the MetricReportCollection resource: /redfish/v1/TelemetryService/MetricReports, it is then available to be consumed via GET request to the /redfish/v1/TelemetryService/MetricReports/{MetricReportId} resource which contains the information a client is interested in, or, can be subscribed to via an EventDestination subscription with EventTypes of MetricReport, which specifies the MetricReportDefinition resource that describes the reports a client wishes to receive.

14.4.1 MetricReportCollection Resource

The MetricReports Collection resource provides a collection of all generated MetricReport resources. If a MetricReportDefinition contains the “LogToMetricReportsCollection” value in its ReportActions list, the MetricReport it generates will be propagated to the /redfish/v1/TelemetryService/MetricReports collection, and will be persistently stored, and updated as defined by the MetricReportDefinition which generates it.

Schema: https://redfish.dmtf.org/schemas/v1/MetricReportCollection.json

Example GET request to the MetricReportCollection resource:

```
]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/TelemetryService/MetricReports | jq
{
  "@odata.context": "MetricReportCollection.MetricReportCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricReports",
  "@odata.type": "MetricReportCollection.v1_0_0.MetricReportCollection",
  "Description": "MetricReportCollection published by API services",
  "Members": [
    {
```

```
    "@odata.id": "/redfish/v1/TelemetryService/MetricReports/kjlmo17n000-IPMI-MXE3000018VVS001-
EnvironmentalStats"
   },
   {
    "@odata.id": "/redfish/v1/TelemetryService/MetricReports/kjlmo17n000-LinuxStats"
   },
...
   {
    "@odata.id": "/redfish/v1/TelemetryService/MetricReports/kjlmo17n008-LinuxStats"
   },
   {
    "@odata.id": "/redfish/v1/TelemetryService/MetricReports/kjlmo17n009-LinuxStats"
   }
  ],
  "Members@odata.count": 26,
  "Name": "MetricReportCollection"
}
```


Details of the response:

- The Members field of this resource holds the collection of all supported MetricReport resource endpoints
- The Members@odata.count shows how many total MetricReport resources exist in the collection

14.4.2 MetricReport Resource

The MetricReport resource contains a report of a set of Metrics defined by an associated MetricReportDefinition resource. It will contain a MetricReportDefinition field which defines that MetricReportDefinition which generated the MetricReport, as well as a MetricValues list which contains all of the individual Metrics contained within the report in the form of MetricValue schemas (as seen in the MetricReport Schema below). Each MetricValue will have:

- MetricId – the name or identifier of the metric
- MetricValue – the value of the Metric
- Oem – section containing units and other miscellaneous data or metadata about the metric

MetricReport.Id – the MetricReport.Id must identify the hostname which generated the report, and the MetricDefintion the report contains. It will be of the form: “<hostname>-<metric definition id>”, for most MetricDefinitions, with the exception of

EnviromentalStatus MetricDefinition, which will have the form: “<hostname>-<enclosure type>-<enclosure serial number>-<MetricDefinition>”

Schema: https://redfish.dmtf.org/schemas/v1/MetricReport.v1_5_2.json

Example GET request of a LinuxStats MetricReport from a single hostname:

```
]# curl -Ssk -H "X-Auth-Token: $RFSF_API_TOKEN"
https://localhost:8081/redfish/v1/TelemetryService/MetricReports/kjlmo17n004-LinuxStats | jq
{
  "@odata.context": "MetricReport.MetricReport",
  "@odata.id": "/redfish/v1/TelemetryService/MetricReports/kjlmo17n004-LinuxStats",
  "@odata.type": "MetricReport.v1_4_2.MetricReport",
  "Id": "kjlmo17n004-LinuxStats",
  "MetricReportDefinition": {
    "@odata.id": "/redfish/v1/TelemetryService/MetricReportDefinitions/LinuxStats"
  },
  "MetricValues": [
    {
      "MetricId": "TotalMemory",
      "MetricValue": "250.6876",
      "Oem": {
        "category": "performance",
        "hostname": "kjlmo17n004",
        "origin": "kjlmo17n004",
        "type": "node",
        "units": "GB",
        "value": "linux"
      },
      "Timestamp": "1752617892"
    },
    {
      "MetricId": "AvailableMemory",
      "MetricValue": "130.5104",
      "Oem": {
        "category": "performance",
        "hostname": "kjlmo17n004",
        "origin": "kjlmo17n004",
        "type": "node",
        "units": "GB",
        "value": "linux"
      },
      "Timestamp": "1752617892"
    },
...
```

```
    {
      "MetricId": "InterruptsPerSecond",
      "MetricValue": "4369.9943",
      "Oem": {
        "category": "performance",
        "hostname": "kjlmo17n004",
        "origin": "kjlmo17n004",
        "type": "node",
        "units": "",
        "value": "linux"
      },
      "Timestamp": "1752617892"
    }
  ],
  "Name": "kjlmo17n004-LinuxStats MetricDefinition",
  "Oem": {},
  "Timestamp": "2025-07-15T16:19:12-06:00"
}
```


14.6 Triggers Resources

The **Triggers** resource provides a centralized way to define and manage the threshold rules for monitoring specific metric properties, i.e. TelemetryService/MetricDefinitions/{MetricDefinitionId}. These triggers allow you to set upper or lower limits on metric values and define automated actions when those limits are reached. For example, In ClusterStor when a threshold is exceeded, the system can generate an alert via the Event Service. Instead of configuring thresholds individually within each resource that contains a metric, you can create a common set of triggers and apply them to multiple metric properties. This approach simplifies configuration, ensures consistency, and reduces duplication across resources.

14.6.1 TriggersCollection Resource

The Triggers Collection resource contains all supported Triggers resources. Each Triggers resource describes thresholds and the actions that subscribers will receive when those thresholds are exceeded. Currently, only the GET method is supported for the Triggers Collection and Triggers are created only for the LinuxStats MetricDefinition resource. The TriggerId follows the format <MetricDefinitionId>-<TriggerName>-<NodeRole>. For example: LinuxStats-CpuUsage-All.

Schema: https://redfish.dmtf.org/schemas/v1/TriggersCollection.json

Example TriggersCollection GET request:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/Triggers | jq
{
  "@odata.context": "MetricDefinitionCollection.MetricDefinitionCollection",
  "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions",
  "@odata.type": "MetricDefinitionCollection.v1_0_0.MetricDefinitionCollection",
  "Description": "MetricDefinitionCollection defines the collection of supported MetricDefinition resources,
which describe all supported API Metrics available.",
  "Members": [
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LustreStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/LinuxStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/JobStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/EnvironmentalStats"
    },
    {
      "@odata.id": "/redfish/v1/TelemetryService/MetricDefinitions/TieringStats"
    }
  ],
  "Members@odata.count": 5,
  "Name": "MetricDefinitionCollection"
}
```

Details of the response:

- The Members field of this resource holds the collection of all supported Triggers resource endpoints
- The Members@odata.count shows how many total Triggers resources exist in the collection

14.6.2 Triggers Resources

The Triggers resource includes the NumericThresholds property, which defines the thresholds for a numeric trigger to activate. Currently, four numeric thresholds are

supported: LowerCaution, LowerWarning, UpperCaution, and UpperWarning. When a trigger is activated, a Redfish event is sent to the subscribers.

Schema: https://redfish.dmtf.org/schemas/v1/Triggers.v1_3_2.json

Example Triggers GET request:

```
[root@kjlmo12-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/TelemetryService/Triggers/LinuxStats-CpuUsage-All
 | jq
{
  "@odata.context": "Triggers.Triggers",
  "@odata.id": "/redfish/v1/TelemetryService/Triggers/LinuxStats-CpuUsage-All",
  "@odata.type": "Triggers.v1_1_4.Triggers",
  "Id": "LinuxStats-CpuUsage-All",
  "Links": {},
  "MetricType": "Numeric",
  "Name": "LinuxStats-CpuUsage-All",
  "NumericThresholds": {
    "LowerCaution": {},
    "LowerCautionUser": {},
    "LowerCritical": {},
    "LowerCriticalUser": {},
    "LowerFatal": {},
    "UpperCaution": {
      "DwellTime": "300",
      "Reading": 80
    },
    "UpperCautionUser": {},
    "UpperCritical": {
      "DwellTime": "300",
      "Reading": 90
    },
    "UpperCriticalUser": {},
    "UpperFatal": {}
  },
  "Status": {},
  "TriggerActions": [
    "RedfishEvent"
  ]
}
```

Details of the response:

- UpperCaution: is the reading is above normal range.
- UpperCritical: is the reading is above normal range and requires attention
- LowerCaution: is the reading is below normal range
- LowerCritical: is the reading is below normal range and requires attention
- Reading: value of the threshold

- DwellTime: is the duration in Seconds that value must be over the threshold before the trigger is tripped.

Example Event when a Triggers resource is tripped:

```
[root@kjlmo9-oem rest_api]# curl -k -H "X-Auth-Token: ${AUTH}"
https://kjlmo900.hpc.amslabs.hpecorp.net:8081/redfish/v1/Events/fef1d0836-2663-49bc-9215-55cf7cd74421 | jq
```


```
{
  "@odata.context": "/redfish/v1/$metadata/#Event.Event",
  "@odata.type": "#Event.v1_5_0.Event",
  "Context": "hw-ec",
  "Events": [
    {
      "Actions": {},
      "EventId": "ef1d0836-2663-49bc-9215-55cf7cd74421",
      "EventTimestamp": "2025-06-12T05:42:20-06:00",
      "EventType": "Alert",
      "MemberId": "0",
      "Message": "ComputerSystem kjlmo9:Node-R1C1-22U-A has CpuUsage over 90.00% for 300s",
      "MessageArgs": [
        "CPU Utilization"
      ],
      "MessageId": "TelemetryService.Triggers.CpuUsage",
      "OriginOfCondition": {
        "@odata.id": "/redfish/v1/StorageSystems/kjlmo9:Node-R1C1-22U-A"
      },
      "Severity": "Critical"
    }
  ],
  "Events@odata.count": 1,
  "Id": "ef1d0836-2663-49bc-9215-55cf7cd74421",
  "Name": "hw-ec Event"
}
```


15. UpdateService Resource

The UpdateService resource lives directly under the ServiceRoot /redfish/v1 and can be seen by issuing a GET request to /redfish/v1/UpdateService. It provides an entry point to the SoftwareInventory Collection to see version information about various API software components, as well as top-level API version information.

Schema: https://redfish.dmtf.org/schemas/v1/UpdateService.v1_11_2.json

Example UpadateService GET request:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" https://kjlmo1200:8081/redfish/v1/UpdateService | jq
{
  "@odata.context": "UpdateService.UpdateService",
  "@odata.id": "/redfish/v1/UpdateService",
  "@odata.type": "UpdateService.v1_9_0.UpdateService",
  "Description": "UpdateService for RFSF API",
  "HttpPushUriOptions": {
    "HttpPushUriApplyTime": {}
  },
  "Id": "UpdateService",
  "Name": "UpdateService",
  "ServiceEnabled": true,
  "SoftwareInventory": {
    "@odata.id": "/redfish/v1/UpdateService/SoftwareInventory"
  },
  "Status": {}
}
```

Details of the response: There is not much detail provided to see in the response output. The important thing to note is that it provides the path to the SoftwareInventory Collection to work further down the resource tree to discover the software version information of the top-level API as well as all API service components.

15.1 SoftwareInventoryCollection Resource

The SoftwareInventory Collection resource provides a collection of individual SoftwareInventory resource odata.ids in the SoftwareInventory.Members field. This holds the endpoints of all SoftwareInventory resources that contain Software version information about the API.

Schema: https://redfish.dmtf.org/schemas/v1/SoftwareInventoryCollection.json

Example SoftwareInventoryCollection GET request:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}" https://kjlmo1200:8081/redfish/v1/UpdateService/SoftwareInventory |
jq
{
  "@odata.context": "SoftwareInventoryCollection.SoftwareInventoryCollection",
  "@odata.id": "/redfish/v1/UpdateService/SoftwareInventory",
  "@odata.type": "SoftwareInventoryCollection.v1_0_0.SoftwareInventoryCollection",
  "Description": "SoftwareInventoryCollection for RFSF API",
  "Members": [
    {
```

```
    "@odata.id": "/redfish/v1/UpdateService/rfsf-hw-ec"
  },
  {
    "@odata.id": "/redfish/v1/UpdateService/rfsf-base-libs"
  },
  {
    "@odata.id": "/redfish/v1/UpdateService/rfsf-raid-mgmt-ec"
  },
  {
    "@odata.id": "/redfish/v1/UpdateService/rfsf-monitoring-ec"
  },
  {
    "@odata.id": "/redfish/v1/UpdateService/rfsf-dm-mgmt-ec"
  },
  {
    "@odata.id": "/redfish/v1/UpdateService/rfsf-fs-mgmt-ec"
  },
  {
    "@odata.id": "/redfish/v1/UpdateService/rfsf-dp-api"
  },
  {
    "@odata.id": "/redfish/v1/UpdateService/api"
  }
 ],
 "Members@odata.count": 8,
 "Name": "SoftwareInventoryCollection"
}
```


Details of the response:The Members@odata.count field shows how many SoftwareInventory resources exist in the collection.

- The Members field holds the collection of all individual SoftwareInventory resource endpoints.
- The Members@odata.count field shows how many SoftwareInventory resources exist in the collection.

15.2 SoftwareInventory Resource

The SoftwareInventory resource contains version information about a particular piece of Software that makes up the RFSF API. In this implementation, there is a SoftwareInventory resource for each rpm installedd on the system to make up the API, as well as a resource that contains top-level version information about the API as a whole. There is not aton of detail in these resources, but can be useful to examine them to know which versions of the components of the API are running on the system, as well as the top-level API version.

Schema: https://redfish.dmtf.org/schemas/v1/SoftwareInventory.v1_9_0.json

Example SoftwareInventory GET request for individual API component:

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200:8081/redfish/v1/UpdateService/SoftwareInventory/rfsf-dp-api | jq
{
  "@odata.context": "SoftwareInventory.SoftwareInventory",
  "@odata.id": "/redfish/v1/UpdateService/SoftwareInventory/rfsf-dp-api",
  "@odata.type": "SoftwareInventory.v1_4_0.SoftwareInventory",
  "Description": "A RESTful API server that provides routers to redfish/swordfish endpoints, as well as request
handlers to populate certain higher level redfish/swordfish models to represent clusters, nodes, and enclosures",
  "Id": "rfsf-dp-api",
  "Manufacturer": "HPE",
  "Name": "rfsf-dp-api SoftwareInventory",
  "Status": {},
  "Version": "1.0.x.6.9.999-11.733b7284"
}
```

Details of the response:

- Description: description of the rpm package
- Id: Resource Id that matches the rpm package name name
- Manufacturer: producer of the rpm package
- Name: resource name that contains rpm package name
- Version: combination of rpm package version and release fields

Example SoftwareInventory GET request for top-level API

```
# curl -Ssk -H "X-Auth-Token: ${AUTH}"
https://kjlmo1200:8081/redfish/v1/UpdateService/SoftwareInventory/api | jq
{
  "@odata.context": "SoftwareInventory.SoftwareInventory",
  "@odata.id": "/redfish/v1/UpdateService/SoftwareInventory/api",
  "@odata.type": "SoftwareInventory.v1_4_0.SoftwareInventory",
  "Description": "Top level RFSF API version information. Holds the over-arching API version info to represent the
API as a whole.",
  "Id": "api",
  "Manufacturer": "HPE",
  "Name": "api SoftwareInventory",
  "Status": {},
  "Version": "v1.0.0"
}
```

Details of the response: the details of the response are the same as the previous example, however the information doesn’t derive directly from rpm packages, but rather describes API as a whole.

16. rfsf-api-client: Client-side libraries to interact with the RFSF API

The rfsf-api-client package provides a set of client-side libraries, examples, config files, etc. that can be used for programmatic setup and interaction with the RFSF API on any given target system. It is a python-based library that facilitates much of the manual setup described above, by providing libraries to:

- Define a user and API destination to target throughout the monitoring life cycle
- Authenticate that user with prompt for user password and internally store the token for all future requests
- Discover the full resource tree that exists in the API, including all endpoints of all sub-trees
- Subscribe to receive general Events at a specified EventDestination
- Subscribe to receive MetricReport Events at a specified EventDestination for: LustreStats, LinuxStatus and JobStats metrics.
- Example EventReceiver services
- Event based monitoring example
- Periodic Interval of resources monitoring example

The following sections will step through the code, config files, and runtime examples setup and use the rfsf-api-client libraries as described in the above bullets.

16.1. Populating the base config file to setup API destination and API user

The rfsf-api-client package provides a “base” config file in order to specify both the destination hostname or ip address of the Cluster where the API you’d like to interact with is running, as well as the API user you’d like to authenticate to use the API. The config file is located here: /etc/rfsf-api-client-base.config

By default the fields are empty, and must be populated:

```
[root@kjlmo17-oem hpc-sp-rfsf-api-client]# cat /etc/rfsf-api-client-base.config
# RFSF API setup configuration file

## specify information about target cluster here
[system]
# FQDN or IP address to connect to API on
destination =
```

```
## specify information about user to authenticate here
[authentication]
# username to authenticate for API use
user =
```

Example population of the rfsf-api-client-base.config file for destination: kjlmo1200.hpc.amslabs.hpecorp.net, user: dmatthews

```
[root@kjlmo17-oem hpc-sp-rfsf-api-client]# cat /etc/rfsf-api-client-base.config
# RFSF API setup configuration file

## specify information about target cluster here
[system]
# FQDN or IP address to connect to API on
destination = kjlmo1200.hpc.amslabs.hpecorp.net

## specify information about user to authenticate here
[authentication]
# username to authenticate for API use
user = dmatthews
```

The RfsfClient object initialization will read this file in to populate both the user and destination fields for the remainder of its lifetime, so they can be referenced and used in later steps. Example:

```
from rfsf_api_client.client.rfsf_api_client import RfsfClient, CONFIG_FILE_BASE
    rfsf_client = RfsfClient()
    print("Initialized base RfsfClient from config file: {0}".format(CONFIG_FILE_BASE))
    print("API destination...{0}".format(rfsf_client.destination))
    print("API user...{0}".format(rfsf_client.user))
```

16.2 Authenticate a user

Using the rfsf_api_client.authentication library

Once the RfsfClient object is initialized, you can then use the rfsf_api_client.authentication library to authenticate the RfsfClient.user which was previously defined and loaded from the base config file. To authenticate a user successfully, they must already be an existing cscli admins user on the cluster. See section 3 “Token based API Authentication” for more details. To perform the authentication, the rfsf_api_client.authentication.Auth object must be initialized with the RfsfClient.user from before. Once initialized, the “authenticate()” method may be called on the object. If successful, a token attribute will be populated automatically on the Auth object, this token attribute can then be assigned to the top-level RfsfClient.token field for user for the remainder of the lifetime of the RfsfClient object. As indicated in

the comments, the below code assumes you have already initialized the RfsfClient object for a target user and API destination.

Code example:

```
import sys
from rfsf_api_client.authentication.auth import Auth
    # note, the below assume you have already initialized the RfsfClient object as seen above
    # note, the below assumes you have already authenticated the rfsf_client.user and received a valid
rfsf_client.token
    response_content = None
    err_msg = None
    auth = Auth(rfsf_client.user)
    response_content, err_msg = auth.authenticate()
    if err_msg:
        print("Failed to authenticate user: {0}. {1}... exiting".format(rfsf_client.user, err_msg))
        sys.exit(1)
    else:
        print("Successfully authenticated user: {0} ".format(rfsf_client.user))
        print(response_content)
    # Assign auth token to top-level rfsf_client object
    rfsf_client.token = auth.token
```

The authenticate() method seen above, will prompt the user for a password to be entered to generate a Session request to send to the API. The username and password will be checked against local cluster LDAP instance to validate an existing admins user, for more details, again see section 3 “Token based API Authentication”. The authenticate method will either return a successful response or an error message. If request it sends is successful, the response will be the /redfish/v1/SessionService/Sessions/{SessionId} resource it created. If it is not successful, the error message will contain a description.

Here are some examples of running the code above and output you’d expect to see.

Example output when username / password cannot be validated **– as implied before, the output is shown to initialize RfsfClient:**

```
Initilized base RfsfClient from config file: /etc/rfsf-api-client-base.config
API destination...kjlmo1200.hpc.amslabs.hpecorp.net
API user...dmatthews
Password for user dmatthews:
Failed to authenticate user: dmatthews. "Bad response code 401 from
https://kjlmo1200.hpc.amslabs.hpecorp.net:8081/redfish/v1/SessionService/Sessions! {'error': {'code': '401',
'message': 'Failed to authorize Session for user: dmatthews. LDAP Result Code 49 \"Invalid Credentials\": '}}"...
exiting
```

Example output when username / password are valid and a Session resource is returned – as implied before, the output is shown to initialize RfsfClient as well:

```
Initialized base RfsfClient from config file: /etc/rfsf-api-client-base.config
API destination...kjlmo1200.hpc.amslabs.hpecorp.net
API user...dmatthews
Password for user dmatthews:
Successfully authenticated user: dmatthews
{
    "@odata.context": "Session.Session",
    "@odata.id": "/redfish/v1/SessionService/Sessions/K6NzwzgpkskbwHT8t9yCVV",
    "@odata.type": "Session.v1_3_0.Session",
    "Description": "Session for user dmatthews",
    "Id": "K6NzwzgpkskbwHT8t9yCVV",
    "Name": "dmatthews Session",
    "Oem": {
        "StartTime": "Mon Feb 13 19:26:31 UTC 2023",
        "ExpirationTime": "Tue Feb 14 07:26:31 UTC 2023",
        "TimeLeft": "11h59m59.999973521s"
    },
    "UserName": "dmatthews"
}
```

The above output indicates that the user 'dmatthews' has been authenticated to use the API and received a token to pass with future requests, details of the session are also returned as seen above.

16.3 Rfsf_api_client.inventory resource_tree discovery

Once a user is authenticated as seen above, one of the next important things to do is to discover the full resource tree that the API is tracking. To do this, the rfsf_api_client.inventory library provides a ResourceTree object which provides a get_resource_tree() method used to send several requests to the API to discover and build out a full tree of endpoints, such that every resource tree endpoint known to the API is discovered and tracked in the ResourceTree object. The ResourceTree object provides a tree field to hold the entire resource tree structure, but also provides fields for some of the "sub-tree" structures to make indexing into the tree to find a particular resource easier. The entire populated resource_tree object should then be assigned to the top-level RfsfClient.resource_tree field to be used throughout the monitoring lifecycle. The following code shows an example of initializing the ResourceTree object, populating it with get_resource_tree, and assigning that to the top-level RfsfClient object. As indicated in the comments, the below code assumes you have already initialized the RfsfClient object for a target user and API destination, as well as authenticated that user and received a valid token:

```
import sys
from rfsf_api_client.inventory.resource_tree import ResourceTree
    # note, the below assume you have already initialized the RfsfClient object as seen above
```

```
    # note, the below assumes you have already authenticated the rfsf_client.user and received a valid
rfsf_client.token
    resource_tree = ResourceTree(rfsf_client.token)
    err_msg = resource_tree.get_resource_tree()
    if err_msg:
        print("Failed to discover API resource tree with error: {0}... exiting".format(err_msg))
        sys.exit(1)
    else:
        print("Successfully discovered API resource tree!")
    # resource tree object assigned here
    # can now reference rfsf_client.resource_tree attributes
    rfsf_client.resource_tree = resource_tree
```

If successful, the rfsf_client.resource_tree can **then be used for monitoring in various ways, some examples will be shown below.**

Example output when resource_tree discovery is successful **– as implied before, the output is shown to initialize RfsfClient and authenticate a user as well:**

```
Initialized base RfsfClient from config file: /etc/rfsf-api-client-base.config
API destination...kjlmo1200.hpc.amslabs.hpecorp.net
API user...dmatthews
Password for user dmatthews:
Successfully authenticated user: dmatthews
{
    "@odata.context": "Session.Session",
    "@odata.id": "/redfish/v1/SessionService/Sessions/K6NzwzgpkskbwHT8t9yCVV",
    "@odata.type": "Session.v1_3_0.Session",
    "Description": "Session for user dmatthews",
    "Id": "K6NzwzgpkskbwHT8t9yCVV",
    "Name": "dmatthews Session",
    "Oem": {
        "StartTime": "Mon Feb 13 19:26:31 UTC 2023",
        "ExpirationTime": "Tue Feb 14 07:26:31 UTC 2023",
        "TimeLeft": "11h59m59.999973521s"
    },
    "UserName": "dmatthews"
}
Successfully discovered API resource tree!
```

16.4 Event Subscription with rfsf_api_client.subscription library

Another important piece of functionality to setup for monitoring using the RFSF API is to subscribe to receive Events from the API EventService. For more details on the EventService and Events in RFSF API, please see section 10 “RFSF API EventService and Events”.

The rfsf_api_client.subscription library provides an EventSubscription object that can be used to subscribe for events and metric reports. After initialization, it must load a config file with information about the subscription requirements in order to generate and send a Subscription request to API EventService. For generic Event subscription, this information is specified in the “/etc/rfsf-api-client-event-subscription.config” file.

By default, the file comes un-populated:

```
[root@kjlmo17-oem hpc-sp-rfsf-api-client]# cat /etc/rfsf-api-client-event-subscription.config
## specify general Event subscription specific information here:
[eventsubscription]
# EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService
event_destination =
# Unique ID for EventDestination
event_destination_id =
```

The file must be populated with an event_destination, which is the target destination endpoint you would like RFSF API to send events to, as well as a unique event_destination_id, which provides a unique id to the EventDestination resource the subscription request will create. Example which will create a subscription to receive events on event_destination: http://172.16.0.1:8082/eventreceiver/, with event_destination_id: “test-event-receiver":

```
[root@kjlmo17-oem hpc-sp-rfsf-api-client]# cat /etc/rfsf-api-client-event-subscription.config
## specify general Event subscription specific information here:
[eventsubscription]
# EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService
event_destination = http://172.16.0.1:8082/eventreceiver/
# Unique ID for EventDestination
event_destination_id = test-event-receiver
```

Once the request is sent to subscribe to receive events, a client side EventReceiver service must be setup to listen for Events sent to the target event_destination by RFSF API EventService. Please see section ”10.7 Subscribers Receiving Events” for more details.

As mentioned previously, the rfsf_api_client.subscription.EventSubscription must load a config file after initialization, so once the /etc/rfsf-api-client-event-subscription.config file is populated, the next step is to initialize the EventSubscription object, and call the load_event_subscription_config(<file>) with this file passed in. Once the object is initialized and

config is loaded, you may use it to call the subscribe_to_events() method which will build and send a subscription request to the API. If the request is successful, the resulting EventSubscription.destination_id should be added to the rfsf_client.event_subscriptions list for tracking. The following code example does all of this. As indicated in the comments, the below code assumes you have already initialized the RfsfClient object for a target user and API destination, as well as authenticated that user and received a valid token:

```
import sys
from rfsf_api_client.subscription.subscribe import EventSubscription, CONFIG_FILE_EVENT_SUBCRIPTION
    # note, the below assume you have already initialized the RfsfClient object as seen above
    # note, the below assumes you have already authenticated the rfsf_client.user and received a valid
rfsf_client.token
    response_content = None
    err_msg = None
    event_subscription = EventSubscription(rfsf_client.token)
    event_subscription.load_event_subscription_config(CONFIG_FILE_EVENT_SUBCRIPTION)
    response_content, err_msg = event_subscription.subscribe_to_events()
    if err_msg:
        print("Failed to subscribe for Event Destination: {0}. {1}... exiting".format(event_subscription.event_destination,
err_msg))
        sys.exit(1)
    else:
        print("Successfully subscribed to receive Events on destination: {0}, Id:
{1}".format(event_subscription.event_destination, event_subscription.destination_id))
        print(response_content)
    # Track event subscription endpoint in top-level rfsf_client object
    rfsf_client.event_subscriptions.append(event_subscription.destination_id)
```

The subscribe_to_events() method will either return a valid response or an error message. If the request it sends is successful, the response content will be that of the /redfish/v1/EventService/Subscriptions/{EventDestinationId} resource that was created. If it is not successful, the error message will indicate why.

Here is an example of a EventDestination response from a successful subscribe_to_events() call - as implied, the output is shown to initialize RfsfClient and authenticate a user as well:

```
Initilized base RfsfClient from config file: /etc/rfsf-api-client-base.config
API destination...kjlmo1200.hpc.amslabs.hpecorp.net
API user...dmatthews
Password for user dmatthews:
Successfully authenticated user: dmatthews
{
    "@odata.context": "Session.Session",
    "@odata.id": "/redfish/v1/SessionService/Sessions/men84q38zDVourFj6V4YgY",
    "@odata.type": "Session.v1_3_0.Session",
    "Description": "Session for user dmatthews",
```

```
    "Id": "men84q38zDVourFj6V4YgY",
    "Name": "dmatthews Session",
    "Oem": {
        "StartTime": "Mon Feb 13 19:23:11 UTC 2023",
        "ExpirationTime": "Tue Feb 14 07:23:11 UTC 2023",
        "TimeLeft": "11h59m59.999964083s"
    },
    "UserName": "dmatthews"
}
Successfully subscribed to receive Events on destination: http://172.16.0.1:8082/eventreceiver/, Id:
/redfish/v1/EventService/Subscriptions/test-event-receiver
{
    "@odata.id": "/redfish/v1/EventService/Subscriptions/test-event-receiver",
    "@odata.type": "",
    "Context": "Public",
    "Destination": "http://172.16.0.1:8082/eventreceiver/",
    "HttpHeaders": [
        {
            "Content-Type": "Application/JSON",
            "OData-Version": "4.0"
        }
    ],
    "Id": "test-event-receiver",
    "Name": "",
    "SNMP": {},
    "Status": {},
    "SubscriptionType": ""
}
```


16.5 LustreStats MetricReport Events Subscription

Using the rfsf_api_client.subscription library

Another important piece of functionality to setup for monitoring using the RFSF API is to subscribe to receive MetricReport Events from the API EventService, which provides a data streaming capability for various system Metrics. For more details on the MetricReport Events in RFSF API, please see section 11 “MetricReport Event Subscriptions for Metric data Streaming”.

The same rfsf_api_client.subscription library and EventSubscription object is used to subscribe to receive MetricReport Events in the same way that it is to subscribe for generic Events, the difference comes in the specific config file that is loaded after object initialization. To receive LustreStats MetricReport Events, the config file used is: “/etc/rfsf-api-client-lustrestats-metric-report-subscription.config”.

By default, the file comes un-populated, except for the data_set – which is predefined to LustreStats and event_types – which is predefined to MetricReport:

```
[root@kjlmo17-oem hpc-sp-rfsf-api-client]# cat /etc/rfsf-api-client-lustrestats-metric-report-subscription.config
## specify general Event subscription specific information here:
[eventsubscription]
# EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService
event_destination =
# Unique ID for EventDestination
event_destination_id =

## Next specify MetricReport specific information
[metricreport]
# Type of events to subscribe to, MetricReport value is required here for dp-api MetricReport setup
event_types = MetricReport
# Number of seconds which dp-api will send MetricReport events to EventDestination.Destination endpoint
send_interval =
# Type of MetricReport data to be sent, options are: LustreStats, JobStats, LinuxStats. Only one may be specified at
a time
data_set = LustreStats
```

The file must be populated with an event_destination, and event_destination_id the same as the generic event subscription config file, however, some additional metric report specific fields exist that must be populated or are pre-populated in the [metricreport] section of the config file.

event_types: must be set to MetricReport and is pre-populated

send_interval: must specify the interval client would like to receive these Events in seconds.

data_set: specifies the target data_set for the MetricReport events, and is pre-populated to LustreStats in this file. The following example will subscribe to receive MetricReports on event_destination: http://172.16.0.1:8082/metricsreceiver/, with event_destinaton_id: lustre-stats-metric-receiver, at a send_interval of 5 seconds:

```
[root@kjlmo17-oem hpc-sp-rfsf-api-client]# cat /etc/rfsf-api-client-lustrestats-metric-report-subscription.config
## specify general Event subscription specific information here:
[eventsubscription]
# EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService
event_destination = http://172.16.0.1:8082/metricsreceiver/
# Unique ID for EventDestination
event_destination_id = lustre-stats-metric-receiver

## Next specify MetricReport specific information
[metricreport]
# Type of events to subscribe to, MetricReport value is required here for dp-api MetricReport setup
event_types = MetricReport
# Number of seconds which dp-api will send MetricReport events to EventDestination.Destination endpoint
send_interval = 5
```

```
# Type of MetricReport data to be sent, options are: LustreStats, JobStats, LinuxStats. Only one may be specified at a time
data_set = LustreStats
```

Once the request is sent to subscribe to receive MetricReport events, a client side EventReceiver service must be setup to listen for Events sent to the target event_destination by RFSF API EventService. Please see section ”10.7 Subscribers Receiving Events” for more details.

As mentioned previously, the rfsf_api_client.subscription.EventSubscription must load a config file after initialization, so once the /etc/rfsf-api-client-lustrestats-metric-report-subscription.config” file is populated, the next step is to initialize the EventSubscription object, and call the load_event_subscription_config(<file>) with this file passed in. Once the object is initialized and config is loaded, you may use it to call the subscribe_to_events() method which will build and send a subscription request to the API. If the request is successful, the resulting EventSubscription.destination_id should be added to the rfsf_client.event_subscriptions list for tracking. The following code example does all of this. As indicated in the comments, the below code assumes you have already initialized the RfsfClient object for a target user and API destination, as well as authenticated that user and received a valid token:

```
import sys
from rfsf_api_client.subscription.subscribe import EventSubscription,
CONFIG_FILE_LUSTRESTATS_METRIC_REPORT_SUBSCRIPTION
    # note, the below assume you have already initialized the RfsfClient object as seen above
    # note, the below assumes you have already authenticated the rfsf_client.user and received a valid
rfsf_client.token
    response_content = None
    err_msg = None
    lustrestats_subscription = EventSubscription(rfsf_client.token)

lustrestats_subscription.load_event_subscription_config(CONFIG_FILE_LUSTRESTATS_METRIC_REPORT_SUBSCRIPTI
ON)
    response_content, err_msg = lustrestats_subscription.subscribe_to_events(metric_report=True)
    if err_msg:
        print("Failed to subscribe for {0} MetricReport Event Destination: {1}. {2}...
exiting".format(lustrestats_subscription.data_set, lustrestats_subscription.event_destination, err_msg))
        sys.exit(1)
    else:
        print("Successfully subscribed to receive {0} MetricReport Events on destination: {1}, Id:
{2}".format(lustrestats_subscription.data_set, lustrestats_subscription.event_destination,
lustrestats_subscription.destination_id))
        print(response_content)
    # Track event subscription endpoint in top-level rfsf_client object
    rfsf_client.event_subscriptions.append(lustrestats_subscription.destination_id)
```

The subscribe_to_events() method will either return a valid response or an error message. If the request it sends is successful, the response content will be that of the /redfish/v1/EventService/Subscriptions/{EventDestinationId} resource that was created. If it is not successful, the error message will indicate why.

Here is an example of a EventDestination response from a successful subscribe_to_events() call - as implied, the output is shown to initialize RfsfClient and authenticate a user as well:

```
Initialized base RfsfClient from config file: /etc/rfsf-api-client-base.config
API destination...kjlmo1200.hpc.amslabs.hpecorp.net
API user...dmatthews
Password for user dmatthews:
Successfully authenticated user: dmatthews
{
    "@odata.context": "Session.Session",
    "@odata.id": "/redfish/v1/SessionService/Sessions/i4n3CYMZx5TuKjZJvhA3BP",
    "@odata.type": "Session.v1_3_0.Session",
    "Description": "Session for user dmatthews",
    "Id": "i4n3CYMZx5TuKjZJvhA3BP",
    "Name": "dmatthews Session",
    "Oem": {
        "StartTime": "Mon Feb 13 19:15:47 UTC 2023",
        "ExpirationTime": "Tue Feb 14 07:15:47 UTC 2023",
        "TimeLeft": "11h59m59.999965836s"
    },
    "UserName": "dmatthews"
}
Successfully subscribed to receive LustreStats MetricReport Events on destination:
http://172.16.0.1:8082/metricsreceiver/, Id: /redfish/v1/EventService/Subscriptions/lustre-stats-metric-receiver
{
    "@odata.id": "/redfish/v1/EventService/Subscriptions/lustre-stats-metric-receiver",
    "@odata.type": "",
    "Context": "Public",
    "Destination": "http://172.16.0.1:8082/metricsreceiver/",
    "EventTypes": [
        "MetricReport"
    ],
    "HttpHeaders": [
        {
            "Content-Type": "Application/JSON",
            "OData-Version": "4.0"
        }
    ],
    "Id": "lustre-stats-metric-receiver",
    "MetricReportDefinitions": [
        {
            "data_set": "LustreStats",
```

```
            "send_interval": "5"
        }
    ],
    "Name": "",
    "SNMP": {},
    "Status": {},
    "SubscriptionType": ""
}
```


16.6 LinuxStats MetricReport Events Subscription

rfsf_api_client.subscription library

Same as the mechanism to subscribe to receive LustreStats MetricReport Events, you can also subscribe to receive MetricReport Events on other data sets, one of which being: LinuxStats. LinuxStatus MetricReport subscriptions and Events work the same as MetricReports for other data sets, just provide different data. For more details on the MetricReport Events in RFSF API, please see section 11 “MetricReport Event Subscriptions for Metric data Streaming”.

The same rfsf_api_client.subscription library and EventSubscription object is used to subscribe to receive MetricReport Events in the same way that it is to subscribe for generic Events, the difference comes in the specific config file that is loaded after object initialization. To receive LinuxStats MetricReport Events, the config file used is: “/etc/rfsf-api-client-linuxstats-metric-report-subscription.config”.

By default, the file comes un-populated, except for the data_set – which is predefined to LinuxStats and event_types – which is predefined to MetricReport:

```
[root@kjlmo17-oem ~]# cat /etc/rfsf-api-client-linuxstats-metric-report-subscription.config
## specify general Event subscription specific information here:
[eventsubscription]
# EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService
event_destination =
# Unique ID for EventDestination
event_destination_id =

## Next specify MetricReport specific information
[metricreport]
# Type of events to subscribe to, MetricReport value is required here for dp-api MetricReport setup
event_types = MetricReport
# Number of seconds which dp-api will send MetricReport events to EventDestination.Destination endpoint
send_interval =
# Type of MetricReport data to be sent, options are: LustreStats, JobStats, LinuxStats. Only one may be specified at
a time
```

```
data_set = LinuxStats
```

The file must be populated with an event_destination, and event_destination_id the same as the generic event subscription config file, however, some additional metric report specific fields exist that must be populated or are pre-populated in the [metricreport] section of the config file.

event_types: must be set to MetricReport and is pre-populated

send_interval: must specify the interval client would like to receive these Events in seconds.

data_set: specifies the target data_set for the MetricReport events, and is pre-populated to LustreStats in this file. The following example will subscribe to receive MetricReports on event_destination: http://172.16.0.1:8082/metricsreceiver/, with event_destinaton_id: linux-stats-metric-receiver, at a send_interval of 15 seconds:

```
[root@kjlmo17-oem ~]# cat /etc/rfsf-api-client-linuxstats-metric-report-subscription.config
## specify general Event subscription specific information here:
[eventsubscription]
# EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService
event_destination = http://172.16.0.1:8082/metricsreceiver/
# Unique ID for EventDestination
event_destination_id = linux-stats-metric-receiver

## Next specify MetricReport specific information
[metricreport]
# Type of events to subscribe to, MetricReport value is required here for dp-api MetricReport setup
event_types = MetricReport
# Number of seconds which dp-api will send MetricReport events to EventDestination.Destination endpoint
send_interval = 15
# Type of MetricReport data to be sent, options are: LustreStats, JobStats, LinuxStats. Only one may be specified at
a time
data_set = LinuxStats
```

Once the request is sent to subscribe to receive MetricReport events, a client side EventReceiver service must be setup to listen for Events sent to the target event_destination by RFSF API EventService. Please see section ”10.7 Subscribers Receiving Events” for more details.

As mentioned previously, the rfsf_api_client.subscription.EventSubscription must load a config file after initialization, so once the /etc/rfsf-api-client-linuxstats-metric-report-subscription.config” file is populated, the next step is to initialize the EventSubscription object, and call the load_event_subscription_config(<file>) with this file passed in. Once the object is initialized and config is loaded, you may use it to call the subscribe_to_events() method which will build and send a subscription request to the API. If the request is successful, the resulting EventSubscription.destination_id should be added to the rfsf_client.event_subscriptions list for

tracking. The following code example does all of this. As indicated in the comments, the below code assumes you have already initialized the RfsfClient object for a target user and API destination, as well as authenticated that user and received a valid token:

```
import sys
from rfsf_api_client.subscription.subscribe import EventSubscription,
CONFIG_FILE_LINUXSTATS_METRIC_REPORT_SUBSCRIPTION
    # note, the below assume you have already initialized the RfsfClient object as seen above
    # note, the below assumes you have already authenticated the rfsf_client.user and received a valid
rfsf_client.token
    response_content = None
    err_msg = None
    linuxtats_subscription = EventSubscription(rfsf_client.token)

linuxtats_subscription.load_event_subscription_config(CONFIG_FILE_LINUXSTATS_METRIC_REPORT_SUBSCRIPTION)
    response_content, err_msg = linuxtats_subscription.subscribe_to_events(metric_report=True)
    if err_msg:
        print("Failed to subscribe for {0} MetricReport Event Destination: {1}. {2}...
exiting".format(linuxtats_subscription.data_set, linuxtats_subscription.event_destination, err_msg))
        sys.exit(1)
    else:
        print("Successfully subscribed to receive {0} MetricReport Events on destination: {1}, Id:
{2}".format(linuxtats_subscription.data_set, linuxtats_subscription.event_destination,
linuxtats_subscription.destination_id))
        print(response_content)
    # Track event subscription endpoint in top-level rfsf_client object
    rfsf_client.event_subscriptions.append(linuxtats_subscription.destination_id)
```

The subscribe_to_events() method will either return a valid response or an error message. If the request it sends is successful, the response content will be that of the /redfish/v1/EventService/Subscriptions/{EventDestinationId} resource that was created. If it is not successful, the error message will indicate why.

Here is an example of a EventDestination response from a successful subscribe_to_events() call - as implied, the output is shown to initialize RfsfClient and authenticate a user as well:

```
Initialized base RfsfClient from config file: /etc/rfsf-api-client-base.config
API destination...kjlmo1200.hpc.amslabs.hpecorp.net
API user...dmatthews
Password for user dmatthews:
Successfully authenticated user: dmatthews
{
    "@odata.context": "Session.Session",
    "@odata.id": "/redfish/v1/SessionService/Sessions/aFdaRaprBmCkHrhvYrU5rW",
    "@odata.type": "Session.v1_3_0.Session",
```

```
    "Description": "Session for user dmatthews",
    "Id": "aFdaRaprBmCkHrhvYrU5rW",
    "Name": "dmatthews Session",
    "Oem": {
        "StartTime": "Mon Feb 13 18:45:06 UTC 2023",
        "ExpirationTime": "Tue Feb 14 06:45:06 UTC 2023",
        "TimeLeft": "11h59m59.999966217s"
    },
    "UserName": "dmatthews"
}
Successfully subscribed to receive LinuxStats MetricReport Events on destination:
http://172.16.0.1:8082/metricsreceiver/, Id: /redfish/v1/EventService/Subscriptions/linux-stats-metric-receiver
{
    "@odata.id": "/redfish/v1/EventService/Subscriptions/linux-stats-metric-receiver",
    "@odata.type": "",
    "Context": "Public",
    "Destination": "http://172.16.0.1:8082/metricsreceiver/",
    "EventTypes": [
        "MetricReport"
    ],
    "HttpHeaders": [
        {
            "Content-Type": "Application/JSON",
            "OData-Version": "4.0"
        }
    ],
    "Id": "linux-stats-metric-receiver",
    "MetricReportDefinitions": [
        {
            "data_set": "LinuxStats",
            "send_interval": "15"
        }
    ],
    "Name": "",
    "SNMP": {},
    "Status": {},
    "SubscriptionType": ""
}
```


16.7 JobStats MetricReport Subscription

Using rfsf_api_client.subscription library

Same as the mechanism to subscribe to receive LustreStats and LinuxStats MetricReport Events, you can also subscribe to receive MetricReport Events on other data sets, one of which being: JobStats. JobStats MetricReport subscriptions and Events work the same as MetricReports for other data sets, just provide different data. For more details on the MetricReport Events in RFSF API, please see section 11 "MetricReport Event Subscriptions for Metric data Streaming".

The same rfsf_api_client.subscription library and EventSubscription object is used to subscribe to receive MetricReport Events in the same way that it is to subscribe for generic Events, the difference comes in the specific config file that is loaded after object initialization. To receive LinuxStats MetricReport Events, the config file used is: "/etc/rfsf-api-client-jobstats-metric-report-subscription.config".

By default, the file comes un-populated, except for the data_set – which is predefined to JobStats and event_types – which is predefined to MetricReport:

```
[root@kjlmo17-oem hpc-sp-rfsf-api-client]# cat /etc/rfsf-api-client-jobstats-metric-report-subscription.config
## specify general Event subscription specific information here:
[eventsubscription]
# EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService
event_destination =
# Unique ID for EventDestination
event_destination_id =

## Next specify MetricReport specific information
[metricreport]
# Type of events to subscribe to, MetricReport value is required here for dp-api MetricReport setup
event_types = MetricReport
# Number of seconds which dp-api will send MetricReport events to EventDestination.Destination endpoint
send_interval =
# Type of MetricReport data to be sent, options are: LustreStats, JobStats, LinuxStats. Only one may be specified at
a time
data_set = JobStats
```

The file must be populated with an event_destination, and event_destination_id the same as the generic event subscription config file, however, some additional metric report specific fields exist that must be populated or are pre-populated in the [metricreport] section of the config file.

event_types: must be set to MetricReport and is pre-populated

send_interval: must specify the interval client would like to receive these Events in seconds.

data_set: specifies the target data_set for the MetricReport events, and is pre-populated to JobStats in this file.

The following example will subscribe to receive MetricReports on event_destination: http://172.16.0.1:8082/metricsreceiver/, with event_destinaton_id: job-stats-metric-receiver, at a send_interval of 30 seconds:

```
[root@kjlmo17-oem hpc-sp-rfsf-api-client]# cat /etc/rfsf-api-client-jobstats-metric-report-subscription.config
## specify general Event subscription specific information here:
[eventsubscription]
# EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService
event_destination = http://172.16.0.1:8082/metricsreceiver/
# Unique ID for EventDestination
event_destination_id = job-stats-metric-receiver

## Next specify MetricReport specific information
[metricreport]
# Type of events to subscribe to, MetricReport value is required here for dp-api MetricReport setup
event_types = MetricReport
# Number of seconds which dp-api will send MetricReport events to EventDestination.Destination endpoint
send_interval = 30
# Type of MetricReport data to be sent, options are: LustreStats, JobStats, LinuxStats. Only one may be specified at
a time
data_set = JobStats
```

Once the request is sent to subscribe to receive MetricReport events, a client side EventReceiver service must be setup to listen for Events sent to the target event_detination by RFSF API EventService. Please see section ”10.7 Subscribers Receiving Events” for more details.

As mentioned previously, the rfsf_api_client.subscription.EventSubscription must load a config file after initialization, so once the /etc/rfsf-api-client-jobstats-metric-report-subscription.config” file is populated, the next step is to initialize the EventSubscription object, and call the load_event_subscription_config(<file>) with this file passed in. Once the object is initialized and config is loaded, you may use it to call the subscribe_to_events() method which will build and send a subscription request to the API. If the request is successful, the resulting EventSubscription.destination_id should be added to the rfsf_client.event_subscriptions list for tracking. The following code example does all of this. As indicated in the comments, the below code assumes you have already initialized the RfsfClient object for a target user and API destination, as well as authenticated that user and received a valid token:

```
import sys
from rfsf_api_client.subscription.subscribe import EventSubscription,
CONFIG_FILE_JOBSTATS_METRIC_REPORT_SUBSCRIPTION
    # note, the below assume you have already initialized the RfsfClient object as seen above
    # note, the below assumes you have already authenticated the rfsf_client.user and received a valid
rfsf_client.token
    response_content = None
    err_msg = None
    jobstats_subscription = EventSubscription(rfsf_client.token)
    jobstats_subscription.load_event_subscription_config(CONFIG_FILE_JOBSTATS_METRIC_REPORT_SUBSCRIPTION)
    response_content, err_msg = jobstats_subscription.subscribe_to_events(metric_report=True)
```

```
    if err_msg:
        print("Failed to subscribe for {0} MetricReport Event Destination: {1}. {2}...
exiting".format(jobstats_subscription.data_set, jobstats_subscription.event_destination, err_msg))
        sys.exit(1)
    else:
        print("Successfully subscribed to receive {0} MetricReport Events on destination: {1}, Id:
{2}".format(jobstats_subscription.data_set, jobstats_subscription.event_destination,
jobstats_subscription.destination_id))
        print(response_content)
    # Track event subscription endpoint in top-level rfsf_client object
    rfsf_client.event_subscriptions.append(jobstats_subscription.destination_id)
```

The subscribe_to_events() method will either return a valid response or an error message. If the request it sends is successful, the response content will be that of the /redfish/v1/EventService/Subscriptions/{EventDestinationId} resource that was created. If it is not successful, the error message will indicate why.

Here is an example of a EventDestination response from a successful subscribe_to_events() call - as implied, the output is shown to initialize RfsfClient and authenticate a user as well:

```
Initialized base RfsfClient from config file: /etc/rfsf-api-client-base.config
API destination...kjlmo1200.hpc.amslabs.hpecorp.net
API user...dmatthews
Password for user dmatthews:
Successfully authenticated user: dmatthews
{
    "@odata.context": "Session.Session",
    "@odata.id": "/redfish/v1/SessionService/Sessions/dZkPfRQbByqjSw6yZ7umQi",
    "@odata.type": "Session.v1_3_0.Session",
    "Description": "Session for user dmatthews",
    "Id": "dZkPfRQbByqjSw6yZ7umQi",
    "Name": "dmatthews Session",
    "Oem": {
        "StartTime": "Mon Feb 13 15:32:09 UTC 2023",
        "ExpirationTime": "Tue Feb 14 03:32:09 UTC 2023",
        "TimeLeft": "11h59m59.999939216s"
    },
    "UserName": "dmatthews"
}
Successfully subscribed to receive JobStats MetricReport Events on destination:
http://172.16.0.1:8082/metricsreceiver/, Id: /redfish/v1/EventService/Subscriptions/job-stats-metric-receiver
{
    "@odata.id": "/redfish/v1/EventService/Subscriptions/job-stats-metric-receiver",
    "@odata.type": "",
    "Context": "Public",
    "Destination": "http://172.16.0.1:8082/metricsreceiver/",
    "EventTypes": [
```

```
        "MetricReport"
    ],
    "HttpHeaders": [
        {
            "Content-Type": "Application/JSON",
            "OData-Version": "4.0"
        }
    ],
    "Id": "job-stats-metric-receiver",
    "MetricReportDefinitions": [
        {
            "data_set": "JobStats",
            "send_interval": "30"
        }
    ],
    "Name": "",
    "SNMP": {},
    "Status": {},
    "SubscriptionType": ""
}
```


16.8 Resource Tree Iteration Examples

Once the entire resource tree that the RFSF API tracks has been discovered and populated in the top-level RfsfClient object, it is important to understand the structure of the tree and how to iterate in order to reach all available API resource endpoints and use them for some sort of periodic monitoring via GET requests to each endpoint. As stated in section 4 “Redfish/Swordfish API ServiceRoot” - all resources live underneath the service root: /redfish/v1, thus – this is the entry point to begin iterating through the structure of the tree. Then, as suggested in section various sub-sections of section 2 “High level usage”, the next level to traverse down the tree is the top-level resource collection endpoints seen just underneath the ServiceRoot. From there, you look at each resource within the collections to determine additional resource collections within that, and so on.

The following shows a couple of code examples that can be used to traverse into the resource tree, starting both at the service root, as well as various service collection entry points, to discover and print all resource endpoints within.

First, we will look at the rfsf_client.resource_tree object attributes that hold the service root endpoint, as well as all top-level service entry point endpoints:

```
rfsf_client.resource_tree.service_root # /redfish/v1
rfsf_client.resource_tree.event_service_uri # /redfish/v1/EventService
rfsf_client.resource_tree.subscriptions # /redfish/v1/EventService/Subscriptions
```

```
rfsf_client.resource_tree.event_collection_uri # /redfish/v1/Events
rfsf_client.resource_tree.session_service_uri # /redfish/v1/SessionService
rfsf_client.resource_tree.session_collection_uri # /redfish/v1/SessionService/Sessions
rfsf_client.resource_tree.chassis_collection_uri # /redfish/v1/Chassis
rfsf_client.resource_tree.storage_service_collection_uri # /redfish/v1/StorageServices
rfsf_client.resource_tree.storage_system_collection_uri # /redfish/v1/StorageSystems
```

Next, we'll look at how to pull out the service root tree from the full rfsf_client.tree structure. This service root tree, can then be used to further index into the structure with the top-level URIs:

```
# pull ServiceRoot and below out of full ResourceTree
service_root_tree = rfsf_client.resource_tree.tree[rfsf_client.resource_tree.service_root]
```

The following will then show examples of either: 1) indexing into the top-level service_root_tree or 2) indexing into a starting resource collection substructure, to print out all collected resource endpoints that the API tracks. Since the output can be very long, only code examples will be shown. Please see rfsf_api_client.monitoring.rfsf_api_monitoring library for a full set of examples to run.

/redfish/v1/EventService and /redfish/v1/EventService/Subscriptions

```
    ## 1) index into full resource_tree to pull it out
    print("########## EventService Resource (Subscriptions) {0} FROM FULL TREE
##########".format(rfsf_client.resource_tree.event_service_uri))
    event_service_tree = service_root_tree[rfsf_client.resource_tree.event_service_uri]
    print(json.dumps(event_service_tree, indent=4))
    ## 2) reference the collection attribute collected directly
    print("########## Subscriptions Collection Resource ##########")
    print(json.dumps(rfsf_client.resource_tree.subscriptions, indent=4))
```

/redfish/v1/SessionService and /redfish/v1/SessionService/Sessions

```
    ## 1) index into full resource_tree to pull it out
    print("########## SessionService Resource (Sessions) {0} FROM FULL TREE
##########".format(rfsf_client.resource_tree.event_service_uri))
    session_service_tree = service_root_tree[rfsf_client.resource_tree.session_service_uri]
    print(json.dumps(session_service_tree, indent=4))
    ## 2) reference the collection attribute collected directly
    print("########## Sessions Collection Resource ##########")
    print(json.dumps(rfsf_client.resource_tree.sessions, indent=4))
```

/redfish/v1/Events

```
    ## 1) index into full resource_tree to pull it out
```

```
    print("########## Event Collection Resource {0} FROM FULL TREE
 ##########".format(rfsf_client.resource_tree.event_collection_uri))
    event_collection_tree = service_root_tree[rfsf_client.resource_tree.event_collection_uri]
    print(json.dumps(event_collection_tree, indent=4))
    ## 2) reference the collection attribute collected directly
    print("########## Event Collection Resource ##########")
    print(json.dumps(rfsf_client.resource_tree.event_collection_members, indent=4))
```

/redfish/v1/Chassis

```
  ## 1) index into full resource_tree to pull it out
  print("########## Chassis Collection Resource Tree {0} FROM FULL TREE
##########".format(rfsf_client.resource_tree.chassis_collection_uri))
  # pull ChassisCollection and below out of full ResourceTree
  chassis_tree = service_root_tree[rfsf_client.resource_tree.chassis_collection_uri]
  for chassis_member in rfsf_client.resource_tree.chassis_members:
     for k, v in chassis_member.items():
        print("Chassis: {0}".format(v))
        chassis_resources = chassis_tree[v]
        print(json.dumps(chassis_resources, indent=4))
  ## 2) reference the collection attribute collected directly
  print("########## Chassis Collection Resource Tree ##########")
  print(json.dumps(rfsf_client.resource_tree.chassis_resources, indent=4))
```

/redfish/v1/StorageServices

```
    ## 1) index into full resource_tree to pull it out
    print("########## StorageService Collection Resource Tree {0} FROM FULL TREE
 ##########".format(rfsf_client.resource_tree.storage_service_collection_uri))
    # pull StorageServiceCollection and below out of full ResourceTree
    storage_service_tree = service_root_tree[rfsf_client.resource_tree.storage_service_collection_uri]
    for storage_service_member in rfsf_client.resource_tree.storage_service_members:
       for k, v in storage_service_member.items():
          print("StorageService: {0}".format(v))
          storage_service_resources = storage_service_tree[v]
          print(json.dumps(storage_service_resources, indent=4))
    ## 2) reference the collection attribute collected directly
    print("########## StorageService Collection Resource Tree ##########")
    print(json.dumps(rfsf_client.resource_tree.storage_service_resources, indent=4))
```

/redfish/v1/StorageSystems

```
    ## 1) index into full resource_tree to pull it out
    print("########## StorageSystem Collection Resource Tree {0}  FROM FULL TREE
 ##########".format(rfsf_client.resource_tree.storage_system_collection_uri))
    # pull StorageSystem Collection and below out of full ResourceTree
    storage_systems_tree = service_root_tree[rfsf_client.resource_tree.storage_system_collection_uri]
```

```
for storage_system_member in rfsf_client.resource_tree.storage_system_members:
    for k, v in storage_system_member.items():
        print("StorageSystem: {0}".format(v))
        storage_system_resources = storage_systems_tree[v]
        print(json.dumps(storage_system_resources, indent=4))
## 2) reference the collection attribute collected directly
print("########## StorageSystem Collection Resource Tree ##########")
print(json.dumps(rfsf_client.resource_tree.storage_system_resources, indent=4))
```

16.9 Example Event based monitoring setup

Once you have subscribed to receive generic Events on resources at some destination that was appended to the rfsf_client.event_subscriptions list, you can programmatically start up an EventReceiver service as a background thread to listen for Events broadcasted from RFSF API EventService. This example starts the POST request handler in a daemon thread, and in the main loop receives Events from the thread off of a queue. Once it receives and event off the queue, it will parse the resource id which the Event occurred and check the state of the Event with a GET request to that Event. For more details on the event receiver started in the thread, please see rfsf_api_client.event_receiver.generic_event_receiver library.

The following code shows the example described above.

```
"""
    Example to start EventReceiver and process / react to Events:
    The following code shows an example use case of how to start up an EventReceiver, listening on the
EventDestination endpoint specified in
    the generic Event subscription config file: /etc/rfsf-api-client-event-subscription.config. It relies on the
event_subscription object previously
    created to track to start and track a daemon thread that will start the EventReceiver service. A Queue is then
used to process Events received
    from the API on the EventDesitnation endpoint the receiver is listening on and react to them in some way.

    This particular example, will parse out the Event.Oem.ResourceId field from the Event resource body. This
ResourceId field indicates which resource
    in the RfsfClient.resource_tree the Event occurred on. Since an event was received on this ResourceId, a GET
request is issued to the resource endpoint
    in an atttempt to see what changed in the resource. The functionality can be expanded to do as the client sees
fit to react to Events on resources.
"""
 # setup event Queue for communication with event receiver threads
rfsf_client.event_queue = queue.Queue()
# create thread for event receiver
generic_event_receiver_thrd = threading.Thread(target=start_event_receiver,
name=event_subscription.destination_id, args=(event_subscription.event_destination, rfsf_client.event_queue))
```

```
generic_event_receiver_thrd.daemon = True
# Track event_destination to start event receiver on
rfsf_client.event_receivers[event_subscription.event_destination] = generic_event_receiver_thrd
generic_event_receiver_thrd.start()
while True:
    event_dict = {}
    resources = []
    if not rfsf_client.event_queue.empty():
        event_str = rfsf_client.event_queue.get()
        # TODO add try/except here
        try:
            event_dict = json.loads(event_str)
            print("Received Event {0}, type: {1}".format(event_dict, type(event_dict)))
            event_list = event_dict.get("Events", [])
            for event in event_list:
                event_oem = event.get("Oem", {})
                resource_id = event_oem.get("ResourceId")
                if resource_id:
                    resources.append(resource_id)
            for rsrc in resources:
                print("Checking Status Event Resource {0}".format(rsrc))
                response_content, err_msg = check_resource(rsrc, rfsf_client.destination, rfsf_client.token)
                if err_msg:
                    print(err_msg)
                else:
                    print(response_content)
        except Exception as e:
            print("Failed to process Event. {0}".format(e))
```

Appendix 1. Client Notes

Client location on ClusterStor
So to start, the "rfsf-api-client" package is available on the local cluster repo, and can be downloaded from either mgmt node and then scp'd off of the system to wherever they'd like to run it.

n00 # ~ dnf download rfsf-api-client

^^ this should download the rpm to the CWD and they should be able to copy it off of the system over the EAN network on pub0 to another server or location in their lab.

Once the rpm is copied to another server, so long as python3 is available on that server, they should be able to install it without any dependency issues.

Next step would be to correctly populate the config files they'd like to use, I will show an example of the two required (which they already noted) with comments – first the base.config file:

[root@c5ftc03-oem ~]# cat /etc/rfsf-api-client-base.config

RFSF API setup configuration file
specify information about target cluster here

[system]

FQDN or IP address to connect to API on

destination = kjlmo1200.hpc.amslabs.hpecorp.net # <-- destination FQDN or IP address of mgmt=primary host on EAN (login IP or FQDN)

specify information about user to authenticate here

[authentication]

username to authenticate for API use

user = dmatthews # <-- user to authenticate to use the API with (previously created with cscli admins add as documentation describes)

Next, the linuxstats-metric-report-subscription.config file:

[root@c5ftc03-oem ~]# cat /etc/rfsf-api-client-linuxstats-metric-report-subscription.config

specify general Event subscription specific information here:

[eventsubscription]

EventReceiver service EventDestination endpoint to receive Events broadcasted from dp-api EventService

event_destination = http://172.16.0.1:9085/metricsreceiver/ # <-- example client endpoint that EventReceiver service is listening on

Unique ID for EventDestination

event_destination_id = test-linux-stats # <-- some unique Id for the Subscription

Next specify MetricReport specific information

[metricreport]

Type of events to subscribe to, MetricReport value is required here for dp-api MetricReport setup

event_types = MetricReport

Number of seconds which dp-api will send MetricReport events to EventDestination.Destination endpoint

send_interval = 60 # <-- 60s is the default SensingInterval for LinuxStats defined by /redfish/v1/TelemetryService/MetricDefinitions/LinuxStats resource, this can be changed or updated with a PUT or PATCH request to that resource, but subscription request requires that the send_interval is equal to or greater than the SensingInterval time defined in TelemetryService/MetricDefinitions for each data set

Type of MetricReport data to be sent, options are: LustreStats, JobStats, LinuxStats. Only one may be specified at a time

data_set = LinuxStats

Once those two files are in place, then the example code they provided is pretty close to what would be required to create the subscription. I'll share an example derived from "/usr/share/rfsf-api-client/examples/monitoring/rfsf_api_monitoring.py":

```
import sys

import json

from rfsf_api_client.client.rfsf_api_client import RfsfClient, CONFIG_FILE_BASE

from rfsf_api_client.authentication.auth import Auth

from rfsf_api_client.subscription.subscribe import EventSubscription,
CONFIG_FILE_LINUXSTATS_METRIC_REPORT_SUBSCRIPTION


rfsf_client = RfsfClient()

print("Initilized base RfsfClient from config file: {0}".format(CONFIG_FILE_BASE))

print("API destination...{0}".format(rfsf_client.destination))

print("API user...{0}".format(rfsf_client.user))


response_content = None

err_msg = None

auth = Auth(rfsf_client.user)

response_content, err_msg = auth.authenticate()
```

```
if err_msg:

    print("Failed to authenticate user: {0}. {1}... exiting".format(rfsf_client.user, err_msg))

    sys.exit(1)

else:

    print("Successfully authenticated user: {0} ".format(rfsf_client.user))

    print(json.dumps(response_content, indent=4))

# Assign auth token to top-level rfsf_client object

rfsf_client.token = auth.token



response_content = None

err_msg = None

linuxtats_subscription = EventSubscription(rfsf_client.token)

linuxtats_subscription.load_event_subscription_config(CONFIG_FILE_LINUXSTATS_METRI
C_REPORT_SUBSCRIPTION)

response_content, err_msg = linuxtats_subscription.subscribe_to_events(metric_report=True)

if err_msg:

    print("Failed to subscribe for {0} MetricReport Event Destination: {1}. {2}...
exiting".format(linuxtats_subscription.data_set, linuxtats_subscription.event_destination,
err_msg))

    sys.exit(1)

else:

    print("Successfully subscribed to receive {0} MetricReport Events on destination: {1}, Id:
{2}".format(linuxtats_subscription.data_set, linuxtats_subscription.event_destination,
linuxtats_subscription.destination_id))

    print(json.dumps(response_content, indent=4))
```

```
# Track event subscription endpoint in top-level rfsf_client object

rfsf_client.event_subscriptions.append(linuxtats_subscription.destination_id)
```

The above code simply creates the subscription, prints the response, and exists – whether that is successful or not. The next piece is setting up a client receiver. Using the example above where the EventDestination.Destination was set as “http://172.16.0.1:9085/metricsreceiver/” – I’ll show starting an EventRecevier listing on port 9085 using the rfsf-api-client examples:

```
[root@kjlmo17-oem ~]# python3 /usr/share/rfsf-api-
client/examples/metrics_receiver/test_metrics_receiver.py 9085

INFO:root:Starting httpd...
```

The above is now setup to receive MetricReport’s from dp-api EventService in the form of a POST request from dp-api-server -> the client Destination EventRecevier. I’ll note that “/usr/share/rfsf-api-client/examples/monitoring/rfsf_api_monitoring.py” also provides an example of setting up an event receiver thread and processing Events received on a queue that they could take a look at to expand the basic example receiver, code is starting here:

```
    """

        Example to start EventReceiver and process / react to Events:

        ...
```

With all of this being said, I’d like to emphasize that rfsf-api-client is largely only intended to provide examples. There are few uses of it in production by cscli, but at a very limited capacity. The goal was to provide base libraries to authenticate, discover the resource tree, subscribe to receive events, and objects to store info in. The /usr/share/rfsf-api-client/examples in the package are intended to show examples of using the libraries, but are meant to be expanded upon, i.e. we do not provide a fully capable client side library, just enough to get people started and exposed to programmatically interacting with the API.